Box2D：从数学物理到源码实现，再到优化

> 一套面向“想知道做了什么、怎样做、为什么这样做”的分章教材。 阅读顺序：**物理现象 → 数学模型 → 手算 → 朴素实现 → 当前源码 → 优化与代价。** 本目录独立于 `readmegrok/`，不修改原文，也不修改引擎源码。

1. 本书对应哪份代码

- 仓库：当前工作区的 Box2D。
- 源码基线：`dc67c40ba2ce1ad384b0c985dbc1b539cba9221b`。
- 提交日期：2026-09-03；提交标题：`pkgconfig support (#1102)`。
- `CMakeLists.txt` 第 4–9 行声明版本 **3.2.0**。这里不据此声称它是某个官方发布标签。
- 文档核对日期：2026-09-10。
- 研究对象：`src/`、`include/box2d/`、`test/`，并参考 `readmegrok/` 的主题组织。

后续源码变化后，行号可能移动。请优先按函数名定位，再核对分支。 本书说明的是这一工作树，不把旧版本 Box2D 的实现直接套到当前代码上。

2. 和原文相比，重点补了什么

1. 不只给出公式：说明每个核心量的单位、方向、使用前提，以及公式怎么从前一步推出来。
2. 不只说“打一脚”：把冲量怎样同时改变平动与转动、怎样改变接触点速度算完。
3. 不只说“更稳定”：说明积分误差、约束耦合、位置修正注入速度以及 Relax 的作用。
4. 不把论文算法等同源码：明确区分**物理定律、数值模型、教学简化、当前实现。**
5. 先给单线程、标量、少量物体的正确方案，最后才解释动态树、缓存、睡眠、SIMD 与并行。
6. 修正原文中自由落体算例、有效质量名称、恢复 bias 符号、热启动步骤、CCD 范围等问题。
7. 用完整阶段表、算例、练习答案、源码索引和实验判据，帮助读者自查，而非只凭动画判断。

3. 完整目录

第一篇：建立数学与物理基础

章节	内容	学完能做什么
00 · 约定与学习路线	单位、符号、正负号、阅读方法、各层模型的边界	不再把位置、速度、力、冲量混为一谈
01 · 二维数学与坐标变换	点积、叉积、旋转、局部坐标、质心与原点	手算一个接触点的位置和速度

章节	内容	学完能做什么
02a · 转动惯量的定义与由来	定义、平行轴推导、多块合并的质心与惯量	能推出 $I_O = I_C + md^2$，并能把两块焊成一个 I_C
02 · 质量质心与转动惯量	几何积分、平行轴、复合刚体、源码近似	明白物体为什么有不同的转动响应
03 · 力力矩冲量与积分	牛顿定律、冲量、欧拉积分、阻尼、旋转更新	从连续运动方程走到每步更新
04 · 从零实现最小物理世界	最小数据、单线程步骤、实现里程碑与验收	先搭出可验证的朴素引擎，而非照搬优化框架

概念串讲：不看碰撞检测，先把一步模拟讲清

章节	内容	学完能做什么
22 · 二维物理模拟与约束冲量	分析力学用了什么、半隐式欧拉、关节如何变约束、$\mathbf{V}, u, J, M, m_{\text{eff}}$ 推导	用一句话说明求解器在算哪一档速度、打多大冲量

第 22 章是概念串讲，不替代第 03、09、13 章的数字表和源码对照。建议在第 03 章之后、或开始第 09 章之前阅读；碰撞几何仍走第二篇。

第二篇：碰撞几何——先回答在哪里接触

章节	内容	学完能做什么
05 · 基础碰撞与接触流形	总流程、5 种形状、12 种配对、每种数字例	先指出当前形状对走哪一格，再算出接触几何
06 · SAT裁剪与链段接触	投影、参考边、入射边、裁剪、单面链段	理解一个或两个接触点从何而来
07 · GJK距离与形状查询	支撑函数、Minkowski 差、单纯形、凸包、查询	跟踪距离算法的一次迭代
08 · TOI与连续碰撞	扫掠、求根、TOI、fast/bullet 的实际调度	分清推测接触、形状扫掠和刚体 CCD

第三篇：接触求解——再回答怎样改变运动

章节	内容	学完能做什么
09 · 碰撞冲量与约束方程	动量、恢复、Jacobian、有效质量、耦合	从接触目标推导冲量公式
10 · 累计冲量摩擦与热启动	累计投影、缓存施加、摩擦、滚阻	知道为何不能只钳制增量或只保存旧冲量
11 · 软约束子步进与Relax	弹簧阻尼离散化、三系数、推测、修正、松弛	把数值稳定化与真实物理区分开

章节	内容	学完能做什么
12 · 接触求解器逐行导读与完整算例	Prepare → WarmStart → Solve → 积分 → Relax → Store	对照标量源码完成一整个子步

第四篇：关节——用约束描述机械结构

章节	内容	学完能做什么
13 · 距离关节与统一关节框架	硬杆、弹簧、限位、马达、统一阶段	从一个约束目标构造求解过程
14 · 旋转棱柱轮与焊接关节	二维有效质量矩阵、滑轨、悬挂、焊接	知道各关节保留和消除了什么自由度
15 · 马达mover与pogo关节	目标速度、目标位姿、逐轴限力、方向弹簧、filter	分清驱动关节和角色几何查询

第五篇：回到完整引擎，最后学习优化

章节	内容	学完能做什么
16 · 世界步进与数据生命周期	实际调用顺序、接触迁移、缓存跨帧、事件与收尾	沿一个物体和一个接触追踪整步数据变化
17 · 宽阶段动态树与几何缓存	从全配对到树、过滤、胖包围盒、三类缓存	明白怎样少做不必要的几何工作
18 · 岛睡眠与数据布局	稳定身份、稠密数组、冷热数据、整岛睡眠	明白怎样少处理物体、少访问内存
19 · 图着色SIMD与多线程	写冲突、颜色、lane、任务块、同步	明白如何并行而不破坏求解依赖
20 · 确定性精度与性能验证	浮点、相对坐标、稳定提交、回放、基准实验	不把“更快”“更准”“可复现”当成同一个指标
21 · 源码勘误与实验手册	原文修正、关键数值答案、现有测试、故障定位	用计算和证据验证理解

4. 不必一次读完：三条推荐路线

路线 A：数学基础一般，希望真正理解

按 `00 → 01 → 02a → 02 → 03 → 22 → 04 → 05 → 09 → 10 → 11 → 12` 阅读。

第一遍暂时跳过 GJK、CCD、复杂关节和所有并行实现。先能解释“一个圆落地为什么停住”，再追求完整引擎。

路线 B：已有物理基础，主要想读源码

按 `00 → 16 → 09 → 11 → 12 → 06 → 07 → 08 → 13～15 → 17～20` 阅读。

先看 `b2World_Step` 的主干，再精读 `b2SolveContacts_Overflow`。不要从 SIMD gather/scatter 或调度原子变量开始。

路线 C：希望自己写一个二维刚体引擎

以第 04 章的里程碑为主线，第 21 章的实验为验收。

先完成质点积分、圆碰撞、偏心冲量、盒子流形、累计投影与摩擦、堆叠、距离关节；之后再加动态树、睡眠与并行。**性能优化不能代替正确的几何和约束。**

5. 最常用的公式到章节索引

想搞清的计算	去哪一章
世界点、局部点、质心杆臂怎样互转	01
转动惯量如何定义、平方从哪来	02a
平行轴 $I_O = I_C + md^2$ 怎样推出来	02a、02
几块合并时质心怎么加权、惯量怎么加	02a、02
形状怎样提供质量和惯量	02
力和冲量的单位、重力怎样积分	03
半隐式欧拉与显式欧拉差在哪	03、22
分析力学里哪些进了求解器、哪些没有	22
关节如何写成 C、速度层约束做什么	22、13 ~ 15
$\mathbf{V}, u, J, M, K, m_{\text{eff}}$ 逐步展开	22、09
12 种形状对、AABB、点到线段、圆与多边形接触点	05
多边形投影、裁剪交点	06
GJK 支撑点与重心坐标	07
碰撞时刻、扫掠插值、根求解	08
接触点速度、Jacobian、有效质量	09
累计冲量投影、摩擦上限、滚动阻力	10
`biasRate`、`massScale`、`impulseScale`	11
子步分离估计与完整求解数值表	12
关节矩阵、马达和上下限冲量	13 ~ 15
步长、阶段计数、事件集合变化	16

想搞清的计算	去哪一章
配对数量、包围盒周长、缓存阈值	17
睡眠速度、数组搬迁和内存规模	18
SIMD 利用率、任务块、并行上限	19 ~ 20

6. 阅读与验证约定

- 公式使用 `\( … \)` 和 `\[ … \]`；需要 Markdown 阅读器支持数学渲染。即使未渲染，变量说明和计算步骤仍可对照阅读。
- 源码片段的围栏标明“起止行号与文件”；教学伪代码明确标注，不冒充当前公开 API。
- “教学算例”为了隔离一个因素可能关闭重力、摩擦、阻尼、睡眠或恢复；**不能把它的结果直接当作默认世界的逐位输出。**
- 算术复核与完整引擎测试是两回事。本次文档工作对关键算例做独立算术校验，不声称已执行全部 C 测试或测得性能提升。
- 没有“一读就能省略所有练习”的捷径。每章先算一个数字例，再找对应变量，比连续阅读公式更有效。
- 物理定律、碰撞几何、SAT、GJK 和冲量公式旁配有 SVG 示意图，放在 `images/`。先看图对符号，再代入该节数字例。

7. 示意图索引

主题	图	所在章
世界坐标、法线、三个接触符号	图 0-1、0-2	00
点积、叉积、旋转、点速度	图 1-1 ~ 1-7	01
质量与 I 成对、平行轴三项、多块合并	图 2a-1 ~ 2a-4	02a
质心逐步手算：圆、三角形、胶囊、多边形	图 2-1 ~ 2-9	02
牛顿定律、冲量、欧拉、自由落体	图 3-1 ~ 3-4	03
检测与响应、AABB、圆圆、点到线段	图 5-1 ~ 5-5	05
总流程、12 种配对、胶囊/多边形/链段算例	图 5-6 ~ 5-13	05
SAT 投影、最浅穿透、参考边、裁剪	图 6-1 ~ 6-5	06
Minkowski 差、支撑点、单纯形、GJK 迭代	图 7-1 ~ 7-6	07
隧穿、圆圆扫掠 TOI	图 8-1、8-2	08
相对速度、对心/偏心冲量、Jacobian、有效质量、双接触	图 9-1 ~ 9-8	09

建议现在打开：00 · 约定与学习路线。

00 • 约定与学习路线：先知道自己正在计算什么

返回总目录

> 本章目标：建立后续公式共用的词汇、单位和正负号。 不要求先懂矩阵、微积分或 SIMD；这些内容会在真正需要时展开。

1. 一次物理模拟究竟要回答哪些问题

假设把一个盒子放到地面上方。要得到下一时刻的状态，至少要回答：

1. **盒子是什么？** 形状、质量、质心、转动惯量。
2. **现在怎样运动？** 质心位置、朝向、线速度、角速度。
3. **外界怎样影响它？** 重力、用户施力、阻尼。
4. **会在哪里碰到地面？** 接触法线、接触锚点、表面分离量。
5. **接触允许怎样运动？** 可以离开地面，不可以继续穿进去。
6. **怎样修改速度才能近似满足这个条件？** 计算并施加约束冲量。
7. **速度决定怎样的位置变化？** 数值积分。
8. **哪些数据下次还能用？** 接触 ID、冲量缓存、几何缓存。

动态树、SIMD、图着色都不是这八个问题的物理答案。它们解决的是：当物体多时，如何更便宜地完成这些工作。

因此，本书先讲答案，再讲怎样减少计算。

2. 四层内容，不能混在一起

层次	回答的问题	典型例子	怎么验证
物理定律	连续世界中什么关系成立	动量定理、角动量定理	单位、守恒、物理前提
物理模型	用什么近似真实物体	刚体、库仑摩擦、恢复系数	模型适用范围与实验现象
数值方法	计算机如何用有限步逼近	半隐式欧拉、顺序冲量、软约束	收敛、误差、稳定区间
工程实现	如何存储和调度运算	SolverSet、动态树、SIMD	输出一致性、耗时、内存

例如“盒子稍微陷入地面后被推开”：

- 不穿透是刚体接触模型的目标。
- 用速度 bias 慢慢修复位置误差是数值方法。
- 把这条计算放入四路向量是工程实现。
- 它们都不能解释为自然界真的存在一个名叫 `biasRate` 的物理常数。

2.1 本书怎样标识教学简化

如果只研究一次法向碰撞，会明确说明“无摩擦、无外力、无软度、单接触点”。

这样的算例可能一次就满足法向约束。真实盒子存在两个接触点，摩擦又与旋转耦合，解完一条约束可能重新影响另一条。因此不能把单点结论推广为“整个世界一遍求解就精确”。

2.2 “源码如此”也不等于“物理只能如此”

本版 Box2D 选择了子步进、接触软度、Relax 和特定 CCD 调度。别的引擎可以采用不同的有效方案。

本书会先给朴素方案，再解释当前源码的选择；不会把某个实现细节包装成唯一正确的物理定律。

3. 坐标、方向与时间的统一约定

3.1 二维坐标

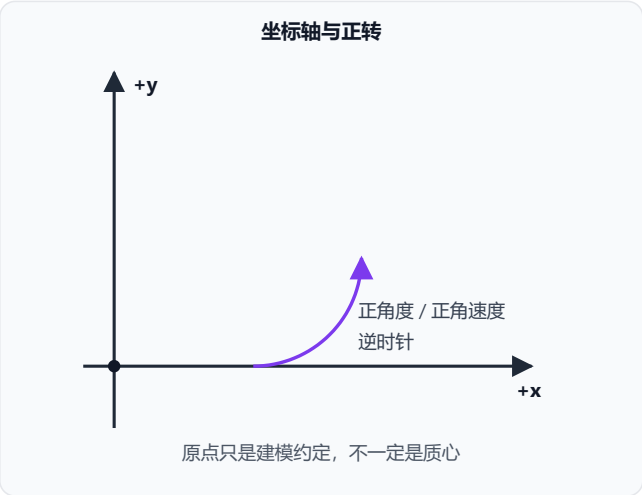
- 世界坐标：向右为正 x，向上为正 y。
- 逆时针旋转为正角度、正角速度。
- 屏幕坐标若 y 向下，在渲染时转换；不要因此只修改物理公式的一半符号。
- 接触法线 n 从 A 指向 B，并且是单位向量。
- 接触切向采用右垂线，与当前接触求解器一致。

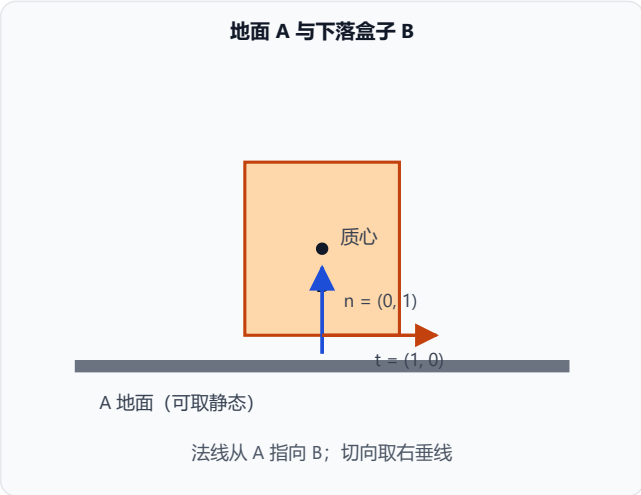
例： 地面是 A，盒子是 B。朝向盒子的法线为 $n = (0, 1)$，切向为 $t = (1, 0)$。盒子向下运动时，法向相对速度为负。

若交换 A/B，法线也应翻转。保持整套符号一致，物理结果才能不变。

图 0-1　世界坐标：向右 +x，向上 +y，逆时针为正

后续所有公式都按这套符号。屏幕若 y 向下，只在渲染时翻转，不要改一半物理公式。





看图： 右上为正转。地面是 A、箱子是 B 时，法线竖直向上，切向水平向右。箱子向下掉，相对速度在法线上为负。

3.2 大步与子步

本书用：

- Δt：一次公开世界步进的总时间，单位秒。
- N：本次大步内的子步数，无量纲正整数。
- h：每个子步的时长，单位秒。

计算为：

$$h = \frac{\Delta t}{N}.$$

作用： 让若干小次积分总共推进同一段时间。

代入： 物理大步为 $1/60$ 秒，子步数为 4，则：

$$h = \frac{1/60}{4} = \frac{1}{240}\ \mathrm{s} \approx 0.004166667\ \mathrm{s}.$$

四次小步加起来仍是约 0.016666667 秒，不是把模拟速度变成四倍。

子步时长的倒数为：

$$h^{-1} = 240\ \mathrm{s}^{-1}.$$

它用于把位置误差换算成速度，以及把**一个子步的冲量**换算成该子步的等效平均力。若统计的是整个大步的 `totalNormalImpulse`，则应除以 Δt，而不是 h。例如 4 个子步合计 0.2 N·s、大步 0.02 s，平均力是 10 N；误除以 0.005 s 会得到错误的 40 N。看到 `inv_h`，先想“每秒”，不要只想“一个优化除法的变量”。

对应源码：`src/physics_world.c`，`b2World_Step` 第 956–980 行。

3.3 帧、步、迭代不是同义词

- 渲染帧：画面刷新一次。
- 物理大步：调用一次 `b2World_Step`。
- 子步：大步内部真实推进一小段时间。
- 约束迭代：对约束再扫描一遍，未必推进时间。

一帧可以执行零次、一次或多次物理大步。把同一批固定几何约束扫四遍，不等于执行四个包含位置积分的子步。

4. 全书最常用的量

符号	含义	默认 SI 单位	常见源码名
p	刚体原点位置	m	`transform.p`
c	世界质心位置	m	`center`
c_l	相对刚体原点的局部质心	m	`localCenter`

符号	含义	默认 SI 单位	常见源码名
r	质心到作用点的杆臂	m	`anchorA`、`anchorB`，注意所在阶段
v	质心线速度	m/s	`linearVelocity`
ω	角速度	rad/s	`angularVelocity`
m	质量	kg	`mass`
I	绕质心的转动惯量	kg·m²	`inertia`；对其他质量数据还需检查参考点
I^{-1}	绕质心的逆转动惯量	(kg·m²)⁻¹	`invInertia`
F	力	N = kg·m/s²	`force`
τ	力矩	N·m	`torque`
P	线冲量向量	N·s	求解时的临时冲量向量
λ_n	累计法向冲量标量	N·s	`normalImpulse`
λ_t	累计切向冲量标量	N·s	`tangentImpulse`
s	有符号表面分离	m	`separation`
K	线性约束的逆有效质量	1/kg	`kNormal` 等局部变量
m_{eff}	有效质量	kg	`normalMass`、`tangentMass`
b	速度层的偏置项	m/s	`velocityBias`
μ	库仑摩擦系数	无量纲	`friction`
e	恢复系数	无量纲	`restitution`
f	频率	Hz = 1/s	`hertz`
ζ	阻尼比	无量纲	`dampingRatio`

注意三个名字陷阱：

1. 接触 `normalMass` 是有效质量，不是逆有效质量。
2. `b2ManifoldPoint.anchorA` 在求解准备时是世界方向、相对质心的杆臂，不是绝对世界点。
3. 密度、滚动阻力长度系数不要都用一个没有上下文的 ρ。各章会重新说明；材质滚阻输入与求解器内部乘上形状尺度后的系数也不是同一单位。

角约束的 Jacobian 与线性约束不同，因此其有效质量具有转动惯量的单位，不能直接套上表中的 kg。

5. 先用单位判断公式有没有写错

5.1 力作用一段时间才形成冲量

对于该时间内恒定的力：

$$P = Fh.$$

变量： F 是力，h 是作用时间，P 是冲量。

作用： 把连续的力作用转换成一步的动量变化。

代入： 12 N 的水平力作用 0.01 秒，产生：

$$P = 12 \times 0.01 = 0.12 \ \mathrm{N{\cdot}s}.$$

对于 2 kg 的刚体，速度变化为：

$$\Delta v = P/m = 0.12/2 = 0.06 \ \mathrm{m/s}.$$

单位检查：N·s 除以 kg 正好是 m/s。如果把 12 N 直接当成 12 N·s，效果会大 100 倍；这是常见的“每帧施力太猛”错误。

5.2 为什么不直接把穿透长度加到速度上

假设误差长度为 0.01 m，希望在 0.02 秒的量级修正它。对应速度尺度是：

$$u = \frac{0.01}{0.02} = 0.5 \ \mathrm{m/s}.$$

长度必须除以时间，才能参与速度方程。但“怎样修、修多少、是否封顶”是数值设计，不由单位检查决定。第 11 章会推导软约束，不能看到这个式子就每步强制修完全部穿透。

5.3 一个检查有效质量的简单例子

仅考虑两个质点，质量分别为 2 kg 和 3 kg。单位方向冲量同时给 A 减速、给 B 加速，相对速度变化的比例为：

$$K = 1/2 + 1/3 = 5/6 \ \mathrm{kg}^{-1}.$$

所以：

$$m_{eff} = 1/K = 6/5 = 1.2 \ \mathrm{kg}.$$

若需要增加 1 m/s 的相对分离速度，所需冲量是：

$$\lambda = m_{eff} \times 1 = 1.2 \ \mathrm{N{\cdot}s}.$$

分别检查：A 的速度减少 0.6 m/s，B 的速度增加 0.4 m/s，相对速度总共增加 1 m/s。

这里没有转动；真正刚体还必须加上杆臂和转动惯量，见第 09 章。

6. 接触的三个符号要一次记对

6.1 分离量

- $s > 0$：表面之间还有间隙。
- $s = 0$：理想刚好接触。
- $s < 0$：几何上已经重叠。

正分离的 manifold 不一定是错误：Box2D 会在一定邻近范围内预先建立推测接触。

6.2 法向相对速度

定义为 B 的接触点速度减去 A 的接触点速度，再投影到 A 指向 B 的法线。

因此：

- 负值：正在接近。
- 正值：正在分离。

例：静止地面法线朝上，盒子向下 4 m/s，法向相对速度就是 −4 m/s。

6.3 法向冲量

对于无黏附接触，累计法向冲量不能为负：接触只能推开，不能主动把物体吸回去。

但**单次增量可以为负**：它可能只是撤销前面多施加的一部分正冲量。第 10 章用数字解释为什么这不等于吸力。

6.4 用一个推测例子统一三个符号

只做一阶分离预测时：

$$s_{next} \approx s + hv_n.$$

前提： 子步内法线近似固定，运动按当前接触点法向速度线性预测。

作用： 判断这一小步是否会把正间隙消耗成穿透。

取 $s = 0.01$ m，$h = 0.01$ s：

- 若 $v_n = -0.5$ m/s，预测间隙为 0.005 m，不必阻止。
- 若 $v_n = -2$ m/s，预测间隙为 −0.01 m，需要约束。

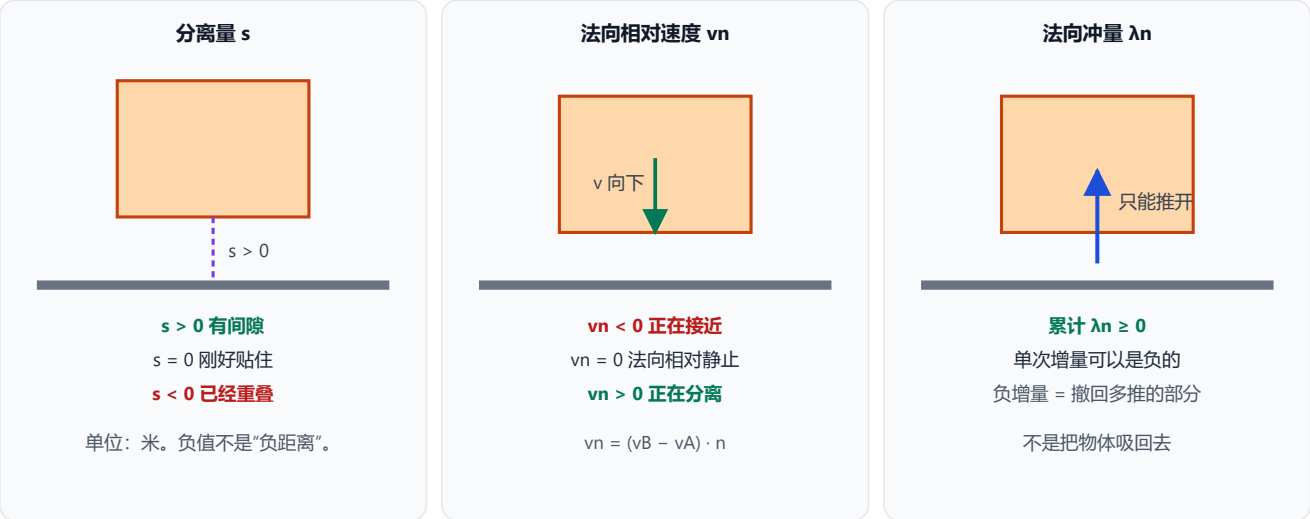
不穿透条件可整理为：

$$v_n + s/h \geq 0.$$

这里 $s/h = 1$ m/s，所以允许的接近速度不小于 −1 m/s。**不是只要有正间隙就把接近速度立刻变成零。**

图 0-2　接触里三个最容易搞反的符号

先记住：正分离 = 还有空隙；负速度 = 正在靠近；正冲量 = 把双方推开。



看图记三个符号： 空隙用 s，靠近用负的 v_n，接触只能推开。后面 SAT、GJK、冲量公式都沿用这一套。

7. 默认尺度与参数：是容差，不是宇宙常数

下表按默认长度单位“1 单位 = 1 米”理解：

参数	基线值	学习时怎样理解
`B2_LINEAR_SLOP`	0.005 m	数值容差量级，不代表每个位置修正公式都显式减它
`B2_SPECULATIVE_DISTANCE`	0.02 m	有限推测接触距离，不是 CCD 的替代品
`B2_CONTACT_RECYCLE_DISTANCE`	0.05 m	接触几何回收阈值之一，不是通用穿透许可
`B2_CONTACT_RECYCLE_COS_ANGLE`	0.98	角变化判据，实际还检查其他量
`B2_MAX_AABB_MARGIN`	0.05 m	树代理余量上限，小形状还受尺寸比例限制
`B2_TIME_TO_SLEEP`	0.5 s	持续满足睡眠判据的时间尺度
`contactHertz`	30 Hz	输入接触软度频率，还会按子步长限制
`contactDampingRatio`	10	接触数值恢复采用强阻尼，不等于材质恢复系数
`contactSpeed`	3 m/s	重叠修正速度尺度上限
`restitutionThreshold`	1 m/s	很慢的接近不触发恢复目标

来源：`include/box2d/constants.h` 和 `src/types.c` 的 `b2DefaultWorldDef` 第 8–25 行。

长度相关常量很多会乘 `b2GetLengthUnitsPerMeter()`；但不要假设所有输入都自动换算。例如此基线默认重力 y 值在代码中直接写为 −10。自定义单位时，用户仍须一致设置位置、速度、重力和密度等量。

缩放例： 若用厘米数值表示长度，1 m 对应 100 个长度单位，则 0.005 m 应表示为 0.5 个单位，0.02 m 表示为 2 个单位。重力 −10 m/s² 则应表示为 −1000 单位/s²，而不是原样 −10。

8. 读一段源码时，用六个问题拆开它

以接触准备为例：

1. **输入是什么？** 两个刚体质量、初始杆臂、manifold。
2. **坐标系是什么？** 杆臂的方向在世界坐标，长度相对各自质心。
3. **要输出什么？** 有效质量、基础分离、初始累计冲量。
4. **它改什么？** 准备约束数据，尚未施加接触冲量。
5. **多久执行一次？** 当前大步开始一次，而非每个子步完整重算。
6. **下一阶段怎样消费？** WarmStart 施加缓存，Solve 用有效质量计算新冲量。

遇到 `indexA` 时，先搞清是公开 ID、对象池索引、稠密数组索引，还是求解器的“下标加一”。不分清索引语义，容易把数据搬迁误读为物理变化。

9. 练习与答案

题 1：一个大步分成 8 个子步，世界是否快了 8 倍？

没有。若大步是 1/60 秒，每个子步是 1/480 秒，总时间仍为 1/60 秒。

题 2：盒子离地面 1 cm，接近速度 −0.2 m/s，子步为 1/100 秒，一定需要法向冲量吗？

在线性预测下，下一步间隙为 $0.01-0.002=0.008$ m，尚未穿透。存在接触记录不代表一定有非零冲量。

题 3：静态物体的逆质量为零，是因为浮点数不能表示无穷大吗？

不是。IEEE 浮点支持无穷大。使用零逆质量是为了统一“约束冲量不能改变其运动”的计算，也避免无穷大参与运算的特殊情况。

题 4：为什么“法向求解后没有穿透速度”还不能证明整个子步结束也如此？

因为随后另一个接触点、关节或摩擦可能改变同一个刚体的线速度与角速度。要看完整求解顺序和最终残差。

题 5：应该先读动态树还是标量接触求解器？

若目标是懂物理，先读标量接触求解器。动态树只筛候选，不决定碰撞冲量。

10. 接下来怎样学习

看图时先对符号，再代入该节数字例。总目录第 7 节有全部示意图索引；SAT 在第 06 章，GJK 在第 07 章，冲量在第 09 章。

下一章从二维向量开始，把“投影”和“转动”两件事讲清楚。之后每引入一种计算，都走完整链条：

这个物理现象 → 需要什么量 → 怎么算这个量 → 数值代入 → 谁读写它 → 当前源码为何多出这些分支。

下一章：01 · 二维数学与坐标变换 · 返回总目录

01 • 二维数学与坐标变换

返回总目录

本章先建立能够手算的数学约定，再把每个约定落到当前源码。 后续碰撞、质量、力矩和约束，都只是这些运算的组合。 阅读时不要先背函数名：先问输入是什么、结果是什么、单位是否一致。

1. 点、向量和单位不能混为一谈

约定世界坐标向右为 $+x$，向上为 $+y$，逆时针旋转为正。 屏幕坐标可以向下，但那应是渲染转换，不应在一半物理公式里偷偷改号。 点描述位置，例如 $P = (3, 2)\,\mathrm{m}$。 向量描述位移、速度或力，例如 $d = (1, -2)\,\mathrm{m}$。 二者可能都存成两个数，但物理意义不同。 两个点相减得到位移；点加位移得到新点；两个点相加通常没有直接物理意义。

$$d = P_B - P_A = (x_B - x_A, y_B - y_A).$$

若 $P_A = (2, 1)$，$P_B = (5, 5)$，则 $d = (3, 4)\,\mathrm{m}$。 d 从 A 指向 B；颠倒减法得到 $-d$。 长度使用勾股定理：

$$\|d\| = \sqrt{d_x^2 + d_y^2} = \sqrt{9 + 16} = 5\,\mathrm{m}.$$

单位方向向量是：

$$\hat{d} = \frac{d}{\|d\|} = (0.6, 0.8).$$

分子分母都有米，因此单位向量无量纲。 不要对零向量直接除以长度：$0/0$ 不代表任意方向，而是未定义。 源码 `b2GetLengthAndNormalize` 会把极小向量的长度和方向都返回零。 这是数值防护，不是说零向量具有有效接触法线。 教学碰撞器遇到完全同心的圆，必须另外制定确定性备用方向或拒绝这种初始状态。 定位：数学头文件，`b2GetLengthAndNormalize`，361–374 行。

图 1-1　点描述位置，两点相减才得到位移

例：PA=(2,1) m，PB=(5,5) m，则 d=PB−PA=(3,4) m，长度 5 m。



看图： P_A 到 P_B 的箭头才是向量。长度 5 m，单位方向 $(0.6, 0.8)$ 没有单位。零长度没有方向，不能拿来当接触法线。

2. 点积：把一个运动投影到某个方向

定义：

$$a \cdot b = a_x b_x + a_y b_y = \|a\|\|b\| \cos\theta.$$

a, b 为同一坐标系中的向量；θ 为二者夹角。 若 $b = n$ 是单位法线，那么 $a \cdot n$ 就是 a 沿法线的有符号分量。 正号表示同向，负号表示反向，零表示垂直。 点积结果的单位是两输入单位相乘。 速度与无量纲法线点积得到 $\mathrm{m/s}$；力与位移点积得到 $\mathrm{N \cdot m}$。

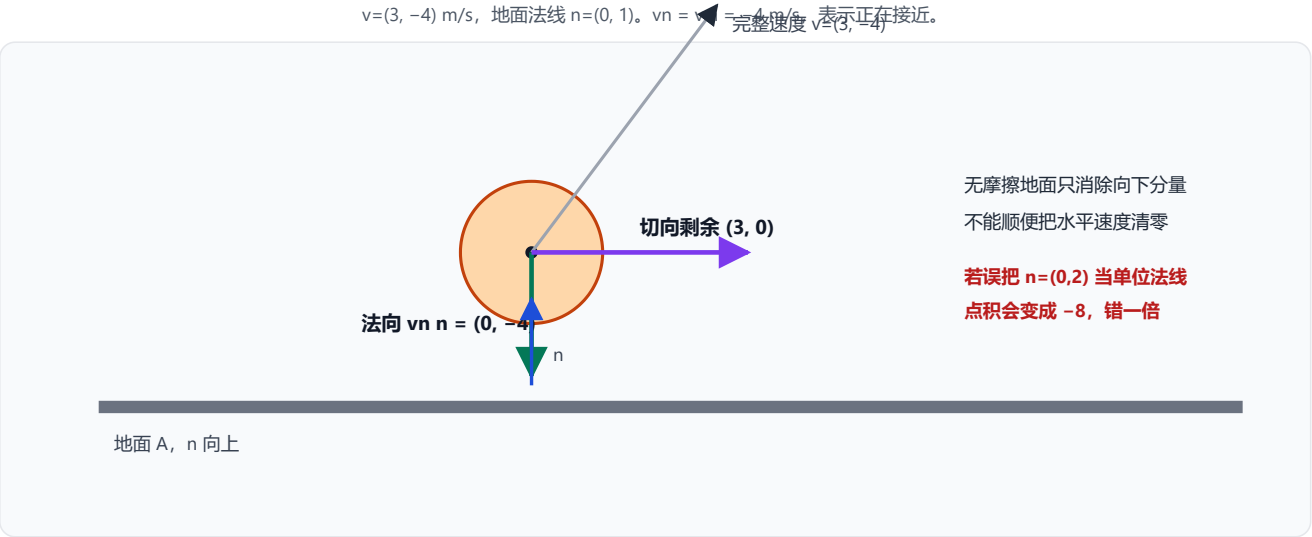
例如物体速度 $v = (3, -4)\,\mathrm{m/s}$，地面向上法线 $n = (0, 1)$：

$$v_n = v \cdot n = 3 \times 0 + (-4) \times 1 = -4\,\mathrm{m/s}.$$

负号表示向地面运动，不是“速度长度为负”。 沿法线的速度向量为 $v_n n = (0, -4)$。 切向剩余速度为 $v - v_n n = (3, 0)$。 因此无摩擦地面冲量应消除向下分量，而不能顺便把水平速度清零。

图 1-2　点积：把速度拆成“撞向地面”和“沿地面滑”

v=(3, −4) m/s，地面法线 n=(0, 1)。vn = v · n = −4 m/s，表示正在接近。



看图： 绿色箭头是“正在撞地”，紫色箭头是“沿地面滑”。无摩擦只处理绿色那一段。

2.1 为什么必须保证法线单位长度

如果误把 $n = (0, 2)$ 当单位法线，点积得到 -8，不是实际法向速度。 再次乘 n 得 $(0, -16)$，误差进一步放大。 非单位向量上的正确投影为：

$$\mathrm{proj}_b(a) = \frac{a \cdot b}{b \cdot b} b.$$

分母不能省略，除非已知 $b \cdot b = 1$。 求解器通常事先保存单位法线，避免每次求解再除一次。 这里的优化依赖前置不变量，不是任意删掉除法。 定位：数学头文件，`b2Dot`，204–207 行。

3. 叉积：二维中的有向面积和旋转趋势

把二维向量看成三维中的 $(a_x, a_y, 0)$。 三维叉积只有 z 分量不为零，因而二维可只保存这个标量：

$$a \times b = a_x b_y - a_y b_x.$$

从 a 转向 b 的较小有向角为逆时针时，通常得到正值。 例如 $(1,0) \times (0,1) = 1$，与逆时针为正的约定一致。 交换顺序会变号：$b \times a = -(a \times b)$。 若两向量共线，叉积为零；这既可能同向，也可能反向。

3.1 为什么叉积能算面积

以 a 为底的平行四边形高度为 $\|b\| \sin\theta$。 因此有向面积为：

$$A_{\text{para}} = \|a\|\|b\| \sin\theta = a \times b.$$

三角形面积是它的一半。 取 $a = (2,0)\,\text{m}$，$b = (0,3)\,\text{m}$，得到 $a \times b = 6\,\text{m}^2$。 三角形面积为 $3\,\text{m}^2$。 逆时针顶点顺序使这套面积求和为正。 顺时针顺序会给出负的有向面积，不能靠盲目对每一项取绝对值修复任意非法多边形。 后者可能掩盖自交、错误拓扑和错误三角剖分。

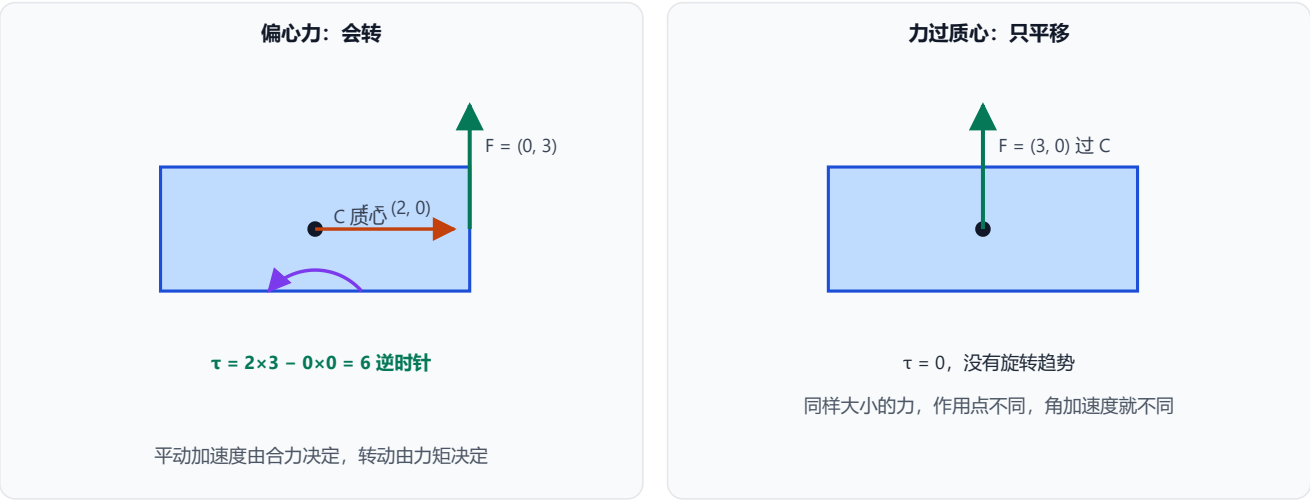
3.2 为什么叉积能算力矩

$$\tau = r \times F = r_x F_y - r_y F_x.$$

r 是从质心指向受力点的位移，单位米。 F 是力，单位牛顿；τ 单位 $\text{N} \cdot \text{m}$。 $r = (2,0)$，$F = (0,3)$ 时，$\tau = 6$：向右端施加向上的力，使物体逆时针转。 若 $F = (3,0)$，则 $\tau = 0$：力的作用线穿过质心，没有旋转趋势。 同样大小的力，作用点不同，平动加速度相同，角加速度可能不同。 定位：数学头文件，`b2Cross`，210–213 行。

图 1-3　叉积 τ = r × F：力不经过质心才会转

左图：右端向上推，τ=6 N·m，逆时针。右图：同样的力穿过质心，τ=0。



看图： 左图右端向上推，物体逆时针转；右图力穿过质心，只平移。这就是后面“边角碰撞会转、正撞不转”的几何原因。

4. 左垂线、右垂线，以及标量叉积的顺序

向量逆时针转 90° 得左垂线：

$$L(v) = (-v_y, v_x).$$

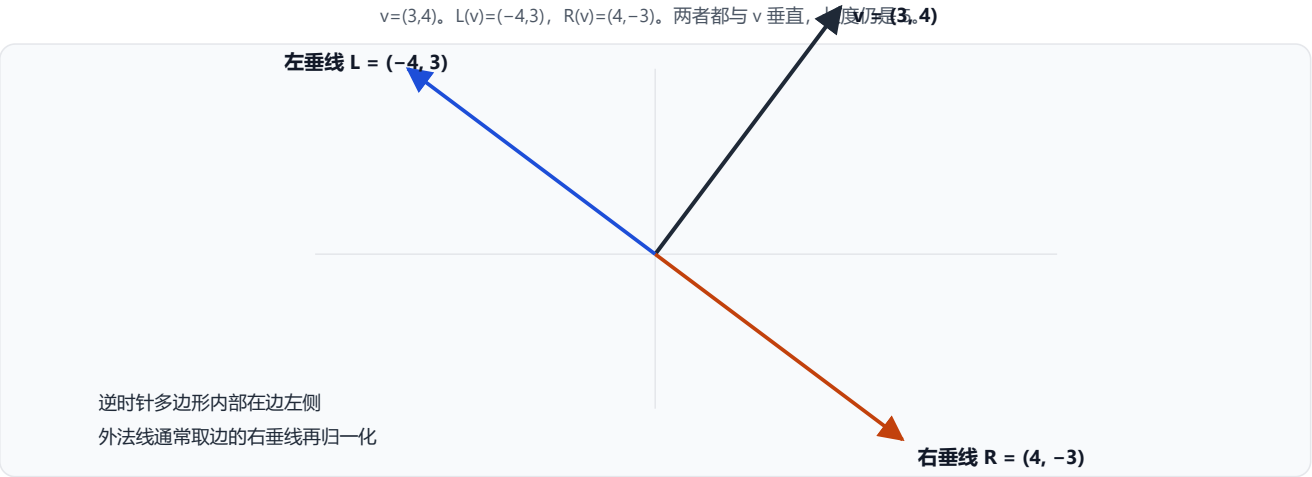
顺时针转 90° 得右垂线：

$$R(v) = (v_y, -v_x).$$

取 $v = (3, 4)$，得到 $L(v) = (-4, 3)$、$R(v) = (4, -3)$。 检查垂直性：$v \cdot L(v) = 3(-4) + 4(3) = 0$。 检查长度：两个垂线都长 5，不会自动成为单位向量。 逆时针多边形的内部位于有向边左侧，因此外法线通常取边的右垂线再归一化。

图 1-4　左垂线逆时针转 90°，右垂线顺时针转 90°

v=(3,4)。L(v)=(−4,3)，R(v)=(4,−3)。两者都与 v 垂直，长度仍是 5。



看图： SAT 里多边形外法线就是边向量的右垂线。方向反了，分离量符号会整章跟着错。

标量 s 可视为三维 z 分量，则：

$$s \times v = (-sv_y, sv_x) = sL(v), \qquad v \times s = (sv_y, -sv_x) = sR(v).$$

两者互为相反数，不能只看“输入一个数和一个向量”就混用。 源码 `b2CrossSV` 是标量在前；`b2CrossVS` 是向量在前。

```
B2_INLINE b2Vec2 b2CrossVS( b2Vec2 v, float s )
{
	return B2_LITERAL( b2Vec2 ){ s * v.y, -s * v.x };
}

/// Perform the cross product on a scalar and a vector. In 2D this produces a vector.
B2_INLINE b2Vec2 b2CrossSV( float s, b2Vec2 v )
{
	return B2_LITERAL( b2Vec2 ){ -s * v.y, s * v.x };
}
```

定位：数学头文件， `b2CrossVS` / `b2CrossSV` / `b2LeftPerp` / `b2RightPerp` ，216–237 行。

5. 旋转矩阵不是需要背诵的魔法

局部 x 轴 $(1,0)$ 旋转 θ 后变为 $(\cos\theta, \sin\theta)$。 局部 y 轴 $(0,1)$ 同时变为 $(-\sin\theta, \cos\theta)$。 任意向量等于 v_x 倍 x 轴加 v_y 倍 y 轴。 把两个旋转后的轴按同样系数组合，即得：

$$Rv = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} v_x \\ v_y \end{pmatrix} = \begin{pmatrix} cv_x - sv_y \\ sv_x + cv_y \end{pmatrix}.$$

这里 $c = \cos\theta$、$s = \sin\theta$，均无量纲。 旋转不应改变向量的长度和单位。 取 $\theta = 90°$，$(c, s) = (0, 1)$，$v = (2, 1)$ 变成 $(-1, 2)$。 平方长度从 $2^2 + 1^2 = 5$ 变为 $(-1)^2 + 2^2 = 5$。

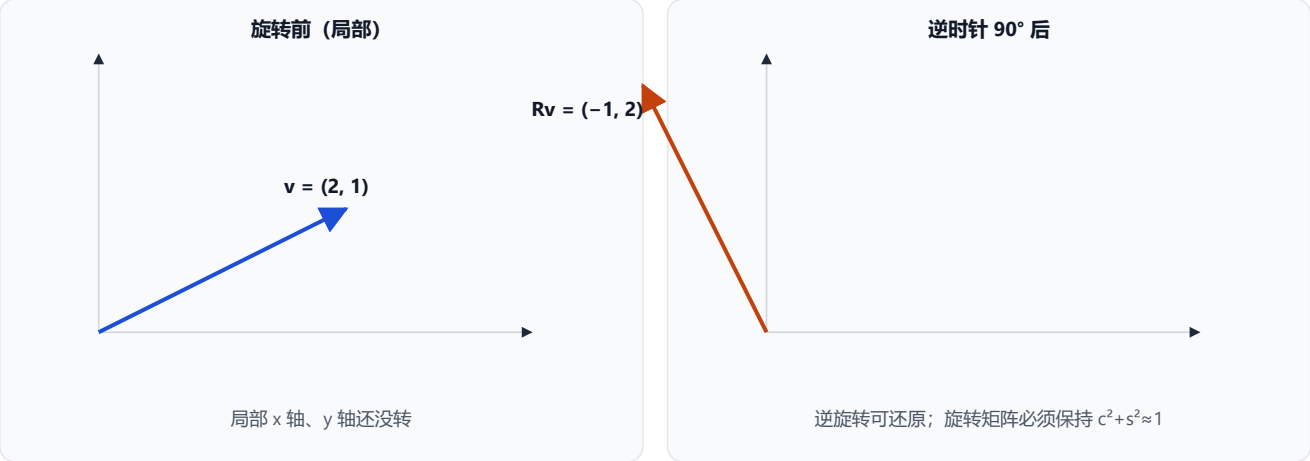
因为旋转矩阵正交，逆矩阵等于转置：

$$R^{-1}v = R^T v = (cv_x + sv_y, -sv_x + cv_y).$$

把 $(-1, 2)$ 逆旋转 $90°$，得到 $(2, 1)$，可作为实现单元测试。

图 1-5　旋转只改方向，不改长度

局部向量 (2,1) 逆时针转 90° 后变成 (−1,2)。平方长度都是 5。



源码用 `b2Rot` 保存 (c, s)，而不是每次从角度计算三角函数。 但它依赖 $c^2 + s^2 \approx 1$；若不归一化，变换会混入缩放。 定位：数学头文件，`b2Rot`，37–41 行；`b2RotateVector` / `b2InvRotateVector`，558–567 行。

6. 点变换要平移，方向变换不要平移

设刚体原点世界位置为 p，局部点为 x_l，姿态为 R。 先旋转局部位移，再放到原点：

$$x_w = p + Rx_l.$$

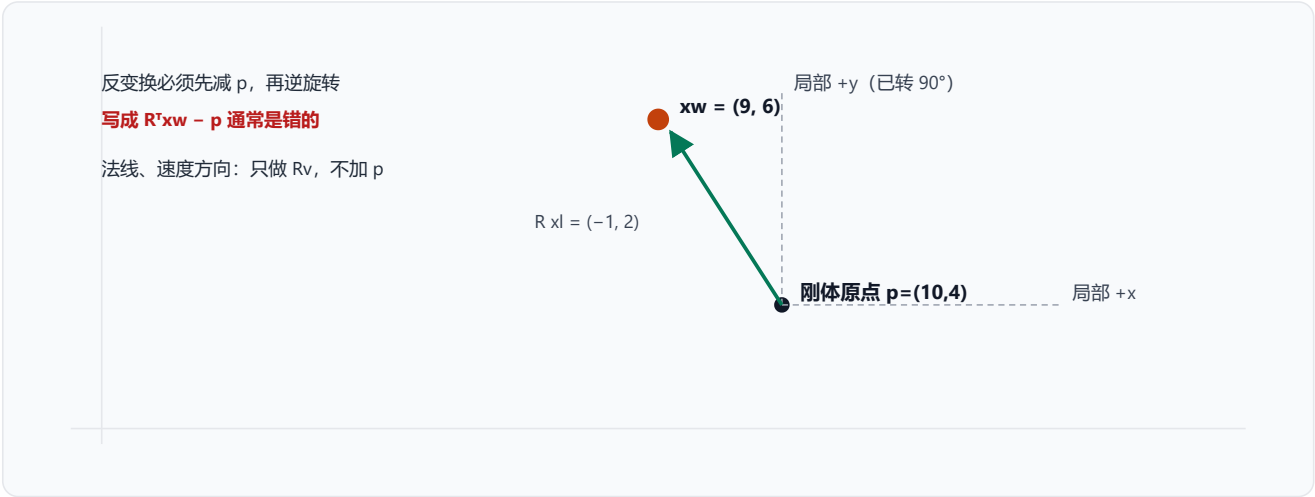
反变换必须先减原点，再逆旋转：

$$x_l = R^T(x_w - p).$$

例：$p = (10, 4)$，R 为 $90°$，$x_l = (2, 1)$。 旋转结果 $(-1, 2)$，所以 $x_w = (9, 6)$。 反算先得 $(9, 6) - (10, 4) = (-1, 2)$，再逆旋转得 $(2, 1)$。 如果逆变换写成 $R^T x_w - p$，通常是错的，因为 p 也需要处在相同旋转关系中。 对局部速度方向或法线，只做 $v_w = Rv_l$，不能加位置 p。 定位：数学头文件，`b2TransformPoint` / `b2InvTransformPoint`，570–584 行。

图 1-6　点要旋转加平移，方向只旋转

xw = p + R xl。例：p=(10,4)，转 90°，xl=(2,1) → 旋转后 (−1,2) → 世界点 (9,6)。



看图： 橙色点是世界位置。法线这种“方向”只跟着旋转走，不要再加上 p。

6.1 两个变换如何合成

B 的局部点先进入 A，再进入世界：

$$x_w = p_A + R_A(p_B + R_B x_B) = (p_A + R_A p_B) + (R_A R_B)x_B.$$

因此复合位置是 $p_A + R_A p_B$，不是简单的 $p_A + p_B$。 复合旋转是 $R_A R_B$。

接下来改变已知量的定义：**假设 A、B 的变换都相对于世界。**此处 p_B, R_B 不再是上式的 B→A 局部变换，而是 B→世界变换。若要把 B 的点直接表达在 A 坐标中：

$$x_A = R_A^T(p_B - p_A) + R_A^T R_B x_B.$$

这正是相对变换，碰撞检测常用它把两个形状放到一个近处坐标系。 定位：数学头文件，`b2MulTransforms` / `b2InvMulTransforms`，591–608 行。

7. 刚体原点不一定是质心

原点是建模约定；质心由质量分布决定。 设局部质心为 c_l，世界质心为 c_w，刚体原点世界位置为 p：

$$c_w = p + Rc_l, \qquad p = c_w - Rc_l.$$

动力学积分通常推进质心，而绘制、局部形状变换需要刚体原点。 二者相等仅在 $c_l = (0,0)$ 时成立。例：$c_w = (10,4)$，$c_l = (2,0)$，姿态 $90°$。 旋转后的局部质心偏移为 $(0,2)$，故原点 $p = (10,2)$。 若错误把原点直接设为质心，整个形状会平移错 2 米。

当前实现的落点：

```
                sim->center = b2OffsetPos( sim->center, state->deltaPosition );
                sim->transform.q = b2NormalizeRot( b2MulRot( state->deltaRotation, sim-
>transform.q ) );
```

随后 `b2FinalizeBodiesTask` 在 621 行用质心减去旋转后的 `localCenter`，重建 `transform.p`。 定位：求解器，`b2FinalizeBodiesTask`，604–621 行。

7.1 接触点速度为什么多一项

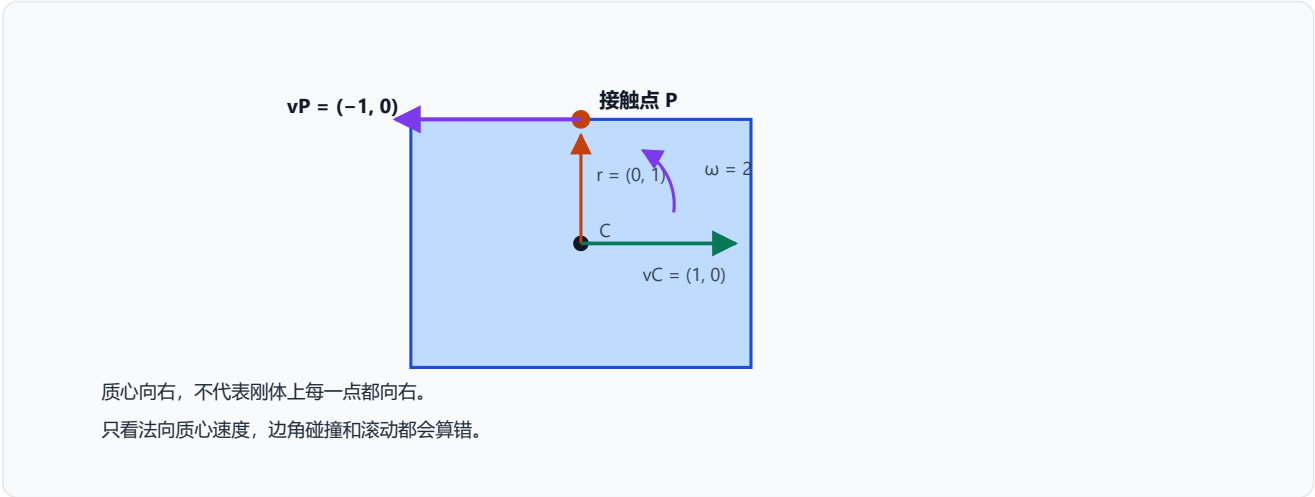
刚体上相对质心的世界位移为 r。 质心速度为 v_C，角速度为 ω，单位 $\mathrm{rad/s}$。 刚体点的速度：

$$v_P = v_C + \omega \times r = v_C + (-\omega r_y, \omega r_x).$$

这里的 $\times$ 是标量叉向量：ω 是标量而不是平面向量，结果仍是向量，不是第 3 节向量叉乘得到的标量；原因见 7.2。 原因是 $r = Rr_l$，对时间求导得到 $\dot{r} = \omega \times r$。 角度的弧度在量纲上视为无量纲，因此 ωr 单位是米每秒。 例：$v_C = (1, 0)$，$r = (0, 1)$，$\omega = 2$。 转动部分为 $(-2, 0)$，故该点速度是 $(-1, 0)$。 质心向右运动，不代表刚体上每一点都向右运动。 接触求解若只看质心速度，就无法正确处理滚动和偏心碰撞。 定位：接触求解器，`b2SolveContacts_Overflow`，296–299 行。

图 1-7　接触点速度：vP = vC + ω × r

质心向右 1 m/s，杆臂向上 1 m，ω=2 rad/s。转动项 (−2, 0)，该点实际向左 1 m/s。



看图： 转动项 $\omega \times r$ 把顶边点的速度拧向左边。第 09 章的接触速度就是这个公式。

7.2 为什么 $\omega \times r$ 是向量，不是标量

第 3 节的 $a \times b$ 得标量，这里却得向量，差别在操作数类型而不在符号本身。 二维的“×”有三种用法，结果类型由谁在平面内、谁在平面外决定：

写法	操作数	二维结果	典型单位	源码
$a \times b = a_x b_y - a_y b_x$	向量 × 向量	标量	m^2、$\mathrm{N \cdot m}$	`b2Cross`
$\omega \times r = (-\omega r_y, \omega r_x)$	标量 × 向量	向量	$\mathrm{m/s}$	`b2CrossSV`
$r \times \omega = (\omega r_y, -\omega r_x)$	向量 × 标量	向量，与上一行反号	$\mathrm{m/s}$	`b2CrossVS`

补成三维 $(x, y, 0)$ 最清楚。 角速度是三维向量 $(0, 0, \omega)$ 的 z 分量，与 $r = (r_x, r_y, 0)$ 叉乘：

$$(0, 0, \omega) \times (r_x, r_y, 0) = (-\omega r_y,\ \omega r_x,\ 0).$$

z 分量恰好为零，剩下的就是二维保留的向量。 而两个平面向量叉乘：

$$(a_x, a_y, 0) \times (b_x, b_y, 0) = (0,\ 0,\ a_x b_y - a_y b_x),$$

只有 z 分量非零，于是缩成一个标量。 两个平面向量叉乘，结果被挤到 z 轴上；角速度本来就在 z 轴上，叉乘后 z 分量为零，结果留在平面内。

量纲也要求它是向量：ω 为 $\mathrm{rad/s}$（弧度视为无量纲），r 为 m，乘积为 $\mathrm{m/s}$，正好与 v_C 相加。 标量（如 m^2）既没有方向，也不能与速度相加。

几何上 $\omega \times r = \omega L(r)$：把 r 逆时针转 90° 再乘 $|\omega|$。 结果垂直于 r，沿该点绕质心转动的切线方向；$\omega > 0$ 时指向逆时针一侧。 它与 v_C 的性质不同：某一瞬间 v_C 与 ω 对整块刚体是同一个值，而 r 取决于“问的是哪个点”，所以 $\omega \times r$ 逐点变化，不是常量。 同一刚体上 $r = (0, 1)$ 与 $r = (0, -1)$ 两点，转动项分别是 $(-\omega, 0)$ 与 $(\omega, 0)$，大小相同、方向相反。 对确定点、确定瞬间，它是确定的向量；随点、随时间变化的是它的值，不是运算规则。

同一函数里两种叉积同时出现，返回类型就是最直接的判据：

```
float rnA = b2Cross( rA, normal );
float rnB = b2Cross( rB, normal );
float kNormal = mA + mB + iA * rnA * rnA + iB * rnB * rnB;
cp->normalMass = kNormal > 0.0f ? 1.0f / kNormal : 0.0f;
```

```
// relative normal velocity at contact
b2Vec2 vrA = b2Add( vA, b2CrossSV( wA, rA ) );
b2Vec2 vrB = b2Add( vB, b2CrossSV( wB, rB ) );
float vn = b2Dot( b2Sub( vrB, vrA ), normal );
```

`rA` 与 `normal` 都是平面向量，叉乘得 `float`，用于有效质量； `wA` 是标量，与 `rA` 叉乘得 `b2Vec2`，用于接触点速度。 第 4 节已说明 $s \times v$ 与 $v \times s$ 反号：这里必须用标量在前的 `b2CrossSV`，若写成 `b2CrossVS( rA, wA )`，转动项整体反号。 第 09 章的恒等式 $\mathbf{n} \cdot (\omega \times \mathbf{r}) = \omega(\mathbf{r} \times \mathbf{n})$ 是这两种叉积的接口：左边先做标量叉向量得到向量、再与 $\mathbf{n}$ 点积，右边直接做向量叉乘得到标量。

8. 先做朴素版，再处理大世界误差

教学初版可直接存世界位置和局部顶点，按前面的公式变换。 等出现“移到远处就抖动”时，再理解为什么改坐标表示。 单精度浮点数的有效二进制位有限，绝对值越大，相邻可表示数间距越大。 在 10^8 附近，float 的相邻数间距约为 8。 所以 100000000.00 与 100000000.25 先各自转成 float 后，很可能完全相等。 再相减时，原先 0.25 米的间距已经丢失，任何算法都无法从零恢复它。

正确策略之一是：世界位置用 double；先在 double 中相减；小差值再转 float。

$$\mathrm{float}(100000000.25 - 100000000.00) = 0.25.$$

这不同于先转换两个绝对位置再相减。 本版 `BOX2D_DOUBLE_PRECISION` 开启时， `b2Pos` 的平移为 double；旋转、向量、求解仍主要为 float。 关闭时 `b2Pos` 是 `b2Vec2` 的别名，因此不能笼统说“本版全是双精度”。 该构建选项默认关闭，见 构建配置 第67行。

```
B2_INLINE b2Vec2 b2SubPos( b2Pos a, b2Pos b )
{
	return B2_LITERAL( b2Vec2 ){ (float)( a.x - b.x ), (float)( a.y - b.y ) };
}
```

定位：数学头文件，类型定义50–70行；`b2SubPos` 649–652行；`b2InvMulWorldTransforms` 683–689行。

8.1 小增量是第二层保护，不是双精度的替代品

每个大步开始时设置 $\Delta p = (0,0)$，$\Delta q = (1,0)$。 子步中累加本大步的位移和旋转，最后一次写回世界位置。 小增量参与接触间距计算，可避免每次子步都对很大的绝对位置反复加减。 `b2BodyState` 专门保存这两项，见 刚体内部结构，153–169行。 此外，多边形碰撞先把 B 变到 A 的坐标，并把 A 的第一个顶点附近作为局部原点。 定位：流形生成，`b2CollidePolygons`，704–730行。

但局部偏移本身若很大，float 仍可能失精度。 相对坐标不是无限世界、任意大小物体、任意尺度混用的万能保证。 优化的目标是把高频计算的数值尽可能限制在物体尺度附近。

9. 手算题与验收

题一：点积与叉积

给定 $a = (3,4)$、$b = (4,-3)$，求点积、叉积和夹角性质。 答案：$a \cdot b = 12 - 12 = 0$；$a \times b = -9 - 16 = -25$。 它们垂直，b 是 a 的右垂线；叉积负号符合顺时针方向。

题二：原点和质心

局部质心 $(1,0)$，世界质心 $(3,5)$，姿态 90°，求刚体原点。 答案：旋转偏移 $(0,1)$，原点 $(3,4)$。 若局部点为 $(2,0)$，其世界坐标为 $(3,6)$。

题三：点速度

$v_C = (2,1)\,\mathrm{m/s}$，$\omega = -3\,\mathrm{rad/s}$，$r = (2,0)\,\mathrm{m}$。 答案：$\omega \times r = (0,-6)$，所以 $v_P = (2,-5)\,\mathrm{m/s}$。 负角速度表示顺时针，右侧点向下运动，符合直观。

题四：辨认叉积的结果类型

$\omega = 2\,\mathrm{rad/s}$，$r = (3,0)\,\mathrm{m}$，$F = (0,5)\,\mathrm{N}$。 $r \times F$ 与 $\omega \times r$ 各是什么类型、什么值、什么单位？ 答案：$r \times F = 15\,\mathrm{N \cdot m}$ 是标量（力矩）；$\omega \times r = (0,6)\,\mathrm{m/s}$ 是向量（点速度）。 若误写成 $r \times \omega = (0,-6)$，符号相反：标量在前的顺序不能换。

代码验收条件

- 非零向量归一化后平方长度接近1。

- 左右垂线与原向量点积接近0，且长度保持。
- 正变换后再逆变换，应在容差内恢复原点。
- 旋转应保持长度和点积。
- 平移世界中所有刚体，不应改变相对位移、碰撞法线和内部动力学。
- 完全同心、极小向量和非法旋转必须有明确策略，不能让 NaN 继续传播。

下一章先弄清转动惯量是什么、平方从哪来；随后第02章用积分算出具体形状的数值。第03章再把这些量用于力与积分。 下一章：02a · 转动惯量的定义与由来 · 返回总目录

02a • 转动惯量的定义与由来

返回总目录

第01章已经能描述位置和旋转。 第02章会用积分算出圆、多边形、胶囊的具体惯量数字。 本章先停一步，把三件事讲清楚：**转动惯量是什么**，**换平行轴时为什么变成** $I_C + md^2$，以及 **几块焊成一个刚体时质心怎么加权、惯量怎么加。** 读完应能用一句话说清 I，从定义推出平行轴定理，并能对两块刚体算出总质心和总惯量。

形状积分、胶囊的两次移轴、以及 Box2D 里两遍累加的源码，见 02 · 质量、质心与转动惯量。

1. 它回答什么问题

质量回答：同样的力，谁更不愿意被推着走。
转动惯量回答：同样的力矩（拧的劲），谁更不愿意转起来。

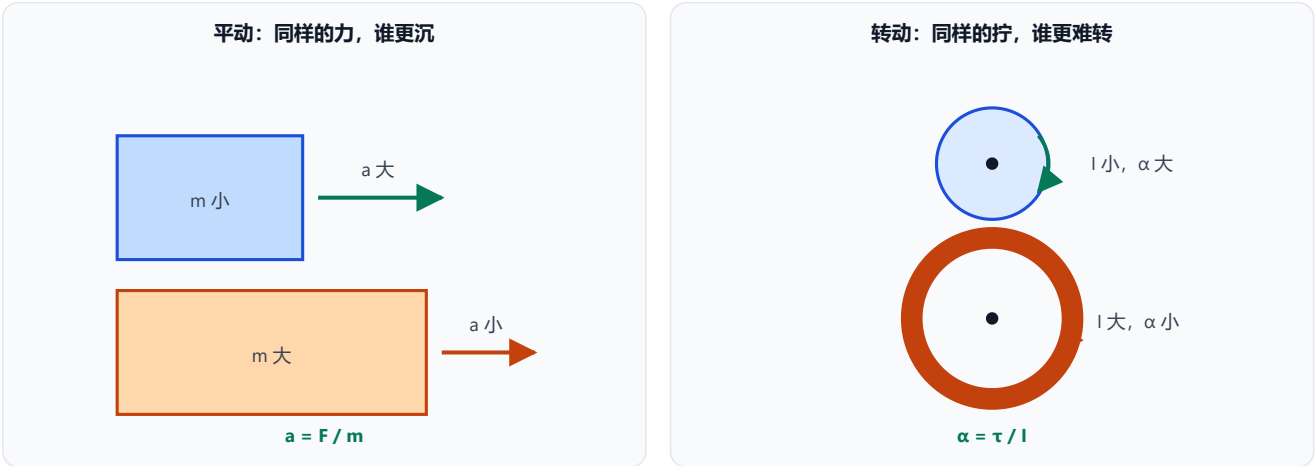
二者成对，但不是同一个量，也不能互相代替。

	平动	转动
惯性有多大	质量 m	转动惯量 I
谁来改变运动	力 F	力矩 τ
运动改多快	$a = F/m$	$\alpha = \tau/I$
动能	$\frac{1}{2}mv^2$	$\frac{1}{2}I\omega^2$
SI 单位	kg	$\mathrm{kg}\cdot\mathrm{m}^2$

质量相同，转动响应仍可以差很多：质量往外摊，就更难转；往中心收拢，就更容易转。 滑冰的人把手臂收回去会转得更快，不是质量变了，而是质量离转轴近了，I 变小。角动量 $I\omega$ 近似守恒时，ω 就会变大。

图 2a-1　质量管平动，转动惯量管转动

F=ma 与 τ=Iα 成对。I 不是第二个质量，单位也不是千克。



看图： 左边两块都受同样的水平力，重的加速度小。右边两个质量可以相同，但细环把质量放在边缘，同样力矩下角加速度更小。

2. 定义：按到转轴距离的平方加权

对一根固定转轴，刚体分成许多小块，第 i 块质量为 m_i，到轴的**垂直距离**为 r_i。定义

$$I = \sum_i m_i r_i^2.$$

连续物体把求和换成积分：

$$I = \int r^2 \, dm.$$

单位是 $\mathrm{kg \cdot m^2}$。它不是质量，也不是“长宽乘积”。 一个质量为 m 的点放在距轴 d 处，贡献就是 md^2。距离扩大两倍，贡献扩大四倍。

本书讨论二维均匀薄片。除特别说明，惯量都指**垂直于纸面、穿过质心的 z 轴**。微元相对质心的位置为 $r = x - c$，于是

$$I_C = \int_A \|r\|^2 \, dm = \rho \int_A (r_x^2 + r_y^2) \, dA.$$

这就是第02章一开始直接写出的那个积分。本章只解释它为什么长这样。

例： 1 kg 的点质量放在距轴 0.5 m 处，

$$I = 1 \times (0.5)^2 = 0.25 \,\mathrm{kg \cdot m^2}.$$

若改放到 1 m 处，$I = 1 \,\mathrm{kg \cdot m^2}$，是原来的四倍，不是两倍。

3. 平方为什么自然出现

可以先不背公式，用动能把平方逼出来。

刚体在转，角速度是 ω。离轴 r 的那一小块，线速度大小是

$$v = |\omega| \, r.$$

同一刚体上 ω 处处相同，但离轴远的点跑得更快。该微元动能

$$\frac{1}{2} m v^2 = \frac{1}{2} m (\omega r)^2 = \frac{1}{2} (m r^2) \omega^2.$$

把所有小块加起来：

$$E_{\mathrm{rot}} = \frac{1}{2} \Big(\sum_i m_i r_i^2 \Big) \omega^2 = \frac{1}{2} I \omega^2.$$

平动动能是 $\frac{1}{2} m v^2$。转动动能长得一模一样，只是 m 换成了 I，v 换成了 ω。 **I 就是为了让这句话成立，才被单独拎出来的系数。** 严格说，ω^2 前面的系数是 $I/2$，不是 I。

例如 $I = 2 \,\mathrm{kg \cdot m^2}$、$\omega = 3 \,\mathrm{rad/s}$，旋转动能为 $9 \,\mathrm{J}$。

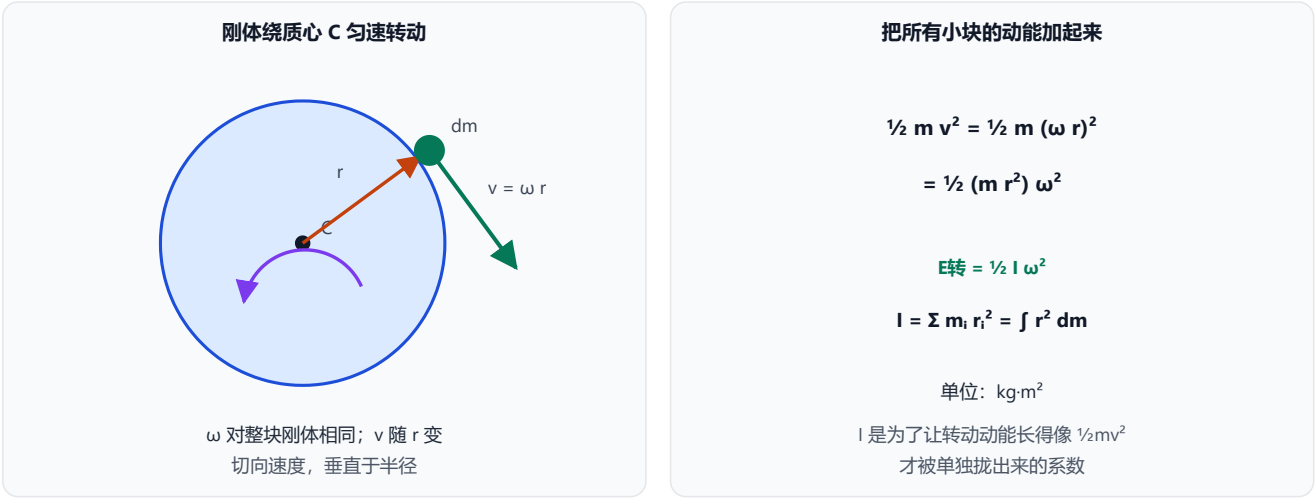
力矩那边走的是同一件事。对垂直轴，角动量 $L = I\omega$，牛顿定律的转动版就是

$$\tau = I\alpha,$$

与 $F = ma$ 成对。后面碰撞冲量、关节约束，用的就是这个 I，以及它的倒数 I^{-1}。

图 2a-2　绕轴转动时，速率与距离成正比，动能带出 r²

同一 ω 下，离轴越远的小块跑得越快。把 ½mv² 加起来，系数就是 I。



看图： 绿色箭头是切向速度 $v = \omega r$，不是质心平动速度。把每块 $\frac{1}{2}mv^2$ 加总，自然出现 $\sum mr^2$。

角动量也可以独立推出同一个平方。垂直轴上，点质量的角动量大小是

$$L = r \cdot (mv) = mr(\omega r) = mr^2\omega.$$

求和后仍是 $L = I\omega$。动能与角动量两条路给出同一个 I，不是两套定义。

4. 历史：先有钟摆问题，后有这个名字

转动惯量不是先发明一个抽象符号，再去找用途。它是被一件很具体的事逼出来的。

4.1 更早：杠杆里已经有“力矩”

古希腊静力学已经知道：同样重的东西，挂得离支点越远，越容易撬动。 英文 moment 来自拉丁语 momentum，在静力学里就是**力乘力臂**。 后面把惯性也按“离轴远近”加权，才出现 moment of inertia：字面意思是**惯性的力矩**，不是“一瞬间的惯性”。

中文译成“转动惯量”，其实更直白：旋转方向上的惯性有多大。

4.2 惠更斯，1673：给实物钟摆找等效长度

17 世纪要做准钟。理想单摆太干净：一根无质量绳子，底下一个点质量。 真实钟摆是一根有质量的杆，质量散布在各处。惠更斯（Christiaan Huygens）在 1673 年的《钟表论》（*Horologium Oscillatorium*）里问：

> 这根“质量摊开的实物摆”，等价于多长的理想单摆？

他发现：不能只看总质量和质心在哪。必须把每一小块的质量，按**到转轴距离的平方**加起来。 那个等效摆长就是现在写的

$$\ell = \frac{I}{md},$$

其中 d 是转轴到质心的距离，m 是总质量。 他当时还没有今天这个标准符号 I，但 $\sum mr^2$ 这个量已经在公式里了。 他把轴从质心挪开时惯量要加 md^2，也是这条线索里长出来的。完整展开见本章第 6 节；第02章把它用到胶囊和复合刚体。

一句话：**转动惯量是为了给实物钟摆找一个“等效单摆”才被算清楚的。**

4.3 欧拉，约 1750–1765：写成刚体定律

牛顿的《原理》把平动讲透了：$F = ma$。 刚体还会转。欧拉（Leonhard Euler）在 1765 年前后的刚体力学里，把这个量定义成旋转惯性，并写出

$$\tau = I\alpha.$$

三维里还不够一个数：绕不同轴、质量分布不对称时，要用**惯量张量**（一组互相耦合的分量）。 二维游戏物理走运：物体只在平面里转，张量塌缩成**一个标量** I_z。 Box2D 里的 `rotationalInertia` 就是这个数。

时间线可以记成：

阶段	谁	做了什么
杠杆时代	阿基米德一脉	知道“离支点远就更有劲”
1673	惠更斯	为钟摆算出 $\sum mr^2$，找到等效摆长
1765 前后	欧拉	把它定义成刚体的转动惯性，写出 $\tau = I\alpha$
后来工程与教材		固定名称 moment of inertia / 转动惯量

5. 定义时必须说清的三件事

否则公式会对、意思却乱。

5.1 绕哪根轴

I 不是贴在物体身上的一个绝对标签。换轴就换值。 物理引擎默认用**过质心、垂直于运动平面**的那根；铰链、转盘则用各自的转轴。 第02章的形状函数返回的是形状自身质心惯量；刚体再把各形状搬到总质心上相加。

5.2 r 是到轴的垂直距离

二维里就是 $\sqrt{x^2 + y^2}$，不是沿某条边量出来的长度，也不是到某个角点的距离。 点在轴上则 $r = 0$，该点对这根轴的惯量贡献为零：它只跟着转，自己并不“绕着跑”。

5.3 不要和材料力学的“惯性矩”混

工程里梁截面常用 $I = \int y^2\, dA$。那个积分**没有质量**，单位是 m^4，描述的是截面抗弯。 刚体动力学的转动惯量带质量，单位是 $\mathrm{kg}\cdot\mathrm{m}^2$。 中文有时都叫“惯性矩”，看书时先看单位，再看它有没有密度或质量。

换轴时惯量怎么变，下一节从定义推出来，不另引入新的物理定律。

6. 平行轴定理：换一根平行轴，惯量怎么变

要证明的就是

$$I_O = I_C + m\|d\|^2.$$

I_C 是绕质心轴的惯量，I_O 是绕新轴的惯量，m 是总质量，d 是质心到新轴的垂直距离（二维里就是质心 C 到轴脚 O 的向量）。 前提只有两条：**两轴平行**，并且 **其中一根必须穿过质心。** 它不是新实验定律，只是把定义 $I = \int r^2\, dm$ 换了一个原点再展开。

6.1 先把三个点摆清楚

纸面里画三个点：

- O：新轴与纸面的交点，新轴垂直于纸面。
- C：质心。
- P：刚体上任意一小块，质量为 dm。

约定向量

$$d = C - O, \qquad r = P - C.$$

d 对所有小块都一样；r 随 P 变。于是小块相对新轴的位置是

$$P - O = r + d.$$

按定义，绕新轴的惯量就是把这个距离平方再对质量积分：

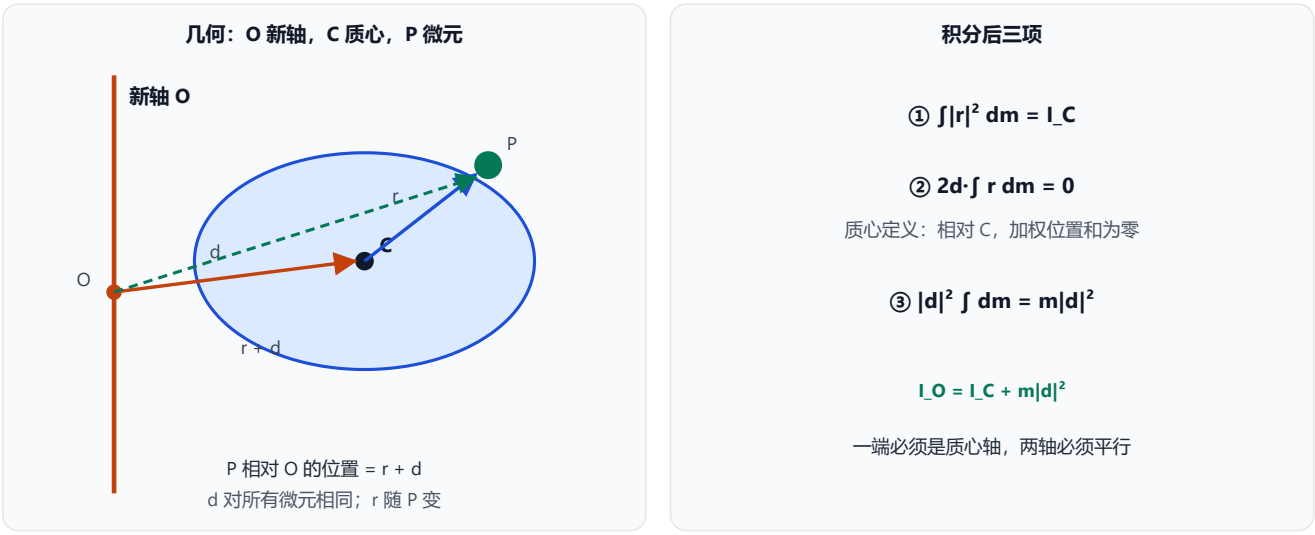
$$I_O = \int \|r + d\|^2\, dm.$$

绕质心轴则是

$$I_C = \int \|r\|^2\, dm.$$

图 2a-3　把 |r+d|² 拆成三项，中间那项积完是零

新轴 O 与质心轴 C 平行。P 是一小块。只有质心定义能让交叉项消失。



看图： 橙色虚线是 $r + d$，也就是 P 到新轴的杆臂。平方一旦展开，会出现三项；只有中间那项会在积分后消失。

6.2 平方展开成三项

二维里 $\|a\|^2 = a_x^2 + a_y^2 = a \cdot a$，所以

$$\|r + d\|^2 = (r + d) \cdot (r + d) = \|r\|^2 + 2r \cdot d + \|d\|^2.$$

写成分量也是同一件事：

$$(r_x + d_x)^2 + (r_y + d_y)^2 = r_x^2 + r_y^2 + 2(r_x d_x + r_y d_y) + d_x^2 + d_y^2.$$

代入积分，d 与每个小块无关，可以提出去：

$$I_O = \int \|r\|^2 \, dm + 2d \cdot \int r \, dm + \|d\|^2 \int dm.$$

三项分别是：

项	积分后	意思
$\int \|r\|^2 \, dm$	I_C	质量相对质心摊开的那部分
$2d \cdot \int r \, dm$	下面证明是 0	交叉项
$\|d\|^2 \int dm$	$m\|d\|^2$	把全部质量当成堆在质心上的一个点

第一项和第三项没有悬念。整个定理的关键是交叉项为什么消失。

6.3 交叉项为什么是零：那就是质心的定义

把 $r = x - c$ 写回去，其中 x 是微元的位置，c 是质心位置：

$$\int r \, dm = \int (x - c) \, dm = \int x \, dm - c \int dm.$$

质心本来就定义为质量加权平均位置

$$c = \frac{1}{m}\int x\,dm, \qquad m = \int dm,$$

所以 $\int x\,dm = mc$，于是

$$\int r\,dm = mc - c\cdot m = 0.$$

人话：**站在质心上看，左边“质量×向左”和右边“质量×向右”正好抵消。** $2d$ 是常量，点乘一个零向量仍是零。交叉项整段蒸发。

因此

$$I_O = I_C + m\|d\|^2.$$

第三项永远非负，所以 **新轴只要离开质心，惯量只能变大，不能变小。**绕质心转是最轻省的。 反向要用减法：已经知道某根非质心轴上的 I_O，才能写 $I_C = I_O - m\|d\|^2$。减完若得到负数，多半是轴认错了，或把不平行的轴硬套进来。

这个结论也叫惠更斯–施泰纳定理（Huygens–Steiner theorem）。惠更斯在钟摆里已经用到移轴，后来施泰纳把它写成刚体惯量的标准形式。

6.4 用第02章那两个点质量把数字走完

第02章第1节的例子：左端 $m_1 = 1\,\mathrm{kg}$ 在 $x = 0$，右端 $m_2 = 3\,\mathrm{kg}$ 在 $x = 2\,\mathrm{m}$。总质量 $m = 4\,\mathrm{kg}$，质心

$$c_x = \frac{1\cdot 0 + 3\cdot 2}{4} = 1.5\,\mathrm{m}.$$

先按定义算绕质心的惯量。两块到质心的距离分别是 $1.5\,\mathrm{m}$ 和 $0.5\,\mathrm{m}$：

$$I_C = 1\cdot(1.5)^2 + 3\cdot(0.5)^2 = 2.25 + 0.75 = 3\,\mathrm{kg\cdot m^2}.$$

再按定义算绕原点轴 O（$x = 0$）：

$$I_O = 1\cdot 0^2 + 3\cdot 2^2 = 12\,\mathrm{kg\cdot m^2}.$$

平行轴公式给出

$$I_C + m\|d\|^2 = 3 + 4\cdot(1.5)^2 = 3 + 9 = 12.$$

两边相同。交叉项在离散求和里也看得到：

$$\sum_i m_i r_i = 1\cdot(0-1.5) + 3\cdot(2-1.5) = -1.5 + 1.5 = 0.$$

不是凑巧，是质心定义逼它为零。

6.5 为什么不能对任意两轴直接加 md^2

任意一根通过点 A 的平行轴，都可以先搬到质心再搬到 B：

$$I_A = I_C + m\|c - A\|^2, \qquad I_B = I_C + m\|c - B\|^2.$$

两式相减：

$$I_B = I_A + m\big(\|c-B\|^2 - \|c-A\|^2\big).$$

只有当 A 就是 C 时，括号里剩下 $\|c-B\|^2 = \|d\|^2$，才回到 $I_B = I_A + md^2$。 若 A、B 都不是质心，中间还藏着一项 $-2m(c-A)\cdot(B-A)$，不能省。

把上一小节的轴从原点错搬到右端 $x=2$ 可以立刻验出来。绕右端按定义：

$$I_{x=2} = 1\cdot(0-2)^2 + 3\cdot 0^2 = 4.$$

正确用法是从质心搬：$I_C + m(2-1.5)^2 = 3 + 4\cdot 0.25 = 4$。 若把 $I_O = 12$ 当成质心惯量再加 $m\cdot 2^2$，会得到 $12+16=28$，错得离谱。

口诀：一端必须是质心轴；两轴必须平行。 复合刚体先算各块绕自己质心的 I_C，再搬到总质心，用的就是这一条。

6.6 白话图像

绕质心转：只反映质量相对质心摊得有多开，这是 I_C。 把轴挪开：相当于再加一个点质量 m 放在质心上、绕新轴转，这是 md^2。 两者相加，不多不少。质量并没有真的堆到质心上；交叉项为零，才允许把“摊开”和“整体挪轴”这样分开写。

第02章胶囊要把半圆质心先移回圆心、再移到胶囊质心，连续用两次，漏掉 $2Ha$ 就是漏了第二次平行轴。下一节把同一公式用到“好几块焊成一个刚体”。

7. 几个物体合并：质心怎么加权，惯量怎么加

这里的“合并”是指焊成**一个刚体**：此后只有一个质心、一个绕质心的 I_C、一套质心速度和角速度。 不是几个独立刚体用关节连在一起。关节连接仍然是多个刚体，质量和惯量各算各的。

每块已经有自己的 $(m_i,\, c_i,\, I_{C_i})$。c_i 是第 i 块自己的质心，I_{C_i} 是绕**它自己质心**的惯量。问题是：整体的 m、C、I_C 怎么从这些数得出来。

7.1 质量直接相加

质量是标量，与位置无关：

$$m = \sum_i m_i.$$

单位千克。零密度、传感器形状这一步贡献 0，不进入后面的质心和惯量。 若总质量仍为 0，质心公式分母为零，动态刚体不应走到这一步。

7.2 质心是质量加权平均，不是几何中点

整体质心的定义仍是 $C = \dfrac{1}{m}\displaystyle\int x\,dm$。 把积分拆到各块上（块与块之间若不重叠，这就是区域的无交并）：

$$\int x\,dm = \sum_i \int_{\text{第 } i \text{ 块}} x\,dm = \sum_i m_i c_i.$$

因此

$$C = \frac{\sum_i m_i c_i}{m}.$$

分子单位是千克·米，除以千克，结果是米。 **重的一块把 C 往自己那边拉。** 不能把各块质心直接平均，也不能在密度不同时拿总面积中心代替总质心。

两块时写开就是杠杆平衡：

$$m_1(c_1 - C) + m_2(c_2 - C) = 0.$$

C 把 $c_1 c_2$ 按质量反比分割。三块及以上只是把同一加权再加一项。

取本章后面要用的二维例子：

块	质量 m_i	自身质心 c_i	自身惯量 I_{C_i}
A	$2\,\mathrm{kg}$	$(0, 0)\,\mathrm{m}$	$0.25\,\mathrm{kg}\cdot\mathrm{m}^2$
B	$4\,\mathrm{kg}$	$(3, 1)\,\mathrm{m}$	$0.50\,\mathrm{kg}\cdot\mathrm{m}^2$

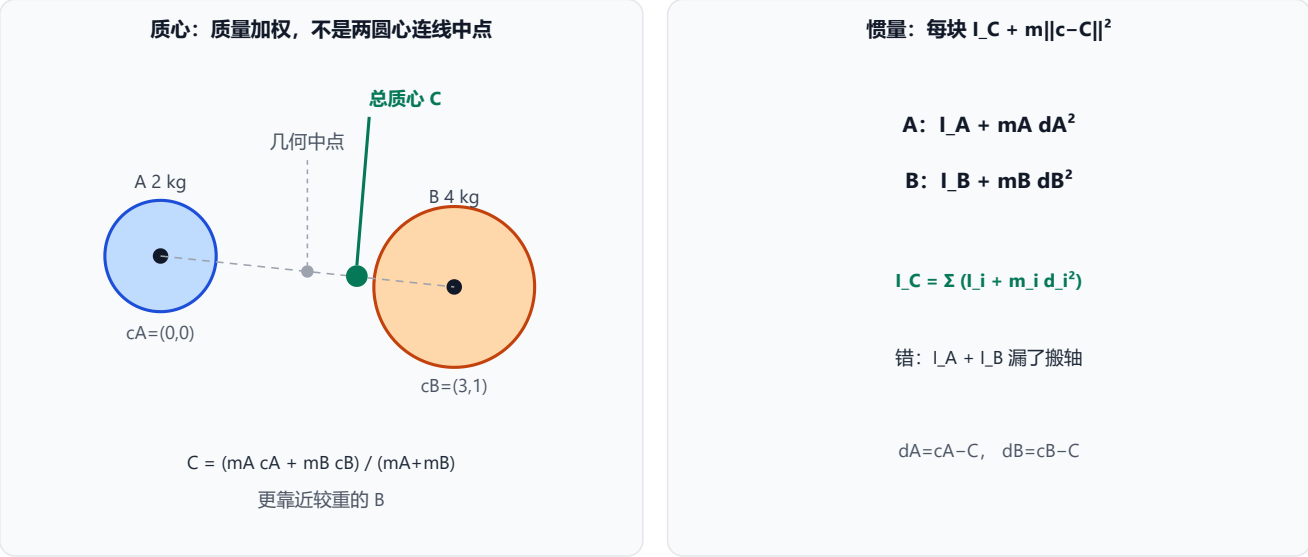
总质量 $m = 6\,\mathrm{kg}$。总质心

$$C = \frac{2\cdot(0,0) + 4\cdot(3,1)}{6} = \frac{(12,4)}{6} = \left(2, \frac{2}{3}\right)\mathrm{m}.$$

A、B 圆心连线的几何中点是 $(1.5,\ 0.5)$，比 C 更靠左、更靠下。质心被 4 kg 那块拉向 B。

图 2a-4　合并：质心按质量加权，惯量先搬轴再相加

第一遍求 C。第二遍把每块的 I_C 用 m d² 搬到总质心。不能把各块自己的 I_C 直接加。



看图： 绿点是 C，灰点才是连线中点。右边公式里每块都要加 md^2，不是把 I_A 和 I_B 加起来就结束。

7.3 惯量不能把各块的 I_{C_i} 直接相加

I_{C_A} 的轴穿过 A 的质心，I_{C_B} 的轴穿过 B 的质心。合并之后整块刚体绕**同一个**总质心 C 转。 两根轴不是同一根，数字没有资格直接相加。

直接把 $0.25 + 0.50 = 0.75$ 当成总惯量，等于假装两块都已经堆在 C 上。实际上它们离 C 还有一段距离，这段距离对转动惯性的贡献正是平行轴项 $m_i\|c_i - C\|^2$。漏掉它，总惯量会严重偏小，同样力矩下角加速度会被高估。

7.4 从定义推出加法：每块先搬到 C，再相加

整体绕 C 的惯量按定义是

$$I_C = \int \|x - C\|^2 \, dm = \sum_i \int_{\text{第 } i \text{ 块}} \|x - C\|^2 \, dm.$$

对第 i 块，把它相对自己质心的位置写出来：

$$x - C = (x - c_i) + (c_i - C).$$

令 $r_i = x - c_i$，$d_i = c_i - C$。d_i 在第 i 块内部是常量。平方展开与第 6 节完全相同：

$$\|r_i + d_i\|^2 = \|r_i\|^2 + 2r_i \cdot d_i + \|d_i\|^2.$$

积分后：

$$\int_{\text{第 } i \text{ 块}} \|x - C\|^2 \, dm = I_{C_i} + 2d_i \cdot \int_{\text{第 } i \text{ 块}} r_i \, dm + m_i\|d_i\|^2.$$

第 i 块自己的质心定义迫使 $\int r_i \, dm = 0$，交叉项再次消失。于是每一块贡献

$$I_{C_i} + m_i\|c_i - C\|^2,$$

全部加起来：

$$I_C = \sum_i \left(I_{C_i} + m_i\|c_i - C\|^2 \right).$$

两项都不能省：

- I_{C_i}：这一块相对**自己**质心摊开得有多开。
- $m_i\|c_i - C\|^2$：这一块的质心离**总**质心有多远。

每一项都非负，所以把质量往外接（质心离总质心更远）只会让整体更难转。

7.5 必须先求 C，再求 I_C：两遍

$d_i = c_i - C$ 里有总质心。第一遍还不知道 C，第二遍才能搬轴。顺序是：

1. 累加 $m = \sum m_i$ 和 $\sum m_i c_i$，然后 $C = (\sum m_i c_i)/m$。
2. 再对每块加 $I_{C_i} + m_i\|c_i - C\|^2$。

数学上也可以先把所有惯量搬到同一个遥远原点 O，得到 $I_O = \sum (I_{C_i} + m_i\|c_i\|^2)$，再减 $m\|C\|^2$。与两遍公式等价。浮点里若 C 很大，I_O 和 $m\|C\|^2$ 都是大数，相减会吃掉有效数字。本版 Box2D 因此走两遍，每项只加非负数。这是运算顺序，不是另一条物理定律。

7.6 把第 7.2 节的两块数字走完

已有 $m = 6\,\mathrm{kg}$，$C = (2,\ 2/3)\,\mathrm{m}$。

A 相对总质心：

$$d_A = (0,0) - \left(2, \frac{2}{3}\right) = \left(-2, -\frac{2}{3}\right), \qquad \|d_A\|^2 = 4 + \frac{4}{9} = \frac{40}{9}.$$

B 相对总质心：

$$d_B = (3,1) - \left(2, \frac{2}{3}\right) = \left(1, \frac{1}{3}\right), \qquad \|d_B\|^2 = 1 + \frac{1}{9} = \frac{10}{9}.$$

核对加权位移是否抵消（第 7.2 节的杠杆式）：

$$2 \cdot d_A + 4 \cdot d_B = 2\left(-2, -\frac{2}{3}\right) + 4\left(1, \frac{1}{3}\right) = (-4, -\tfrac{4}{3}) + (4, \tfrac{4}{3}) = (0,0).$$

惯量：

$$I_C = 0.25 + 2 \cdot \frac{40}{9} + 0.50 + 4 \cdot \frac{10}{9} = \frac{3}{4} + \frac{80}{9} + \frac{40}{9} = \frac{3}{4} + \frac{40}{3} = \frac{169}{12} \approx 14.0833\,\mathrm{kg \cdot m^2}.$$

若漏掉平行轴，只加 $0.25 + 0.50 = 0.75$，会小估大约 19 倍。同样力矩下角加速度会被高估同样倍数。

7.7 五种用错的方式

错误做法	错在哪
$I_C = I_{C_A} + I_{C_B}$	两轴不是同一根，漏了 md^2
把 c_A, c_B 直接平均当 C	忽略了质量不同
密度不同时用总面积中心当 C	质心是质量加权，不是面积加权
先搬到遥远原点再减 $m\|C\|^2$ 却在大坐标下算	数学对，浮点可能抵消
两块几何重叠仍当两份材料	质量被算两次；Box2D 当前就是这样，它不是几何并集

重叠这一点不是公式写错，是模型选择：教学和本版引擎把各形状的质量独立累加。若形状交叠，交叠区域的密度被加了两遍。

关节连着的两个刚体也不走这一节。那是两个质心、两套惯量，靠约束冲量耦合，不是先合成一个 I_C。

第02章第 2.6 节先算多形状质心，第 10 节把两遍累加落到 `b2UpdateBodyMassData`。

8. 两个对照数字，用来钉住定义

质量同为 $m = 2\,\mathrm{kg}$、外半径同为 $R = 0.5\,\mathrm{m}$：

形状	质量怎么摊	I_C
均匀实心圆盘	从圆心铺到边缘	$\frac{1}{2}mR^2 = 0.25\,\mathrm{kg \cdot m^2}$
薄圆环	全部集中在半径 R	$mR^2 = 0.50\,\mathrm{kg \cdot m^2}$

圆环更“转不动”，因为每一小块的 r 都等于最大的那个 R。 圆盘公式里的 $\frac{1}{2}$ 来自积分，不是经验系数；推导见第02章第5节。 边角碰撞时，惯量大的物体更不愿意被撞着转开，后面第09章的有效质量会把这一点算进去。

同一图在第02章标为图 2-2，这里只借用结论：离轴越远，权重按平方增加。

9. 收到引擎里，它变成什么

对每个形状先算自己绕自身质心的 I_C：圆是 $\frac{1}{2}mR^2$，矩形是 $m(w^2+h^2)/12$，多边形拆成三角形再积分。 一个刚体上有好几块形状时，按第 7 节做两遍：先加权求出总质心，再把各块用平行轴搬到总质心上相加。 求解器真正频繁用的是 I^{-1}：力矩除以惯量得到角加速度。惯量越大，同样一拧，角速度变得越慢。 静态、运动学刚体，以及锁定旋转的刚体，用零逆惯量表示“拧不动”，不是在内存里存无穷大。

形状积分和 `b2UpdateBodyMassData` 的两遍循环属于第02章。本章保证三件事：看到 `inertia` 想到 $\int r^2\,dm$；换轴用 $I_C + md^2$；合并时先求 C，再对每块加 $I_{C_i} + m_i\|c_i - C\|^2$。

10. 练习

题 1

用一句话区分质量和转动惯量。

答案：质量衡量同样的力能产生多大平动加速度；转动惯量衡量同样的力矩能产生多大角加速度，并且还取决于质量离转轴有多远。

题 2

2 kg 的点质量距轴 0.5 m。I 是多少？若距离改为 1 m，I 变成几倍？

答案：$I = 2 \times 0.25 = 0.5\,\mathrm{kg\cdot m^2}$。距离加倍，$I$ 变为 $2 \times 1 = 2\,\mathrm{kg\cdot m^2}$，是原来的 4 倍。

题 3

$I = 0.5\,\mathrm{kg\cdot m^2}$，$\omega = 4\,\mathrm{rad/s}$。旋转动能是多少？若有人把动能写成 $I\omega^2 = 8\,\mathrm{J}$，错在哪？

答案：$E_{\mathrm{rot}} = \frac{1}{2} \times 0.5 \times 16 = 4\,\mathrm{J}$。漏掉了 $\frac{1}{2}$。I 是 ω^2 前面整体系数的两倍。

题 4

为什么惠更斯研究钟摆会碰到 $\sum mr^2$，而不是 $\sum mr$？

答案：摆动时每块质量的速率与到转轴距离成正比。动能和等效摆长里都会出现 $v^2 \propto r^2$，所以惯性必须按距离平方加权。若只按 $\sum mr$ 加权，得到的是质心，不是转动惯性。

题 5

梁截面惯性矩和刚体转动惯量能否直接代入 $\tau = I\alpha$？

答案：不能。前者单位是 m^4，没有质量；后者单位是 $\mathrm{kg}\cdot\mathrm{m}^2$。$\tau = I\alpha$ 里的 I 必须是质量二次矩。

题 6

平行轴公式里的交叉项 $2d\cdot\int r\,dm$ 为什么会消失？如果新轴不平行于质心轴，还能不能用 $I_O = I_C + md^2$？

答案：r 是相对质心的位置，质心定义迫使 $\int r\,dm = 0$。二维本书的轴都垂直于纸面，彼此已经平行。三维里若两轴不平行，平方展开会混进惯量积，不能再用这个标量公式。

题 7

沿用第 6.4 节：$1\,\mathrm{kg}$ 在 $x=0$，$3\,\mathrm{kg}$ 在 $x=2\,\mathrm{m}$。绕 $x=2$ 的惯量是多少？若有人用 $I_O + m\cdot 2^2$、其中 $I_O = 12$，错在哪？

答案：按定义是 $4\,\mathrm{kg}\cdot\mathrm{m}^2$。从质心搬：$3 + 4\cdot(0.5)^2 = 4$。$I_O + m\cdot 2^2 = 28$ 错在把原点轴当成了质心轴；定理的一端必须是质心。

题 8

两块焊成一个刚体：A 为 $1\,\mathrm{kg}$，$c_A = (0,0)$，$I_A = 0.2$；B 为 $3\,\mathrm{kg}$，$c_B = (2,0)$，$I_B = 0.6$。单位分别为 kg、m、kg·m²。总质心和绕总质心的惯量各是多少？若只把 $0.2 + 0.6$ 相加，漏了什么？

答案：$m = 4\,\mathrm{kg}$，$C_x = 1.5\,\mathrm{m}$。 $I_C = 0.2 + 1\cdot(1.5)^2 + 0.6 + 3\cdot(0.5)^2 = 0.2 + 2.25 + 0.6 + 0.75 = 3.8\,\mathrm{kg}\cdot\mathrm{m}^2$。 只加 0.8 漏了两块相对总质心的平行轴项 $2.25 + 0.75 = 3$。

题 9

为什么惯量必须第二遍才算，不能和质心在同一轮循环里加完？

答案：平行轴项里的 $d_i = c_i - C$ 需要已经知道总质心 C。第一遍只能累加质量和 $m_i c_i$。

11. 一句话收束

转动惯量 = 把“质量如何摊在转轴周围”压缩成的一个数。 平方来自 $v = \omega r$；历史上来自惠更斯给钟摆找等效长度，欧拉再把它写成和 $F = ma$ 配套的转动定律。 换一根平行轴时，把 $\|r + d\|^2$ 展开，交叉项被质心定义消掉，剩下 $I_O = I_C + md^2$。 几块焊成一个刚体时，质心按质量加权；惯量把每块自己的 I_{C_i} 用平行轴搬到总质心再相加，不能直接把各块的 I_{C_i} 加起来。

下一章从许多小质量块出发，算出质心，再对圆、三角形、胶囊、多边形真正积分出 I_C，并用两遍算法把各块搬到总质心。 下一章：02 · 质量、质心与转动惯量 · 返回总目录

02 • 质量、质心与转动惯量

返回总目录

第01章说明了怎样描述位置和旋转。 02a · 转动惯量的定义与由来 说明 $I = \int r^2\, dm$ 为什么长这样、平方从动能来、以及惠更斯和欧拉怎样把它独立出来。 本章接着解释同样的力为什么让不同物体产生不同的加速度，以及为什么质量相同也可能难以转动；从第 3 节起直接用定义做形状积分，第 10 节把多块焊成一个刚体。 全部讨论采用二维均匀薄片模型，密度单位为 $\mathrm{kg/m^2}$，不是三维体密度。 除特别说明，惯量都指垂直于二维平面、穿过质心的 z 轴转动惯量。

1. 从许多小质量块开始，而不是先背公式

把刚体分成小块，第 i 块质量为 m_i，位置为 x_i。 总质量：

$$m = \sum_i m_i.$$

质量是平动惯性的尺度，单位千克。 同样的力作用下，质量越大，速度改变越慢。 质心是质量加权平均位置：

$$c = \frac{\sum_i m_i x_i}{m}.$$

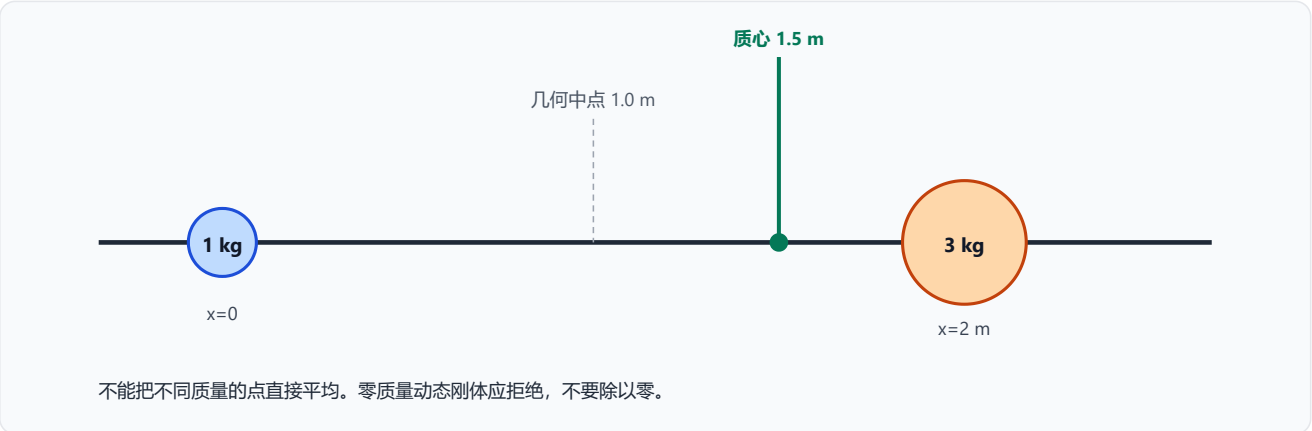
分子单位为千克·米，除以千克，结果为米。 “加权”意味着重的一侧贡献更大，不能把不同质量的点直接平均。 例如左端 $m_1 = 1\,\mathrm{kg}$、$x_1 = 0$，右端 $m_2 = 3\,\mathrm{kg}$、$x_2 = 2\,\mathrm{m}$：

$$m = 4\,\mathrm{kg}, \qquad c_x = \frac{1 \times 0 + 3 \times 2}{4} = 1.5\,\mathrm{m}.$$

质心更靠近较重的一侧，符合直观。

图 2-1　质心更靠近较重的一侧，不是两端中点

m1=1 kg 在 x=0，m2=3 kg 在 x=2 m。cx = (0+6)/4 = 1.5 m。



看图： 右边 3 kg 把质心从 1 m 拉到 1.5 m。后面刚体原点可以不是这个点；动力学积分推进的是质心。

若所有质量都为零，分母为零，质心公式不再定义一个唯一物理答案。 教学引擎应拒绝零质量动态刚体，或制定独立的明确约定，不要靠除以零处理。

2. 连续薄片：求和如何变为积分

面积微元为 dA，面密度为 ρ，微元质量：

$$dm = \rho\, dA.$$

因此：

$$m = \int_A \rho\, dA, \qquad c = \frac{1}{m}\int_A x\rho\, dA.$$

均匀形状中 ρ 为常量，可移到积分外。 质量等于密度乘面积；质心退化为面积加权中心。 多形状组成的刚体，各形状可以有不同密度，不能拿总面积中心直接代替总质心。 非均匀密度函数不属于本章简单形状公式的前提。

2.1 这个积分在 Box2D 里怎样变成数字

引擎**不会**对 $\int x\rho\, dA$ 做网格求积，也不会随机撒点。 二维均匀密度下，连续积分恒等于有限块的质量加权平均：

$$c = \frac{1}{m}\int_A x\rho\, dA \quad\Longleftrightarrow\quad c = \frac{\sum_i m_i c_i}{\sum_i m_i}.$$

因此实现只做两件事：把形状拆成能闭式求质心的块，再按质量加权。 调度在 `b2ComputeShapeMass`：圆、胶囊、多边形各走一条公式；线段和链段返回零质量，不进入质心。 第 2.2–2.6 节按圆、三角形、胶囊、多边形、多形状刚体把加减乘除逐步走完。

形状	质心怎么得到	源码
圆	直接取圆心	`b2ComputeCircleMass`
胶囊	两端圆心的中点	`b2ComputeCapsuleMass`
凸多边形	从第一个顶点扇形剖分，三角形面积加权	`b2ComputePolygonMass`
多形状刚体	各形状 (m_i, c_i) 再质量加权	`b2UpdateBodyMassData`

 图 2-4　积分公式在引擎里变成有限次加权平均

看图： 左边圆心即质心；中间胶囊关于中心线对称，整体质心在两端中点；右边才真正“积分”，而且积的是三角形，不是像素。 具体数字步骤从第 2.2 节的圆开始，到第 2.5 节的梯形结束。

返回值 `b2MassData.center` 是形状局部坐标中的质心，单位米。 刚体再把它聚合成 `localCenter`，世界质心为 $c_w = p + Rc_l$。 碰撞用的 `polygon.centroid` 是核心顶点的几何形心，创建多边形时就算好；质量路径在有圆角时会先外推顶点，二者不必逐位相同。

下面按**圆 → 三角形 → 胶囊 → 多边形**把源码里的加减乘除逐步走完。 惯量推导仍在第 5、7、8 节；本节只把质心算明白。

2.2 第一步：均匀圆，质心就是圆心

取

$$\text{center} = (3,1)\,\text{m}, \qquad R = 0.5\,\text{m}, \qquad \rho = 2\,\text{kg/m}^2.$$

步骤 1. 半径平方：

$$rr = R^2 = 0.25\,\text{m}^2.$$

步骤 2. 面积乘密度得质量：

$$m = \rho\pi R^2 = 2 \cdot \pi \cdot 0.25 = \frac{\pi}{2} \approx 1.570796\,\text{kg}.$$

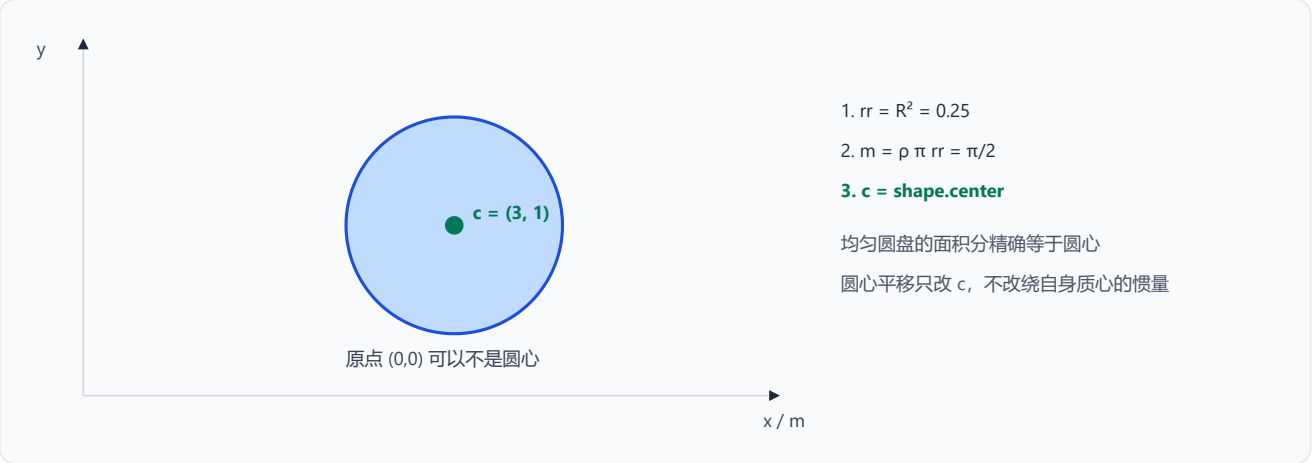
步骤 3. 均匀圆盘对圆心对称，面积分 $\int x\,dA$ 的平均值就是圆心：

$$c = \texttt{shape.center} = (3,1)\,\text{m}.$$

没有循环，也没有“对圆面积做数值积分”。圆心可以不是刚体原点；原点只是建模约定。 把圆平移到 $(10,4)$，质心跟着变成 $(10,4)$，绕自身质心的惯量不变。

图 2-6　圆：质量用 πR²，质心直接取圆心

例：center=(3,1) m，R=0.5 m，ρ=2 kg/m²。m=π/2 kg，c=(3,1) m。没有循环。



看图： 绿色点就是 c。源码对应三行赋值：`mass = density * B2_PI * rr`，`center = shape->center`。定位：几何质量计算，`b2ComputeCircleMass`，221–233行。

2.3 第二步：一个三角形，三个顶点平均

多边形路径在 `count==3` 且半径为零时，循环只跑一轮，结果必须等于三个顶点的平均。 先用这个事实核对公式，再推广到更多顶点。

取逆时针顶点

$$v_0 = (1,1), \quad v_1 = (5,1), \quad v_2 = (1,4), \qquad \rho = 2\,\text{kg/m}^2.$$

步骤 1. 参考点取第一个顶点，减小远处大坐标相减的误差：

$$r = v_0 = (1,1).$$

步骤 2. 两条相对边：

$$e_1 = v_1 - r = (4,0), \qquad e_2 = v_2 - r = (0,3).$$

步骤 3. 有向面积（叉积）再取一半：

$$D = e_1 \times e_2 = 4 \cdot 3 - 0 \cdot 0 = 12, \qquad A = \frac{D}{2} = 6\,\mathrm{m}^2.$$

逆时针时 $D > 0$。若得到负面积，顶点顺序反了，质心公式会整章跟着错。

步骤 4. 标准三角形的质心相对参考点是

$$\frac{e_1 + e_2}{3} = \left(\frac{4}{3}, 1\right).$$

加回参考点：

$$c = r + \frac{e_1 + e_2}{3} = (1,1) + \left(\frac{4}{3}, 1\right) = \left(\frac{7}{3}, 2\right)\mathrm{m}.$$

步骤 5. 用三个顶点平均独立核对：

$$c = \frac{v_0 + v_1 + v_2}{3} = \frac{(1,1) + (5,1) + (1,4)}{3} = \left(\frac{7}{3}, 2\right).$$

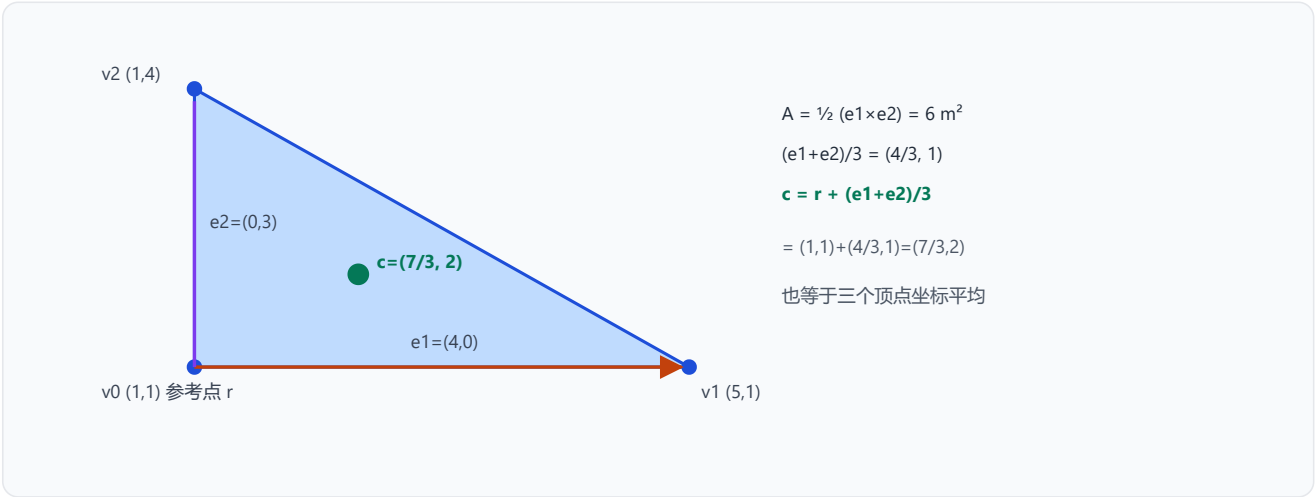
两式相同。质量为

$$m = \rho A = 2 \times 6 = 12\,\mathrm{kg}.$$

源码累加的是 $A \cdot (e_1 + e_2)/3 = (8, 6)$，再除以面积 6，得到相对质心 $(4/3, 1)$，最后加 r。 这就是 `b2MulAdd(center, triangleArea * inv3, b2Add(e1,e2))` 再 `center *= 1/area`。

图 2-7　三角形：三个顶点平均，也等于 (e1+e2)/3 加回 v0

v0=(1,1)，v1=(5,1)，v2=(1,4)。c=(v0+v1+v2)/3=(7/3, 2)。这是多边形循环在 count=3 时的全部工作。



看图： 橙色、紫色箭头是 e_1, e_2。绿色点既是三顶点平均，也是源码那一轮循环的输出。

2.4 第三步：胶囊，两端中点；半圆外偏会抵消

胶囊不是圆，也还不是多边形。源码把它拆成**中间矩形 + 两个半圆**（合计一个整圆）。 取水平胶囊

$$p_1 = (0,0), \quad p_2 = (4,0), \quad R = 1\,\mathrm{m}, \qquad \rho = 2\,\mathrm{kg/m}^2.$$

步骤 1. 中心线长度：

$$L = \|p_2 - p_1\| = 4\,\mathrm{m}.$$

步骤 2. 两块质量（注意矩形宽是直径 $2R$，长是 L）：

$$m_{\text{圆}} = \rho\pi R^2 = 2\pi\,\text{kg}, \qquad m_{\text{箱}} = \rho(2R)L = 16\,\text{kg}.$$

总质量 $m = 16 + 2\pi \approx 22.283185\,\text{kg}$。

步骤 3. 三块各自的质心：

块	质量	质心
中间矩形	16	$(2, 0)$
左半圆	π	$p_1 - (a, 0) = (-a, 0)$
右半圆	π	$p_2 + (a, 0) = (4 + a, 0)$

其中半圆质心相对圆心外偏

$$a = \frac{4R}{3\pi} = \frac{4}{3\pi} \approx 0.424413\,\text{m}.$$

步骤 4. 质量加权：

$$c_x = \frac{16 \cdot 2 + \pi(-a) + \pi(4 + a)}{16 + 2\pi} = \frac{32 + 4\pi}{16 + 2\pi} = 2.$$

a 被左右抵消。$c_y = 0$。因此

$$c = \frac{p_1 + p_2}{2} = (2, 0).$$

这就是源码直接写 `0.5f * (p1 + p2)` 的原因：对称胶囊不必每次先算 a。 a 仍然要留给惯量的平行轴，质心这一栏用不到它。 若 $L = 0$，胶囊退化为圆，中点就是圆心，与第 2.2 节一致。

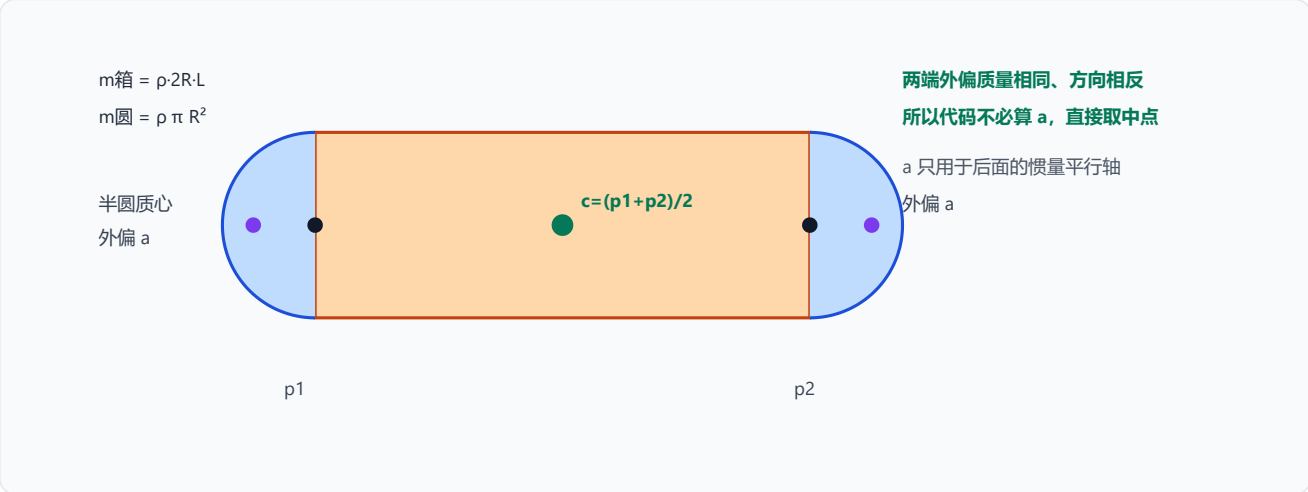
倾斜胶囊同样取中点。例如 $p_1 = (1, 1)$、$p_2 = (4, 5)$：

$$c = \big(2.5,\ 3\big), \qquad L = 5.$$

两端外偏沿中心线方向相反，加权后仍然抵消。

图 2-8　胶囊：拆成矩形加两个半圆，整体质心在两端中点

p1=(0,0)，p2=(4,0)，R=1。左半圆质心左偏 4R/(3π)，右半圆右偏同样多，加权后抵消。



看图： 绿色点是代码输出的 c。两侧紫色点才是半圆自己的质心；质量相同、方向相反，所以整体仍落在中点。 定位：几何质量计算，`b2ComputeCapsuleMass`，235–250行。

2.5 第四步：多边形，把第 2.3 节的三角形加起来

凸多边形从 v_0 扇形剖分成 $n-2$ 个三角形，每个三角形按第 2.3 节计算，再按面积加权。 这就是把积分 $\int x\,dA$ 拆成有限块之和。

取零圆角梯形，顶点逆时针：

$$v_0=(0,0),\quad v_1=(6,0),\quad v_2=(4,3),\quad v_3=(0,3).$$

密度仍取 $\rho=2$。$n=4$，循环两次。 初始化：$r=v_0=(0,0)$，累加器 $center=(0,0)$，$area=0$。

第一轮 $i=1$，三角形 (v_0,v_1,v_2)：

$$e_1=(6,0),\quad e_2=(4,3),\quad D=18,\quad A_1=9,$$
$$\frac{e_1+e_2}{3}=\left(\frac{10}{3},1\right),\qquad A_1\cdot\frac{e_1+e_2}{3}=(30,9).$$

累加器变为 $center=(30,9)$，$area=9$。

第二轮 $i=2$，三角形 (v_0,v_2,v_3)：

$$e_1=(4,3),\quad e_2=(0,3),\quad D=12,\quad A_2=6,$$
$$\frac{e_1+e_2}{3}=\left(\frac{4}{3},2\right),\qquad A_2\cdot\frac{e_1+e_2}{3}=(8,12).$$

累加器变为 $center=(38,21)$，$area=15$。

收尾 与源码相同：先除以总面积，再加回参考点。

$$\bar{c}=\frac{(38,21)}{15}=\left(\frac{38}{15},\frac{7}{5}\right)=\left(2.5\overline{3},\,1.4\right),\qquad c=r+\bar{c}=\left(\frac{38}{15},\frac{7}{5}\right)\text{m}.$$

质量 $m=\rho A=2\times15=30\,\text{kg}$。

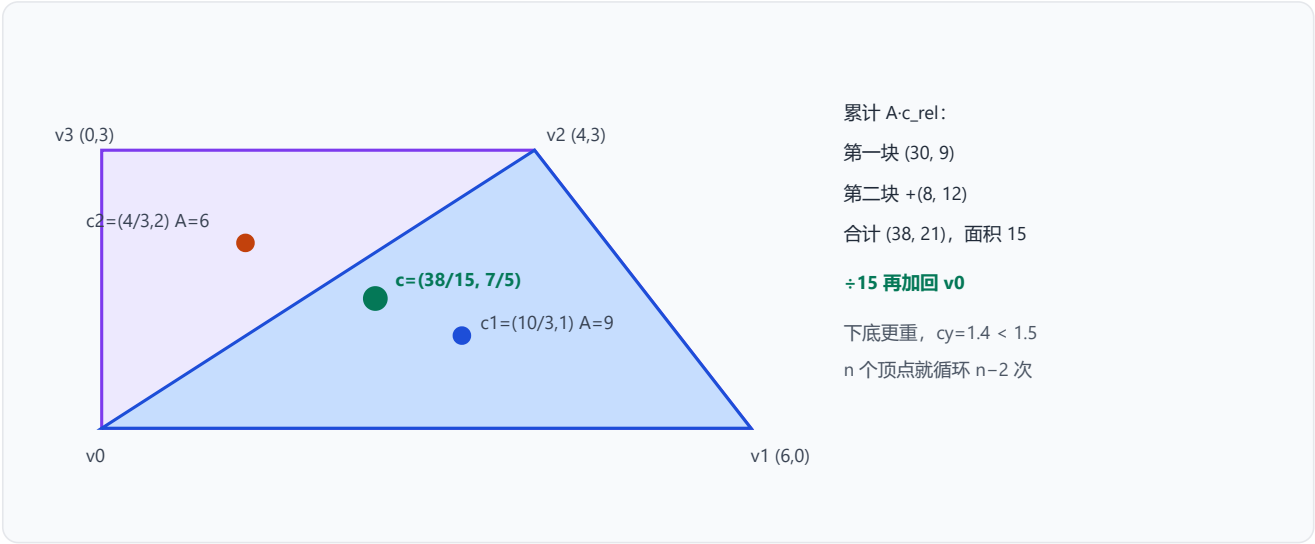
轮	e_1	e_2	A	本轮 $A(e_1+e_2)/3$	累计 `center`	累计面积
1	$(6,0)$	$(4,3)$	9	$(30,9)$	$(30,9)$	9
2	$(4,3)$	$(0,3)$	6	$(8,12)$	$(38,21)$	15
除以面积					$(38/15,\ 7/5)$	

下底长 6、上底长 4，所以质心应低于高度中点 1.5，并偏左于 $x=3$。 $c_y=1.4<1.5$、$c_x\approx2.533<3$，与直观一致。

五个顶点只是循环多一轮，公式不变：n 个顶点生成 $n-2$ 个三角形。 `count==1` 转圆，`count==2` 转胶囊，见 `b2ComputePolygonMass` 的 302–317 行。

图 2-9　多边形：从 v0 扇形剖分，按面积把各三角形质心加权

顶点 (0,0)、(6,0)、(4,3)、(0,3)。下底更长，所以质心低于高度中点，并偏左。



看图： 蓝色块面积 9、橙色块面积 6。绿色点是加权结果，不是某个顶点，也不是包围盒中心。

若参考点不是原点，例如把整个梯形平移 $(10, 4)$，累加器算的是相对 v_0 的 $\bar{c}$，最后 `b2Add(r, center)` 会加回 $(10, 4)$，质心变为 $(10 + 38/15,\ 4 + 7/5)$。

2.6 第五步：一个刚体上的多个形状

每个形状先走完上面某一条路径，得到 (m_i, c_i)。 刚体局部质心再做一次第 1 节那种加权。推导见 02a 第 7 节：整体质心定义是 $\int x\, dm$，把积分拆到各块上就是

$$C = \frac{\sum_i m_i c_i}{\sum_i m_i}.$$

重的一块把 C 往自己那边拉。不能把各块质心直接平均；密度不同时也不能拿总面积中心代替总质心。这一步还没有转动惯量。各块的 I_{C_i} 绕的不是同一根轴，要等第 10 节用平行轴搬到 C 上再加。

例：同一刚体上一个圆、一个三角形，$\rho = 1$。

- 圆：$R = 1$，圆心 $(0, 0)$。$m_1 = \pi$，$c_1 = (0, 0)$。
- 三角形：$(2, 0), (5, 0), (2, 3)$。$r = (2, 0)$，$e_1 = (3, 0)$，$e_2 = (0, 3)$，$A = 4.5$，$c_2 = (3, 1)$，$m_2 = 4.5$。

$$C = \frac{\pi(0, 0) + 4.5(3, 1)}{\pi + 4.5} = \frac{(13.5,\ 4.5)}{\pi + 4.5} \approx (1.767,\ 0.589)\ \mathrm{m}.$$

密度为零的形状这一步质量为 0，不拉质心。 两个形状若重叠，重叠区域会被算两次；这不是几何并集积分。 静态、运动学刚体不走这条质量和质心路径。 完整聚合代码在第 10 节。

3. 转动惯量：离轴越远，权重是距离平方

定义、平方的来源和历史由来见 02a。本节起把它当成计算工具。

以质心为轴，令微元相对质心的位置为 $r = x - c$。 定义：

$$I_C = \int_A \|r\|^2 \, dm = \rho \int_A (r_x^2 + r_y^2) \, dA.$$

I_C 单位为 $\mathrm{kg \cdot m^2}$。 它不是质量，也不是“长宽乘积”。 一个质量为 m 的点放在距轴 d 处，贡献 md^2。距离扩大两倍，惯量贡献扩大四倍。 把质量向中心收拢，在总质量不变时也能减小转动惯量。

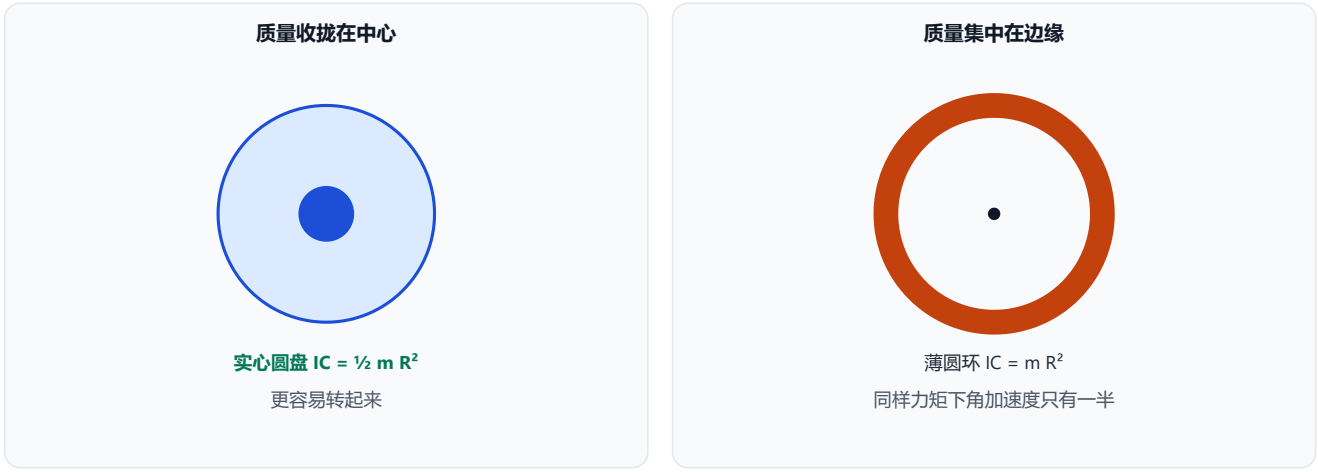
为什么平方自然出现？ 匀速转动时微元速率为 $v = |\omega| \|r\|$。 把每块的动能相加：

$$E_{\text{rot}} = \int \frac{1}{2} v^2 \, dm = \frac{1}{2} \omega^2 \int \|r\|^2 \, dm = \frac{1}{2} I_C \omega^2.$$

所以惯量是旋转动能写成 $\frac{1}{2} I_C \omega^2$ 时的惯性参数；严格说，ω^2 前的系数是 $I_C/2$，不是 I_C。 例如 $I_C = 2\,\mathrm{kg \cdot m^2}$、$\omega = 3\,\mathrm{rad/s}$，旋转动能为 $9\,\mathrm{J}$。

图 2-2　同样质量，离轴越远越难转动

点质量贡献 md²。距离扩大 2 倍，惯量贡献扩大 4 倍。



看图： 实心圆盘是 $\frac{1}{2} mR^2$，薄环是 mR^2。边角碰撞时，惯量大的物体更不愿意转开，有效质量也就更大。

4. 平行轴定理：为什么移轴要加一项

完整的人话推导见 02a 第 6 节。本节把同一条代数再走一遍，因为后面胶囊、多边形移轴、复合刚体都要反复用它。

设新轴通过 O，质心为 C，$d = C - O$。微元相对质心的位置是 r，相对新轴的位置就是 $r + d$。按定义

$$I_O = \int \|r + d\|^2 \, dm.$$

平方展开：

$$\|r + d\|^2 = \|r\|^2 + 2r \cdot d + \|d\|^2.$$

d 对所有微元相同，积分后

$$I_O = I_C + 2d \cdot \int r \, dm + \|d\|^2 \int dm.$$

质心的定义就是 $c = \frac{1}{m} \int x \, dm$，而 $r = x - c$，所以

$$\int r\,dm = \int x\,dm - c\int dm = mc - cm = 0.$$

交叉项消失，$\int dm = m$，故

$$I_O = I_C + m\|d\|^2.$$

新轴远离质心时，惯量只能增加，不能减少。反过来，已知非质心轴惯量才能减去 md^2 得到质心惯量。不能把任意两轴都当成“直接加 md^2”；定理的一端必须是质心轴。任意两平行轴之间其实是

$$I_B = I_A + m\big(\|c-B\|^2 - \|c-A\|^2\big),$$

只有 A 或 B 之一是质心，才会退化成加上（或减去）一项 md^2。

用第 1 节那两个点质量核对。$m_1 = 1$ 在 $x = 0$，$m_2 = 3$ 在 $x = 2$，$c = 1.5$，$m = 4$：

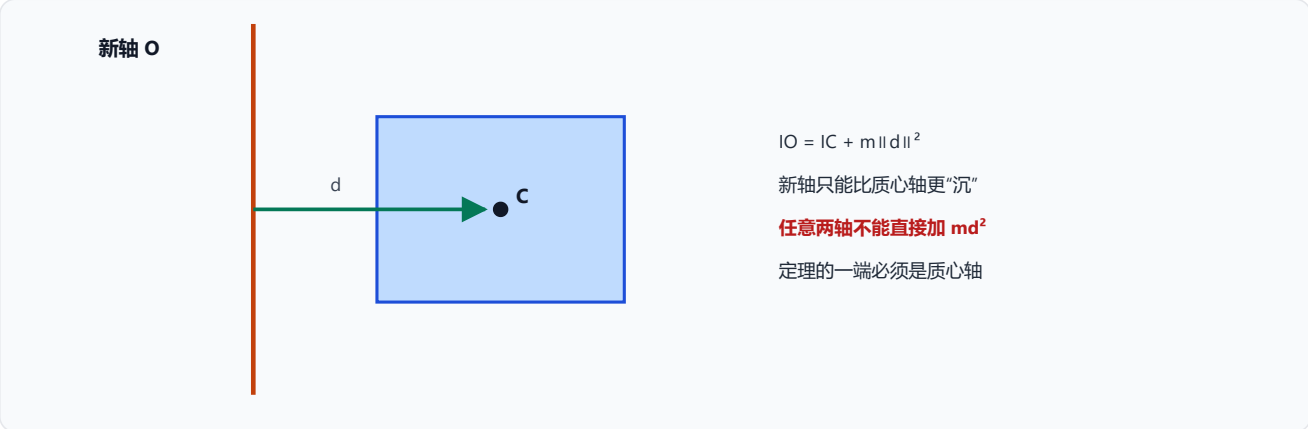
$$I_C = 1\cdot(1.5)^2 + 3\cdot(0.5)^2 = 3, \qquad I_{x=0} = 3\cdot 2^2 = 12, \qquad 3 + 4\cdot(1.5)^2 = 12.$$

再搬到右端 $x = 2$：$I_C + 4\cdot(0.5)^2 = 4$，与按定义 $1\cdot 2^2 = 4$ 相同。若误用 $12 + 4\cdot 2^2 = 28$，就是把原点轴当成了质心轴。

例：$I_C = 0.25$、$m = 2$、$d = 1$，则 $I_O = 2.25\,\mathrm{kg\cdot m^2}$。 如果把 I_O 错当 I_C，同样力矩下角加速度会被低估 9 倍。

图 2-3　平行轴：绕新轴的惯量 = 绕质心惯量 + md²

例：IC=0.25，m=2 kg，d=1 m，则 IO=2.25。错把 IO 当 IC，角加速度会被低估 9 倍。



看图： 新轴 O 离质心越远，转动越沉。复合刚体先算各块绕自己质心的惯量，再平行轴搬到总质心。第 7 节胶囊会连续用两次；第 10 节第二遍累加写的就是 $I_{C_i} + m_i\|c_i - C\|^2$。

5. 圆盘：一步步积分出一半

半径为 R，均匀密度为 ρ。 取半径 r、厚度 dr 的细圆环。 圆环面积约为周长乘厚度：$dA = 2\pi r\,dr$。 质量：

$$m = \int_0^R \rho 2\pi r\,dr = 2\pi\rho\frac{R^2}{2} = \pi\rho R^2.$$

每个圆环到轴的距离都是 r，故惯量：

$$I_C = \int_0^R r^2 \rho 2\pi r \, dr = 2\pi\rho\frac{R^4}{4} = \frac{1}{2}\pi\rho R^4 = \frac{1}{2}mR^2.$$

这里的 $1/2$ 来自积分，不能拿薄圆环公式 mR^2 代替实心圆盘。 圆盘的质量分布在轴到边缘之间，当然比所有质量都集中在边缘更容易转动。

取 $R = 0.5\,\mathrm{m}$，$\rho = 8/\pi\,\mathrm{kg/m^2}$：

$$m = \pi\frac{8}{\pi}(0.5)^2 = 2\,\mathrm{kg}, \qquad I_C = \frac{1}{2}\times 2 \times 0.25 = 0.25\,\mathrm{kg\cdot m^2}.$$

圆心即质心。圆心在局部坐标中不为零，也不会改变绕圆自身质心的惯量。 质心的逐步计算见第 2.2 节；下面只补质量和惯量积分。 所以第 2.1 节的圆路径没有循环：`massData.center = shape->center`。 本版函数返回的正是质心惯量：

```
        b2MassData massData;
        massData.mass = density * B2_PI * rr;
        massData.center = shape->center;

        // inertia about the center of mass
        massData.rotationalInertia = massData.mass * 0.5f * rr;
```

定位：几何质量计算，`b2ComputeCircleMass`，221–233行。

6. 矩形：长宽平方为什么相加

矩形以质心为原点，完整宽度为 w、完整高度为 h。 积分范围 $x \in [-w/2, w/2]$、$y \in [-h/2, h/2]$。 面积 $A = wh$，质量 $m = \rho wh$。

$$I_C = \rho\int_{-h/2}^{h/2}\int_{-w/2}^{w/2}(x^2 + y^2)\,dx\,dy.$$

把两项分开：

$$\int_{-w/2}^{w/2} x^2\,dx = \frac{w^3}{12}, \qquad \int_{-h/2}^{h/2} y^2\,dy = \frac{h^3}{12}.$$

第一项乘高度，第二项乘宽度：

$$I_C = \rho\left(\frac{hw^3}{12} + \frac{wh^3}{12}\right) = \frac{m(w^2 + h^2)}{12}.$$

若程序存的是半宽 $a = w/2$、半高 $b = h/2$，等价公式是 $I_C = m(a^2 + b^2)/3$。 把半宽直接代入完整宽公式，会小估4倍。 例：$w = 4\,\mathrm{m}$、$h = 2\,\mathrm{m}$、$\rho = 2\,\mathrm{kg/m^2}$。 质量 $m = 16\,\mathrm{kg}$，惯量 $I_C = 16(16 + 4)/12 = 80/3 \approx 26.666667$。 本版矩形通常由多边形表示，质量通过通用多边形积分得到，而不是另有必须调用的盒子质量接口。

7. 胶囊：矩形加两个半圆，不能漏掉半圆质心

胶囊由一段长 L 的中心线与半径 R 组成。 中间矩形的完整长宽为 L 和 $2R$。 两端各接半圆，中心线端点不是胶囊外端点；总长度为 $L + 2R$。 整体质心为中心线两端点的中点。 两端半圆外偏相互抵消，中间矩形也对称，因此质量代码直接写：

$$c = \frac{p_1 + p_2}{2}.$$

质心逐步抵消见第 2.4 节。半圆质心相对圆心外偏 $4R/(3\pi)$，只进入后面的惯量平行轴，不改这个 c。

矩形质量与两半圆总质量分别为：

$$m_b = 2\rho R L, \qquad m_c = \rho \pi R^2.$$

所以总质量 $m = m_b + m_c$。 关键是半圆质心相对其圆心向外偏移，不在圆心上。

7.1 半圆质心的推导

考虑朝 $+x$ 的半圆，极角 $\phi \in [-\pi/2, \pi/2]$，径向坐标 $r \in [0, R]$。 坐标 $x = r\cos\phi$，面积微元 $dA = r\,dr\,d\phi$。

$$\int x\,dA = \left(\int_0^R r^2\,dr\right)\left(\int_{-\pi/2}^{\pi/2} \cos\phi\,d\phi\right) = \frac{R^3}{3} \times 2.$$

除以半圆面积 $\pi R^2/2$：

$$a = \frac{4R}{3\pi}.$$

a 为向外质心偏移，单位米。 设中心线半长为 $H = L/2$，半圆自己的质心距离整体质心为 $H + a$。

7.2 为什么需要连续使用两次平行轴定理

单个半圆质量为 $m_c/2$。 绕其圆心的惯量为 $\frac{1}{2}(m_c/2)R^2$。 先从圆心轴移到半圆质心轴，需要减去 $(m_c/2)a^2$。 再移到胶囊整体质心轴，需要加上 $(m_c/2)(H + a)^2$。 两个半圆合计：

$$I_c = m_c\left(\frac{1}{2}R^2 - a^2 + (H + a)^2\right) = m_c\left(\frac{1}{2}R^2 + H^2 + 2Ha\right).$$

注意额外项 $2Ha$ 不能省略。 把两端半圆粗暴当成两个“半质量圆盘”放在端点，只加 $m_c H^2$，会漏掉这项。 矩形惯量：

$$I_b = \frac{m_b(L^2 + 4R^2)}{12}, \qquad I_C = I_c + I_b.$$

取 $R = 1$、$L = 2$、$\rho = 1$： $m_c = \pi$，$m_b = 4$，$H = 1$，$a = 4/(3\pi)$。 总质量 $m = 4 + \pi \approx 7.141593\,\text{kg}$。

$$I_c = \frac{3}{2}\pi + \frac{8}{3}, \qquad I_b = \frac{8}{3}, \qquad I_C = \frac{3}{2}\pi + \frac{16}{3} \approx 10.045722.$$

令 $L = 0$，矩形消失，结果自然退化为圆盘质量和惯量。 定位：几何质量计算，`b2ComputeCapsuleMass`，235–271行，尤其261–269行。

8. 多边形：三角剖分加积分

先考虑按逆时针排列的有效凸多边形，暂时令圆角半径为零。 质心逐步手算见第 2.3 节（一块三角形）和第 2.5 节（多块加权）。本节补剖分约定和惯量。 选择第一个顶点 r 为参考原点。 第 i 个三角形用顶点 (r, v_i, v_{i+1})。 相对边向量：$e_1 = v_i - r$，$e_2 = v_{i+1} - r$。 有向面积因子：$D = e_1 \times e_2$，三角形面积 $A_i = D/2$。 对于正确逆时针凸多边形，这些面积应为非负，合计为正。

8.1 用标准三角形替代任意三角形

把三角形内的相对位置写成：

$$x(u, v) = u e_1 + v e_2, \qquad u \geq 0,\ v \geq 0,\ u + v \leq 1.$$

面积变换的雅可比是 D，因此 $dA = D\, du\, dv$。 u, v 无量纲，D 单位平方米。 标准三角形上需要的积分为：

$$\int 1\, du\, dv = \frac{1}{2}, \quad \int u\, du\, dv = \int v\, du\, dv = \frac{1}{6}.$$

例如 $\int u\, du\, dv = \int_0^1 (1-v)^2/2\, dv = 1/6$。 所以三角形相对质心为 $(e_1 + e_2)/3$。 多边形相对质心通过面积加权：

$$\bar{c} = \frac{\sum_i A_i (e_{1i} + e_{2i})/3}{\sum_i A_i}, \qquad c = r + \bar{c}.$$

这里 $\bar{c}$ 相对参考点 r，c 则在形状局部坐标中。 这就是第 2 节面积分的离散形式：每个三角形贡献 $A_i \cdot (e_1 + e_2)/3$，再除以总面积。

当前循环从第一个顶点扇形剖分，i 跑 1 到 $count - 2$，对应三角形 (v_0, v_i, v_{i+1})。 参考点取 v_0 而不是坐标原点，是为了减小远处大坐标相减的舍入。

```
b2Vec2 r = vertices[0];

const float inv3 = 1.0f / 3.0f;

for ( int i = 1; i < count - 1; ++i )
{
          // Triangle edges
          b2Vec2 e1 = b2Sub( vertices[i], r );
          b2Vec2 e2 = b2Sub( vertices[i + 1], r );

          float D = b2Cross( e1, e2 );

          float triangleArea = 0.5f * D;
          area += triangleArea;

          // Area weighted centroid, r at origin
          center = b2MulAdd( center, triangleArea * inv3, b2Add( e1, e2 ) );
```

累加结束后除以总面积，再加回参考点：

```
B2_ASSERT( area > FLT_EPSILON );
float invArea = 1.0f / area;
center.x *= invArea;
center.y *= invArea;
massData.center = b2Add( r, center );
```

创建多边形时，`b2ComputePolygonCentroid` 用同一套扇形加权，结果写入 `polygon.centroid`，供碰撞与范围查询使用。 质量函数在半径大于零时会先外推顶点再积分，因此 `massData.center` 与 `polygon.centroid` 在圆角情况下一般不同。

8.2 不跳过惯量积分

展开平方：

$$\|ue_1 + ve_2\|^2 = u^2\|e_1\|^2 + 2uv(e_1 \cdot e_2) + v^2\|e_2\|^2.$$

标准三角形上的二次矩：

$$\int u^2\,du\,dv = \int v^2\,du\,dv = \frac{1}{12}, \qquad \int uv\,du\,dv = \frac{1}{24}.$$

第一式可由 $\int_0^1 (1-v)^3/3\,dv = 1/12$ 得到。 交叉项为 $\int_0^1 v(1-v)^2/2\,dv = 1/24$。 因此三角形关于参考点 r 的惯量为：

$$I_{r,i} = \frac{\rho D_i}{12}\left(\|e_{1i}\|^2 + e_{1i}\cdot e_{2i} + \|e_{2i}\|^2\right).$$

把三角形贡献相加，再把轴移到多边形质心：

$$I_C = \sum_i I_{r,i} - m\|\bar{c}\|^2.$$

减掉的是相对参考点的质心偏移 $\bar{c}$，不是世界坐标，也不是未经平移的局部质心 c。 定位：几何质量计算，`b2ComputePolygonMass`，349–397行。

8.3 一个完整三角形算例

顶点为 $(0,0),(2,0),(0,1)$，密度 $\rho = 3$。 $e_1 = (2,0)$、$e_2 = (0,1)$，$D = 2$，面积 $A = 1$。 质量 $m = 3$，质心 $c = (2/3, 1/3)$。 参考点惯量：

$$I_r = \frac{3\times 2}{12}(4 + 0 + 1) = \frac{5}{2}.$$

平移修正：$m\|c\|^2 = 3(4/9 + 1/9) = 5/3$。 最终：

$$I_C = \frac{5}{2} - \frac{5}{3} = \frac{5}{6} \approx 0.833333.$$

若算出了负惯量，优先检查顶点绕序、交叉项、密度、面积和移轴用的坐标。 不能把负惯量静默取绝对值然后宣称物理已经正确。

8.4 矩形手算：把源码循环走完

取零圆角矩形顶点：

$$v_0 = (0,0), \quad v_1 = (4,0), \quad v_2 = (4,2), \quad v_3 = (0,2).$$

参考点 $r = v_0$。循环产生两个三角形。

第一块 (v_0, v_1, v_2)：

$$e_1 = (4,0), \quad e_2 = (4,2), \quad D = 8, \quad A_1 = 4,$$
$$c_1 = \frac{e_1 + e_2}{3} = \left(\frac{8}{3}, \frac{2}{3}\right).$$

第二块 (v_0, v_2, v_3)：

$$e_1 = (4,2), \quad e_2 = (0,2), \quad D = 8, \quad A_2 = 4,$$
$$c_2 = \frac{e_1 + e_2}{3} = \left(\frac{4}{3}, \frac{4}{3}\right).$$

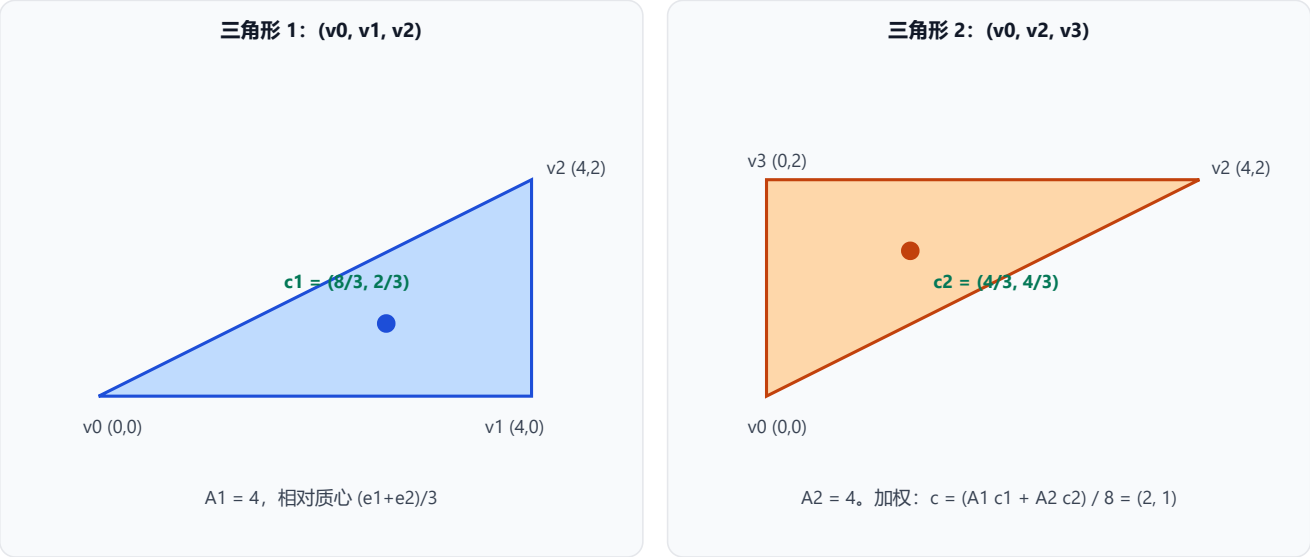
面积加权：

$$\bar{c} = \frac{4 \cdot c_1 + 4 \cdot c_2}{8} = (2,1), \qquad c = r + \bar{c} = (2,1).$$

这正是矩形几何中心。均匀密度下它也是质心。 源码累加的是 $A_i(e_1 + e_2)/3$，最后除以总面积，与上式相同。

图 2-5　多边形质心：扇形三角形的面积加权，不是网格积分

顶点 (0,0)、(4,0)、(4,2)、(0,2)。参考点取 v0。两个三角形面积都是 4，加权后 c=(2,1)。



看图： 不要对矩形画格子再求和。引擎只生成 $n - 2$ 个三角形，每个用闭式质心，再按面积加权。

9. 圆角多边形：精确几何与本版近似必须分开说

理论上，对凸核心多边形向外扩张半径 R，其精确面积为：

$$A_{\text{exact}} = A_{\text{core}} + P_{\text{core}} R + \pi R^2.$$

它来自核心面积、沿边矩形条和合计一整圆的圆角扇形。 这是精确圆角几何的面积定理，不是当前质量代码所执行的算法。 当前 `b2ComputePolygonMass` 对三个及以上顶点、半径大于零的情况，先把顶点向外推，再套普通多边形积分。 推顶点方向为相邻外法线之和的单位方向：

$$\hat{m}_i = \frac{n_{i-1} + n_i}{\|n_{i-1} + n_i\|}, \qquad v_i' = v_i + 1.412R\hat{m}_i.$$

注意源码常量实际是 `1.412f`，变量名虽为 `sqrt2`，却不是精确 $\sqrt{2}$。 不能把它改写成精确扇形积分，也不能承诺近似误差始终低估或始终高估。

```
if ( radius > 0.0f )
{
	// Approximate mass of rounded polygons by pushing out the vertices.
	float sqrt2 = 1.412f;
	for ( int i = 0; i < count; ++i )
	{
		int j = i == 0 ? count - 1 : i - 1;
		b2Vec2 n1 = shape->normals[j];
		b2Vec2 n2 = shape->normals[i];

		b2Vec2 mid = b2Normalize( b2Add( n1, n2 ) );
		vertices[i] = b2MulAdd( shape->vertices[i], sqrt2 * radius, mid );
	}
```

例如核心正方形为 $[-1,1]^2$，$R = 0.5$。 精确圆角面积为 $4 + 8(0.5) + \pi(0.5)^2 \approx 8.785398$。 推顶点近似得到正方形半边长 $1 + 1.412(0.5)/\sqrt{2}$，面积约 8.9906。 这是一个能手算看到近似误差的反例，不意味着源码误把几何碰撞形状也改成了该正方形。 质量近似与碰撞几何是不同计算路径。 本函数顶点数1时转圆，顶点数2时转胶囊，见302–317行。

10. 多形状刚体：两遍累加比大数相减更稳

概念推导见 02a 第 7 节。各形状已经有 (m_i, c_i, I_{C_i})。 焊成一个刚体之后，只有一个质心、一个绕该质心的惯量。质量直接相加；质心按质量加权；惯量必须先搬到总质心再相加。

第一遍求：

$$m = \sum_i m_i, \qquad C = \frac{\sum_i m_i c_i}{m}.$$

第二遍求：

$$I_C = \sum_i \left(I_{C_i} + m_i \|c_i - C\|^2 \right).$$

I_{C_i} 是第 i 块绕**自己**质心的惯量，不能和别的块直接相加。第二项把轴从 c_i 搬到总质心 C。每一项都非负；本版因此避免先把所有惯量转到遥远原点、再减去一个巨大的 $m\|C\|^2$。 后者数学上等价，浮点计算中却可能严重抵消有效数字。 这里的优化不是少算一个乘法，而是改变运算顺序以保留精度。

10.1 二维两块，把加减乘除走完

与 02a 第 7.6 节同一组数。A：$m = 2$，$c = (0, 0)$，$I_C = 0.25$。B：$m = 4$，$c = (3, 1)$，$I_C = 0.50$。

$$m = 6, \qquad C = \frac{(12, 4)}{6} = \left(2, \frac{2}{3}\right).$$

$$d_A = \left(-2, -\frac{2}{3}\right),\ \|d_A\|^2 = \frac{40}{9}; \qquad d_B = \left(1, \frac{1}{3}\right),\ \|d_B\|^2 = \frac{10}{9}.$$

$$I_C = 0.25 + 2 \cdot \frac{40}{9} + 0.50 + 4 \cdot \frac{10}{9} = \frac{169}{12} \approx 14.0833.$$

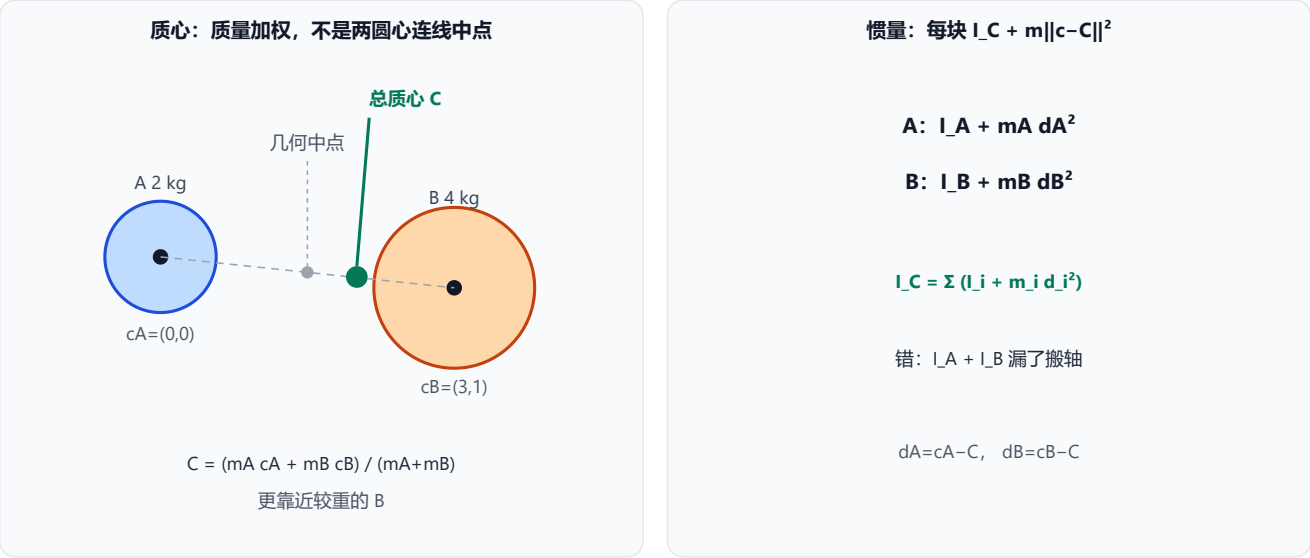
只加 $0.25 + 0.50 = 0.75$ 漏了平行轴，会小估约 19 倍。

一维核对仍可用第 1 节那两块：质量 1 和 3，局部质心 0 和 2，自身惯量 0.2 和 0.6。总质量 4，总质心 1.5，

$$I_C = 0.2 + 1(0 - 1.5)^2 + 0.6 + 3(2 - 1.5)^2 = 3.8.$$

图 2a-4　合并：质心按质量加权，惯量先搬轴再相加

第一遍求 C。第二遍把每块的 I_C 用 m d² 搬到总质心。不能把各块自己的 I_C 直接加。



看图： 绿点才是 C。右边两项都要留下：自身摊开 I_{C_i}，加上离总质心的 $m_i d_i^2$。

10.2 源码就是这两遍

第一遍对应：密度为零的形状写入空的 `b2MassData` 后跳过；其余调用 `b2ComputeShapeMass`，累加 m 与 $m_i c_i$，最后 $C = (\sum m_i c_i)/m$。

```
                if ( s->density == 0.0f )
                {
                        masses[shapeIndex] = (b2MassData){ 0 };
                        shapeIndex += 1;
                        continue;
                }

                b2MassData massData = b2ComputeShapeMass( s );
                body->mass += massData.mass;
                localCenter = b2MulAdd( localCenter, massData.mass, massData.center );

                masses[shapeIndex] = massData;
                shapeIndex += 1;
        }

        // Compute center of mass.
        if ( body->mass > 0.0f )
        {
                bodySim->invMass = 1.0f / body->mass;
                localCenter = b2MulSV( bodySim->invMass, localCenter );
```

第二遍把缓存下来的各块 (m_i, c_i, I_{C_i}) 搬到刚算出来的 `localCenter`：

```
        // Second loop to accumulate the rotational inertia about the center of mass
        for ( shapeIndex = 0; shapeIndex < shapeCount; ++shapeIndex )
        {
                b2MassData massData = masses[shapeIndex];
                if ( massData.mass == 0.0f )
                {
                        continue;
                }

                // Shift to center of mass. This is safe because it can only increase.
                b2Vec2 offset = b2Sub( localCenter, massData.center );
                float inertia = massData.rotationalInertia + massData.mass * b2Dot( offset,
offset );
                body->inertia += inertia;
        }
```

`offset` 是 $C - c_i$，点积就是 $\|c_i - C\|^2$。注释里的 “can only increase” 就是平行轴第三项非负。

静态和运动学刚体在此函数前半段直接把世界中心设为原点变换，不走质量和质心积分。 定位：刚体质量聚合， `b2UpdateBodyMassData`，524–613行。 若两个形状重叠，本质量聚合会将重叠材料重复计入；它不是几何并集的质量积分器。 两个刚体用关节连着，也不走这一节：那是两个质心、两套惯量。

11. 逆质量、逆惯量与质心改变后的速度

力和冲量求解频繁需要除以质量，因此预存：

$$m^{-1} = \frac{1}{m}, \qquad I^{-1} = \frac{1}{I}.$$

单位分别为 kg^{-1} 和 $(\mathrm{kg}\cdot\mathrm{m}^2)^{-1}$。 静态、运动学对象的求解响应用零逆质量/零逆惯量表示，不是在内存中存真正的无穷大质量。 运动学对象仍可由给定速度移动，不能只凭逆质量零就认定它必然静止。 固定旋转时本版会把 `invInertia` 设零，同时在此质量更新路径把公开记录的惯量也设为零。 定位：刚体质量聚合，541–563行、594–598行、620–628行。

修改形状后质心从 C_{old} 移到 C_{new}，同一刚体速度场中的新质心速度应为：

$$v_{new} = v_{old} + \omega \times (C_{new} - C_{old}).$$

这不是凭空产生冲量，而是改变“被记录速度的物质点”。 本版在630–641行更新世界质心，并使用该公式修正线速度。 例：质心向右移动1米，$\omega = 2$，则线速度记录增加 $(0, 2)\ \mathrm{m/s}$。

12. 先实现什么，为什么后面再优化

1. 先明确每个形状返回的惯量轴，建议统一返回形状自身质心惯量。
2. 先实现圆、矩形和手算单元测试，不急于支持任意多边形。
3. 加入复合形状时用两遍算法求整体质心和惯量。
4. 多边形先支持合法逆时针凸形，严格拒绝退化和非法输入。
5. 对照三角形积分加入多边形公式，再使用靠近顶点的参考原点减少抵消。
6. 圆角近似必须记录其模型和误差，不要为了“看上去更高级”假装精确。
7. 只有力与冲量求解成为高频路径后，才会真正感受到预存逆量的价值。

13. 手算题与答案

题一：圆盘和圆环

相同质量2千克、半径0.5米，实心圆盘与薄圆环惯量各多少？ 答案：圆盘0.25，圆环0.5，单位均为千克·平方米。 圆环惯量更大，因为所有质量都在最外侧。

题二：两个偏心圆

两个圆各质量2，半径0.5，局部圆心分别为 $(-1, 0)$ 和 $(1, 0)$。 答案：总质量4，质心原点，每圆自身惯量0.25，总惯量 $2(0.25 + 2 \times 1^2) = 4.5$。 这就是刚体第一遍质量加权：$C = (2\cdot(-1) + 2\cdot 1)/4 = 0$。

题二补充：多边形扇形质心

零圆角顶点 $(0,0),(6,0),(6,3),(0,3)$，按 `b2ComputePolygonMass` 的循环求质心。 答案：两块面积都是 9，相对质心分别为 $(4,1)$ 与 $(2,2)$，加权得 $(3,1.5)$，即矩形中心。

题二再练：梯形质心

零圆角顶点 $(0,0),(6,0),(4,3),(0,3)$，$\rho=2$。按第 2.5 节两轮循环求 c 和 m。 答案：面积 15，$c=(38/15,7/5)$，$m=30$。下底更长，所以 $c_y=1.4$ 低于高度中点。

题三：矩形尺寸缩放

相同密度下把矩形所有长度扩大2倍，质量与惯量各扩大多少倍？ 答案：面积、质量扩大4倍；距离平方再扩大4倍，惯量扩大16倍。 若改成保持质量不变，则惯量只扩大4倍；必须先说清哪个量固定。

题四：平行轴必须过质心

第 1 节两点：1 kg 在 $x=0$，3 kg 在 $x=2$。已算得绕原点 $I_O=12$。绕 $x=2$ 的惯量是多少？能否写成 $12+4\cdot 2^2$？

答案：$I_{x=2}=4$。正确路径是 $I_C+m(0.5)^2=3+1=4$。$12+16=28$ 非法，因为原点不是质心。推导见 02a 第 6 节。

题五：两块合并的质心与惯量

A：$m=2$，$c=(0,0)$，$I_C=0.25$。B：$m=4$，$c=(3,1)$，$I_C=0.50$。求总质心和绕总质心的惯量。

答案：$m=6$，$C=(2,2/3)$，$I_C=169/12\approx 14.0833$。推导见 02a 第 7 节 和第 10.1 节。不能写成 $0.25+0.50=0.75$。

验收清单

- 平移形状局部坐标，返回的质心跟着平移，但绕其自身质心的惯量不变。
- 旋转形状不应改变质量和 z 轴质心惯量。
- 均匀矩形用扇形三角形加权，必须得到几何中心，而不是某个顶点。
- 有圆角时不要要求 `polygon.centroid` 与 `massData.center` 逐位相等。
- 零密度形状不贡献质量，但不能据此假定它不参与碰撞。
- 多边形顶点绕序、有效面积和非负惯量有断言或错误处理。
- 胶囊 $L=0$ 应退化到圆公式。
- 复合刚体离质心越远的附加质量，应增加惯量，而不是减小。

下一章把质量与惯量代入牛顿定律、冲量定理和数值积分。 返回总目录

03 • 力、力矩、冲量与积分

返回总目录

第01章提供向量运算，02a 说明转动惯量的定义，第02章提供质量和惯量的数值。 本章把它们连成时间推进公式，并明确区分：连续物理方程、教学离散算法、当前 Box2D 的实际实现。 这三者有关联，但不应写成完全相同的东西。

1. 先给状态变量一个明确含义

量	含义	单位	本章符号约定
x	世界质心位置	m	向右、向上为正
v	质心线速度	m/s	分量可正可负
a	质心加速度	m/s²	决定速度变化
m	质量	kg	动态物体应为正
F	外力合力	N	$1\,N = 1\,kg \cdot m/s^2$
θ	姿态角	rad	逆时针为正
ω	角速度	rad/s	逆时针为正
τ	关于质心的合力矩	N·m	逆时针为正
I	关于质心的转动惯量	kg·m²	非负
h	一个积分子步的时长	s	应为正且有限

速度不是位置，加速度也不是速度。 因此 $x+ = a$ 在量纲上就错了；$v+ = ha$、$x+ = hv$ 才能匹配单位。 代码检查不能替代量纲检查，两者应同时存在。

2. 牛顿第二定律如何变成加减乘

平动定律：

$$F = ma, \qquad a = \frac{F}{m}.$$

合力为零时是加速度为零，不是速度为零。 匀速直线运动无需持续“维持速度”的力；现实里必须持续推车，是因为存在阻力。 转动版本：

$$\tau = I\alpha, \qquad \alpha = \frac{\tau}{I}, \qquad \alpha = \frac{d\omega}{dt}.$$

它们在平动与转动上结构相同，但质量和转动惯量的单位不同，不能互相代替。

作用点为 P，质心为 C，力臂 $r = P - C$：

$$\tau = r \times F = r_x F_y - r_y F_x.$$

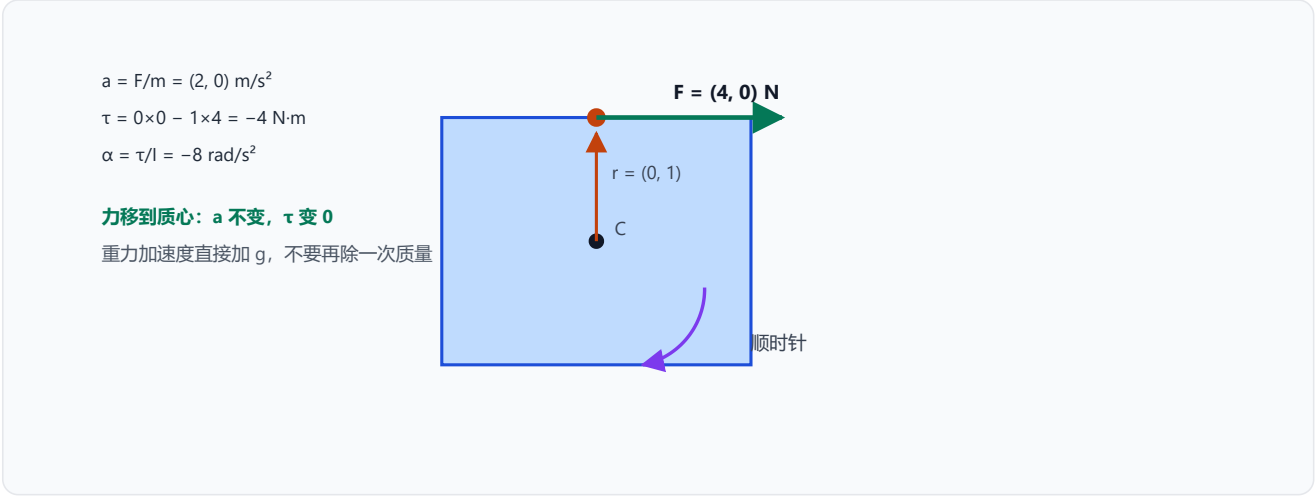
取 $r = (0, 1)\,\mathrm{m}$、$F = (4, 0)\,\mathrm{N}$。 $\tau = 0 \times 0 - 1 \times 4 = -4\,\mathrm{N \cdot m}$，即从上边缘向右推会顺时针转。 若 $m = 2\,\mathrm{kg}$、$I = 0.5\,\mathrm{kg \cdot m^2}$：

$$a = (2, 0)\,\mathrm{m/s^2}, \qquad \alpha = -8\,\mathrm{rad/s^2}.$$

相同的力移到质心，平动加速度不变，但力矩变零。

图 3-1　同一份力：平动看合力，转动看力臂

r=(0,1) m，F=(4,0) N，m=2 kg，I=0.5 kg·m²。从上边缘向右推，会顺时针转。



看图手算： $a = F/m = (2, 0)$，$\tau = -4$，$\alpha = -8$。重力不要再除一次质量，否则轻的会掉得更快。

源码 `b2Body_ApplyForce` 累加力并同时累加力矩，见 刚体实现，935–955行。 非动态或禁用对象会提前返回；对睡眠对象是否起作用还取决于唤醒和 awake 状态。 不要把“API 调用成功”理解成任何对象一定响应。

3. 重力为什么不再乘一次逆质量

近地匀强重力为 $F_g = mg$。 代入牛顿定律：

$$a_g = \frac{F_g}{m} = \frac{mg}{m} = g.$$

质量约掉了；自由落体中的加速度与质量无关，忽略空气阻力时轻重物体同样加速。 实现中直接加 hg，不是 $h\,m^{-1}g$。 后者会错误地让轻物体掉得更快。 `gravityScale` 是无量纲缩放系数，可以改变这个刚体的重力响应：

$$a = m^{-1}F + \mathrm{gravityScale} \cdot g.$$

这里的 F 不应再包含同一份已由 world gravity 添加的重力，否则会重复施加。 当前积分器在 `invMass == 0` 时将该重力缩放置零，特别用于运动学对象。 定位：求解器，`b2IntegrateVelocitiesTask`，97–102行。

4. 冲量：短时间内的力作用总量

线动量定义为 $p = mv$。 由 $F = dp/dt$，在时间区间积分：

$$J = \int_{t_0}^{t_1} F(t)\,dt = \Delta p = m\Delta v.$$

冲量 J 的单位为 $\mathrm{N}\cdot\mathrm{s} = \mathrm{kg}\cdot\mathrm{m/s}$。 若力在时长 h 内恒定，$J = hF$。 因此：

$$\Delta v = m^{-1}J.$$

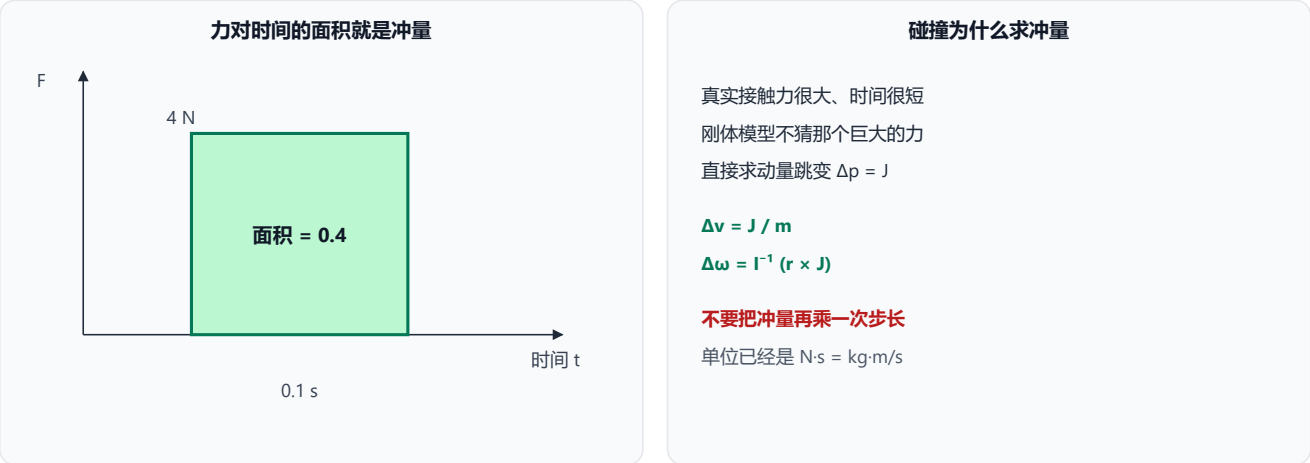
点冲量还产生角冲量：

$$\Delta L = r \times J, \qquad \Delta\omega = I^{-1}(r \times J).$$

ΔL 单位为 $\mathrm{kg}\cdot\mathrm{m}^2/\mathrm{s}$。 不要把冲量再乘一次 h，否则单位和效果都会错。 例如 $F = (4, 0)\ N$ 持续0.1秒，等效冲量为 $J = (0.4, 0)\ N\cdot s$。 对前面的 $m = 2$、$I = 0.5$、$r = (0, 1)$： $\Delta v = (0.2, 0)$，$\Delta\omega = -0.8$。 同样大小的力若只持续0.05秒，效果减半；已经给定的冲量则不应再随子步数量缩放。

图 3-2　冲量 J = ∫F dt：短时间里的动量变化

恒力 4 N 作用 0.1 s，J=0.4 N·s。对 m=2 kg：Δv=0.2 m/s。已经是冲量就不要再乘 h。



看图： 碰撞力很大、时间很短，刚体模型不猜那个力，直接求动量跳变。J 已经是 N·s，再乘步长就会错 100 倍量级。

4.1 力缓冲与冲量 API 不同

`b2Body_ApplyForce` 修改待积分的 `force` 和 `torque`。 `b2Body_ApplyLinearImpulse` 直接修改当前线速度和角速度，并在该路径调用速度限制。

```
                    state->linearVelocity = b2MulAdd( state->linearVelocity, bodySim->invMass,
impulse );
                    state->angularVelocity += bodySim->invInertia * b2Cross( b2SubPos( point,
bodySim->center ), impulse );

                    b2LimitVelocity( state, world->maxLinearSpeed );
```

定位：刚体实现，`b2Body_ApplyLinearImpulse`，1015–1041行。 碰撞求解使用冲量，正是因为需要直接改变速度，而不用构造极大、极短的碰撞力。 这并不意味着冲量是另一套违背牛顿定律的物理；它是牛顿定律的时间积分形式。

5. 连续积分与两种欧拉法

连续方程为：

$$\frac{dv}{dt} = a, \qquad \frac{dx}{dt} = v.$$

真实一步位置需要积分整个区间内的速度：

$$x(t+h) = x(t) + \int_t^{t+h} v(s)\, ds.$$

计算机用某个代表速度乘 h 来近似积分。 选择旧速度得到显式欧拉；选择先更新的速度得到半隐式欧拉。

5.1 显式欧拉必须使用同一份旧状态

$$v_{n+1} = v_n + h\, a(x_n, v_n), \qquad x_{n+1} = x_n + h v_n.$$

这里 n 是离散步编号，不是时间单位。 若代码先覆盖 x，再用覆盖后的 x 求弹簧力，那已经不是这里定义的显式欧拉。 最稳妥的教学实现是保存旧位置、旧速度，然后算两个新值。

5.2 半隐式欧拉先更新速度

$$v_{n+1} = v_n + h\, a(x_n, v_n), \qquad x_{n+1} = x_n + h v_{n+1}.$$

加速度仍可取旧状态，因此“半隐式”不等于求解一般非线性隐式方程。 在无耗散的合适哈密顿系统中，这种方法有辛结构；加上阻尼、冲量约束和钳制后，不宜笼统宣称整个游戏世界严格辛或严格守恒。

以下为教学伪代码，不是可编译代码，也不是 Box2D API 示例：

```
oldPosition = body.center
acceleration = body.force / body.mass + gravity
body.velocity = body.velocity + h * acceleration
body.center = oldPosition + h * body.velocity
```

图 3-3　两种欧拉：用哪一份速度去走位置

自由落体里两者速度相同，位置分别偏高和偏低。弹簧问题才能看出稳定性差异。



看图： Box2D 位置积分用的是约束修正后的新速度，属于半隐式这一边。它更稳，但不是“怎样都不会炸”。

6. 自由落体：完整计算60步，而不是只看第一步

设 $y_0 = 10\,m$、$v_0 = 0$、$g = -10\,m/s^2$、$h = 1/60\,s$。 不启用碰撞、阻尼、速度钳制和其他力。 解析解：

$$v(t) = v_0 + gt, \qquad y(t) = y_0 + v_0 t + \frac{1}{2} g t^2.$$

1秒后 $v = -10\,m/s$、$y = 5\,m$。 两种欧拉法的离散速度，在实数算术下都为 $v_n = v_0 + ngh = -n/6$。实际 float 实现会有小量舍入误差，不应把“理论精确”误写成“每个平台位级精确”。

6.1 显式欧拉的求和

位置每步使用旧速度 v_k，因此：

$$y_N = y_0 + h \sum_{k=0}^{N-1} v_k = y_0 + Nhv_0 + gh^2 \sum_{k=0}^{N-1} k.$$

等差数列之和为 $N(N-1)/2$，于是：

$$y_N^{\text{explicit}} = y_0 + Nhv_0 + gh^2 \frac{N(N-1)}{2}.$$

代入 $N = 60$：

$$y_{60} = 10 - 10\frac{1}{3600}\frac{60 \times 59}{2} = 10 - \frac{59}{12} = \frac{61}{12} \approx 5.083333.$$

第一步确实没有下落，因为 $v_0 = 0$；但第二步已经使用负速度，之后仍会不断下落。 因此“显式欧拉60步后仍然在10米”是错误结论，不应继续传播。

6.2 半隐式欧拉的求和

每步使用新速度 v_{k+1}，所以：

$$y_N^{\text{semi}} = y_0 + h\sum_{k=1}^{N} v_k = y_0 + Nhv_0 + gh^2\frac{N(N+1)}{2}.$$

代入60步：

$$y_{60} = 10 - 10\frac{1}{3600}\frac{60\times 61}{2} = 10 - \frac{61}{12} = \frac{59}{12} \approx 4.916667.$$

6.3 用中间状态排查代码

步号 n	理论速度 v_n	显式位置	半隐式位置	解析位置
0	0	10	10	10
1	-0.166667	10	9.997222	9.998611
2	-0.333333	9.997222	9.991667	9.994444
30	-5	8.791667	8.708333	8.750000
60	-10	5.083333	4.916667	5.000000

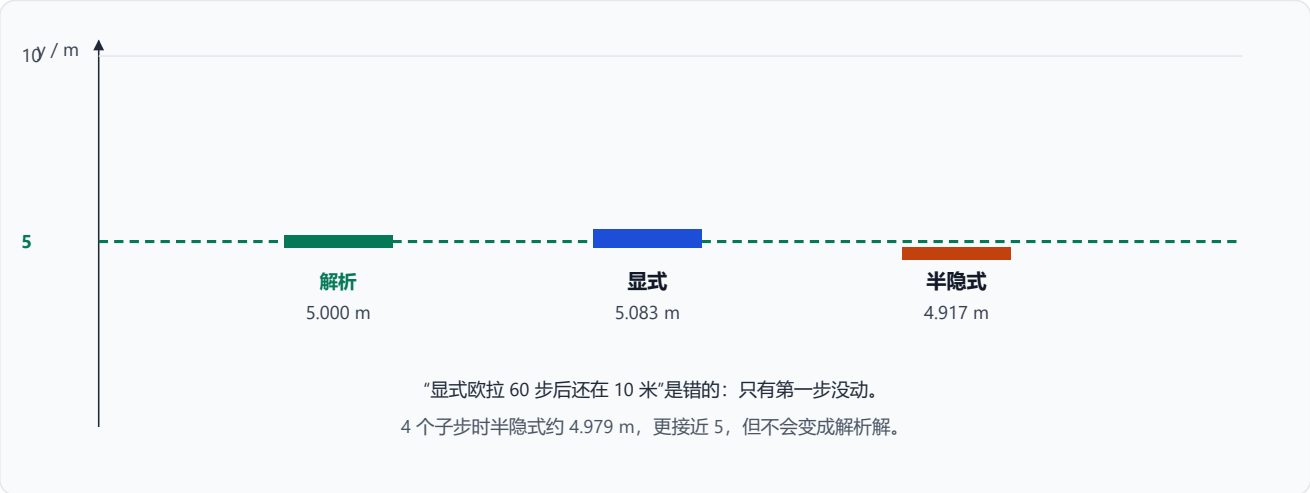
恒定加速度下，两者位置误差恰好在解析值两侧：

$$y_N^{\text{explicit}} - y(t) = -\frac{1}{2}gth, \qquad y_N^{\text{semi}} - y(t) = \frac{1}{2}gth, \quad t = Nh.$$

由于 g 为负，显式偏高、半隐式偏低。 减半 h 会把这个问题中的位置误差减半，但不会让半隐式成为解析解。 若每个1/60秒大步分成4个子步，一秒共240步，半隐式高度为 $4.979167\,m$。

图 3-4　从 10 m 自由落下 1 秒：三种高度不要记混

g=−10 m/s²，h=1/60 s，共 60 步。速度都到 −10 m/s；位置误差恰好在解析值两侧。



看图： 三种高度不要记混。第一步显式欧拉确实没动，但 60 步后绝不是还停在 10 m。

7. 半隐式为什么常更适合刚体模拟，但不是无条件稳定

自由落体不能充分展现方法的稳定性差异，因为加速度不依赖位置。 更有代表性的例子是无阻尼弹簧：

$$a = -\omega_0^2 x, \qquad \omega_0 = \sqrt{k/m}.$$

k 为弹簧刚度，单位 N/m，ω_0 为固有角频率，单位 s^{-1}。 负号表示偏离平衡点越远，力越指向平衡点。

显式欧拉对应矩阵：

$$\begin{pmatrix} x_{n+1} \\ v_{n+1} \end{pmatrix} = \begin{pmatrix} 1 & h \\ -h\omega_0^2 & 1 \end{pmatrix} \begin{pmatrix} x_n \\ v_n \end{pmatrix}.$$

特征值为 $1 \pm ih\omega_0$，模长 $\sqrt{1+h^2\omega_0^2} > 1$。 理想无阻尼弹簧的振幅会持续增长；步长很小也只是增长慢些。 半隐式矩阵是：

$$\begin{pmatrix} 1-h^2\omega_0^2 & h \\ -h\omega_0^2 & 1 \end{pmatrix}.$$

其行列式为1，迹为 $2 - h^2\omega_0^2$。 在 $0 < h\omega_0 < 2$ 时，特征值位于单位圆，可保持有界振荡。 它通常表现为能量有界摆动，不是每一步都严格保能量。 边界 $h\omega_0 = 2$ 不能一般地当作安全稳定点；超过2通常会发散。 所以“用了半隐式就不会炸”仍然错误：再好的离散方法也有模型和步长适用范围。

8. 阻尼：先解物理模型，再看近似

采用线性阻尼模型：

$$\frac{dv}{dt} = -cv.$$

c 单位 s^{-1}，不是摩擦系数，要求 $c \geq 0$。 物理阻力写成 $F_d = -bv$ 时，$c = b/m$；引擎配置直接使用 c，并不保证不同质量物体共享同一个 b。 分离变量：$dv/v = -c\,dt$，积分得：

$$v(t+h) = e^{-ch} v(t).$$

令无量纲量 $z = ch$。 显式离散给出 $v_{n+1} = (1-z)v_n$。 它的稳定性必须分区说清：

z范围	系数	结果
$0 < z < 1$	0到1	同向衰减
$z = 1$	0	一步归零
$1 < z < 2$	-1到0	翻向但幅值衰减
$z = 2$	-1	反复翻向，不衰减
$z > 2$	小于-1	翻向且发散

因此不能把所有 $ch > 1$ 都说成“速度越来越大”。 其中1到2之间是非物理翻向的衰减，而非幅值增长。

8.1 当前实现的有理近似

后向欧拉对纯阻尼给出：

$$\frac{v_{n+1}-v_n}{h} = -cv_{n+1}, \qquad v_{n+1} = \frac{v_n}{1+ch}.$$

它也可看作指数函数的低阶 Padé 近似：$e^{-z} \approx 1/(1+z)$。 对于 $c \geq 0, h \geq 0$，系数在0到1之间，不翻向且不放大。 稳定不等于精确：例如 $v = 10$、$c = 2$、$h = 0.1$，近似为 $10/1.2 = 8.333333$，解析值约 8.187308。 步长增大时，该近似通常会低估纯阻尼衰减程度。 负 c 会变成反阻尼，且 $1 + ch = 0$ 时分母失效；不要把稳定结论用于非法参数。

9. 本版“阻尼加外力”的顺序必须忠于代码

本版实际速度更新为：

$$v_{n+1} = \frac{v_n}{1+hc_v} + h(m^{-1}F + g_s g), \qquad \omega_{n+1} = \frac{\omega_n}{1+hc_\omega} + hI^{-1}\tau.$$

c_v, c_ω 分别是线性与角阻尼，g_s 为有效重力缩放。 只有旧速度乘阻尼系数，本步外力和重力增量在阻尼外加上。 它不是 $(v_n + ha)/(1 + hc)$，也不是整个受力阻尼方程的完整后向欧拉。

```
                    float linearDamping = 1.0f / ( 1.0f + h * sim->linearDamping );
                    float angularDamping = 1.0f / ( 1.0f + h * sim->angularDamping );

                    // Gravity scale will be zero for kinematic bodies
                    float gravityScale = sim->invMass > 0.0f ? sim->gravityScale : 0.0f;

                    // lvd = h * im * f + h * g
                    b2Vec2 linearVelocityDelta = b2Add( b2MulSV( h * sim->invMass, sim->force ),
b2MulSV( h * gravityScale, gravity ) );
                    float angularVelocityDelta = h * sim->invInertia * sim->torque;

                    v = b2MulAdd( linearVelocityDelta, linearDamping, v );
                    w = angularVelocityDelta + angularDamping * w;
```

定位：求解器，`b2IntegrateVelocitiesTask`，66–112行。 数值对比：$v_n = 10$、$a = 3$、$c = 2$、$h = 0.1$。 本版得到 $10/1.2 + 0.3 = 8.633333$。 若把整个分子除以1.2，会得到 $10.3/1.2 = 8.583333$，这不是相同结果。

若长期保持恒定 a 和 c，当前递推的固定点满足：

$$v_* = \frac{v_*}{1+hc} + ha \quad \Longrightarrow \quad v_* = \frac{a}{c}(1+hc).$$

连续模型的终端速度是 a/c，因此此分步近似的终端速度也有步长误差。 这不是要否定实现，而是说明为什么调阻尼时仍要保持固定步长，并通过减小子步改善一致性。

10. 力必须持续一个大步，不能每个子步都清零

设一个大步 Δt 分成 N 个子步，$h = \Delta t/N$。 同一外力每个子步贡献 hF/m，N次合计 $\Delta t F/m$。 如果第一个子步后立即清空 force，剩下 N-1 步没有这份力，效果会缩小到原来的 $1/N$。 如果每个子步都重复调用并累加同一份外力而不重置，又会越积越大。 因此应明确所有权：游戏逻辑在大步前提供外力，子步读取同一份缓冲，大步收尾才清零。 本版在 `b2FinalizeBodiesTask` 的631–633行清空力和力矩。 定位：求解器，`b2FinalizeBodiesTask`，617–633行。

冲量则是一次性速度跳变，不应为了 N 个子步重复施加 N 次。 若希望模拟持续推进器，使用每个大步持续提交的力通常比每帧随意提交固定冲量更符合时间尺度。

11. 旋转积分：归一化只修长度，不会修正所有角度误差

将姿态保存为 $q = (c, s) = (\cos\theta, \sin\theta)$。 由链式法则：

$$\dot{c} = -\omega s, \qquad \dot{s} = \omega c.$$

取 $d = h\omega$，显式一步得到：

$$\tilde{c} = c - ds, \qquad \tilde{s} = s + dc.$$

若原来 $c^2 + s^2 = 1$，展开平方得：

$$\tilde{c}^2 + \tilde{s}^2 = 1 + d^2.$$

不归一化会让旋转二元组越变越长，最终把缩放混入旋转。 因此实现除以模长：

$$q_{new} = \frac{(c - ds, s + dc)}{\sqrt{(c - ds)^2 + (s + dc)^2}}.$$

定位：数学头文件，`b2IntegrateRotation`，388–399行。 这避免了每个子步从角度重算正弦余弦，但仍包含平方根与归一化成本。

11.1 实际转角是 arctan，而不是精确 d

从单位姿态 $(1, 0)$ 出发，结果为 $(1, d)/\sqrt{1 + d^2}$。 其角度为 $\arctan d$，而目标角度为 d。 当 d 很小时：

$$\arctan d = d - \frac{d^3}{3} + O(d^5).$$

例如 $d = 0.1$ 时实际增量约0.09966865弧度，略小于0.1。 $d = 1$ 时实际只有 $\pi/4 \approx 0.785398$，误差已明显。 所以单位长度稳定并不意味着任意角速度、任意步长都精确。 输入本来是零旋转二元组时，归一化也不会凭空恢复正确姿态；必须维持合法初始状态。

11.2 本版角速度上限使用大步时长

`b2IntegratePositionsTask` 在应用运动锁后限制线速度和角速度，再积累位置与旋转增量。 角速度上限为：

$$|\omega| \le \frac{B2_MAX_ROTATION}{\Delta t}, \qquad B2_MAX_ROTATION = 0.25\pi.$$

这里分母是整个大步 Δt，不是子步 h。 $\Delta t = 1/60$ 时，上限约47.12389弧度每秒，除非允许快速旋转。此限制服务于数值鲁棒性及连续碰撞假设，不应把它解释为自然界的旋转极限。 定位：常量，第50行；求解器，`b2IntegratePositionsTask`，121–158行。

12. 当前真实阶段顺序

一次大步先建立接触并准备约束，然后每个子步执行：

1. 积分外力、重力、力矩和阻尼，得到预测速度。
2. 热启动，把已有冲量估计应用到速度。
3. Solve，带偏置地求解约束并改变速度。
4. 使用修正后的速度积分位置和旋转增量。
5. Relax，关闭重叠偏置的相应影响，并在当前接触实现中处理恢复、摩擦和滚动阻力。

全部子步结束后，再执行一次接触冲量存储阶段，然后进行刚体收尾等工作；不是每个子步都执行 Store。 定位：求解器，`b2SolverTask`，1082–1169行。 `b2SolveContacts_Overflow` 的恢复分支位于 接触求解器，301–305行，仅当 `useBias == false` 执行。 不要把 `b2SolverTask` 前面的历史阶段注释当成真正调用序列；其中仍列有并未在当前循环单独执行的 Restitution 阶段名称。

13. 手算练习与验收

题一：偏心冲量

$m = 2$、$I = 0.5$、$r = (0, 1)$、$J = (2, 0)$，求速度变化。 答案：$\Delta v = (1, 0)\, m/s$，$\Delta \omega = (0 \times 0 - 1 \times 2)/0.5 = -4\, rad/s$。 不乘 h，因为输入已经是冲量。

题二：阻尼系数的稳定性

$h = 0.1$、$c = 15$，纯阻尼显式法和本版有理近似各乘多少？ 答案：显式乘 $1 - 1.5 = -0.5$，翻向但幅值衰减；有理近似乘 $1/2.5 = 0.4$，同向衰减。 把显式结果说成“幅值发散”是不正确的。

题三：固定步长自由落体

$y_0 = 10$、$g = -10$，用半隐式 $h = 1/120$ 跑1秒，位置多少？ 答案：$5 - 5/120 \approx 4.958333\, m$，速度理论值为-10。 与1/60秒步长相比，位置误差减半。

验收清单

- 无力、无阻尼、无碰撞时，速度保持不变。
- 重力自由落体不应随质量改变加速度。
- 给定一次冲量的效果不应随子步数成倍改变。

- 一份恒力跨 N 个子步的总效果应与 $\Delta t F/m$ 一致，忽略其他效应。
- 纯阻尼不应因正常正参数反向或放大速度。
- 姿态归一化后长度接近1，仍要检查大转角的精度。
- 自由落体测试明确区分解析5、显式5.083333、半隐式4.916667。

下一章将这些公式组织成一个可逐步实现、可观测、可验证的单线程物理世界。 返回总目录

04 • 从零实现最小物理世界

返回总目录

本章不是把 Box2D 换个名字重新抄一遍，而是建立一个能不断增加功能、每一步都能验证的教学引擎。 先做正确的单线程版本；只有测量显示性能不足，才添加树、睡眠、数据拆分、并行和 SIMD。 力和积分的示意图见第 03 章，圆与 AABB 见第 05 章，多边形 SAT 见第 06 章，距离查询见第 07 章，冲量公式见第 09 章。本章只把它们按可验证的顺序串起来。 本章所有普通代码围栏均为教学伪代码，不是可编译程序，也不是当前 Box2D API 调用示例。 真实源码引用使用带行号和文件路径的专用格式。

1. 第一份设计只回答四个问题

1. 当前世界保存哪些状态?
2. 一个时间步读取什么输入?
3. 每个阶段允许修改哪些状态?
4. 怎样判断这一步的结果对不对?

如果无法回答第3点，就很容易把力清零、碰撞冲量和位置更新放错时机。 如果无法回答第4点，画面“看着差不多”会掩盖符号、尺度和能量错误。

2. 最小状态表：先不要复制全部内部结构

字段	含义	单位	生命周期
`type`	静态、动态、运动学	枚举	通常长期保持
`center`	世界质心位置	m	每步更新
`rotation`	单位旋转二元组	无量纲	每步更新
`linearVelocity`	质心线速度	m/s	外力和冲量改变
`angularVelocity`	角速度	rad/s	力矩和冲量改变
`force`	本大步待积分合力	N	大步收尾清零
`torque`	本大步待积分合力矩	N·m	大步收尾清零
`invMass`	逆质量	1/kg	形状或质量改变时更新
`invInertia`	逆质心惯量	1/(kg·m²)	质量或旋转锁改变时更新
`localCenter`	质心相对刚体原点	m	质量分布改变时更新
`shape`	圆半径或局部顶点	m	通常长期保持

最初只有圆时，可以将圆心同时作为刚体原点和质心，令 `localCenter=0`。 这只是 M1/M2 的简化，不是一般刚体永远满足的定律。 静态物体不移动、不接受力和冲量响应。 运动学物体按给定速度移动，但

不因力和碰撞冲量改变速度。 动态物体由动力学推进，质量应为正。 因此后面加入运动学物体时，不能只用 `invMass==0` 跳过所有位置积分。

对应源码的结构不是一份大结构，而是分开组织的。 `b2BodyState` 保留高频速度和增量，见 刚体内部结构，153–169行。 `b2BodySim` 保留变换、质心、力、逆量等，见175–209行。 教学阶段先合并存放，等访问热点明确后再拆分。

3. 固定步长：物理时间不等于渲染帧时长

先取物理步长 $h = 1/60\,s$。 显示器可以每秒144帧，也可能某帧卡顿；物理仍以固定 h 推进。 累积器的含义是：现实经过的时间暂存起来，够一个 h 才推进一次物理。

```
accumulator += elapsedRealTime
steps = 0
while accumulator >= h and steps < stepBudget:
    StepPhysics(h)
    accumulator -= h
    steps += 1
alpha = clamp(accumulator / h, 0, 1)
renderFromKnownStates(alpha)
```

`stepBudget` 防止一次卡顿后无限补步拖垮后续帧。 这份教学伪代码选择保留未处理的物理时间积压，只将渲染插值系数钳在 0～1：例如积压为 $2h$，原始比例为 2，钳制后为 1，显示最新已知状态而不对未知未来外推。它不修改累积器，因此没有暗中丢弃待模拟时间。 到达预算后也可另选丢弃部分时间，但会牺牲实时一致性；保留积压则会使模拟落后。 不能一边丢弃时间，一边宣称完整模拟了全部真实经过的秒数。 用于渲染的插值不应写回物理状态。 初期不需要插值，直接绘制最新物理状态即可。 验收时按固定步数测试，而不是根据主观经过了“差不多一秒”。

4. M1：只有质心、重力和积分

4.1 输入和输出

输入是上一步 (x_n, v_n)、质量 m、外力 F、重力 g、步长 h。 输出是新状态 (x_{n+1}, v_{n+1})。 先不加碰撞、阻尼、睡眠和速度限制，以便与第03章的手算结果逐项比较。

```
for each dynamic body:
    acceleration = invMass * force + gravity
    linearVelocity += h * acceleration
for each dynamic body:
    center += h * linearVelocity
for each body:
    force = zero
    torque = zero
```

这是 N=1 的版本，力在完整大步结束时清零。 后面加子步时要把清零移到子步循环外，不能保留在每个子步中。

4.2 验收数据必须精确到具体方法

$y_0 = 10$、$v_0 = 0$、$g = -10$、$h = 1/60$，走60步。 半隐式欧拉应约为 $y = 4.916667$、$v_y = -10$。 显式欧拉应约为 $y = 5.083333$，解析解为5。 这里不允许启用地板，否则测试对象已不再是纯自由落体。 若 y 一直不变，检查是否漏掉位置写回或时间步为零；不要把它直接归咎于显式欧拉。

4.3 临时地板只是一块脚手架

可以暂时令圆心高度不得小于半径 R，并在下穿时把竖直速度置零。 必须写明这是占位逻辑，不是一般碰撞求解。 它不能守恒两物体碰撞动量，不能处理斜面、运动地板、恢复系数和旋转。 M1验收通过后立即用 M2 替换，不要围绕它堆更多特例。 对应：求解器，`b2IntegrateVelocitiesTask` 66–112行；`b2IntegratePositionsTask` 114–162行。

5. M2：先用最容易看懂的离散圆碰撞

教学第一版可采用：积分速度 → 积分位置 → 检测当前重叠 → 施加碰撞冲量 → 有界位置修正。 这是简单的离散后处理流程，不是当前 Box2D 的完整阶段顺序。 它只检查离散时刻，高速圆可能跨过彼此而未被检测；后面需更小步长、推测接触或 CCD。 将这种限制写清，比一开始隐含承诺“不会穿透”更重要。

5.1 暴力配对足够开始

```
for i from 0 to bodyCount - 1:
    for j from i + 1 to bodyCount - 1:
        skip incompatible pair types
        generateCircleContact(i, j)
```

只用 $j > i$，避免重复求解 A-B 和 B-A。 全部不同物体对数量为：

$$N_{pairs} = \frac{n(n-1)}{2}.$$

10个物体只有45对，100个有4950对，2000个有1999000对。 这直接说明为什么初期暴力法足够，而规模大后需要宽阶段。 宽阶段只筛候选，不能把 AABB 重叠当成精确几何接触。

5.2 圆与圆的几何输出

设圆心 C_A, C_B，半径 R_A, R_B。

$$d = C_B - C_A, \quad D = \|d\|, \quad n = d/D, \quad s = D - R_A - R_B.$$

n 从 A 指向 B，无量纲且长度为1。 s 单位米：正数表示间隙，零表示刚好接触，负数表示重叠。 表面上的两点为：

$$Q_A = C_A + R_A n, \qquad Q_B = C_B - R_B n.$$

教学接触点可取 $P = (Q_A + Q_B)/2$。 圆心距离为1.8米、两半径都为1米，则 $s = -0.2\,m$。 n方向不能因求解阶段切换而随意反转，否则所有冲量正负号都会失效。

若 D 为零，法线未定义。 初版可拒绝完全同心的初始圆，或使用固定顺序下的备用法线并明确这是教学策略。 当前 `b2CollideCircles` 调用的归一化函数在零差值时返回零向量，不应声称源码自动选好了任意有效法线。 定位：流形生成，`b2CollideCircles`，36–64行。 本版还可在正间隙小于推测距离时生成流形，并非必须重叠才存在接触数据。

6. 一次法向冲量，从目标速度推出来

在暂不考虑圆的切向摩擦和旋转响应时：

$$v_n = (v_B - v_A) \cdot n.$$

因为 n 从 A 指向 B，所以 $v_n < 0$ 表示接近，$v_n > 0$ 表示分离。 恢复系数 $e \in [0, 1]$ 定义目标：

$$v_n^+ = -e\, v_n^-.$$

上标减号和加号表示碰撞前、碰撞后，不是取绝对正负。 施加标量冲量 j，对 A 为 $-jn$，对 B 为 $+jn$：

$$v_A^+ = v_A^- - m_A^{-1} jn, \qquad v_B^+ = v_B^- + m_B^{-1} jn.$$

相减后沿 n 投影，利用 $n \cdot n = 1$：

$$v_n^+ = v_n^- + (m_A^{-1} + m_B^{-1}) j.$$

代入恢复目标：

$$j = -\frac{(1+e) v_n^-}{m_A^{-1} + m_B^{-1}}.$$

分子为米每秒，分母为每千克，j单位为千克·米每秒，即冲量。 初版只在接近时求这个碰撞冲量；分母为零时必须跳过或拒绝不可响应约束。 已经分离的重叠物体，可以仍做位置修正，但不应为了“有重叠”再给一个吸引冲量。

6.1 等质量的完整手算

两圆质量均为2千克，水平法线 $n=(1,0)$。 碰撞前 $v_A=3$、$v_B=-1$，相对法向速度 $v_n=-4$。 逆质量和 $K=0.5+0.5=1$。 当 $e=1$，$j=8\,N\cdot s$。 碰撞后 $v_A=3-0.5(8)=-1$，$v_B=-1+0.5(8)=3$，交换速度。 总动量碰前碰后均为4，动能均为10焦耳。 当 $e=0$，$j=4$，两者速度都变成1。 总动量仍为4，动能降为2焦耳，损失8焦耳。 因此恢复系数0不是“所有世界速度都清零”，而是法向相对速度消失。

6.2 静态地面例子

令 A 为静态地面，n向上；B质量2千克，速度向下3米每秒。 $K=0+0.5=0.5$，$e=0$，所以 $j=6\,N\cdot s$。 B速度增加 $0.5\times 6=3\,m/s$，变为0。 静态地面是外部固定支撑；只统计动态物体时，动量不必守恒，因为环境接收了反作用。

7. 位置重叠与速度接近是两个问题

冲量改速度，不会自动把已经重叠的位置分开。 教学初版可使用有界、按逆质量加权的位置投影。 令容许重叠为 $\sigma\ge 0$，修正比例为 $\beta\in(0,1]$：

$$p=\max(-s-\sigma,0),\qquad K=m_A^{-1}+m_B^{-1}.$$

这里 p 是要处理的穿透深度，单位米；σ 避免微小误差反复修正。 当 $K>0$：

$$\Delta C_A=-\frac{m_A^{-1}}{K}\beta p\,n,\qquad \Delta C_B=+\frac{m_B^{-1}}{K}\beta p\,n.$$

质量较小的对象移动较多，静态对象逆质量为零所以完全不动。 例：两物体质量都2，$s=-0.025\,m$、$\sigma=0.005\,m$、$\beta=0.2$。 p=0.02米，总分离增加0.004米，各移动0.002米。 这只是位置纠偏，并不等价于真实弹性碰撞；过度修正会导致跳变和能量问题。 到稳定约束阶段，应明确选择位置纠偏、速度偏置或更完整约束策略，不要同时无节制叠加。

8. M3：让圆变成能转动的盒子

矩形先用局部顶点 $(-a,-b),(a,-b),(a,b),(-a,b)$ 表示。 a、b为半宽半高，单位米。 按第01章公式，将局部点变到世界：$P=p+Rv_l$。 按第02章计算质量和质心惯量。 按第03章积分角速度和姿态。 碰撞点相对质心的力臂为 $r_A=P-C_A$、$r_B=P-C_B$。

接触点速度不再只是质心速度：

$$u_A=v_A+\omega_A\times r_A,\qquad u_B=v_B+\omega_B\times r_B.$$

相对法向速度为 $v_n=(u_B-u_A)\cdot n$。 冲量除了改线速度，还必须改角速度：

$$\Delta\omega_A=-I_A^{-1}(r_A\times jn),\qquad \Delta\omega_B=+I_B^{-1}(r_B\times jn).$$

因此有效逆质量变为：

$$K_n=m_A^{-1}+m_B^{-1}+I_A^{-1}(r_A\times n)^2+I_B^{-1}(r_B\times n)^2.$$

平方来自：冲量先经力臂产生角速度，再由角速度经同一力臂反馈到接触点法向速度。 每个转动项的单位都是每千克，与平动逆质量一致。 漏掉它就等于假定偏心冲量不会通过转动改变接触点速度。 定位：接触准备，`b2PrepareContacts_Overflow`，116–131行。

9. 区分几何触地、流形候选点与实际支撑

先做盒子对水平地面，可以计算每个顶点到地面有符号距离。 再推广到凸多边形 SAT、参考边、入射边和裁剪。 SAT用于寻找分离轴或参考面，完整流形生成还需要确定接触点，不能只输出“碰撞/不碰撞”。 有效流形通常包含1或2个点，但候选点数量不等于当前几何触地点数量，也不等于产生正法向冲量的点数。

例如半宽1、半高0.5的盒子逆时针倾斜20度。 两个底角相对质心的 y 坐标分别为：

$$y_L = -\sin 20^\circ - 0.5\cos 20^\circ \approx -0.811866,$$

$$y_R = +\sin 20^\circ - 0.5\cos 20^\circ \approx -0.127826.$$

当左下角刚好接触地面时，右下角仍高约0.68404米。 较高角尚未几何触地，但这不等于流形必须删除较高点。当前 `b2ClipPolygons` 的双点裁剪路径会保留裁剪得到的两个候选点，第589–618行没有逐点按0.02米推测距离筛掉较大的正间隙。

该候选点是否产生法向冲量，还要结合间距和接近速度，由推测约束处理。正间隙候选点不等于已经接地，更不能无条件给它着地支持冲量。 例如暂不考虑转动，取较高点间距 $s = 0.68404$ m、$h = 1/60$ s、接近速度 $v_n = -1$ m/s，则 $v_n + s/h = -1 + 41.0424 = 40.0424$ m/s，满足推测不穿透条件；从零累计冲量开始时，这个点不需要新增法向冲量。

单角首先接触后，重力与接触冲量通常让盒子转向平放。正确验收应同时检查候选几何、`separation`、实际冲量和转动方向，而不是要求20度斜盒子永远保持20度，也不是仅凭高差强制流形只留一点。 本版 `b2CollidePolygons` 包含单点顶点情形与裁剪情形。 定位：流形生成，702–1028行；例如832–847行为一个单点分支，1015–1019行为重叠裁剪路径，589–618行为双点构造。完整正分离候选算例见第06章。

10. M4：把一次冲量升级为顺序冲量

一个盒子可能同时接触两个物体，一个接触点更新速度会改变其他接触点看到的相对速度。 因此一次遍历通常不能同时满足所有约束。 顺序冲量采用迭代：每次处理一个约束，立刻修改刚体速度，让后面的约束看到最新结果。 它是迭代求解耦合系统，不是“每遍把完整碰撞冲量重复打一次”。

10.1 接触缓存最小状态

字段	含义
`bodyA/bodyB`	稳定配对关系
`featureId`	几何特征标识，用于匹配接触点

字段	含义
`normal`	从A到B的单位法线
`anchorA/anchorB`	相对质心的锚点
`separation`	有符号间距
`normalMass`	$1/K_n$
`normalImpulse`	当前累计法向冲量
`tangentImpulse`	当前累计切向冲量

接触点不能仅凭数组下标认定与上一帧相同。 物体移动或参考边切换后，点的顺序可能改变；错误复用缓存会给错误位置打冲量。 第一版可用稳定的边/顶点特征编号，匹配不确定时宁可清空缓存。

10.2 为什么钳累计量，而不是只钳本次增量

令当前累计冲量为 λ，根据残余法向速度算增量：

$$\Delta\lambda = -M_n(v_n + b), \qquad M_n = 1/K_n.$$

b 是速度偏置，单位米每秒。 零偏置时约束试图消除接近速度；负偏置可产生有限的分离速度以修重叠。 单边接触只能推开，不能拉住，故累计冲量需要满足 $\lambda \geq 0$：

$$\lambda_{new} = \max(\lambda_{old} + \Delta\lambda, 0), \qquad \Delta\lambda_{apply} = \lambda_{new} - \lambda_{old}.$$

实际只应用差值 $\Delta\lambda_{apply}$。 这个差值可以为负，表示撤回之前过大的推力，但累计量仍非负。 如果把每次增量也钳为非负，迭代就无法撤回错误，容易把物体越推越快。 定位：接触求解，`b2SolveContacts_Overflow`，308–328行。

10.3 简单偏置和推测约束的符号

对于重叠，教学偏置可取：

$$b = \frac{\beta}{h}\min(s + \sigma, 0).$$

它非正。目标 $v_n = -b \geq 0$，因此修正方向是分离而不是继续挤压。 对于仍有间隙 $s > 0$ 的推测接触，要求下一步不跨过表面：

$$s + hv_n \geq 0 \quad \Longrightarrow \quad v_n + \frac{s}{h} \geq 0.$$

所以正间隙偏置为 $b = s/h$，它允许一定接近速度，不代表把远处物体立即推开。 本版使用软约束系数和接触速度钳制处理重叠，不能把上述简单硬偏置冒充完整源码。 对应：接触求解，275–294行。

11. 摩擦：等法向冲量可靠后再限制切向

取切向 $t = R(n) = (n_y, -n_x)$，它与法线垂直且为单位向量。 切向相对速度 $v_t = (u_B - u_A) \cdot t$。 通过与法向相同的有效质量推导，可得候选切向增量。 库仑摩擦以累计法向冲量作为上限：

$$|\lambda_t| \le \mu\lambda_n.$$

μ 无量纲，λ_t, λ_n 都是冲量，量纲一致。 使用累计 λ_n，而不是最后一次可能很小甚至为负的法向增量。 例如 $\mu = 0.5$、$\lambda_n = 4\,N\cdot s$，切向累计量必须在 $[-2, 2]\,N\cdot s$ 内。 若候选累计切向量为-3，则钳到-2。 本版接触摩擦在 Relax 分支执行，见 接触求解，331–385行。 教学硬约束版本可以每次法向遍历后求摩擦，但要明确它与本版分阶段实现不同。

12. 热启动：缓存有用，但不应变成重复施力

接触持续时，上一时段的约束冲量通常是下一时段的有用初值。 热启动先把缓存冲量应用到速度，再让当前迭代只修正残差。 当前累计值也必须设置为该缓存值；否则应用过缓存、累计却从零开始，会重复计算。 固定步长下可先直接复用，变步长时需要考虑缓存冲量的时间尺度，不能无条件照搬。 法线或特征变化过大时，应重置或重新匹配。

本版准备阶段将历史冲量按是否启用热启动复制进约束，见 接触准备，112–114行。 热启动实际应用见 `b2WarmStartContacts_Overflow`，184–195行。 关闭热启动应该用于比较收敛速度和残差，不应把“关闭后必须倒塌”当成正确性的硬指标。 充分迭代、简单场景或其他设计下，不热启动也可能稳定。

13. M4成熟后的一步：阶段输入输出表

教学引擎到这一阶段，可以逐步调整为“先生成约束、求好速度、再积分位置”的结构。 对尚未重叠但即将碰撞的对象，需使用合适的候选扩展与推测接触；否则只能下一步才发现新碰撞。 初版也可保留最终有限位置投影，但必须明确它是独立的数值纠偏阶段。

阶段	读取	写入	必须维持的不变量
命令/质量更新	外部操作、形状	质量、逆量、待用力	不在遍历中无序删除对象
宽阶段	形状包围范围	候选物体对	无遗漏才谈减少误报
窄阶段	几何、相对变换	法线、点、间距、特征ID	法线方向一致
准备约束	流形、逆量、缓存	锚点、有效质量、初始累计量	不修改几何造成隐式跳变
外力积分	力、重力、速度	新速度	不在子步中清掉大步外力
热启动	缓存冲量	速度	应用量与累计量一致
迭代求解	最新速度和约束	速度、累计冲量	法向累计量非负
位置积分	求解后的速度	质心、旋转	单位旋转保持有效
收尾	新状态	原点、缓存、事件	清力，合法时机发布事件

画一张每步日志表，记录每阶段之前和之后的状态，比只加更多迭代更容易发现 bug。

14. 用一个落地圆追踪整步状态

A为静态地面，B质量2千克，半径0.5米，圆心高度0.5米，初速度0。 重力 $g_y = -10$，步长 $h = 1/60$，不启用恢复和摩擦。 法线从地面指向圆，为 $(0, 1)$。

阶段	v_y	法向累计冲量	圆心高度
初始	0	0	0.5
重力积分	-1/6	0	0.5
接触求解	0	1/3 N·s	0.5
位置积分	0	1/3 N·s	0.5

因为有效逆质量 $K = 1/2$，冲量 $j = (1/6)/(1/2) = 1/3\, N \cdot s$。 除以步长可还原平均支持力：$j/h = (1/3)/(1/60) = 20\, N = mg$。 这说明静止支撑不是“没有接触冲量”，而是每个时间段的重力作用由向上冲量抵消。 后续帧若启用热启动，支持冲量可作为初始猜测，但不能无条件再额外加同一完整支持量。

15. 当前 Box2D 的阶段位置与教学版本的差别

外层 `b2World_Step` 位于 世界实现，894–1054行。 949–953行更新宽阶段候选；958–965行建立大步和子步时长；986–998行执行窄阶段和求解。 本版求解循环位于 求解器，`b2SolverTask`，1082–1158行。 真实每个子步为：积分速度 → 热启动 → 带偏置Solve → 积分位置 → Relax。 `ITERATIONS` 和 `RELAX_ITERATIONS` 在29–30行均为1，但子步循环会反复推进时间。 这不同于固定几何上重复8次冲量，也不同于教学 M2 的位置推进后一次碰撞后处理。 恢复与摩擦在当前接触 Relax 分支中处理，不另照抄历史阶段注释。 最后 `b2FinalizeBodiesTask` 用增量更新世界质心与旋转，重建刚体原点，并清空力和力矩。 定位：求解器，604–633行。

16. M5以后：按暴露的问题增加优化

观察到的问题	先检查正确性	确认后再优化
单圆落地反向弹飞	法线、相对速度和冲量符号	不要先加迭代掩盖
斜盒子行为奇怪	惯量、力臂、点速度、流形	更稳的接触生成
堆叠收敛慢	累计钳制、缓存匹配、摩擦上限	热启动、子步、软约束
配对数量过大	是否重复A-B/B-A	网格或动态AABB树
静止场景仍很耗时	是否每步重建所有数据	接触复用、睡眠
远离原点后抖动	单位、绝对坐标相减	相对坐标、大世界位置
高速穿透	是否仅测两个离散端点	推测接触、CCD
单核成为瓶颈	热点与内存访问测量	数据拆分、图着色、SIMD

先做子步时，$h = \Delta t/N$，外力在整段 Δt 内保持，不能少施或多施。 先做宽阶段时，拿暴力配对当真值检查是否漏对，而不只比较运行时间。 先做睡眠时，不要让支撑链上相关物体错误地部分睡着；后续需要按连接岛管理唤醒和睡眠。 并行改变读写时序后，必须重新核验结果；“同样公式并行执行”不自动保证同样浮点顺序和位级结果。

17. 可量化验收，而不是凭画面感觉

M1：积分

- 固定步数自由落体与第03章数据表匹配。
- 无力时匀速，质量变化不改变重力加速度。

M2：圆碰撞

- 等质量完全弹性对撞交换速度。
- 非弹性对撞法向相对速度消失，但整体动量保持。
- 已分离对象不出现法向吸引冲量。
- 同心、零质量响应和零时间步有明确处理。

M3：旋转和盒子

- 偏心冲量产生符合叉积符号的角速度。
- 接触点数由实际几何决定，允许单点和双点。
- 盒子静止平放时不因丢掉有效接触而持续无故旋转。

M4：堆叠

固定物体尺寸、质量、摩擦、恢复系数、步长、迭代数、子步数与测试时长。 例如记录10个1米盒子在5秒后的最大穿透、顶部偏移、最大速度与累计冲量残差。 “几厘米”应相对场景单位制定为验收阈值，不是所有尺寸和质量比都通用的保证。 比较开关热启动、迭代次数和子步数的结果，确认变化能解释，而非只挑一张好看的截图。

18. 手算题与答案

题一：配对复杂度

100个物体的全配对数量是多少？ 答案：$100 \times 99/2 = 4950$，不是10000，也不是9900。

题二：接触力与冲量

质量3千克物体在 $g = 10\,m/s^2$ 的地面静止，$h = 1/60$，每步平均支持冲量多少？ 答案：$mgh = 3 \times 10/60 = 0.5\,N \cdot s$。 若分4个子步，每子步理想支持冲量为0.125，总量仍为0.5。

题三：累计冲量撤回

已有法向累计冲量5，本次求得增量-7，应应用多少？ 答案：新累计 $\max(5-7,0)=0$，实际应用 $0-5=-5$。 不能应用-7使累计变负，也不能只允许非负增量而不撤回任何旧冲量。

题四：两个底角是否都碰到地面

半宽1米盒子倾斜20度，一底角刚接触，另一底角离地多少？ 答案：$2\sin 20^\circ \approx 0.68404\,m$。 它说明高角尚未几何触地；不能据此断言当前 Box2D 流形会删除它，也不能给它无条件的着地支持冲量。是否产生冲量，还要结合正分离量和接近速度判断。

完成本章后，再阅读后续碰撞、约束、关节和工程化章节。 此时每个优化都应能对应一个已经看见、已经测量的问题。 返回总目录

22 • 二维物理模拟与约束冲量：概念串讲

返回总目录 · 对照第03章 · 对照第09章 · 对照第13章

> 本章把「二维刚体世界怎样一步步走下去」串成一条不依赖碰撞检测、也不依赖源码阅读的物理主线。 覆盖四件事：Box2D 实际用了哪些分析力学；半隐式欧拉是什么；关节如何变成约束方程；$\mathbf{V}, u, J, M, m_{\text{eff}}$ 从何而来。 源码基准仍是 Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b`。公式与第 03、09、13、14 章一致；需要几何输入或逐步源码时再回到那些章。

本章默认：已经知道刚体有质量、质心、绕质心的转动惯量，以及点速度 $\mathbf{v}_p = \mathbf{v} + \omega \times \mathbf{r}$。这些分别在 02a、02、01。

---

1. 一条可以先记住的主线

每个刚体只有 6 个在动的数：质心位置、角度、线速度、角速度。没有约束时，力和力矩按牛顿定律改速度，再用半隐式欧拉让位置跟着**新速度**走。

关节和接触**不直接改位置公式。**它们先写成几何目标，例如「两锚点距离保持 L_0」。对时间求导，得到「哪些速度组合会破坏这个目标」。求解器再算一个冲量 λ，把这部分速度改掉。

更准确的说法是：

> 约束方程先规定「相对运动里哪一个分量不许错」。 速度层方程 $u = J\mathbf{V}$ 读出这个分量现在是多少。 冲量 λ 沿着 J 的反方向打回去，让这个分量变成目标值（常常是 0）。

「抑制当前破坏约束的速度」对**硬的双侧约束**完全成立。下面这些就不是单纯抑制：

情况	冲量在做什么
硬杆、铰链、滑轨	把约束速度打到 0
接触法向	只阻止继续靠近，已经离开就放手
恢复	不但刹停，还要按 e 反弹
马达	把约束速度**推向**目标，不是刹成 0
软弹簧	只改一部分，留下振动和缓慢纠偏
摩擦	想刹切向滑动，但库仑上限可能刹不住
位置已经错了	只刹速度不够，还要加 bias / 软约束

碰撞检测只负责告诉你「这条临时约束的法线和锚点是什么」。关节把法线换成杆方向、滑轨横向或转角。之后全部是同一套速度约束 + 有效质量 + 冲量。几何算法见第 05 ~ 08 章，本章不再展开。

---

2. Box2D 用了分析力学里的哪些东西

连续刚体力学通常分几条路。Box2D 不是「整本分析力学的数值版」。

路	典型方程	Box2D 有没有
矢量力学	$F = ma$，$\tau = I\alpha$	**有。** 积分、外力、冲量全走这条
分析力学（第二类拉格朗日）	用独立广义坐标消掉约束，写 $L = T - V$	**没有。** 从不把铰链摆写成一个 θ
分析力学（第一类拉格朗日 / 乘子法）	冗余坐标 + $J^T\lambda$	**有。** 接触、关节、马达全是这个
哈密顿力学	q, p 正则方程、辛积分器当主循环	**没有当求解器。** 第 03 章只提半隐式欧拉在无耗散哈密顿系统里有辛结构

所以更准确的说法是：

> 状态是牛顿力学的 (c, θ, v, ω)； 约束力的方向和大小是分析力学的 J 与 λ； 多约束耦合用 $K = JM^{-1}J^T$； 单向接触再加互补条件，变成非光滑接触力学。

第 00 章那四层也要对上：乘子方程是物理模型；顺序冲量、软约束、Relax 是数值方法；动态树和 SIMD 与分析力学无关。

2.1 真正进了引擎的部分

广义坐标与自由度。 每个二维刚体 3 个自由度：

$$\mathbf{q} = (x, y, \theta), \qquad \mathbf{V}_{\text{单体}} = (v_x, v_y, \omega).$$

两个刚体拼成 6 维速度。关节的意义是砍掉**相对**自由度，不是另写一套运动学。引擎**不消元**：始终用冗余坐标 + 乘子。两物体一起平移、一起旋转不应被无故锁死。

约束分类。 教材未必使用 holonomic 一词，但方程就是这些类型：

类型	分析力学名称	Box2D 实例
双侧位置约束	完整双侧 $C(q) = 0$	硬杆、铰链、滑轨、焊接
单侧位置约束	不等式 $C(q) \geq 0$	接触法向、关节限位
位置约束的速度形式	$\dot{C} = JV$	所有关节和接触的求解层
纯速度目标	驱动 / 有界速度约束	马达、滚阻、传送带切向速度
含时 / 驱动	非定常	motor 目标位姿与目标速度

求解器几乎从不直接解 $C(q) = 0$，而是解速度层 $u + b = 0$ 或软形式 $u + \beta c + \gamma\Lambda = 0$。位置误差只作为 bias / 软约束的 c 灌进速度方程。

Jacobian、虚功与乘子。 第 09 章第 6 节用广义微位移说明为什么冲量要乘 J^T：

$$\delta c = J\delta\mathbf{q}, \qquad \lambda\,\delta c = (J^T\lambda)^T\delta\mathbf{q}, \qquad \Delta\mathbf{V} = M^{-1}J^T\lambda.$$

λ 在连续时间是拉格朗日乘子（约束力）；在离散时间是冲量乘子。有恢复和摩擦时，教材明确**不声称**接触总功为零；这里只辨认广义作用方向。

有效质量。 $K = JM^{-1}J^T$ 是约束空间的惯性响应，接触力学里也称 Delassus 算子。$m_{\text{eff}} = 1/K$。偏心、轻、容易转，有效质量都更小。多约束共享刚体时 K 有非对角项，顺序冲量是对这个耦合系统做 Gauss–Seidel。

单侧互补与库仑。 法向 $0 \le \Lambda \perp w \ge 0$ 是速度 / 冲量层的 Signorini 条件。摩擦 $|\Lambda_t| \le \mu\Lambda_n$。法向与切向还通过转动交叉耦合，所以「法向满足后再算摩擦，一遍就都满足」不成立。

动能与角动量。 第 02a 章用 $\frac{1}{2}I\omega^2$ 和 $L = I\omega$ 定义 I。引擎不以 T, V 为状态。机械能守恒只适合孤立无摩擦弹性对心等检查；有地面、马达、阻尼、恢复、bias 时不要要求守恒。

软约束。 一条约束模式看成单自由度弹簧阻尼 $m_{\text{eff}}\ddot{c} + d\dot{c} + kc = 0$，再隐式离散。`biasRate` 不是自然界的物理常数。

马达。 目标是 $\dot{C} = v_{\text{target}}$，能力是力，累计冲量钳在 $\pm hF_{\max}$。力不够就达不到目标，不是求解失败。

2.2 只用了词、没有当求解器的部分

概念	教材怎么用	引擎有没有
虚功原理	解释 J^T	没有对全系统做虚功求和
达朗贝尔原理	隐含在理想双侧约束	有摩擦 / 恢复后明确放弃「总功为零」
辛 / 哈密顿	半隐式欧拉的注释	加阻尼和冲量后不要说整个世界辛
势能 V	重力当 mg，弹簧当 kc	没有总势能函数
Baumgarte	教学偏置 $b = \beta C/h$	本版改成软约束三系数

2.3 完全没用的部分

第二类拉格朗日方程、哈密顿正则方程、最小作用量 / 变分积分器、循环坐标与 Routh 降阶、Kane / Maggi 等解析公式、理想无滑滚动作为第一类非完整约束、三维欧拉方程的陀螺项 $\omega \times I\omega$、机构学符号奇异分析。SAT、GJK、TOI、动态树属于计算几何，只给约束提供 n, r, s。

若把分析力学课本按篇幅算 100%，引擎真正吃进去的是：**约束、Jacobian、乘子、广义质量、单侧互补、约束方向上的弹簧阻尼**；刚体动能 / 惯量作为前置定义。其余正则变换、降阶、分析积分基本为零。

按章节对照：02a、03、09 ~ 11、13 ~ 15 是分析力学含量高的部分；05 ~ 08、17 ~ 20 基本没有。读物理对照源码，路线是 `02a → 03 → 09 → 10 → 11 → 13 → 14 → 15`，或先读完本章再进第 09 章。

---

3. 半隐式欧拉是什么

半隐式欧拉就是：这一小步里先把速度更新完，再用这个新速度去走位置。显式欧拉反过来：位置仍用旧速度走。

连续运动是

$$\frac{dv}{dt} = a, \qquad \frac{dx}{dt} = v.$$

计算机只能用某一个代表速度乘步长 h。差就差在用哪一个 v。

显式欧拉（用旧速度走路）：

$$v_{n+1} = v_n + h\,a(x_n, v_n), \qquad x_{n+1} = x_n + h\,v_n.$$

半隐式欧拉（用新速度走路）：

$$v_{n+1} = v_n + h\,a(x_n, v_n), \qquad x_{n+1} = x_n + h\,v_{n+1}.$$

速度那一行两边一样，都是「旧加速度乘 h」。真正改的是位置：半隐式用的是刚写好的 v_{n+1}。

「半」的意思是：位置对速度是隐式的（用新 v），加速度仍按旧状态算，**并不去解一般非线性隐式方程**。它也叫 symplectic Euler（辛欧拉），和完全隐式欧拉不是一回事。

教学伪代码：

```
velocity = velocity + h * acceleration
position = position + h * velocity    # 这里已经是新速度
```

3.1 自由落体：差在哪

取 $y_0 = 10\,\mathrm{m}$、$v_0 = 0$、$g = -10\,\mathrm{m/s^2}$、$h = 1/60\,\mathrm{s}$，无碰撞无阻尼。解析解 1 秒后是 $v = -10$、$y = 5$。

恒定加速度下，两种方法的速度都对：第 n 步 $v_n = -n/6$。位置不同，因为求和用的速度集合不同。

1 秒后	高度	
解析	$5.000000\,\mathrm{m}$	
显式	$5.083333\,\mathrm{m}$（偏高）	
半隐式	$4.916667\,\mathrm{m}$（偏低）	
第一步时 $v_0 = 0$，显式欧拉位置不动；半隐式第一步就已经用上 $v_1 = -1/60$，所以立刻掉一点。60 步之后两者误差刚好在解析值两侧，大小都是 (\frac12	g	t h)。把 h 减半，这个例子里的位置误差也减半，但半隐式不会因此变成解析解。完整中间状态表见第 03 章第 6 节。

3.2 为什么刚体模拟更常用它

自由落体看不出稳定性，因为 a 不依赖位置。无阻尼弹簧 $a = -\omega_0^2 x$ 才能看出来。

- 显式欧拉：状态转移矩阵特征值模长 $\sqrt{1+h^2\omega_0^2} > 1$。理想弹簧会一步步被数值方法「喂能量」，振幅持续变大；h 再小也只是涨得慢。
- 半隐式：行列式为 1。在 $0 < h\omega_0 < 2$ 时特征值在单位圆上，振荡保持有界。能量会在附近摆，不是每一步严格守恒。

超过 $h\omega_0 = 2$ 通常会散。「用了半隐式就不会炸」不成立，只是比显式欧拉更适合硬弹簧、接触约束这类系统。

3.3 Box2D 每个子步实际怎样插进约束

每个子步不是「算完加速度立刻改位置」，而是：

1. 用外力、重力、阻尼更新速度
2. 用接触和关节冲量再改速度
3. **然后**才用这份已经约束修正过的速度积分位置

位置积分用的是新速度，所以属于半隐式这一边。

和教科书公式还有两处不一样：

1. **阻尼不是整段后向欧拉。** 本版只让旧速度乘 $1/(1+hc)$，本步的 hF/m 和重力加在阻尼外面。例如 $v=10$、$a=3$、$c=2$、$h=0.1$，得到 $10/1.2 + 0.3 = 8.633$，不是 $(10+0.3)/1.2 = 8.583$。见第 03 章第 9 节。
2. **加上接触冲量、摩擦钳制、速度上限之后，整个游戏世界不再是辛系统。** 无耗散的合适哈密顿系统里这种方法有辛结构；有阻尼和冲量后，不要说整个世界严格辛或严格守恒。

---

4. 二维刚体世界本身在模拟什么

一个不受约束的二维刚体有 **3 个自由度**：左右、上下、逆时针转。状态是

$$\mathbf{q} = (x, y, \theta), \qquad \mathbf{V}_{\text{单体}} = (v_x, v_y, \omega).$$

刚体上任意一点的速度不是质心速度，而是

$$\mathbf{v}_p = \mathbf{v} + \omega \times \mathbf{r}, \qquad \omega \times \mathbf{r} = (-\omega r_y, \omega r_x).$$

接触点、关节锚点看的都是点速度，不是质心速度。忽略转动项，是「正撞正常、边角碰撞异常」的常见原因。

无约束时，一个小步 h 只做两件事：

1. 外力改速度：重力、用户施力、阻尼 [\mathbf v\leftarrow\mathbf v+h\mathbf a,\qquad \omega\leftarrow\omega+h\alpha]
2. 新速度改位置（半隐式欧拉） [\mathbf x\leftarrow\mathbf x+h\mathbf v_{\text{新}}]

如果世界里只有自由落体，到这里就结束了。盒子会穿过地板，杆会拉长，轮子会脱轨——因为还没有任何东西禁止这些运动。

约束的全部工作，就是在第 1 步和第 2 步之间插入：算出冲量，改 $\mathbf{v},\omega$，再让位置去跟这份合法速度。

时间推进的是积分；约束方程**不推进时间**，只在这一瞬间改速度。一个子步的物理顺序是：

```
外力 / 重力 / 阻尼   →   先得到「假如没有关节和接触」的速度
约束冲量循环         →   去掉（或改写）被禁止的相对速度
用新速度积分位置
必要时再放松一轮（修恢复、压残差）
```

---

5. 约束方程到底是什么：三层不要混

5.1 位置层：不许怎样摆

每个关节先说一句人话，再写成标量（或几个标量）函数 $C(\mathbf{q}) = 0$。

C 是**约束误差**：现在的几何离目标差多少。单位是米或弧度。

$$C = 0\text{：正好满足，}\quad C \neq 0\text{：已经破坏.}$$

接触是不等式：$C = s \geq 0$（表面分离，负值表示穿进去）。限位也是不等式。

这一层还没有冲量。它只回答：我们究竟锁死了哪一种相对运动。

5.2 速度层：哪些速度会让误差变大

对 C 求导：

$$u = \dot{C} = J\mathbf{V}.$$

J 不是新物理量，就是「每个速度对误差变化率的贡献系数」。 u 才是求解器每一步真正盯着的数：

- $u > 0$：误差正在变大（按该约束的符号约定）
- $u = 0$：这一瞬间误差不再变
- $u < 0$：误差正在变小

硬约束的速度目标通常是

$$u + b = 0.$$

$b = 0$ 时就是：把破坏约束的那一档速度完全刹住。 $b \neq 0$ 时目标不是 0，而是某个偏置速度（修穿透、恢复、预测限位）。

「抑制破坏约束的速度」，指的就是这一层。

5.3 冲量层：用多大一脚改 u

冲量 λ 沿约束方向作用在两个刚体上，作用反作用自动满足。速度变化是

$$\Delta\mathbf{V} = M^{-1}J^T\lambda.$$

于是约束速度的变化是

$$\Delta u = K\lambda, \qquad K = JM^{-1}J^T, \qquad m_{\text{eff}} = \frac{1}{K}.$$

硬约束一眼就能解：

$$\lambda = -m_{\text{eff}}(u + b).$$

这就是「算出需要施加多大的冲量」。五个符号的逐步推导见第 8 节。

5.4 为什么是冲量，不是力

接触力可以极大、极短。刚体模型不猜那个力曲线，直接用动量定理：

$$m\Delta\mathbf{v} = \mathbf{P} = \lambda\mathbf{n}.$$

关节同理：这一小步里需要的约束力对时间的积分，就是 λ。单位是 N·s（线）或 N·m·s（角）。

改完速度再积分位置。所以约束**从不直接把物体传送到合法位置**；它只卖合法速度。位置若已经错了，只能靠 bias / 软约束慢慢借速度修回来。

这也是为什么「只把法向速度刹成 0」时，已经打进地面 2 cm 的盒子会停在穿透处：速度合法了，位置误差还在。

---

6. 关节怎样变成约束方程

统一配方只有四步。每种关节的差别只在第 1 步选了什么 C。

1. 写出几何目标 $C(\mathbf{q}) = 0$（或 ≥ 0）
2. 求导得到 $u = J\mathbf{V}$
3. 用质量和杆臂算 K、m_{eff}
4. 解 λ，把等大反向的冲量加到两个刚体的 $\mathbf{v}, \omega$ 上

二维里一个刚体 3 个自由度，两个刚体一共 6。每多一条独立的 C，就锁掉 1 个**相对**自由度。

6.1 距离关节：一根杆

人话：两个指定锚点的距离保持 L_0。

$$C = L - L_0 = 0, \qquad L = \|p_B - p_A\|.$$

求导：只有沿连线方向的相对速度会改变长度，

$$u = \dot{C} = \mathbf{u} \cdot \mathbf{v}_r.$$

$\mathbf{u}$ 是从 A 指向 B 的单位向量，$\mathbf{v}_r$ 是 B 锚点相对 A 锚点的速度（含转动）。

- 沿杆方向拉长 / 缩短：被锁
- 绕锚点摆动：自由

正 λ 把两锚点推开，负 λ 拉近。详细展开见第 13 章。

6.2 旋转关节（铰链）：两点焊在一起，但可相对转

人话：两个锚点重合。这不是「距离为 0 的杆」——零长杆没有稳定方向。它是 **x、y 两条**约束：

$$C_p = p_B - p_A = \mathbf{0}.$$

速度层是锚点相对速度为 0，二维冲量 (P_x, P_y)。 相对转动不在 C_p 里，所以还能当铰链转。

两条约束通过转动耦合：水平一脚会让物体转，转动又改变竖直方向的锚点速度，所以要解 2×2 的 K。见第 14 章第 2 节。

6.3 棱柱关节（滑轨）：只能沿导轨滑

人话：不能横向脱轨，也不能相对扭转。导轨跟着物体 A 一起转，不是世界里一根死轴。

$$C_1 = \mathbf{n} \cdot d = 0, \qquad C_2 = \theta = 0.$$

$\mathbf{n}$ 是导轨的横向，d 是锚点连线，θ 是相对角。 沿导轨切向 $\mathbf{t}$ 的移动是自由的。

求导时 $\mathbf{n}$ 自己也在转，所以横向速度里会多出导轨转向项。漏掉这一项，等于把导轨当成钉在世界上。见第 14 章第 4 节。

6.4 轮关节：滑轨去掉转角锁

人话：不能横向跑偏，但轮子可以自己转，也可以沿悬挂轴上下。

只保留 $C = \mathbf{n} \cdot d = 0$。 旋转用可选的角马达驱动，悬挂用沿轴的弹簧，那是同一条几何上再挂执行器，不是另一套物理。

6.5 焊接：铰链再锁死相对角

$$C_p = \mathbf{0}, \qquad C_\theta = \theta = 0.$$

三条约束，相对自由度理想下为 0。两个物体仍可作为整体在世界里动。实际常做成软的，否则过刚会抖。

6.6 关节对照表

关节	位置约束 C	锁掉的相对运动	还允许什么
无	无	无	平移 + 转动
距离	$L - L_0 = 0$	沿杆伸缩	绕杆摆
铰链	$d_x = d_y = 0$	锚点错开	相对转

关节	位置约束 C	锁掉的相对运动	还允许什么
棱柱	横向 = 0，相对角 = 0	脱轨、相对转	沿轴滑
轮	横向 = 0	脱轨	沿轴滑 + 相对转
焊接	点重合 + 相对角 = 0	全部相对运动	无（可加软度）

接触可以看成「临时的、单侧的距离关节」：法线方向 $s \geq 0$，切向再视摩擦决定黏住还是滑。一旦有了法线、锚点、分离量，后面的方程和关节是同一套。

---

7. 同一套方程的几种变体

7.1 位置已经错了：bias

只令 $u = 0$，已有的 $C \neq 0$ 会冻在那里。给速度一个修正目标：

$$b = \frac{\beta}{h} C, \qquad \lambda = -m_{\text{eff}}(u + b).$$

杆太长（$C > 0$）时 $b > 0$，需要负冲量拉回来。 这不是新物理定律，是「借一点速度去修位置」。修太猛会弹；本版实际用弹簧阻尼离散成软约束，避免一步修完。见第 11 章。

7.2 接触：单侧

法向只能推，不能吸：

$$\Lambda \geq 0, \quad w = u + b \geq 0, \quad \Lambda w = 0.$$

正在砸向地面：施加正冲量，抑制接近速度。 已经跳开：$\Lambda = 0$，不抑制分离。 若算出负的累计法向冲量，那是在「吸回去」，必须丢掉。

所以接触也是约束方程，但「抑制」只对接近成立。

7.3 恢复：目标不是 0

$$u^{+} = -e u^{-}.$$

接近速度 $-4\,\mathrm{m}/s$、$e = 0.5$，目标分离速度是 $+2$，不是 0。冲量比纯非弹性更大。这是材料模型，不是动量守恒推出来的。本版字段与 Relax 时序见第 12 章，不要把理想碰撞公式直接当成完整源码。

7.4 马达：目标速度，能力是力

距离马达希望 $\dot{L} = v_{\text{target}}$。试算仍是

$$\lambda_{\text{trial}} = m_{\text{eff}}(v_{\text{target}} - u),$$

但累计冲量被钳在 $\pm h F_{\max}$ 里。力不够就达不到目标——**不是求解失败。** 这里冲量在**制造**约束方向上的速度，和「抑制破坏」方向相反。

限位是单侧马达的亲戚：还没越界时，只把这一小步内会冲出边界的速度削掉（预测 $C + hu \geq 0$）；已经越界才修位置。见第 13 章第 9～10 节。

7.5 摩擦：切向约束加上限

切向同样有 $u_t = J_t \mathbf{V}$，同样想把滑动刹成 0，但

$$|\Lambda_t| \leq \mu \Lambda_n.$$

摩擦够：黏着，切向变成等式约束。 摩擦不够：饱和，还在滑。 传送带只是把切向目标从 0 改成皮带速度。

滚阻是相对角速度那一条类似的有界约束，不是切向摩擦。见第 10 章。

7.6 软约束：不要一次刹死

一条约束方向看成弹簧阻尼：

$$m_{\text{eff}} \ddot{C} + d\dot{C} + kC = 0.$$

离散后变成「新速度 + 位置误差项 + 柔度项 = 0」。硬约束是柔度 $\gamma = 0$ 的极限。接触默认很硬但仍有软度，所以堆叠能静得住、又不会因位置修正而狂跳。

7.7 为什么不能一条条各算一次就结束

多个约束共享同一个刚体时，K 有非对角项：左下角的法向冲量会让盒子转，从而改变右下角的法向速度。摩擦冲量也会通过转动改法向速度。第 09 章两个底角的例子是

$$K = \begin{pmatrix} 1.5 & 0.5 \\ 0.5 & 1.5 \end{pmatrix}.$$

求解器是**反复扫**：每次只精确满足当前这一条，立刻改速度，下一条看见最新速度。扫多轮逼近联立解。某一轮的负增量常常只是撤回上一轮推多的部分，不是接触变成了吸力。

---

8. $\mathbf{V}, u, J, M, m_{\text{eff}}$ 一个一个从哪来

以下以接触法向（或杆方向）的**一条标量约束**为默认。铰链是两行 J，棱柱横向加转角也是两行；一行对一个 u，矩阵形式相同。

8.0 先固定场景

两个二维刚体 A、B。每个有：

量	含义	单位
$\mathbf{v} = (v_x, v_y)$	质心线速度	m/s
ω	角速度，逆时针为正	rad/s

量	含义	单位
m	质量	kg
I	绕质心的转动惯量	kg·m²
$\mathbf{r}$	质心指向约束点（接触点或关节锚点）	m

约束方向用单位向量 $\mathbf{n}$（接触法线，或杆的单位方向），从 A 指向 B。刚体上那一点的速度仍是 $\mathbf{v}_p = \mathbf{v} + \omega \times \mathbf{r}$。

8.1 $\mathbf{V}$：把所有未知速度排成一列

约束会同时改两个物体的平动和转动，所以把 6 个数写成一列，只是记账：

$$\mathbf{V} = \begin{bmatrix} v_{Ax} \\ v_{Ay} \\ \omega_A \\ v_{Bx} \\ v_{By} \\ \omega_B \end{bmatrix}.$$

它不是新物理量。没有 $\mathbf{V}$ 时，你照样可以分开写 $v_A, \omega_A, v_B, \omega_B$。有了它，后面的 $u = J\mathbf{V}$、$\Delta\mathbf{V} = M^{-1}J^T\lambda$ 才能写成矩阵乘法。

两个物体一起平移、一起旋转，都在 $\mathbf{V}$ 里；约束要消灭的，只是 $\mathbf{V}$ 的某一种组合，不是整列都变成 0。

8.2 u：约束方向上的相对速度

位置层先有一个误差 C。硬杆是 $C = L - L_0$，接触分离是 $C = s$。 **求解器每一步真正盯的不是 C，而是它的变化率**

$$u = \dot{C}.$$

u 的单位是 m/s（角度约束则是 rad/s）。含义：

- $u < 0$：正在朝破坏约束的方向走（接触在靠近、杆在按符号缩短 / 拉长）
- $u = 0$：这一瞬间误差不再变
- $u > 0$：正在离开破坏方向

以接触法向为例。B 相对 A、沿 $\mathbf{n}$ 的相对速度是

$$\mathbf{v}_{\text{rel}} = (\mathbf{v}_B + \omega_B \times \mathbf{r}_B) - (\mathbf{v}_A + \omega_A \times \mathbf{r}_A),$$

$$u = \mathbf{n} \cdot \mathbf{v}_{\text{rel}}.$$

地面为 A、箱子为 B、$\mathbf{n} = (0, 1)$、箱子以 $4\,\text{m}/s$ 下落：$u = -4\,\text{m}/s$。 这就是「正在破坏法向不穿透」的那档速度。杆的情况把 $\mathbf{n}$ 换成杆方向，公式同一张。

u 是标量。六个速度里只有投影到约束方向的那一档被管；箱子可以边转边滑，只要这一点沿法线不再靠近。

8.3 J：从 $\mathbf{V}$ 里读出 u 的那一行

把 u 展开，用恒等式

$$\mathbf{n}\cdot(\omega\times\mathbf{r})=\omega\,(\mathbf{r}\times\mathbf{n}).$$

（二维叉积 $\mathbf{a}\times\mathbf{b}=a_xb_y-a_yb_x$ 是标量。）

$$\begin{aligned}u&=-\mathbf{n}\cdot\mathbf{v}_A-(\mathbf{r}_A\times\mathbf{n})\,\omega_A+\mathbf{n}\cdot\mathbf{v}_B+(\mathbf{r}_B\times\mathbf{n})\,\omega_B\\&=-n_xv_{Ax}-n_yv_{Ay}-(\mathbf{r}_A\times\mathbf{n})\omega_A+n_xv_{Bx}+n_yv_{By}+(\mathbf{r}_B\times\mathbf{n})\omega_B.\end{aligned}$$

对照 $u=J\mathbf{V}$，系数就是 Jacobian：

$$J=\big[-n_x,\ -n_y,\ -(\mathbf{r}_A\times\mathbf{n}),\ n_x,\ n_y,\ \mathbf{r}_B\times\mathbf{n}\big].$$

J 怎么来的：对位置约束求导，把 $\dot C$ 写成速度的线性组合。 不是另造的物理定律。第 13 章对距离关节把同一过程从 $L=\|d\|$ 重新推了一遍，不要背系数。

J 的列	配对的速度	这一列在问什么
$-n_x,-n_y$	v_A	A 质心沿 $-\mathbf{n}$ 动，u 怎么变
$-(\mathbf{r}_A\times\mathbf{n})$	ω_A	A 转动时，约束点沿法线的快慢
n_x,n_y	v_B	B 质心沿 $\mathbf{n}$
$\mathbf{r}_B\times\mathbf{n}$	ω_B	B 转动

平动列无量纲，转动列带米。不要要求六列单位相同：它们乘的是不同类型的广义速度，乘完每一项都是 m/s。

8.4 M：广义质量，把「踢一脚」变成「速度跳多少」

牛顿定律的冲量形式：

$$m\Delta\mathbf{v}=\mathbf{P},\qquad I\Delta\omega=\mathbf{r}\times\mathbf{P}.$$

写成矩阵，平动两个分量各用一次质量，转动用惯量：

$$M=\operatorname{diag}(m_A,\,m_A,\,I_A,\,m_B,\,m_B,\,I_B),$$
$$M^{-1}=\operatorname{diag}(m_A^{-1},\,m_A^{-1},\,I_A^{-1},\,m_B^{-1},\,m_B^{-1},\,I_B^{-1}).$$

M 的来源就是 $F=ma$ 和 $\tau=I\alpha$ 的时间积分，没有新假设。 静态地面、运动学物体把对应逆质量、逆惯量写成 0：冲量打上去，速度不变。零逆质量是约束响应约定，不是因为浮点不能表示无穷大。

若广义冲量是 $\mathbf{\Gamma}$（与 $\mathbf{V}$ 六个分量配对），则

$$\Delta\mathbf{V}=M^{-1}\mathbf{\Gamma}.$$

下一步只需要知道：约束的标量 λ 怎样变成这六个 $\mathbf{\Gamma}$。

8.5 $J^T\lambda$：标量冲量怎样播回两个刚体

约定：B 沿 $\mathbf{n}$ 受到线冲量 $\lambda\mathbf{n}$，A 受到 $-\lambda\mathbf{n}$（作用反作用）。角冲量是杆臂叉乘线冲量：

$$
\begin{aligned}
\Delta\mathbf{v}_A &= -m_A^{-1}\lambda\mathbf{n},\\
\Delta\omega_A &= -I_A^{-1}(\mathbf{r}_A\times\mathbf{n})\lambda,\\
\Delta\mathbf{v}_B &= m_B^{-1}\lambda\mathbf{n},\\
\Delta\omega_B &= I_B^{-1}(\mathbf{r}_B\times\mathbf{n})\lambda.
\end{aligned}
$$

写成 $\Delta\mathbf{V}$：

$$
\Delta\mathbf{V}=\lambda\begin{bmatrix}
-n_x/m_A\\ -n_y/m_A\\ -(\mathbf{r}_A\times\mathbf{n})/I_A\\
n_x/m_B\\ n_y/m_B\\ (\mathbf{r}_B\times\mathbf{n})/I_B
\end{bmatrix}=M^{-1}J^T\lambda.
$$

最后一步不是凑巧：J^T 正好是

$$
J^T=\begin{bmatrix}
-n_x\\ -n_y\\ -(\mathbf{r}_A\times\mathbf{n})\\
n_x\\ n_y\\ \mathbf{r}_B\times\mathbf{n}
\end{bmatrix},
$$

再左乘 M^{-1} 就是上面那一列。

所以：

- J：用速度去算 u（运动学）
- J^T：把标量冲量按同一套系数播回两个刚体（动力学）
- M^{-1}：冲量 → 速度变化

虚功是同一件事的另一说法。约束微位移 $\delta c = J\delta\mathbf{q}$，乘子做功 $\lambda\delta c = (J^T\lambda)\cdot\delta\mathbf{q}$，所以与广义坐标配对的冲量方向就是 $J^T\lambda$。

两边的线冲量大小相等方向相反，因此动态刚体对的总线动量守恒。静态体不更新速度，它相当于连接了外部支撑。

8.6 K 和 m_{eff}：这一脚能把 u 改多少

施加 λ 之后，新的约束速度是

$$
u' = J(\mathbf{V}+\Delta\mathbf{V}) = u + JM^{-1}J^T\lambda.
$$

定义

$$
K = JM^{-1}J^T.
$$

一条标量约束时 K 是一个数，单位 kg⁻¹：

$$
\Delta u = K\lambda.
$$

把 J 和 M^{-1} 乘开（$\mathbf{n}\cdot\mathbf{n}=1$）：

$$
K = m_A^{-1} + m_B^{-1} + I_A^{-1}(\mathbf{r}_A\times\mathbf{n})^2 + I_B^{-1}(\mathbf{r}_B\times\mathbf{n})^2.
$$

四项的来源：

1. A 被往后推，质心沿 $-\mathbf{n}$ 减速，u 增加 $m_A^{-1}\lambda$
2. B 被往前推，同理 $m_B^{-1}\lambda$
3. A 受到力矩，转动改变约束点法向速度，贡献 $I_A^{-1}(\mathbf{r}_A \times \mathbf{n})^2\lambda$
4. B 同理

有效质量定义为

$$m_{\text{eff}} = \frac{1}{K}.$$

单位是 kg。它回答：**要让这条约束的 u 改变 $1\,\mathrm{m}/s$，需要多大冲量。**

- 两质点对心、无转动：$m_{\text{eff}} = (m_A^{-1} + m_B^{-1})^{-1}$，就是约化质量
- 杆臂越长、越容易转：K 变大，m_{eff} 变小——同样刹住接触点，所需冲量更小，因为物体可以「转开」而不是整块刹停
- 两边都静质量（逆质量全 0）：$K = 0$，这个方向没有可响应的自由度，不能除。代码把有效质量置零，不表示无限大修正冲量

源码里接触的 `normalMass` 存的就是 m_{eff}，不是 K。角约束的有效质量具有转动惯量的单位，不能直接套 kg。

8.7 解出 $\lambda = -m_{\text{eff}}(u + b)$

硬约束希望一步之后

$$u' + b = 0.$$

b 是人为加的速度目标，单位 m/s：

- $b = 0$：只消灭当前的 u（纯非弹性、无位置修正）
- 恢复：希望 $u^+ = -eu_{\text{old}}$，等价于 $b = eu_{\text{old}}$（接近时 $u_{\text{old}} < 0$，b 为负，目标是分开）
- 位置修正：$b \sim C/h$，杆太长就给一个「缩短」的目标速度

代入 $u' = u + K\lambda$：

$$u + K\lambda + b = 0 \quad \Rightarrow \quad \lambda = -\frac{u + b}{K} = -m_{\text{eff}}(u + b).$$

负号的含义：$u + b < 0$（还在按不该走的方向走）时 $\lambda > 0$，沿 $\mathbf{n}$ 把双方推开。

改的是 u，不是把 $\mathbf{V}$ 整列清零。

读公式的顺序永远是：

$$\text{几何 } C \xrightarrow{\text{求导}} u = J\mathbf{V} \xrightarrow{\text{牛顿}} \Delta\mathbf{V} = M^{-1}J^T\lambda \xrightarrow{\text{再左乘 } J} \Delta u = K\lambda \xrightarrow{\text{令 } u'+b=0} \lambda = -m_{\text{eff}}(u + b).$$

关节换的只是 C 的定义，因而 J 的具体数字不同；M、K、m_{eff}、λ 这套箭头不变。马达把目标从 0 换成 v_{target}，接触再加 $\lambda \geq 0$，都是在最后解 λ 那一步改条件，不是另起一套符号。

---

9. 用一个数字把五个量走完

地面 A 静：$m_A^{-1} = I_A^{-1} = 0$。 箱子 B：$m = 1\,\mathrm{kg}$，$I = 0.5\,\mathrm{kg\cdot m^2}$， $\mathbf{r}_B = (0.5, -0.5)\,\mathrm{m}$，$\mathbf{n} = (0, 1)$，$\mathbf{v}_B = (0, -4)$，$\omega_B = 0$。无 bias：$b = 0$。

这是第 09 章第 7.1 节的同一算例。

$\mathbf{V}$（A 的速度全是 0）

$$\mathbf{V} = (0, 0, 0,\ 0, -4,\ 0)^T.$$

J

$$\mathbf{r}_B \times \mathbf{n} = 0.5 \cdot 1 - (-0.5) \cdot 0 = 0.5,$$
$$J = (0, 0, 0,\ 0, 1,\ 0.5).$$

$u = J\mathbf{V}$

$$u = 1 \cdot (-4) + 0.5 \cdot 0 = -4\,\mathrm{m}/s.$$

和 $\mathbf{n} \cdot \mathbf{v}_B$ 一致：正在砸向地面。

M^{-1} 只有 B 的三项非零：$1, 1, 2$（因为 $I^{-1} = 2$）。

K

$$K = m_B^{-1} + I_B^{-1}(\mathbf{r}_B \times \mathbf{n})^2 = 1 + 2 \cdot (0.5)^2 = 1.5\,\mathrm{kg}^{-1}.$$

$m_{\text{eff}} = 1/1.5 = 2/3\,\mathrm{kg}$ 正下方撞击时杆臂叉乘为 0，有效质量会是 $1\,\mathrm{kg}$。撞在角上，箱子能转开，有效质量只有 $2/3$。

λ

$$\lambda = -m_{\text{eff}} u = -\frac{2}{3} \cdot (-4) = \frac{8}{3}\,\mathrm{N\cdot s}.$$

打回去

$$\Delta v_{By} = \frac{8/3}{1} = \frac{8}{3}, \quad \Delta\omega_B = 2 \cdot (0.5) \cdot \frac{8}{3} = \frac{8}{3},$$
$$v_{By}^{+} = -4 + \frac{8}{3} = -\frac{4}{3}, \quad \omega_B^{+} = \frac{8}{3}.$$

新的 u：

$$u' = v_{By}^{+} + 0.5\,\omega_B^{+} = -\frac{4}{3} + \frac{4}{3} = 0.$$

接触点法向停住；质心仍以 $-4/3\,\mathrm{m}/s$ 往下，箱子在转。约束成功的标志是 $u = 0$，不是整块箱子静止。

若把 $\mathbf{r}_B$ 改到质心正下方，J 的最后一列变成 0，$m_{\text{eff}} = 1$，$\lambda = 4$，质心也会被完全刹住——因为这时转动帮不上忙。

硬杆的同一套路见第 13 章第 5 节：约束的是锚点距离变化率，不是质心速度相等，也不是角速度单独相等。

10. 五个量怎么记

符号	是什么	怎么来的	在公式里干什么
$\mathbf{V}$	两个刚体的 6 个速度	把 v, ω 堆成一列	未知运动状态
u	约束方向相对速度 $\dot{C}$	点速度差再点乘 $\mathbf{n}$	现在破坏得有多快
J	1×6 系数行	把 u 对速度展开	$u = J\mathbf{V}$
M	广义质量对角阵	$m\Delta v = P$，$I\Delta\omega = r \times P$	$\Delta\mathbf{V} = M^{-1}(\cdots)$
$J^T\lambda$	播到 6 个通道上的广义冲量	作用反作用 + 力矩臂；或虚功	约束力的方向
$K = JM^{-1}J^T$	冲量到 u 的增益	把上面两步接起来	$\Delta u = K\lambda$
$m_{\text{eff}} = 1/K$	这条约束的等效质量	K 的倒数	$\lambda = -m_{\text{eff}}(u + b)$

11. 用这句话收口

- **约束方程**：先是几何 $C(\mathbf{q}) = 0$ 或 ≥ 0，说明锁死哪一种相对运动。
- **拿到求解器里的**，主要是它的速度形式 $u = J\mathbf{V}$。
- **冲量**：沿着这条约束方向打一脚，让 u 变成目标。

「抑制当前破坏约束的速度」对硬杆、铰链、滑轨、非弹性接触都成立。把它扩成一句就完整了：

> 先读出约束方向上的相对速度 u，再求 $\lambda = -m_{\text{eff}}(u - \text{目标})$； 目标常常是 0，也可以是反弹、马达速度或「这一步别越界」； 单侧约束还要丢掉会吸回去的解； 位置误差另用 bias / 软约束借速度来修。

引擎里每条接触、每条关节、每个马达，最后都是：

$$\underbrace{u = J\mathbf{V}}_{\text{约束的运动学}}, \quad \underbrace{\Delta\mathbf{V} = M^{-1}J^T\lambda}_{\text{乘子 / 虚功}}, \quad \underbrace{K = JM^{-1}J^T}_{\text{约束空间惯性}}, \quad \lambda \text{ 再按等式、不等式或力上限投影.}$$

下一步若要看「几何从哪来」，去第 05 ~ 08 章。若要看「接触怎样迭代、摩擦怎样限幅」，去第 09 ~ 12 章。若要看「每种关节的 C 和 J 逐行写出」，去第 13 ~ 15 章。若要看「积分、阻尼、自由落体完整表」，去第 03 章。

返回总目录 · 第03章 · 力力矩冲量与积分 · 第09章 · 碰撞冲量与约束方程

05 • 基础碰撞与接触流形

> 阅读目标：先记住一次形状对的总流程，再按 12 种已注册配对算出法线、分离量和接触点。
>
> 版本基线：`dc67c40ba2ce1ad384b0c985dbc1b539cba9221b` 对应的 3.2.0 工作树。下文以当前源码为准，不沿用其他版本的退化处理约定。

返回目录 · 下一章：SAT、裁剪与链段接触

本章地图：

1. 检测和响应分开
2. 总流程、五种形状、十二种配对
3. 单位约定
4. 步骤 1：过滤
5. 步骤 2：AABB
6. 步骤 3：相对变换
7. 步骤 4：查表分发
8. 家族 A：距离型
9. 家族 B：两段加双点
10. 家族 C：SAT
11. 步骤 6–7：世界锚点
12. 特征 ID
13. 练习
14. 源码索引

SAT 的轴选择和裁剪逐步推导在第 06 章；本章先把每种配对的入口、分支和数字例算完。

1. 先把碰撞检测和碰撞响应分开

碰撞检测回答几何问题：

- 两个形状是否足够接近？
- 应该沿哪个方向阻止它们继续靠近？
- 在什么位置施加约束，才能得到合理的转动效果？

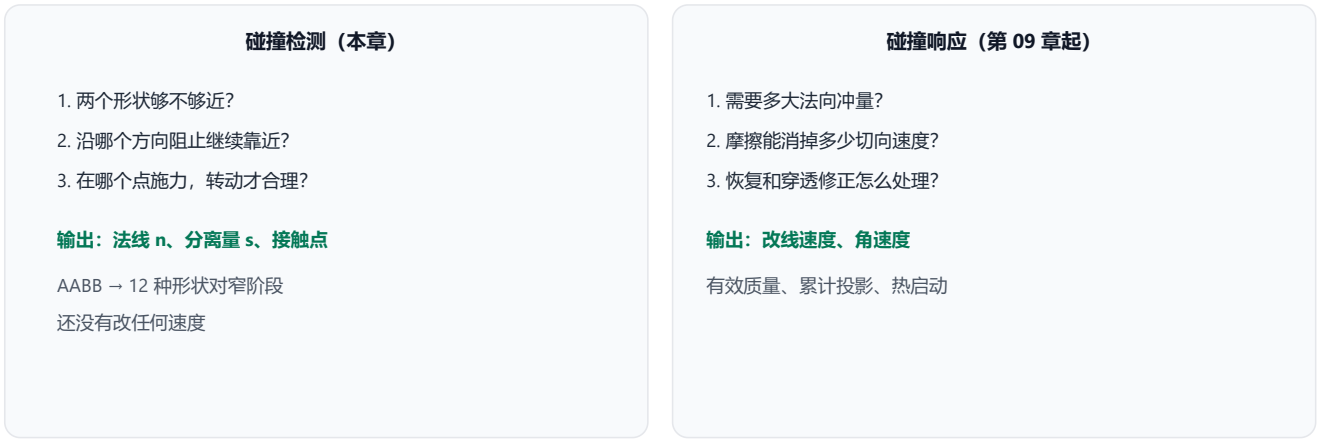
碰撞响应回答动力学问题：

- 需要多大的法向冲量？
- 摩擦可以消除多少切向相对速度？
- 应该怎样处理弹性恢复和穿透修正？

本章只完成第一组任务。

一个正确的接触点不等于已经算出了正确的碰撞冲量。

图 5-1　先几何、后动力学：接触点不等于已经算完冲量



看图： 本章只做左边。SAT、GJK 也属于左边。第 09 章才用冲量改运动。

2. 总流程：先看这张地图

当前引擎对**一个形状对**走下面七步。后面每一节都对应这里的一步或一种分支，不要跳着把公式从不同步骤拼在一起。

图 5-6　总流程：先筛候选，再按形状对做窄阶段

本章只做到局部流形和世界锚点。冲量从第 09 章开始。



每一步会分出的情况

1 过滤：层/组不允许，或双方都是静态 → 不建接触。

2 AABB：任一轴出现 lA > uB 或 lB > uA → 直接拒绝。两轴都重叠 ≠ 实体相交。

4 查表：12 种有函数；线段-线段、链段-线段、链段-链段没有函数，永不配对。

5 窄阶段：距离型最多 1 个点；面接触可 2 个点；分离量超过推测距离 0.02 m 通常返回空流形。

6 局部流形：点、法线、分离量都在 A 的局部系。7 再转成相对质心的世界方向锚点。

mover 走另一条平面查询路径，不是这 12 种接触流形，见第 15 章。

看图按箭头走： 蓝框是筛掉明显不可能的对，橙框才真正算几何，绿框把结果交给求解器。

步骤	问什么	可能的出口
1 过滤	这两类物体允许相碰吗?	不允许 → 不建接触
2 AABB	两个轴对齐盒子叠了吗?	任一轴分开 → 跳过窄阶段
3 相对变换	B 在 A 的局部系里在哪?	所有窄阶段函数都先做这一步

步骤	问什么	可能的出口
4 查表	这是 12 种配对里的哪一种?	3 种组合没有函数，直接不建接触
5 窄阶段	法线、分离量、0/1/2 个点	分离过大 → 空流形
6 局部流形	点在 A 局部系的什么位置?	输出 `b2LocalManifold`
7 世界锚点	相对各自质心的杆臂是什么?	交给约束求解器

这里的流形容量针对**一个形状对**：当前实现最多两个代表接触点。

一个刚体可以安装多个形状，因此两个刚体之间可能存在多个流形、超过两个接触点。

2.1 五种形状

源码枚举是 `b2ShapeType`：

类型	源码名	几何	半径
圆	`b2_circleShape`	一个圆心	圆半径
胶囊	`b2_capsuleShape`	两个端点连成轴段	轴段外再包一层圆盘半径
线段	`b2_segmentShape`	两个端点	半径为 0
多边形	`b2_polygonShape`	凸多边形顶点，逆时针	可带圆角半径
链段	`b2_chainSegmentShape`	链上的一段，外加左右幽灵顶点	半径为 0，并且是单面的

链不是第六种独立窄阶段类型。一条链被拆成许多 `b2_chainSegmentShape`，每段单独和别的形状配对。

角色控制器使用的 mover 查询走 `b2CollideMover*`，输出的是平面而不是接触流形，见第 15 章。不要把它算进下面这 12 种。

2.2 十二种已注册配对，三种永不碰撞

理论上 5×5=25 个有序格子。源码在 `b2InitializeContactRegisters` 里只注册了 **12** 个函数。交换 A/B 时，若 `primary==false`，`b2CreateContact` 会把两个形状对调后再调用同一函数。

另外 **3** 种组合的函数指针是空的，接触根本不会被创建：

- 线段–线段
- 链段–线段
- 链段–链段

图 5-7　5 种形状：12 种已注册，3 种永不碰撞

行是接触里的 A，列是 B。绿色=primary 函数；橙色=翻转 A/B 后再调同一函数；红叉=没有函数。



看图： 绿格是 primary 入口，橙格只是把 A/B 翻转后走同一条绿格，红格没有实现。

12 种配对按算法家族分成三组：

家族	配对	核心想法	点数
A 距离型	圆–圆、胶囊–圆、线段–圆、多边形–圆、链段–圆	先找最近点或最近边，再减半径	0 或 1
B 两段加双点	胶囊–胶囊、线段–胶囊	先求两轴段最近点，平行重叠再裁两点	0、1 或 2
C SAT 加裁剪	多边形–多边形、多边形–胶囊、线段–多边形、链段–多边形、链段–胶囊	最浅分离轴 + 参考边裁剪	0、1 或 2

2.3 六种配对先改写成已有形状

不是每一种格子都有一套全新公式。有 6 种会先改写，再调用上面已经存在的函数：

配对	实际调用	改写规则
线段–圆	`b2CollideCapsuleAndCircle`	线段当成 $r_A = 0$ 的胶囊
线段–胶囊	`b2CollideCapsules`	线段当成 $r_A = 0$ 的胶囊
线段–多边形	`b2CollidePolygons`	`b2MakeCapsule`，半径 0，两个顶点
多边形–胶囊	`b2CollidePolygons`	把胶囊做成带半径的两顶点多边形
链段–胶囊	`b2CollideChainSegmentAndPolygon`	同样把胶囊做成多边形
链段–圆	专用函数	不改写，但内部仍是点到段

图 5-13　六种配对不是新算法，而是先改写成已有形状再计算

读源码时先看它有没有调用 b2MakeCapsule，再决定该对哪一张表。



看图： 左边把半径改成 0，中间把胶囊升级成一条带半径的边，右边三种组合直接没有函数。

因此本章真正要手算的独立算法是 6 条：

1. 圆–圆
2. 点到线段（胶囊–圆、线段–圆都用它）
3. 多边形–圆
4. 链段–圆（单面 + 幽灵顶点）
5. 两段最近点（胶囊–胶囊的单点路径）
6. SAT + 裁剪（多边形和链段的面接触，详细步骤在第 06 章）

其余配对都是这 6 条的参数变化。

3. 本章的单位约定

量	含义	单位
p, a, b	点的位置	m
d, e, r	位移或力臂	m
R, r_A, r_B	半径	m
l_A, u_A	某一维上盒子 A 的下界、上界	m
l_B, u_B	同一维上盒子 B 的下界、上界	m
n	单位法线，从 A 指向 B	无量纲
s	有符号分离量	m
t	线段插值比例	无量纲
δ	推测距离 `B2_SPECULATIVE_DISTANCE`	默认 0.02 m

量	含义	单位
λ	冲量	kg·m/s

数值例采用默认米制尺度。

如果项目改变长度单位，必须同时检查输入几何、速度与相关阈值；不要理解成所有应用参数都会自动换算。

- $s > 0$：两表面还有间隙。
- $s = 0$：刚好相切。
- $s < 0$：已经穿透。

负分离量不是“负的欧氏距离”。欧氏距离仍非负；分离量额外表达了表面的相对侧别。

当前圆、胶囊、多边形窄阶段的共同门槛是：若 $s > \delta$，返回空流形。默认 $\delta = 0.02$ m。多边形双点裁剪还有自己的筛选，不能据此推出所有输出点都满足同一个硬上限。

4. 步骤 1：碰撞过滤

接触创建之前，先排除不允许相碰的形状对。常见出口有：

情况	结果
碰撞层 / 组过滤失败	不建接触
两个形状属于同一个刚体	不建接触
两个刚体都是静态	不建接触
至少一方是传感器	不走这 12 种流形函数，改走传感器重叠
类型表里函数指针为空	不建接触，见第 2.2 节那 3 格

这一步不计算距离。过滤通过后，宽阶段才用 AABB 继续筛。

5. 步骤 2：AABB 宽阶段

AABB 是与坐标轴平行的包围盒。源码结构是 `b2AABB`：一个下界点 `lowerBound`，一个上界点 `upperBound`。

二维 AABB A 可以写成：

$$A = [l_{Ax}, u_{Ax}] \times [l_{Ay}, u_{Ay}].$$

四个标量都是同一世界坐标系中的位置，单位为 m：

符号	读法	含义	源码字段
l_{Ax}	A 的 x 下界	盒子 A 最左边的 x	`a.lowerBound.x`

符号	读法	含义	源码字段
u_{Ax}	A 的 x 上界	盒子 A 最右边的 x	`a.upperBound.x`
l_{Ay}	A 的 y 下界	盒子 A 最下边的 y	`a.lowerBound.y`
u_{Ay}	A 的 y 上界	盒子 A 最上边的 y	`a.upperBound.y`

字母 l 是 lower（下界），u 是 upper（上界）。下标 A 标明这是哪个盒子，后面的 x 或 y 标明是哪一条坐标轴。盒子 B 同理写成 $l_{Bx}, u_{Bx}, l_{By}, u_{By}$。

有效 AABB 必须满足 $l_{Ax} \le u_{Ax}$ 且 $l_{Ay} \le u_{Ay}$，否则不是正常盒子。

5.1 从区间交集推导判定式

二维相交可以拆成两个独立的一维问题：x 区间是否重叠，以及 y 区间是否重叠。

讨论其中**一条轴**时，不再重复写 x 或 y，只写四个标量：

符号	读法	含义
l_A	A 的下界	盒子 A 在**当前这一维**上的下界
u_A	A 的上界	盒子 A 在**当前这一维**上的上界
l_B	B 的下界	盒子 B 在同一维上的下界
u_B	B 的上界	盒子 B 在同一维上的上界

l_A 不是新的物理量，只是把 l_{Ax} 或 l_{Ay} 的轴下标暂时省略。检查 x 方向时：

$$l_A = l_{Ax}, \quad u_A = u_{Ax}, \quad l_B = l_{Bx}, \quad u_B = u_{Bx}.$$

检查 y 方向时，改成对应的 y 分量即可。

于是当前这一维上，两个闭区间是 $[l_A, u_A]$ 和 $[l_B, u_B]$。它们相交，当且仅当：

$$\max(l_A, l_B) \le \min(u_A, u_B).$$

这又等价于：

$$l_A \le u_B, \qquad l_B \le u_A.$$

左边 $l_A \le u_B$ 的意思是：A 的左端（或下端）没有跑到 B 的右端（或上端）之外。右边 $l_B \le u_A$ 对称。两个方向各检查这两条，四个条件都成立才保留候选。

当前源码 `b2AABB_Overlaps` 写的是上述四个不等式的否定形式：任一方向出现 $l_B > u_A$ 或 $l_A > u_B$，就判定不相交。

使用小于等于号意味着边界刚好贴住也会进入后续检查。

5.2 完整数值例

设：

$$A = [-1, 2] \times [0, 1], \qquad B = [1.5, 3] \times [0.8, 2].$$

也就是：

$$l_{Ax} = -1, \quad u_{Ax} = 2, \quad l_{Ay} = 0, \quad u_{Ay} = 1,$$
$$l_{Bx} = 1.5, \quad u_{Bx} = 3, \quad l_{By} = 0.8, \quad u_{By} = 2.$$

对 x 方向取 $l_A = -1$、$u_A = 2$、$l_B = 1.5$、$u_B = 3$。因为 $-1 \le 3$ 且 $1.5 \le 2$，x 重叠。交集是：

$$[\max(-1, 1.5), \min(2, 3)] = [1.5, 2].$$

对 y 方向取 $l_A = 0$、$u_A = 1$、$l_B = 0.8$、$u_B = 2$。因为 $0 \le 2$ 且 $0.8 \le 1$，y 重叠。交集是：

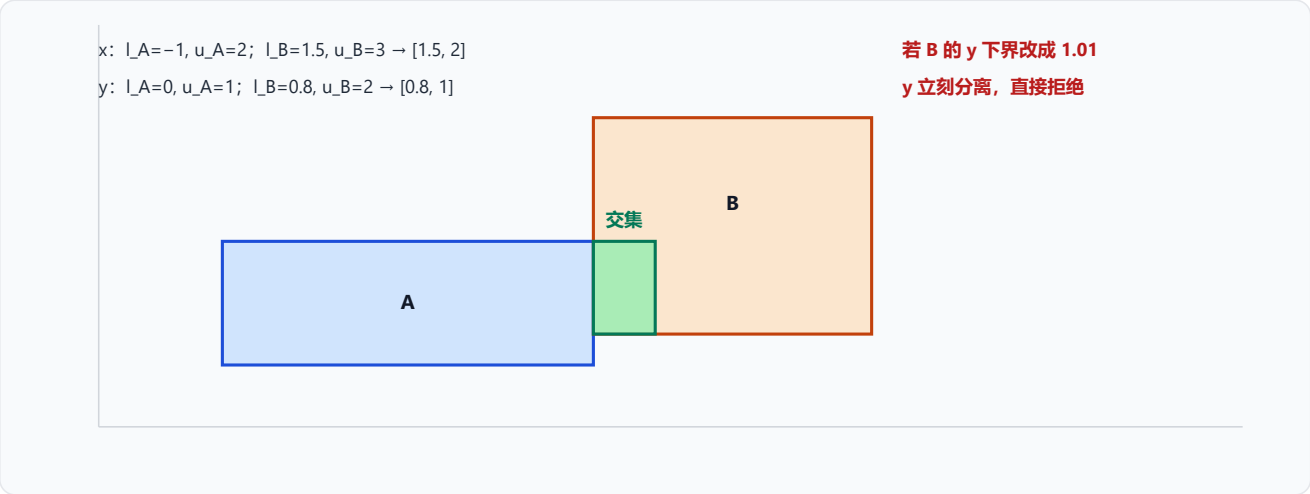
$$[\max(0, 0.8), \min(1, 2)] = [0.8, 1].$$

因此两框相交，应继续做窄阶段。

如果把 B 的 y 下界 l_{By} 改为 1.01，则 y 方向变成 $l_B = 1.01 > u_A = 1$，y 区间立即分离，可以直接拒绝。

图 5-2　AABB：x、y 两个区间都重叠才可能相交

l = 下界，u = 上界。检查某一维时，l_A 就是 A 在该轴上的下界。



看图： 绿色小块是交集 $[1.5, 2] \times [0.8, 1]$。图注里的 l_A, u_A 就是当前轴上 A 的下界和上界。x、y 只要有一边分开，整对形状就可以丢掉。

5.3 AABB 相交不代表实体相交

两个半径均为 1 的圆，圆心分别为：

$$a = (0, 0), \qquad b = (1.8, 1.8).$$

它们的 AABB 在两个方向上都重叠 0.2 m。

但圆心距离为：

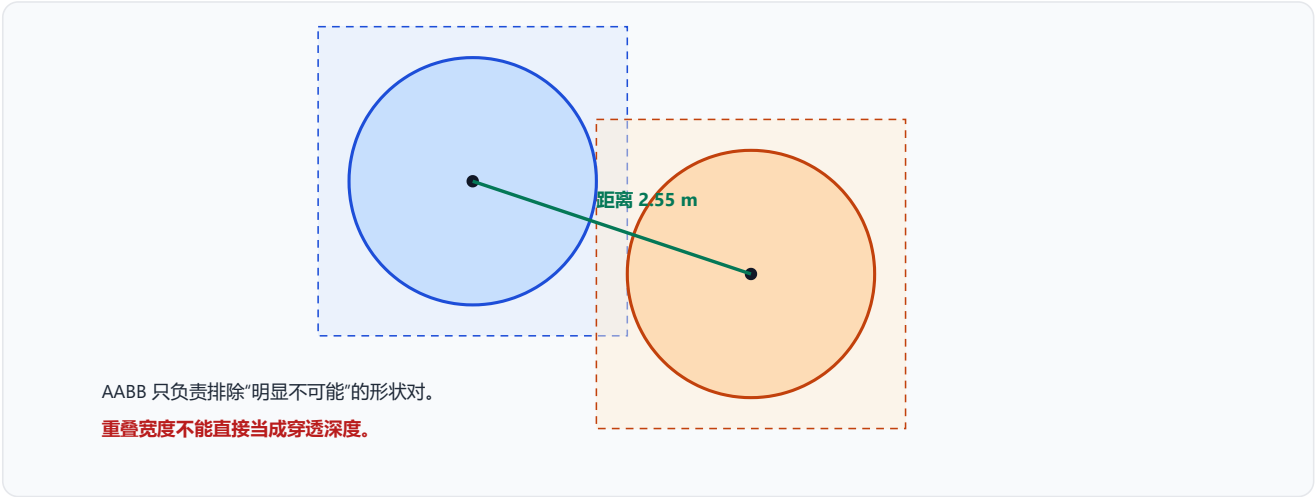
$$\|b - a\| = \sqrt{1.8^2 + 1.8^2} \approx 2.545584 > 2.$$

两个圆仍然分离。

因此 AABB 重叠宽度不能直接当作形状穿透深度。

图 5-3　包围盒相交，圆仍然可以分开

半径都是 1 m，圆心 (0,0) 与 (1.8,1.8)。AABB 各向重叠 0.2 m，但圆心距 ≈2.55 > 2。



看图： 虚线框重叠 0.2 m，圆心距却约 2.55 m。宽阶段只能筛候选，不能代替圆圆、SAT 或 GJK。

5.4 旋转盒为什么会得到更宽的 AABB

设盒子局部半宽、半高为 h_x, h_y，旋转角为 θ。

由旋转公式：

$$x' = \cos\theta\, x - \sin\theta\, y,$$

在 $|x| \le h_x$、$|y| \le h_y$ 中取极值，可得：

$$e_x = |\cos\theta| h_x + |\sin\theta| h_y,$$
$$e_y = |\sin\theta| h_x + |\cos\theta| h_y.$$

其中 e_x, e_y 是世界 AABB 的半尺寸，单位为 m。

代入 $h_x = 2$、$h_y = 1$、$\theta = 30^\circ$：

$$e_x \approx 2.232051, \qquad e_y \approx 1.866025.$$

这是盒子的教学闭式解。

当前通用多边形源码通过变换所有顶点，再逐坐标取最小值和最大值计算边界。

5.5 紧框、推测扩张与树的胖框不要混为一谈

一个包围盒按长度 m 扩张，意味着把它的二维角点沿两个轴各外扩 m：

$$l' = l - (m, m), \qquad u' = u + (m, m).$$

这里的 $l = (l_x, l_y)$、$u = (u_x, u_y)$ 就是源码里的 `lowerBound`、`upperBound`，单位仍是 m。它不是把所有坐标乘以某个倍率。

- 紧框覆盖当前实体。
- 推测距离扩张帮助保留即将接触的候选。

- 树的额外胖框减少代理频繁移动的成本。

更详细的动态树优化应在优化章节学习。

源码入口是 `src/geometry.c` 中的 `b2ComputeCircleFatAABB`、`b2ComputeCapsuleFatAABB`、`b2ComputePolygonFatAABB`。

多边形计算范围为 `430–450` 行；这里还使用更高精度中间量以及朝外舍入，避免包围盒因数值舍入变小。

6. 步骤 3：把 B 放到 A 局部系

所有窄阶段函数的第二个形状都带着相对变换 `xf`。它把 B 的局部点变到 A 的局部系：

$$Q_{AB} = Q_A^{-1} Q_B, \qquad p_{AB} = Q_A^{-1}(o_B - o_A),$$

$$b_A = p_{AB} + Q_{AB} b_L.$$

- o_A, o_B：两个刚体的世界原点。
- Q_A, Q_B：各自局部到世界的旋转。
- b_L：B 的局部点。
- b_A：同一个点在 A 系中的坐标。

先统一坐标，再计算距离。不能把 B 的局部顶点直接减去 A 的世界点。

代入： A 在原点且不旋转，B 的原点是 $(2, 0)$、也不旋转，B 的局部圆心是 $(0.8, 0)$。则 A 系中的圆心是 $(2.8, 0)$。后面圆圆、胶囊圆的数字例都已经默认完成了这一步。

步骤 7 会再把算好的局部接触点转回世界方向、相对质心的锚点。那是另一件事，不要和这里的“先对齐坐标系”混写。

7. 步骤 4：按类型表分发

AABB 通过后，源码用 `s_registers[typeA][typeB].fcn` 找到窄阶段函数。

三种出口：

1. 指针为空：不创建接触。就是那 3 种永不配对。
2. `primary==false`：交换两个形状，递归一次，保证 A 总是表里的第一参数。
3. `primary==true`：调用对应的 `b2Collide*`。

读某个接触时，先看它的 A/B 类型是不是已经按 primary 排好。若你在调试器里看到“圆和多边形”，实际调用的是 `b2CollidePolygonAndCircle`，多边形才是 A。

8. 家族 A：距离型窄阶段，最多一个点

这一族的共同模板是：

1. 在 A 上找到离 B 最近的骨架点或参考边。
2. 沿从 A 指向 B 的方向做单位法线。
3. 分离量 = 中心距离 − 半径和。
4. 若 $s > \delta$，返回空流形。
5. 否则取两个表面点的中点，输出 1 个接触点。

表面见证点和代表点：

$$c_A = a + r_A n, \qquad c_B = b - r_B n, \qquad p = \frac{c_A + c_B}{2}.$$

p 是对两侧表面对称的代表位置，方便建立约束力臂。不要把它误认成“必定同时位于两个真实表面上的点”。

8.1 共享工具：点到线段的三块区域

胶囊–圆、线段–圆、链段–圆都要先回答：圆心离这段骨架最近的点在哪里？

设线段端点为 p_0, p_1，查询点为 q。

$$e = p_1 - p_0, \qquad t_0 = \frac{(q - p_0) \cdot e}{e \cdot e}, \qquad t = \operatorname{clamp}(t_0, 0, 1), \qquad a = p_0 + te.$$

分子、分母都是 m^2，因此 t_0 无量纲。

条件	最近特征	最近点
$t_0 \le 0$	起点区域	p_0
$0 < t_0 < 1$	线段内部	垂足
$t_0 \ge 1$	终点区域	p_1

Voronoi 区域在这里就是“哪些查询位置以哪个特征为最近者”。

内部例： $p_0 = (0, 0)$，$p_1 = (4, 0)$，$q = (1.5, 0.75)$。

$$e = (4, 0), \quad e \cdot e = 16, \quad (q - p_0) \cdot e = 6, \qquad t_0 = \frac{6}{16} = 0.375, \qquad a = (1.5, 0).$$

点到段距离 0.75 m。

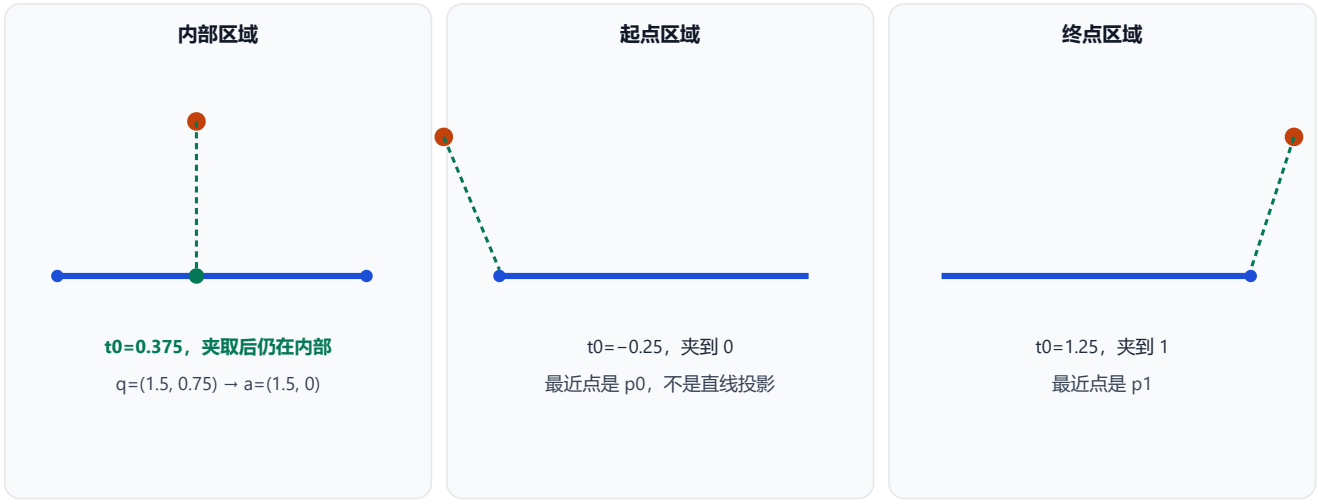
起点例： $q = (-1, 0.75)$，则 $t_0 = -0.25$，必须夹到 p_0。

终点例： $q = (5, 0.75)$，则 $t_0 = 1.25$，必须夹到 p_1。

不做夹取，相当于检测无限直线，会在实际线段之外制造碰撞。

图 5-5　点到线段：先投到无限直线，再夹到 [0,1]

段 p0=(0,0)，p1=(4,0)。内部点投影到段上；段外必须夹到端点，否则等于在检测无限直线。



看图： 中间那张才是常见的“垂足在段上”。左右两张必须夹到端点。

当前胶囊圆函数用两个点积避免多数除法：

$$s_1 = (q - p_0) \cdot e, \qquad s_2 = (p_1 - q) \cdot e.$$

- $s_1 < 0$：用起点。
- $s_2 < 0$：用终点。
- 否则 $t = s_1/(e \cdot e)$。

这里的 s_1, s_2 单位是 m^2，不能当成插值比例。正常胶囊输入应有有效轴长；该专用函数没有替任意零长度轴段保护所有除法。

8.2 配对 1：圆–圆

入口：`b2CollideCircles`，范围 `36–64`。

情况只有两种：

情况	条件	结果
够近	$s \le \delta$	1 个点，`id=0`
太远	$s > \delta$	空流形
同心退化	$\ell = 0$	当前源码法线可能是零向量

设 A、B 的圆心为 a, b，半径为 r_A, r_B，且已经在同一坐标系：

$$d = b - a, \qquad \ell = \sqrt{d \cdot d}, \qquad R = r_A + r_B, \qquad n = \frac{d}{\ell}, \qquad s = \ell - R.$$

代入： $a = (1, 2)$，$b = (2.8, 4.4)$，$r_A = 2$，$r_B = 1.2$。

第一步，中心差与长度：

$$d = (1.8, 2.4), \qquad \ell = \sqrt{1.8^2 + 2.4^2} = 3.$$

第二步，单位法线和分离量：

$$n = (0.6, 0.8), \qquad s = 3 - 2 - 1.2 = -0.2.$$

第三步，两个表面见证点：

$$c_A = (1, 2) + 2(0.6, 0.8) = (2.2, 3.6),$$
$$c_B = (2.8, 4.4) - 1.2(0.6, 0.8) = (2.08, 3.44).$$

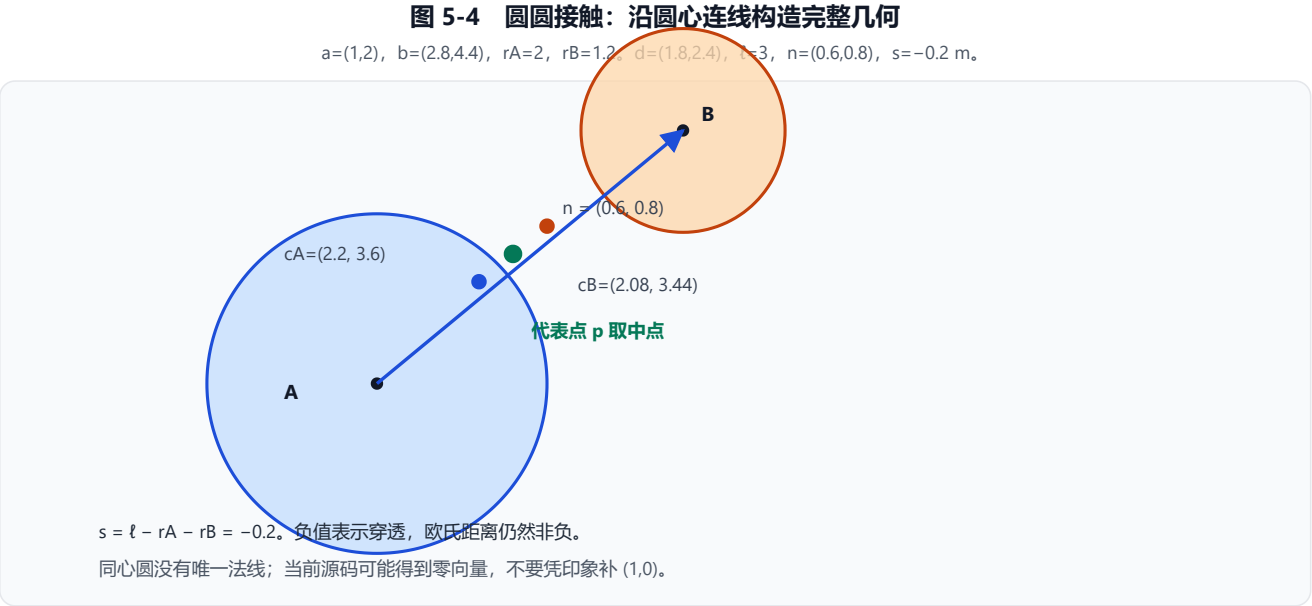
第四步，代表接触点：

$$p = \frac{(2.2, 3.6) + (2.08, 3.44)}{2} = (2.14, 3.52).$$

独立核对：

$$(c_B - c_A) \cdot n = (-0.12, -0.16) \cdot (0.6, 0.8) = -0.2.$$

图 5-4　圆圆接触：沿圆心连线构造完整几何



看图对四个数： $n = (0.6, 0.8)$，$s = -0.2$，见证点 c_A, c_B，绿色中点 p。

当 $a = b$ 时 $\ell = 0$，不存在唯一连线方向。`b2GetLengthAndNormalize` 在长度平方极小时返回长度零、向量 $(0, 0)$。当前工作树不会自动改成 $(1, 0)$。深度重合的初始化不要依赖这条常规接触法线。

8.3 配对 2：胶囊–圆

入口：`b2CollideCapsuleAndCircle`，范围 `68–124`。

三种情况，对应第 8.1 节的三块区域：

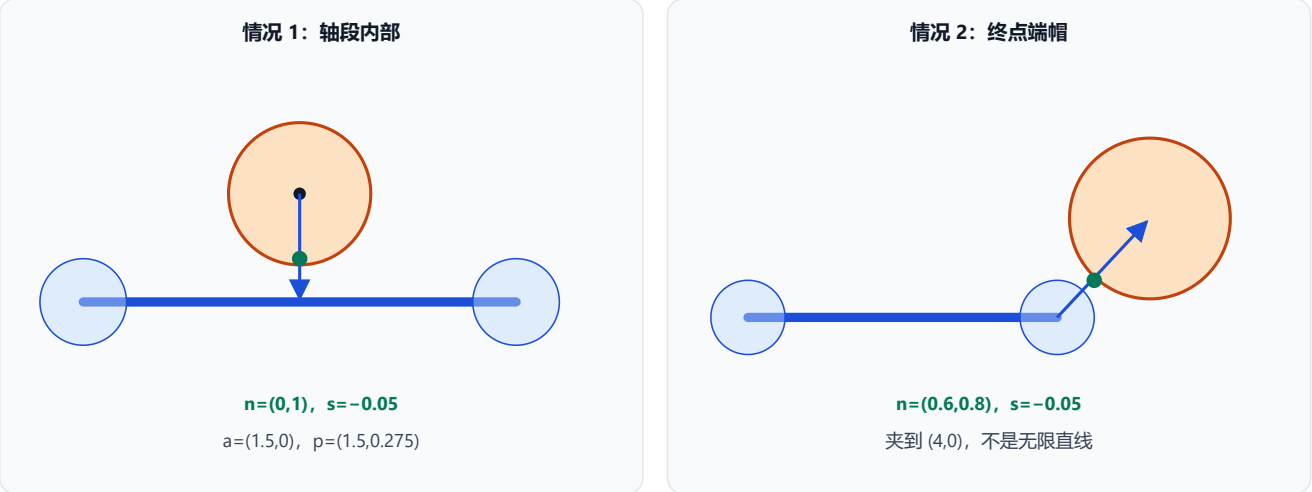
情况	最近骨架点	法线
轴段内部	垂足	通常垂直轴段
起点端帽	p_0	沿圆心到该端点
终点端帽	p_1	沿圆心到该端点

找到骨架点 a 之后，完全按圆圆计算：

$$s = \|q - a\| - r_A - r_B.$$

图 5-8　胶囊-圆：先找轴段最近点，再当圆圆来算

三种 Voronoi：内部、起点外、终点外。这里画出内部和终点两种。



看图： 左图法线竖着，右图法线沿着端点连线，不再垂直胶囊轴。

情况 1，轴段内部。 轴段 $(0,0) \to (4,0)$，$r_A = 0.3$，圆心 $q = (1.5, 0.75)$，$r_B = 0.5$。

由第 8.1 节，$a = (1.5, 0)$。于是：

$$n = (0,1), \qquad s = 0.75 - 0.3 - 0.5 = -0.05,$$

$$c_A = (1.5, 0.3), \quad c_B = (1.5, 0.25), \quad p = (1.5, 0.275).$$

情况 2，终点端帽。 同一胶囊，圆心改为 $(4.6, 0.8)$，$r_B = 0.75$。

$$t_0 = 1.15 \to t = 1, \qquad a = (4,0),$$

$$d = (0.6, 0.8), \quad \ell = 1, \quad n = (0.6, 0.8), \quad s = 1 - 0.3 - 0.75 = -0.05,$$

$$c_A = (4.18, 0.24), \qquad c_B = (4.15, 0.20), \qquad p = (4.165, 0.22).$$

8.4 配对 3：线段–圆

入口：`b2CollideSegmentAndCircle`。它把线段改写成半径 0 的胶囊，然后调用上一节。

情况与胶囊–圆相同，只是 $r_A = 0$，所以 c_A 就落在线段上。

代入： 段 $(0,0) \to (4,0)$，圆心 $(1.5, 0.4)$，$r_B = 0.5$。

最近骨架点 $a = (1.5, 0)$。

$$n = (0,1), \qquad s = 0.4 - 0 - 0.5 = -0.1,$$

$$c_A = (1.5, 0), \quad c_B = (1.5, -0.1), \quad p = (1.5, -0.05).$$

代表点略低于线段，是因为已经穿透：圆的下表面穿过了线段。这仍是中点公式，不是算错。

8.5 配对 4：多边形–圆

入口：`b2CollidePolygonAndCircle`，范围 `127–233`。

先对多边形每条边算圆心到该边的有符号距离，取最大者作为参考边。这就是“最浅分离边”。然后看圆心相对参考边两端的位置，分成三种情况：

情况	判定	法线来自
边区域	圆心投影夹在边两端之间，或圆心已进入多边形	该边外法线
顶点 1	投影落在起点外侧，且核心分离为正	圆心到该顶点
顶点 2	投影落在终点外侧，且核心分离为正	圆心到该顶点

设多边形无圆角 $r_A = 0$，顶点逆时针：

$$(0,0),\ (4,0),\ (4,2),\ (0,2).$$

外法线是 $(0,-1)$、$(1,0)$、$(0,1)$、$(-1,0)$。圆半径 $r_B = 0.5$。

边 i 的分离量是：

$$s_i = n_i \cdot (\text{圆心} - v_i).$$

图 5-9　多边形-圆：先找最浅分离边，再看圆心落在边还是角

盒子 (0,0)–(4,2)。左：边接触 s=−0.1。右：刚好贴在角点 (4,2)。



看图： 左边压在顶边上，右边刚好贴住右上角。

情况 1，边区域。 圆心 $(2, 2.4)$。

四条边的分离量：

$$s = (-2.4,\ -2,\ 0.4,\ -2).$$

最大值 0.4 出现在顶边。半径和加推测距离是 0.52，$0.4 \le 0.52$，继续。

顶边两端 $v_1 = (4,2)$、$v_2 = (0,2)$。两个沿边点积都是 $8 > 0$，圆心夹在边内部。因此用法线 $(0,1)$：

$$s = 0.4 - 0.5 = -0.1,$$

$$c_A = (2, 2), \quad c_B = (2, 1.9), \quad p = (2, 1.95).$$

情况 2，顶点区域。 圆心 $(4.3, 2.4)$。

四条边分离量变成 $(-2.4,\ 0.3,\ 0.4,\ -4.3)$，参考边仍是顶边。但圆心相对 $v_1 = (4, 2)$ 已经偏到外侧。于是改用角点连线：

$$n = \frac{(4.3, 2.4) - (4, 2)}{0.5} = (0.6, 0.8), \qquad s = 0.5 - 0.5 = 0.$$

两个表面点都是 $(4, 2)$，代表点也是 $(4, 2)$。这是刚好贴在角上的相切。

情况 3，圆心在多边形内部。 圆心 $(2, 1)$。四条边分离量都是负数，最大值 -1 出现在底边。源码走边分支，分离量为 -1.5，法线取底边外法线 $(0, -1)$。深度重合时这个结果能推出一个方向，但不适合当作稳定的初始姿态。

8.6 配对 5：链段–圆

入口：`b2CollideChainSegmentAndCircle`，范围 `1042–1122`。

链段比普通线段多两件事：只碰撞法线一侧；端点附近还要看左右幽灵顶点，避免相邻两段抢同一个角点。

法线取边向量的右垂线 `b2RightPerp`：$(x, y) \mapsto (y, -x)$。因此：

- 链从左向右 $(0, 0) \to (4, 0)$ 时，右法线朝下。
- 若地面要托住上方的物体，链必须从右向左画。

下面用一段朝上的地面：$p_1 = (4, 0)$，$p_2 = (0, 0)$，幽灵点 $\text{ghost1} = (6, 0)$，$\text{ghost2} = (-2, 0)$。右法线是 $(0, 4)$，朝上。圆半径 0.5。

情况	条件	结果
背面	圆心在法线另一侧	空流形
正面、段内	投影在开区间内	按点到段计算
正面、落在前一段 Voronoi	起点外侧且 `uPrev≤0`	空流形，交给上一段
正面、落在后一段 Voronoi	终点外侧且下一段认领	空流形，交给下一段

图 5-10　链段-圆：只碰撞法线一侧，端点还要看幽灵顶点

地面链从右向左 (4,0)→(0,0)，右手法线朝上。圆半径 0.5。



看图： 左图能碰，中图从地下穿上来被拒绝，右图这个角点属于虚线的前一段。

情况 1，正面接触。 圆心 $q = (2, 0.4)$。

$$e = (-4, 0), \qquad \text{offset} = (0, 4) \cdot ((2, 0.4) - (4, 0)) = 1.6 > 0.$$

两个沿边点积都是 $8 > 0$，投影在内部。最近骨架点 $p_A = (2, 0)$。

$$n = (0, 1), \qquad s = 0.4 - 0.5 = -0.1, \qquad p = (2, -0.05).$$

情况 2，背面拒绝。 圆心 $(2, -0.4)$。offset 变成 $-1.6 < 0$，直接返回空流形。链不是双边墙。

情况 3，幽灵区拒绝。 圆心 $(4.5, 0.3)$，已经偏到 p_1 右侧。前一段方向 $\text{ghost1} \to p_1$ 的点积 $u_{\text{prev}} = -1 \le 0$，本段放弃，让前一段来处理这个角。

9. 家族 B：两段距离与面接触

胶囊–胶囊和线段–胶囊都先求两条轴段的最近点。若两轴互相都能投影到对方内部，再尝试裁出两个点；否则退回单点。

9.1 共享工具：两条线段的最近点

设：

$$A(s) = p_1 + sd_1, \qquad B(t) = p_2 + td_2, \qquad s, t \in [0, 1].$$

令 $r = p_1 - p_2$，以及

$$a = d_1 \cdot d_1,\ e = d_2 \cdot d_2,\ b = d_1 \cdot d_2,\ c = r \cdot d_1,\ f = r \cdot d_2.$$

这些标量单位都是 m^2。非平行时：

$$s = \frac{bf - ce}{ae - b^2}, \qquad t = \frac{bs + f}{e}.$$

分母为零表示平行，不能套这个式子。

不能把两个无限直线参数各夹一次就结束。正确顺序之一是：

1. 先夹取 s。
2. 根据这个 s 求 t。
3. 若 $t < 0$，设 $t = 0$，再求 $s = \mathrm{clamp}(-c/a, 0, 1)$。
4. 若 $t > 1$，设 $t = 1$，再求 $s = \mathrm{clamp}((b - c)/a, 0, 1)$。

原因是：B 的最近特征一旦变成端点，A 上的最优位置也可能改变。

交叉例： $A : (0,0) \to (4,0)$，$B : (2,1) \to (2,-1)$。

$$d_1 = (4,0),\ d_2 = (0,-2),\ r = (-2,-1),$$
$$a = 16,\ e = 4,\ b = 0,\ c = -8,\ f = 2,$$
$$ae - b^2 = 64, \qquad s = 0.5, \qquad t = 0.5.$$

两个最近点都是 $(2,0)$，距离为零。此时不能用“两点之差除以距离”做法线。

`b2SegmentDistance` 的退化分支：

- 只有第一段退化：当点，求点到第二段。
- 只有第二段退化：当点，求点到第一段。
- 两段都退化：返回两个端点距离。
- 两段都有效：参数求解再重新夹取。

源码用机器精度相关的平方门限识别退化。这不等于“所有物理容差都应直接用机器 epsilon”。

9.2 配对 6：胶囊–胶囊

入口：`b2CollideCapsules`，范围 `237–495`。

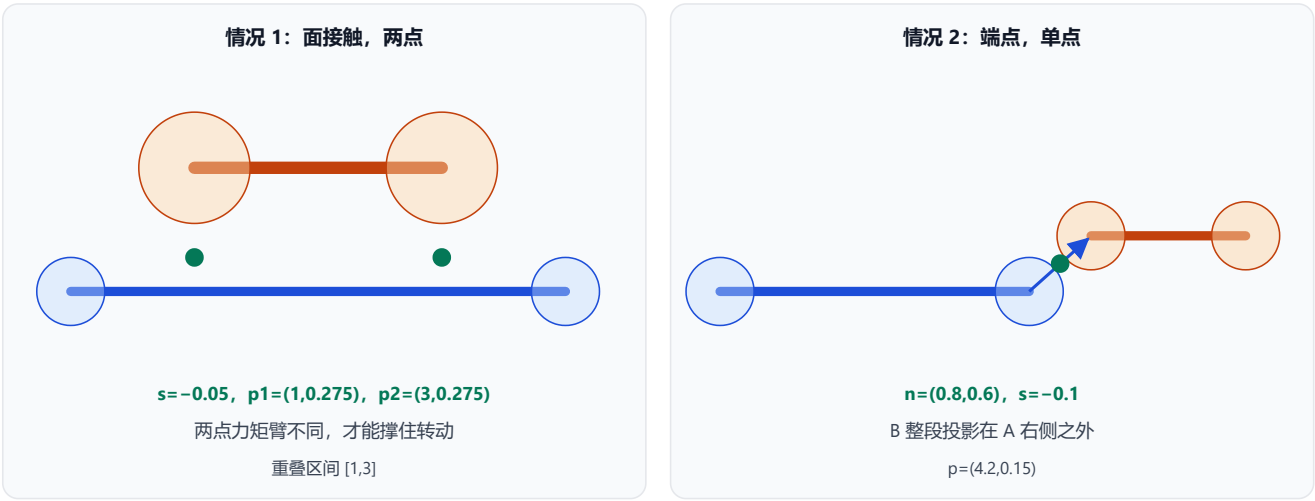
源码步骤：

1. 把 A 的一个轴端点平移到局部原点，减少大数相减。
2. 计算两轴段最近点。
3. 用平方距离拒绝 $s > \delta$ 的明显分离。
4. 检查两轴是否互相投影在对方内部。
5. 若是，用 SAT 选参考轴并裁两点。
6. 裁出来的点分离过大，或两轴并不互相覆盖，就退回单点。
7. 单点时若最近点重合，用法线取 A 轴的左垂线，而不是零向量。

专用函数断言两条轴段有效，不能把它等同于接受任意退化输入的通用段距函数。

图 5-11　胶囊-胶囊：先看两轴是否互相投影在内部

都在内部就尝试裁两点；任一端完全在外就只保留最近点。



看图： 左图两根平行轴重叠一段，必须留两个点；右图 B 整段都在 A 右边，只留端点那一个点。

情况 1，面接触、两个点。

$$A:(0,0)\to(4,0),\ r_A=0.3,\qquad B:(1,0.75)\to(3,0.75),\ r_B=0.5.$$

两轴都水平，距离 0.75 m，半径和 0.8 m。互相投影的重叠区间是 $[1,3]$。

$$s=0.75-0.3-0.5=-0.05,$$

$$p_1=(1,0.275),\qquad p_2=(3,0.275).$$

计算核对：参考法线取 A 的左垂线 $(0,1)$。入射轴两端都不必再裁。每个代表点沿法线从 B 轴走到两侧表面的中点：

$$y=0.75+\tfrac{1}{2}(0.3-0.5-0.75)=0.275.$$

两点力矩臂不同，才能表达这段支撑对转动的影响。只留中点 $(2,0.275)$ 会像把盒子放在一根针上。

情况 2，端点、一个点。

$$A:(0,0)\to(4,0),\ r_A=0.3,\qquad B:(4.4,0.3)\to(5.4,0.3),\ r_B=0.3.$$

B 的两个端点在 A 轴上的投影是 4.4 和 5.4，都大于 A 的轴长 4，所以 `outsideA==true`，不走双点裁剪。

最近骨架点是 A 的终点 $(4,0)$ 与 B 的起点 $(4.4,0.3)$：

$$d=(0.4,0.3),\quad \ell=0.5,\quad n=(0.8,0.6),\quad s=0.5-0.6=-0.1,$$

$$c_A=(4.24,0.18),\quad c_B=(4.16,0.12),\quad p=(4.2,0.15).$$

情况 3，两轴交叉。 第 9.1 节那对线段若都加上半径，最近点差为零。源码不用除零，改取 A 轴左法线。这和圆圆同心时返回零法线的路径不同。

9.3 配对 7：线段–胶囊

入口：`b2CollideSegmentAndCapsule`。线段改写成 $r_A=0$ 的胶囊，然后走上一节。

代入： 段 $(0,0)\to(4,0)$，胶囊 $(1,0.4)\to(3,0.4)$，$r_B=0.5$。

这仍是面接触。轴距 0.4 m，半径和 0.5 m：

$$s=0.4-0.5=-0.1,$$

$$p_1=(1,-0.05),\qquad p_2=(3,-0.05).$$

核对：代表点 $y=0.4+\frac{1}{2}(0-0.5-0.4)=-0.05$。因为线段没有厚度，中点略低于线段，含义与第 8.4 节相同。

10. 家族 C：SAT 与裁剪，本章先算出结果

多边形对、以及被改写成多边形的胶囊，都走 `b2CollidePolygons`。链段对多边形还要额外处理单面和幽灵顶点，入口是 `b2CollideChainSegmentAndPolygon`。

本章只做三件事：说出有哪些情况、用手算得到接触点、指出完整裁剪在第 06 章。不要在这里把 SAT 再推导一遍。

共同骨架：

1. 把 B 变到 A 局部系，并把 A 的一个顶点平移到原点。
2. 对 A、B 的每条边法线算分离量，取各自的最大值 s_A, s_B。
3. 若 $s_A > r_A + r_B + \delta$ 或 $s_B > r_A + r_B + \delta$，空流形。
4. $s_A \geq s_B$ 时 A 做参考形状，否则 B 做参考形状。
5. 在参考边上裁入射边，得到 1 或 2 个点。

边 i 的核心分离量是：

$$s_i=\min_j n_i\cdot(b_j-v_i).$$

先对另一形状的顶点取最小，再对所有边取最大。选的是较浅的那个负数，不是绝对值最大的那个。

10.1 配对 8：多边形–多边形

代入： A 的顶点 $(0,0),(3,0),(3,1),(0,1)$，B 的顶点 $(1,0.8),(2.5,0.8),(2.5,2.3),(1,2.3)$，两边半径都是 0。

A 四条外法线的分离量是 $(-2.3,\ -2,\ -0.2,\ -2.5)$，所以

$$s_A=-0.2,$$

对应顶边 $n=(0,1)$。

B 四条边的分离量是 $(-0.2,\ -2.5,\ -2.3,\ -2)$，所以

$$s_B=-0.2,$$

对应底边。当前源码在相等时偏向 A 做参考。

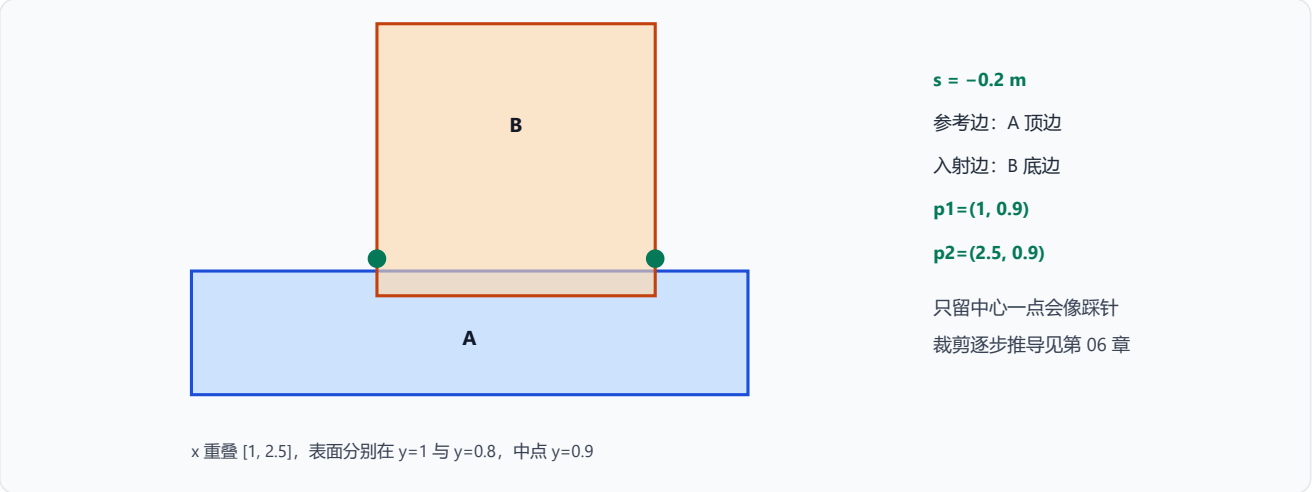
参考边是 A 的顶边 $y=1$，$x\in[0,3]$。入射边是 B 的底边 $y=0.8$，$x\in[1,2.5]$。两段在切向上的重叠是 $[1,2.5]$。

表面分别在 $y = 1$ 和 $y = 0.8$，中点 $y = 0.9$：

$$s = -0.2, \qquad p_1 = (1, 0.9), \qquad p_2 = (2.5, 0.9).$$

图 5-12　多边形-多边形：SAT 选出最浅轴，裁剪得到两点

A=[0,3]×[0,1]，B=[1,2.5]×[0.8,2.3]。两边法线里最浅的是竖直方向 −0.2 m。



看图： 绿色两点撑住这段重叠。若只在 $x = 1.75$ 放一个点，B 仍能绕它左右摇。裁剪的半平面写法见第 06 章第 5 节。

10.2 配对 9：多边形–胶囊

入口：`b2CollidePolygonAndCapsule`。`b2MakeCapsule` 把胶囊变成：

- 两个顶点 = 两个轴端点；
- 两条外法线 = 轴的左右垂线；
- 多边形半径 = 胶囊半径。

它不是把胶囊拆成一个矩形加两个圆，而是一条带半径的边。

代入： 盒子 A 的顶点 $(0,0), (4,0), (4,2), (0,2)$，半径 0。胶囊 B 的轴 $(1, 2.2) \to (3, 2.2)$，半径 0.3。

核心（不含半径）沿竖直方向的分离是 $2.2 - 2 = 0.2$。扣掉胶囊半径后：

$$s = 0.2 - 0.3 = -0.1.$$

重叠区间仍是轴在盒子顶边上的投影 $[1, 3]$。两个代表点取在两侧表面中点 $y = 1.95$：

$$p_1 = (1, 1.95), \qquad p_2 = (3, 1.95).$$

若胶囊竖着戳到盒子顶上，两轴不再互相覆盖，SAT 路径会退化成类似第 9.2 节的单点。

10.3 配对 10：线段–多边形

入口：`b2CollideSegmentAndPolygon`。线段先变成半径 0 的两顶点多边形，再走多边形–多边形。

代入： 盒子顶点 $(0,0), (4,0), (4,2.1), (0,2.1)$，线段 $(1,2) \to (3,2)$。

最浅分离在竖直方向：

$$s = 2 - 2.1 = -0.1.$$

重叠区间 $[1, 3]$，中点高度 2.05：

$$p_1 = (1, 2.05), \qquad p_2 = (3, 2.05).$$

线段没有厚度，代表点会落在线段和盒子顶边之间。

10.4 配对 11、12：链段–多边形，链段–胶囊

链段–多边形是专用函数，情况比普通多边形多：

情况	含义
质心在背面	整段拒绝，单面链
法线落在相邻凸角的跳过锥	本段不接管，避免鬼碰撞
凹角需要贴边	把法线吸附到本段法线
正面 SAT	与多边形类似，但参考边常是链段本身

链段–胶囊不另写一套：先把胶囊做成带半径的两顶点多边形，再走链段–多边形。

链段–胶囊数字例。 仍用第 8.6 节那条朝上的地面 $(4, 0) \to (0, 0)$。胶囊轴 $(1, 0.4) \to (3, 0.4)$，半径 0.5。质心大约在 $(2, 0.4)$，位于法线正面。轴距 0.4 m，半径 0.5 m：

$$s = -0.1, \qquad p_1 = (1, -0.05), \qquad p_2 = (3, -0.05).$$

若把同一胶囊放到 $y = -0.4$，质心在背面，空流形。这和链段–圆的单面规则一致。

链段–多边形数字例。 同一地面，盒子顶点 $(1, -0.1), (3, -0.1), (3, 1.4), (1, 1.4)$。盒子压进地面 0.1 m，质心约 $(2, 0.65)$ 仍在正面：

$$s = -0.1, \qquad p_1 = (1, -0.05), \qquad p_2 = (3, -0.05).$$

鬼碰撞、法线分类和裁剪细节见第 06 章后半。本章只要能分清：背面直接拒绝，正面才进入 SAT。

11. 步骤 6–7：从局部流形到世界方向质心锚点

`b2LocalManifold` 中的点和法线都处于 A 的局部坐标系。

为了避免混淆，记：

- o_A, o_B：两个刚体的世界原点。
- Q_A, Q_B：各自局部到世界的旋转。
- c_A, c_B：两个刚体的世界质心。
- p_L：流形中 A 局部接触点。

此节的 c_A, c_B 是质心，不再表示前面圆表面的见证点。

11.1 最终求解器锚点

代表接触点和世界法线为：

$$p_W = o_A + Q_A p_L, \qquad n_W = Q_A n_L.$$

最终质心相对锚点为：

$$r_A = p_W - c_A, \qquad r_B = p_W - c_B.$$

它们满足：

$$c_A + r_A = c_B + r_B = p_W.$$

这些锚点是**相对质心、沿世界坐标轴表达的位移**。它们不是刚体局部坐标，不应该再额外旋转一次。

11.2 完整数值例

设 A 的原点 $o_A = (10, 20)$，旋转 90°，局部接触点 $p_L = (2, -1)$。

先旋转：

$$Q_A p_L = (1, 2).$$

于是世界接触点为：

$$p_W = (11, 22).$$

若质心分别为 $c_A = (10, 21)$、$c_B = (12, 22)$，则：

$$r_A = (1, 1), \qquad r_B = (-1, 0).$$

两边加回质心都得到 $(11, 22)$。

局部法线 $(1, 0)$ 经同样旋转变成世界法线 $(0, 1)$。

11.3 当前源码分两阶段构造锚点

入口：`b2UpdateContact`，范围 `525–702` 行。

源码先算：

$$\tilde{r}_A = Q_A p_L, \qquad \tilde{r}_B = \tilde{r}_A + (o_A - o_B).$$

这是相对原点的世界方向位移。

随后在 `611–612` 行减去各自的世界方向质心偏移，得到最终 COM 锚点。

这种顺序避免先拼出巨大世界接触坐标、再与巨大质心坐标相减。

需注意内部时序：`577–587` 行的 `PreSolve` 位于减 COM 偏移之前。

阅读内部中间状态时必须说明阶段；不要把中间原点锚点与公开最终 COM 锚点混写。

12. 分离量、特征 ID 与冲量历史

几何数据和动力学状态保存在同一个接触点结构中，但含义不同。

`b2ManifoldPoint` 的主要字段包括：

- `separation`：几何分离量，单位 m。
- `normalImpulse`：法向冲量，单位 kg·m/s。
- `tangentImpulse`：切向冲量，单位 kg·m/s。
- `normalVelocity`：相对法向速度，单位 m/s，负值表示接近。
- `totalNormalImpulse`：跨子步及恢复过程的累计作用信息。
- `id`：当前形状对内的几何特征标识。
- `persisted`：本点是否与前一步的点匹配。

12.1 ID 不是世界唯一接触编号

当前打包方式位于 `src/manifold.c:14`：

$$\mathrm{ID} = (A_{8bit} \ll 8) \mid B_{8bit}.$$

例如 A 特征索引为 2、B 特征索引为 1，则 ID 为：

$$2 \times 256 + 1 = 513.$$

它的语义取决于当前形状对和具体生成路径。

圆圆、胶囊圆、多边形圆、链段圆这些单点路径使用 `id=0`。胶囊–胶囊的双点路径会带上轴端点编号。

12.2 ID 热启动和几何回收是不同机制

`contact.c:603–651` 将新旧点的 ID 比较。

匹配成功就迁移法向、切向冲量并设置 `persisted`。

这属于“新几何已经算好后，继承旧动力学状态”。

另一层接触几何回收可能直接复用旧锚点和法线，减少重新计算窄阶段。

不能把几何回收的位姿门限说成这里 ID 匹配复制冲量的必备条件。

12.3 正分离点不是已经发生撞击

圆圆创建点时允许 $s \le \delta$，所以 $s > 0$ 仍可能有推测接触点。

但多边形双点裁剪具有不同筛选路径，不能据此推出所有输出点都有 $s \le \delta$ 的统一硬上限。

下一章将给出一个实际算法可以输出 $s = 0.15$ m 的例子。

判断是否真正发生冲量作用，需要同时观察速度和冲量状态，而不是仅检查 `pointCount`。

13. 边界练习与答案

题一：Box2D 有多少种形状、多少种碰撞检测函数?

答案：5 种形状，12 种已注册流形函数。 另外 3 种组合（线段–线段、链段–线段、链段–链段）不创建接触。mover 查询不算在这 12 种里。

题二：两个圆的 AABB 相交，是否可以跳过圆圆距离检查?

答案：不可以。 圆心 $(0, 0)$、$(1.8, 1.8)$，半径都为 1 的例子已经构成反例。

题三：中心差是 $(3, 4)$ m，能否直接拿它当单位法线?

答案：不能。 单位法线为 $(0.6, 0.8)$。不归一化会改变投影和冲量的比例。

题四：点段投影比例为 1.4，为什么不能保留这个比例?

答案：线段仅允许 $[0, 1]$。 应夹到终点，不能使用延长线上的虚构点。

题五：线段–圆和胶囊–圆差在哪里?

答案：只差 r_A。 线段就是半径为 0 的胶囊。第 8.4 节把第 8.3 节的 r_A 改成 0，其余步骤相同。

题六：链从左向右画在地面上，为什么箱子会掉下去?

答案：右法线朝下，箱子在背面。 地面链应画成从右向左，或改用双边的线段/多边形。

题七：两轴段交叉时，两胶囊法线一定能由最近点之差归一化得到吗?

答案：不能。 最近点差可能为零，源码改用 A 轴左法线。

题八：两个刚体能否有六个接触点?

答案：可以。 每个刚体安装多个形状时，多个形状对可以各自产生流形。单个形状对最多两个点。

题九：`separation=0.01` m 且累计法向冲量为零，是否能宣称已经撞击?

答案：不能。 它可能只是尚未产生实际约束作用的推测候选。

题十：最终 `anchorA` 加回 A 的世界质心得到什么?

答案：该流形使用的代表接触点。 它不一定等于 A 的真实表面见证点。

题十一：改变多边形顶点编号后，旧特征 ID 和 GJK 索引缓存还应直接复用吗?

答案：不应。 索引对应的几何语义已经改变。

题十二：多边形–胶囊有没有自己的专用距离公式?

答案：没有。 它先把胶囊做成两顶点圆角多边形，再走 SAT。独立手算的是第 10.2 节那种“先改写、再当多边形”的路径。

14. 源码阅读索引

主题	当前入口与范围
类型表注册	`b2InitializeContactRegisters`，`194–212`
创建时翻转 A/B	`b2CreateContact`，`228–239`
圆圆	`b2CollideCircles`，`36–64`
胶囊圆	`b2CollideCapsuleAndCircle`，`68–124`
多边形圆	`b2CollidePolygonAndCircle`，`127–233`
胶囊胶囊	`b2CollideCapsules`，`237–495`
多边形多边形	`b2CollidePolygons`，`702–1028`
线段圆 / 线段胶囊 / 线段多边形	`1030–1040`，分别改写后转发
链段圆	`b2CollideChainSegmentAndCircle`，`1042–1122`
链段胶囊	`b2CollideChainSegmentAndCapsule`，改写后转发
链段多边形	`b2CollideChainSegmentAndPolygon`
把胶囊做成多边形	`b2MakeCapsule`，`16–34`
通用段距	`b2SegmentDistance`，`33–105`
多边形包围盒	`b2ComputePolygonFatAABB`，`430–450`
局部到世界锚点	`b2UpdateContact`，`525–702`
公开接触点字段	`b2ManifoldPoint`，`527–571`
极小向量归一化	`b2GetLengthAndNormalize`，`361–373`

建议阅读顺序：先按图 5-6 走完七步，再手算圆圆和点到线段，然后做多边形–圆的边/角两个数字，最后才进入第 06 章的 SAT 裁剪。

能指出当前形状对属于 12 格中的哪一格、走哪一族算法、会输出几个点，才算把这一章读完。

返回目录 · 下一章：SAT、裁剪与链段接触

06 • SAT、裁剪与链段接触

> 阅读目标：能逐轴算出多边形分离量，选定参考边和入射边，完成两次裁剪，并理解圆角与单面链段为什么还需要额外规则。
>
> 基线：`dc67c40ba2ce1ad384b0c985dbc1b539cba9221b` 对应的 3.2.0 工作树。多边形主流形路径是 SAT 加裁剪，而不是所有形状都统一调用 GJK。第 05 章已经列出 12 种配对和每种的数字结果；本章把 SAT、裁剪和链段鬼碰撞逐步算完。

返回目录 · 上一章：基础碰撞 · 下一章：GJK 与形状查询

1. 先明确输入条件与坐标约定

本章多边形顶点采用逆时针顺序，即 CCW。

参与一次窄阶段运算的顶点都已经放到 A 的局部坐标系。

除特别说明外，位置、半径、距离的单位均为 m。

边向量为：

$$e_i = v_{i+1} - v_i.$$

对于逆时针凸多边形，其单位外法线为：

$$n_i = \frac{(e_{iy}, -e_{ix})}{\|e_i\|}.$$

这表示沿有向边前进时的右侧。

法线无量纲，边向量单位为 m。

假定多边形是凸的、顶点有序、边长有效。

凹多边形应先分解为多个凸形状；不能把后面的凸多边形结论直接套到凹轮廓。

2. 最朴素的办法：枚举边相交还缺什么

一个容易想到的碰撞算法是枚举所有边对，检查线段是否相交。

但若 A 完全包住 B，两者边界可能完全不交，实体仍然重叠。

所以这种办法还要补上点包含检查。

分离轴定理提供了一种更统一的凸形判定方式：把二维问题变成多个一维区间问题。

3. SAT：在一条轴上比较投影区间

给定单位方向 n，A 的投影区间是：

$$[\alpha_{\min}, \alpha_{\max}] = \left[\min_i n \cdot a_i, \max_i n \cdot a_i\right].$$

B 的区间同理：

$$[\beta_{\min}, \beta_{\max}] = \left[\min_j n \cdot b_j, \max_j n \cdot b_j\right].$$

投影标量单位为 m。

若满足任一条件：

$$\alpha_{\max} < \beta_{\min}, \qquad 或 \qquad \beta_{\max} < \alpha_{\min},$$

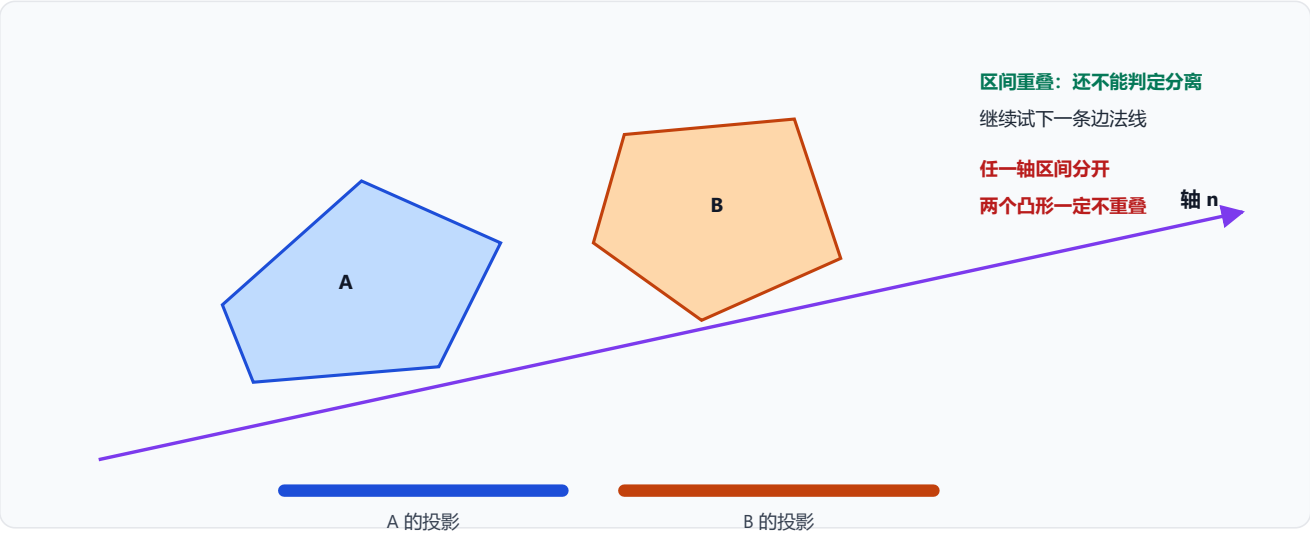
则两个形状一定分离。

对二维未圆角的凸多边形，检查两边所有边的法线即可。

只要找到一条分离轴，就不必继续建立接触流形。

图 6-1　SAT：沿一条轴看两个投影区间是否分开

凸多边形只要找到一条分离轴，就可以停止，不必再建立接触点。



看图想一维： 沿某条轴，A、B 各投影成一段区间。这段分开，二维里就一定分开。所有边法线都分不开，才认为核心可能重叠。

3.1 为什么只检查边法线

两个分开的凸集之间存在支撑分离直线。

对于多边形，极值由边或顶点实现；检查双方边法线可以覆盖凸核相交判定所需的候选方向。

这不意味着任意非凸形状也只需这些轴。

它也不意味着圆角轮廓的所有角角距离都能只靠核心边法线精确表达。

3.2 从投影区间化简为“边对顶点”

A 的一条边取顶点 v_i、外法线 n_i。

因为该边是 A 的支撑面：

$$\max_{a\in A} n_i \cdot a = n_i \cdot v_i.$$

于是该边对应的核心分离量为：

$$s_i = \min_j n_i \cdot (b_j - v_i).$$

第一层最小值找 B 沿这个方向最深入的点。

再对 A 的所有边取最大值：

$$s_A = \max_i s_i.$$

第二层最大值找最能分开两个形状的边。

“先最小、后最大”是两种不同目的，不能交换。

对 B 的所有边重复，得到 s_B。

3.3 负数中应该选择哪一个

设 A 在 x 上覆盖 $[-1,1]$，B 覆盖 $[0.8,2.8]$。

沿 $n=(1,0)$：

$$s = 0.8 - 1 = -0.2.$$

沿 $n=(-1,0)$：

$$s = -2.8 - 1 = -3.8.$$

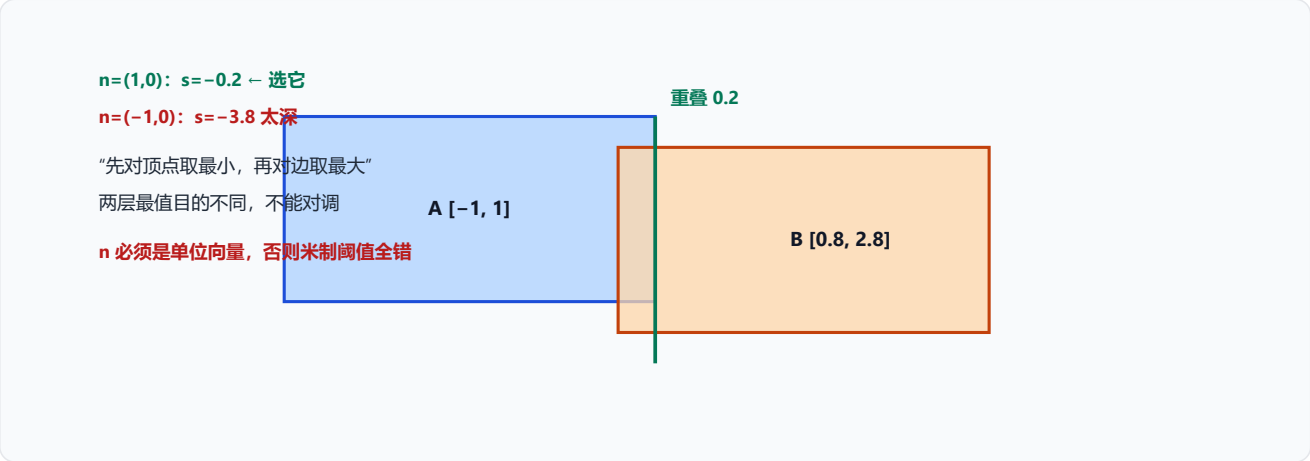
应该选择数值较大的 -0.2，表示较浅的穿透方向。

不是选绝对值最大的 -3.8。

如果错误使用 $n=(10,0)$，分离量会放大十倍，不能再与米制阈值直接比较。

图 6-2　已经重叠时：选“最浅”的穿透方向，不是最深

A 覆盖 [−1,1]，B 覆盖 [0.8,2.8]。向右 s=−0.2，向左 s=−3.8。推开沿 −0.2。



看图： 把 B 往左推 3.8 m 也能分开，但那是穿过整个 A。正确接触法线沿最浅穿透，这里是向右 0.2 m。

3.4 对照当前源码

入口：`b2FindMaxSeparation`，范围 `646–681` 行。

源码对每条参考边遍历另一多边形全部顶点，再选择最大的边分离量。

复杂度约为 $O(mn)$。

当前顶点数很少，简单、紧凑的循环具有良好的工程价值。

`b2CollidePolygons` 还先进行两项坐标处理：

1. 将 A 的第一个顶点平移到原点。
2. 将 B 的顶点和法线变换到这个局部系。

这样可以减少巨大世界坐标相减带来的精度损失。

整对形状的初筛条件是：

$$s_A > r_A + r_B + \delta \quad 或 \quad s_B > r_A + r_B + \delta.$$

其中 r_A, r_B 为圆角半径，δ 为推测距离，单位均为 m。

满足任一条件就返回空流形。

4. 为什么有法线之后还需要裁剪

SAT 可以提供合适的碰撞方向，但方向没有告诉我们在哪里施力。

两个不同位置的法向约束拥有不同力矩臂。

盒子平放时，只保留中心点可以挡住平移，却不能充分表达支撑区间两端对转动的影响。

二维边边接触沿切线方向形成一个区间。

取这个区间的两个端点，可以用少量约束近似面状接触。

这就是 SAT 后还需要裁剪的原因。

4.1 参考边和入射边

- **参考边**：来自当前最大分离的一侧。
- **入射边**：另一形状上法线最反向于参考法线的边。

当前多边形源码比较：

$$s_A \geq s_B.$$

成立时选 A 为参考形状，否则选 B。

相等时偏向 A。

此处不要套用其他版本中常见的比例容差参考边规则；当前 `747–788` 行使用上述直接比较。

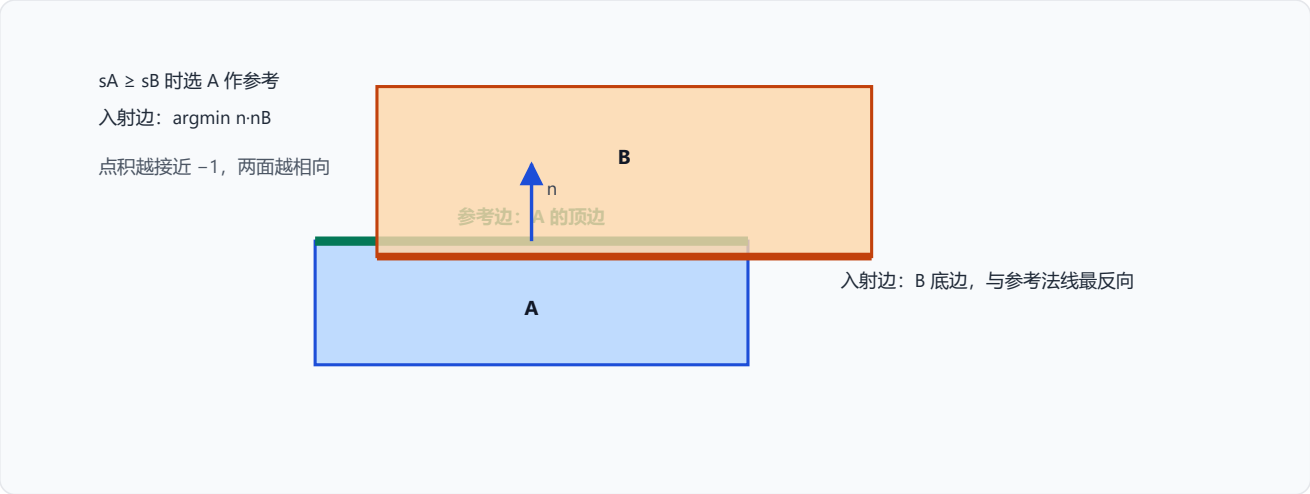
若参考法线为 n，入射边选择：

$$j = \arg\min_k n \cdot n_{Bk}.$$

单位法线点积越接近 -1，两面越相向。

图 6-3　SAT 给出方向后，还要用参考边和入射边找接触点

只挡住中心点能阻止平移，挡不住盒子绕支撑区间两端倾倒。



看图： 绿色粗线是参考边，橙色底边是入射边。只在盒子中心放一个接触点，平放的盒子会像踩在一根针上那样晃。

5. 先写通用的两半平面裁剪

设参考边起点、终点为 r_0, r_1。

定义长度和单位切向：

$$L = \|r_1 - r_0\|, \qquad t = \frac{r_1 - r_0}{L}.$$

候选点 p 应位于参考边两端之间：

$$0 \le t \cdot (p - r_0) \le L.$$

这里的两条裁剪线经过参考边端点，并垂直于参考边。

它们**不一定等于原多边形的两条邻边。**

5.1 一次裁剪的插值公式

入射线段写为：

$$x(u) = a + u(b - a), \qquad 0 \le u \le 1.$$

若裁剪边界要求：

$$t \cdot (x - r_0) = k,$$

代入可得：

$$u = \frac{k - t \cdot (a - r_0)}{t \cdot (b - a)}.$$

分子、分母单位都是 m，因此 u 无量纲。

分别用 $k = 0$ 和 $k = L$ 完成两侧裁剪。

结果仍然是一条线段，最多有两个端点。

5.2 平行与退化情况

若分母为零，入射边平行于当前裁剪边界。

此时：

- 整段在合法半平面内：保留。
- 整段在外：删除。
- 位于边界上：按闭区间约定保留。

不能继续除以零。

若两点非常接近，还需要注意生成两条几乎重复约束的成本和稳定性。

5.3 当前源码将两刀化为区间相交

入口：`b2ClipPolygons`，范围 `511–642` 行。

源码利用 CCW 边的方向关系，取：

$$t = (-n_y, n_x).$$

参考边区间是：

$$[0,\ t \cdot (v_{12} - v_{11})].$$

入射边方向与参考切向相反，所以入射区间写为：

$$[\mathrm{lower}_2, \mathrm{upper}_2] = [t \cdot (v_{22} - v_{11}),\ t \cdot (v_{21} - v_{11})].$$

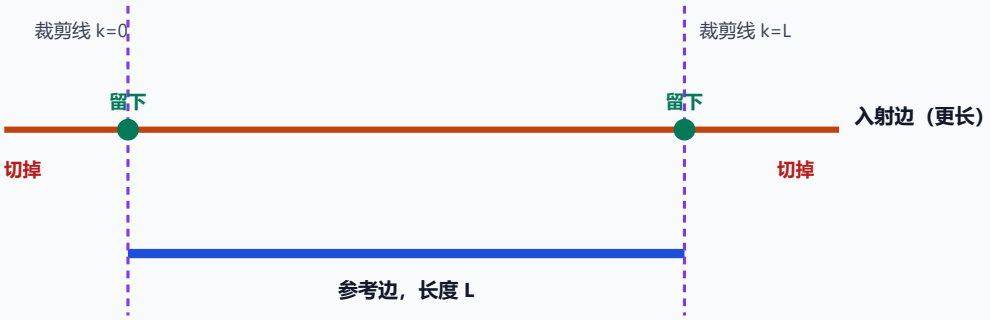
注意：这里 `v22` 对应较小投影端，`v21` 对应较大投影端。

源码在 `562` 行拒绝不相交区间，在 `568–585` 行计算被截断的两端。

这是专门的两边区间裁剪，不是一个通用多边形裁剪库。

图 6-4　裁剪：用参考边两端的“门框”切入射边

候选点必须落在参考边投影区间 [0, L] 内。两刀之后最多剩两个端点。



这两条裁剪线垂直于参考边，不一定等于原多边形的邻边。

平行退化时整段保留或整段删除，不能除以零。

看图： 紫色虚线是参考边两端的垂直门框。入射边更长时，两端切掉，中间留下最多两个接触点。

6. 完整数值例：八条轴、两次裁剪、半径与 ID

本例不跳过任何决定性步骤。

A 的核心顶点按 CCW 排列为：

$$a_0 = (-1, -1), \quad a_1 = (1, -1),$$
$$a_2 = (1, 0), \quad a_3 = (-1, 0).$$

B 的核心顶点为：

$$b_0 = (-2, -0.1), \quad b_1 = (2, -0.1),$$
$$b_2 = (2, 0.9), \quad b_3 = (-2, 0.9).$$

半径和推测距离取：

$$r_A = 0.05, \qquad r_B = 0.10, \qquad \delta = 0.02.$$

6.1 A 的四条轴

四边法线依次为下、右、上、左。

A 边索引	单位外法线	核心分离量
0	$(0, -1)$	-1.9
1	$(1, 0)$	-3
2	$(0, 1)$	-0.1
3	$(-1, 0)$	-3

例如 A 上边位于 $y = 0$：

$$s_2 = \min_j b_{jy} - 0 = -0.1.$$

A 底边位于 $y = -1$，法线向下：

$$s_0 = \min_j[-(b_{jy} + 1)] = -1.9.$$

因此：

$$s_A = -0.1, \qquad \text{edgeA} = 2.$$

6.2 B 的四条轴

B 边索引	单位外法线	核心分离量
0	$(0, -1)$	-0.1
1	$(1, 0)$	-3
2	$(0, 1)$	-1.9
3	$(-1, 0)$	-3

于是：

$$s_B = -0.1, \qquad \text{edgeB} = 0.$$

两侧都没有超过：

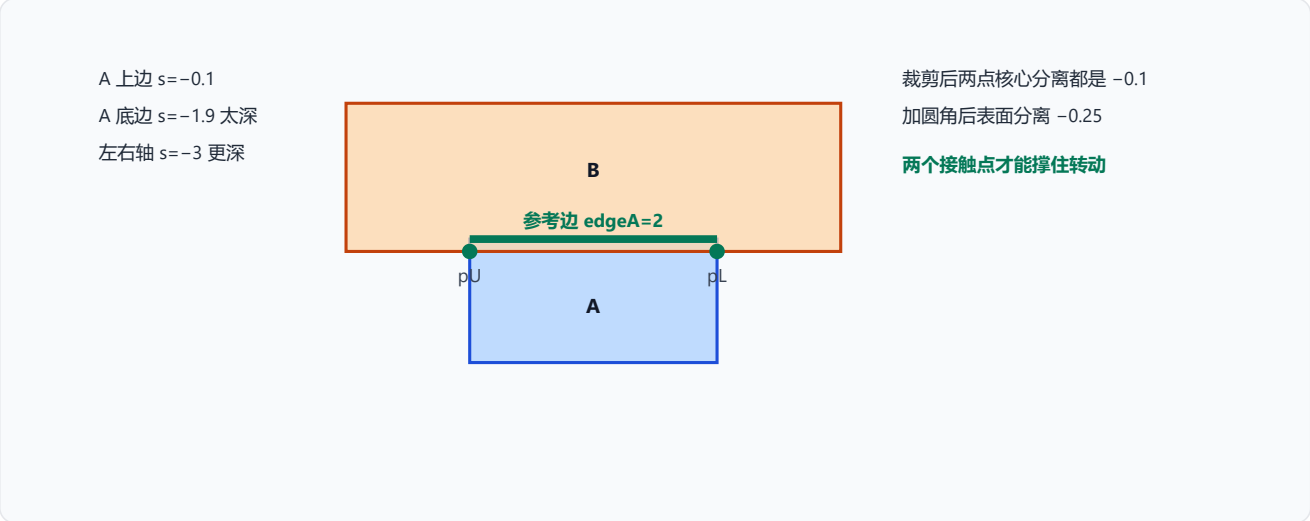
$$r_A + r_B + \delta = 0.17.$$

因为 $s_A = s_B$，选择 A 参考，`flip=false`。

B 底边法线 $(0, -1)$ 与 A 顶边法线 $(0, 1)$ 点积为 -1，因此正好是入射边。

图 6-5　完整算例：宽盒子压在矮盒子顶边上

A 核心 [−1,1]×[−1,0]，B 核心 [−2,2]×[−0.1,0.9]。双方最浅分离都是 −0.1，选 A 顶边作参考。



看图对表： A 上边和 B 底边都是 $s = -0.1$，左右轴 $s = -3$ 太深，扔掉。裁剪后两个绿点撑住转动。

6.3 建立切向区间

参考边顶点为：

$$v_{11} = a_2 = (1, 0), \qquad v_{12} = a_3 = (-1, 0).$$

法线、切向分别是：

$$n = (0, 1), \qquad t = (-1, 0).$$

参考区间上界：

$$\mathrm{upper}_1 = t \cdot (v_{12} - v_{11}) = 2.$$

入射边两点为：

$$v_{21} = b_0 = (-2, -0.1), \qquad v_{22} = b_1 = (2, -0.1).$$

所以：

$$\mathrm{lower}_2 = -1, \qquad \mathrm{upper}_2 = 3.$$

把入射区间 $[-1, 3]$ 裁到参考区间 $[0, 2]$。

6.4 第一刀：裁出较小投影端

插值比例是：

$$u_L = \frac{0 - (-1)}{3 - (-1)} = 0.25.$$

从 `v22` 向 `v21` 插值：

$$v_L = (1 - 0.25)v_{22} + 0.25v_{21} = (1, -0.1).$$

6.5 第二刀：裁出较大投影端

同理：

$$u_U = \frac{2 - (-1)}{3 - (-1)} = 0.75,$$

$$v_U = (1 - 0.75)v_{22} + 0.75v_{21} = (-1, -0.1).$$

两点的核心法向分离都是：

$$g = n \cdot (v - v_{11}) = -0.1.$$

6.6 加入圆角半径

最终表面分离量为：

$$s = g - r_A - r_B = -0.1 - 0.05 - 0.10 = -0.25.$$

参考表面见证点为：

$$c_R = v - gn + r_A n.$$

入射表面见证点为：

$$c_I = v - r_B n.$$

取中点可以化简为：

$$p = \frac{c_R + c_I}{2} = v + \frac{1}{2}(r_A - r_B - g)n.$$

代入后，法向偏移为：

$$\frac{1}{2}(0.05 - 0.10 + 0.10) = 0.025.$$

所以最终接触点是：

$$p_L = (1, -0.075), \qquad p_U = (-1, -0.075).$$

独立校验：参考表面高度 0.05，入射表面高度 -0.20，中点高度确为 -0.075。

6.7 特征 ID

源码在未翻转路径使用：

$$\mathrm{ID}_L = \mathrm{MakeID}(i_{11}, i_{22}),$$
$$\mathrm{ID}_U = \mathrm{MakeID}(i_{12}, i_{21}).$$

本例：

$$\mathrm{ID}_L = 2 \times 256 + 1 = 513,$$
$$\mathrm{ID}_U = 3 \times 256 + 0 = 768.$$

ID 表示几何来源，不是对接触点坐标做量化。

若参考形状是 B，源码会调整法线方向、点顺序和 ID 字段，最终公共法线仍必须从 A 指向 B。

7. 为什么一个输出点可以有较大的正分离量

“推测距离约 2 cm”不能直接改写成“每个流形点最多分离 2 cm”。

当前多边形裁剪在整对形状通过筛选后，并没有对每个输出点执行统一的这个删除条件。

下面给出合法凸形和完整轴检查。

7.1 几何输入

地面核心为：

$$A = [-5, 5] \times [-1, 0].$$

倾斜正方形 B 的顶点为：

$$b_0 = (-0.2, -0.05), \quad b_1 = (0.7, 0.15),$$
$$b_2 = (0.5, 1.05), \quad b_3 = (-0.4, 0.85).$$

双方半径取零。

B 的相邻边向量是 $(0.9, 0.2)$ 与 $(-0.2, 0.9)$。

它们点积为零、长度同为 $\sqrt{0.85}$，所以这确实是一个旋转正方形。

7.2 全部轴检查

轴来源	按边序得到的分离量，单位 m
A 四边	$-2.05, -5.4, -0.05, -5.7$
B 四边	$-1.090075551, -5.813736270, -2.977370534, -5.087019236$

这些数值已通过独立算术计算核对。

最大分离确实是 A 的上表面 -0.05，因此选地面作为参考，不是仅凭图形猜测。

7.3 裁剪输出

B 底边两个点的 x 都在 $[-5, 5]$ 内，不需要侧向截断。

两端核心分离分别为：

$$s_0 = -0.05, \qquad s_1 = +0.15.$$

接触中点是：

$$p_0 = (-0.2, -0.025), \qquad p_1 = (0.7, 0.075).$$

当前 `b2ClipPolygons` 的 `589–618` 行直接构造两点，没有逐点判断 $s > 0.02$ 就删除。

因此第二个点虽然仍高于地面 15 cm，仍可作为该流形的候选点。

候选点存在、几何已经接触、实际产生冲量，是三件不同的事。

求解器还要结合分离量和相对速度决定是否施加冲量。

不要把图中第二个候选点标成“也已经触地”。

8. 圆角多边形：核心加圆盘

圆角凸形可以写为：

$$K = P \oplus D(r).$$

其中 P 是凸核，$D(r)$ 是半径为 r 的圆盘。

沿任意单位方向 n，支撑值满足：

$$h_K(n) = h_P(n) + r.$$

原因是圆盘在任何方向上都能额外伸出 r。

因此面方向上可把核心分离量减去半径和：

$$s = g - r_A - r_B.$$

这不等于简单把每个顶点沿角平分线移动固定距离。

8.1 只检查核心边法线为何还不够

设 A 核心为 $[-1, 1]^2$，B 核心的左下角为 $(1.09, 1.09)$。

取：

$$r_A + r_B = 0.10, \qquad \delta = 0.02.$$

沿 x、y 两轴的核心分离都是：

$$0.09 \le 0.10 + 0.02.$$

所以核心轴初筛通过。

但最近角点距离是：

$$\ell = \sqrt{0.09^2 + 0.09^2} \approx 0.12727922.$$

表面距离为：

$$\ell - 0.10 \approx 0.02727922 > 0.02.$$

这对形状应该被更精确的角角距离检查拒绝。

8.2 当前角角修正路径

`b2CollidePolygons` 在 `794–900` 行处理核心可能分离的情况。

主要步骤是：

1. 对参考边和入射边求线段最近点。
2. 若最近距离减半径和仍超过推测距离，返回空。
3. 尝试通常的边裁剪。
4. 比较段距得到的分离与裁剪点的最小分离。
5. 若角角结果明显更合适，且最近参数确实落在两个端点，改用单点径向法线。

径向法线由两个最近角点之差归一化得到。

当前活跃实现位于 `#if 1` 分支；后面的 `#else` 代码不是本工作树实际执行路径。

9. 单面链段为什么会出现“幽灵碰撞”

假设把若干独立线段拼成平滑地面。

某个物体滑到线段接缝附近时，单独检测其中一段可能认定端点是最近特征。

端点指向物体的法线通常是斜的，于是产生不该存在的水平阻挡。

这就是 ghost collision：不是端点位置错了，而是把邻段已经覆盖的区域当成独立外边界。

9.1 ghost 顶点保存的是邻接信息

一个链段有：

- `ghost1`：前一段提供的邻接顶点。
- `point1`：当前段起点。
- `point2`：当前段终点。
- `ghost2`：后一段提供的邻接顶点。

真正碰撞的实体段只有 `point1 → point2`。

ghost 顶点不额外增加实体长度。

9.2 单面朝向

当前链段法线指向有向边右侧。

若希望水平地面上方可碰，段应从右向左。

例如：

$$p_1 = (2, 0), \qquad p_2 = (0, 0).$$

则：

$$e = (-2, 0), \qquad \mathrm{RightPerp}(e) = (0, 2).$$

这个方向指向上方。

链段圆碰撞先计算：

$$\mathrm{offset} = \mathrm{RightPerp}(e) \cdot (q - p_1).$$

若 $\mathrm{offset} < 0$，直接拒绝背面。

因为这里 e 未归一化，`offset` 单位是 m^2；源码只用符号，不把它当米制穿透量。

10. 完整 ghost 所有权数值例

取共线地面：

$$\mathrm{ghost1} = (4, 0), \quad p_1 = (2, 0),$$
$$p_2 = (0, 0), \quad \mathrm{ghost2} = (-2, 0).$$

圆心为 $q = (2.1, 0.2)$，半径 0.25。

10.1 先检查正面

$$\mathrm{offset} = (0, 2) \cdot (0.1, 0.2) = 0.4 > 0.$$

所以圆在链段正面。

10.2 判定当前段端点区域

$$v = e \cdot (q - p_1) = (-2, 0) \cdot (0.1, 0.2) = -0.2.$$

因为 $v \le 0$，圆位于当前段起点外侧。

若把当前段当孤立线段，最近点会取 p_1。

10.3 检查前段是否拥有这个区域

前段方向为：

$$e_{\text{prev}} = p_1 - \text{ghost1} = (-2, 0).$$

计算：

$$u_{\text{prev}} = e_{\text{prev}} \cdot (q - p_1) = -0.2 \le 0.$$

因此当前段不生成接触，交给前段处理。

10.4 如果没有 ghost 信息会怎样

孤立端点到圆心的向量是 $(0.1, 0.2)$。

其单位法线是：

$$n \approx (0.447214, 0.894427).$$

这在平地上引入了水平阻挡分量。

而前段把该圆视为段内部接触，正确法线是 $(0, 1)$。

10.5 另外两个边界检查

若圆心为 $(-0.1, 0.2)$，位于终点之外：

$$e_{\text{next}} = (-2, 0), \quad e_{\text{next}} \cdot (q - p_2) = 0.2 > 0.$$

当前段交给后段处理。

若圆心为 $(1, -0.2)$，则：

$$\text{offset} = (0, 2) \cdot (-1, -0.2) = -0.4,$$

直接作为背面拒绝。

若圆心为 $(1, 0.2)$，最近轴点是 $(1, 0)$，分离量为：

$$s = 0.2 - 0.25 = -0.05.$$

此时由当前段生成向上的正常接触。

10.6 等号归属不是随手写的

入口：`b2CollideChainSegmentAndCircle`，范围 `1042–1121`。

源码对前段使用 $u_{\text{prev}} \le 0$ 拒绝，对后段使用 $v_{\text{next}} > 0$ 拒绝。

这种边界所有权是非对称的。

不要只为形式对称就改动等号，否则共享边界可能被两个段争抢或同时放弃。

11. 链段多边形：GJK、法线分类、SAT 和裁剪协作

不能把这条路径简写成“链段与多边形只用 GJK”。

入口：`b2CollideChainSegmentAndPolygon`，范围 `1266–1640`。

当前处理可分为四层：

1. 读取前后邻边，判定凸角与单面背面。
2. 用不计半径的 GJK 得到核心距离及特征缓存。
3. 对候选法线执行接纳、跳过或吸附分类。
4. 必要时进行 SAT，并通过线段裁剪生成接触点。

GJK 只知道凸几何距离，不知道接缝所有权。

ghost 与单面拓扑必须在外层补充。

11.1 三种法线分类

`b2ClassifyNormal` 的范围是 `1226–1263` 行。

- `Admit`：接受这个角点或面方向。
- `Skip`：方向不属于当前段的合法区域，应跳过。
- `Snap`：将可能引发凹角内部阻挡的方向约束回主段法线。

这不是简单取相邻两边法线的平均值。

11.2 一个凸角许可例

设单位邻边方向为：

$$e_0 = (1, 0), \qquad e_1 = (0, 1).$$

叉积为：

$$e_0 \times e_1 = 1.$$

它通过源码的凸角门限。

对应前段法线为 $n_0 = (0, -1)$。

候选方向：

$$n = \frac{(1, -1)}{\sqrt{2}}.$$

它朝当前段尾部，并满足：

$$n \times n_0 = -\frac{1}{\sqrt{2}}.$$

因此通过当前凸角尾部方向检查。

若候选换为：

$$n = \frac{(-1,-1)}{\sqrt{2}},$$

则：

$$n \times n_0 = +\frac{1}{\sqrt{2}},$$

超过允许门限，进入 `Skip`。

如果邻接改成凹角，相关候选会进入 `Snap` 路径，而不是按凸角规则接受。

源码中的 `convexTol=0.01` 和 `sinTol=0.01` 作用在归一化方向上，是无量纲角度相关门限，不是 1 cm。

12. 推荐实现顺序

先完成正确、易观察的版本：

1. 验证凸多边形顶点、绕序与边长。
2. 遍历双方全部边法线执行 SAT。
3. 选择参考边和反向入射边。
4. 用通用两半平面裁剪求候选点。
5. 加入圆角半径与中点位置。
6. 检查 A 到 B 法线方向及 ID 来源。
7. 用独立数值例验证角角和面面接触。

正确后再添加：

- A 局部坐标与顶点原点平移。
- 专门的区间裁剪。
- 角角径向距离修正。
- 邻接链段法线许可规则。
- 特征缓存与接触回收。

不要从第一天就把所有优化和退化处理塞进一个不可观察的大函数。

13. 边界练习与答案

题一：SAT 中到底取最大还是最小？

答案：每条轴先取另一形状顶点的最小分离，再跨参考边取最大值。 两层各有不同几何含义。

题二：两个候选轴分离量为 -0.1 和 -2，选哪个？

答案：选 -0.1。 它对应更浅的穿透方向。

题三：裁剪参考边两端所用的线就是原多边形邻边吗？

答案：不是。 它们经过参考端点，并垂直参考切向。

题四：圆角只减半径和，是否就能用核心边法线精确完成所有判定？

答案：不能。 对角间隙例中还需要角角径向距离。

题五：一个流形点有 $s = 0.15$ m，是否一定说明窄阶段写错？

答案：不一定。 当前双点裁剪可能输出这种较远正分离候选；还要看该分支和后续冲量。

题六：链段从 $(0, 0)$ 指向 $(2, 0)$，正面朝哪边？

答案：向右行进的右侧，即 $y < 0$ 一侧。

题七：ghost 顶点是否给链段增加额外实体长度？

答案：不会。 它们提供邻接、法线和所有权信息。

题八：GJK 返回距离零，就可以直接用其法线处理链段吗？

答案：不能保证。 零距离时法线可能无效，链段还需要拓扑规则与 SAT 分支。

14. 源码阅读索引

主题	当前函数与范围
SAT 最大分离	`b2FindMaxSeparation`，`646–681`
多边形主流形	`b2CollidePolygons`，`702–1027`
两边裁剪	`b2ClipPolygons`，`511–642`
链段与圆	`b2CollideChainSegmentAndCircle`，`1042–1121`
链段法线分类	`b2ClassifyNormal`，`1226–1263`
链段与多边形	`b2CollideChainSegmentAndPolygon`，`1266–1640`

先把第 6 节完整算通，再用第 7 节检查“候选点不等于已触地”。

这两个例子比背下所有分支名称更重要。

返回目录 · 下一章：GJK 与形状查询

07 • GJK 距离与形状查询

> 阅读目标：从 Minkowski 差推导 GJK，手算点、线段、三角形单纯形，理解缓存和退化停止，并区分距离、射线、形状扫掠及角色移动平面查询。
>
> 基线：`dc67c40ba2ce1ad384b0c985dbc1b539cba9221b` 对应的 3.2.0 工作树。当前差点采用 A 减 B；射线初始重叠行为也不能照搬旧版资料。

返回目录 · 上一章：SAT 与裁剪 · 下一章：TOI 与连续碰撞

1. 先分清本章几个接口回答的问题

接口类别	问题	主要输出
接触流形	应在哪里建立接触约束?	法线、分离量、代表点
距离查询	两个凸形目前最近有多远?	非负距离、见证点
射线查询	一个零厚度点沿线走，会在哪里碰到形状?	比例、命中点、法线
形状扫掠	一个凸形只做指定平移，会在哪里接近目标?	比例、命中点、法线
mover 查询	胶囊角色应该受到哪些移动平面限制?	扫掠比例或平面
TOI	两个带旋转的 sweep 在何时接近?	状态、时间比例

它们共享几何运算，但不是同一个接口的不同名字。

和第 06 章 SAT 的分工可以先记成：

- **SAT + 裁剪**：已经知道两个凸多边形可能重叠，要找**法线和一两个接触点**，交给求解器。
- **GJK**：要问两个凸形**现在隔多远**，或射线/扫掠何时碰上。它找原点到 Minkowski 差的最近点，不直接给出盒子压在地面上的那两个支撑点。
- 当前多边形主流形路径是 SAT，不是“所有形状统一跑一遍 GJK 再当接触”。

本章所有距离计算默认在 A 局部坐标系中进行。

位置和半径以 m 计，插值比例与重心权重无量纲。

2. 从最朴素的凸形距离算法出发

可以枚举两个多边形的所有边对，计算线段最近距离。

再加上包含判断，便能处理很多凸多边形情况。

但不同形状组合需要不同分支，统一处理圆、胶囊、圆角多边形会越来越复杂。

另一种更统一、但仍很笨的思路是：

1. 生成所有顶点差 $a_i - b_j$。

2. 求这些差点的凸包。
3. 计算原点到这个凸包的距离。

GJK 保留第三步背后的数学关系，却避免真的生成全部差点。

它只按需要查询某个方向上的最远差点。

3. Minkowski 差：把两个形状变成一个原点问题

定义：

$$C = A - B = \{a - b \mid a \in A,\ b \in B\}.$$

如果 A、B 凸，C 也凸。

两个形状相交的条件可以写成：

$$A \cap B \neq \varnothing \quad \Longleftrightarrow \quad 0 \in A - B.$$

原因很直接：共同点满足 $a = b$，于是差为零。

距离关系同样直接：

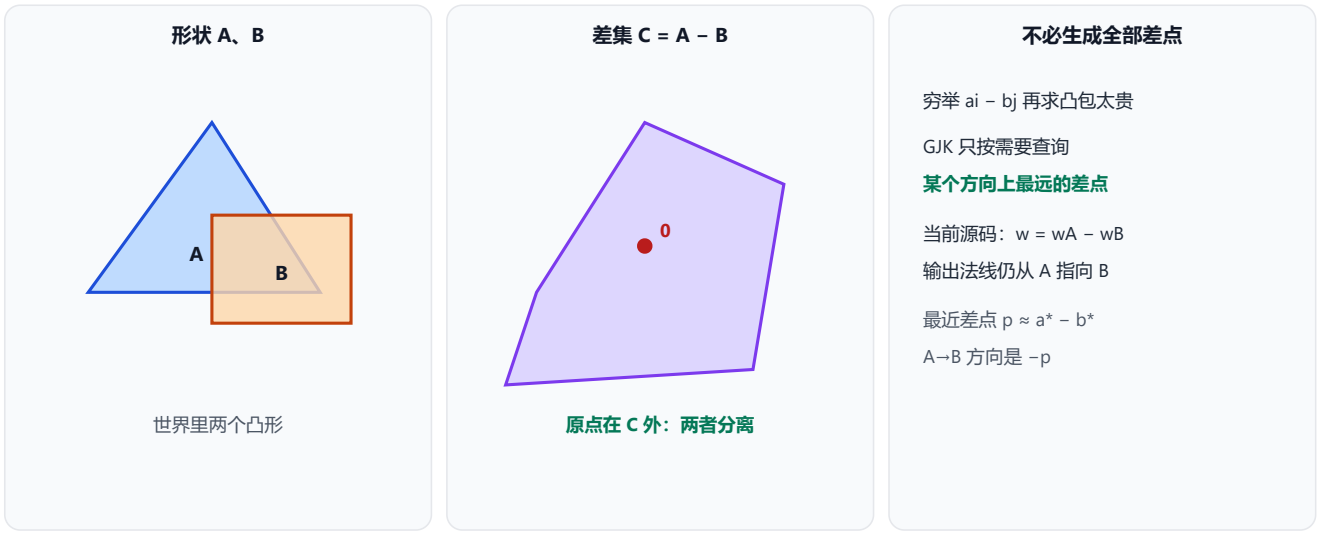
$$\operatorname{dist}(A, B) = \min_{a \in A, b \in B} \|a - b\| = \min_{w \in C} \|w\|.$$

所以我们要找的是差集里离原点最近的点。

所有差点 w 单位仍为 m。

图 7-1　Minkowski 差：两个形状的距离 = 原点到差集的距离

C = A − B。若存在 a=b，则差点为 0，所以相交 ⇔ 原点 ∈ C。



看图换问题： 不再问“A 离 B 多远”，改问“差集 C 离原点多远”。原点进了 C，两个凸核就已经重叠。

3.1 差集符号必须与法线方向一起核对

当前源码采用：

$$w = w_A - w_B.$$

若最近差点为 $p \approx a^* - b^*$，则从 A 指向 B 的方向是：

$$-p \approx b^* - a^*.$$

因此，采用 A 减 B 并不意味着输出法线从 B 指向 A。

当前输出法线仍为 A 到 B。

4. 支撑映射：不生成整个差集也能访问它

形状 A 沿方向 d 的支撑点定义为：

$$\text{support}_A(d) = \arg\max_{a \in A} a \cdot d.$$

对差集展开：

$$\max_{a,b}(a - b) \cdot d = \max_a a \cdot d + \max_b b \cdot (-d).$$

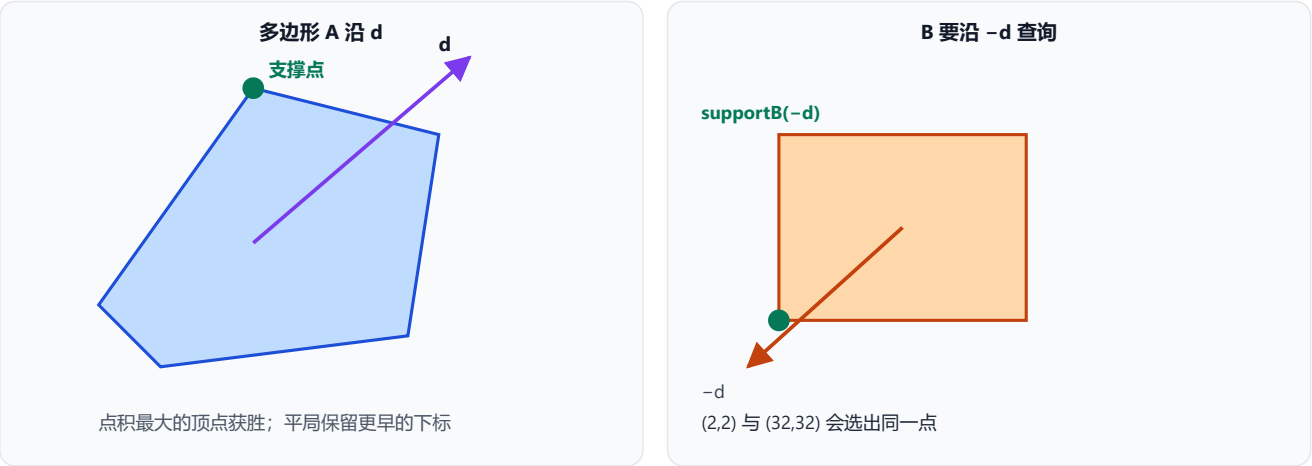
因此：

$$\text{support}_{A-B}(d) = \text{support}_A(d) - \text{support}_B(-d).$$

只需要对 A、B 各查询一次支撑点。

图 7-2　支撑点：沿 d 走到形状最远的那个顶点

差集支撑 = A 沿 d 的最远点 − B 沿 −d 的最远点。方向乘正数不改变结果。



看图： 差集支撑不用真的画出 C。A 沿 d 取最远点，B 沿 $-d$ 取最远点，相减即得。$(2, 2)$ 和 $(32, 32)$ 会选出同一个点。

4.1 搜索方向不必每轮归一化

对任何正数 k：

$$\text{support}_A(kd) = \text{support}_A(d).$$

方向乘正比例不改变点积排序。

因此 $(2, 2)$ 与 $(32, 32)$ 会选出相同支撑特征。

源码可能通过叉积构造非单位方向；其量纲也不必每轮都等于位移量纲。

算法利用的是方向和排序，不应强迫把每个中间方向都解释为“移动多少米”。

4.2 当前支撑查询的平局规则

入口：`b2FindSupport`，范围 `148–165`。

源码只在新点积严格大于当前最佳值时更新索引。

平局保留数组中较早的顶点。

这个细节会影响迭代轨迹和缓存索引，也使后面的数值例能够明确复现。

5. 单纯形：只保留最有用的一点、两点或三点

二维单纯形可以是：

- 一个点。
- 一条线段。
- 一个三角形。

GJK 不保留已经访问过的所有差点。

它不断删去不属于当前最近特征的点。

例如原点离三角形某条边最近，则对面的第三个顶点可以暂时丢掉。

若原点在三角形内部，已经证明两个凸核重叠。

5.1 重心权重的作用

单纯形内一点可以写成：

$$p = \sum_i \alpha_i w_i, \qquad \alpha_i \geq 0, \qquad \sum_i \alpha_i = 1.$$

其中 α_i 是无量纲重心权重。

每个差点都保存来源：

$$w_i = a_i - b_i.$$

因此可以还原原形状上的见证点：

$$p_A = \sum_i \alpha_i a_i, \qquad p_B = \sum_i \alpha_i b_i.$$

两者之差自动满足：

$$p_A - p_B = \sum_i \alpha_i (a_i - b_i) = p.$$

只保存差点而丢掉来源，通常无法恢复原形状上的见证点。

5.2 当前三点重叠输出的特殊处理

`b2ComputeWitnessPoints` 的范围是 `220–246`。

一点直接取来源点，两点按权重插值。

三点内部重叠时，当前代码计算 A 点后直接令 B 点等于 A 点。

这避免有限精度下两侧独立重建产生微小不一致。

不要把所有分支都描述成分别独立加权重建 A、B。

6. 一点单纯形：最简单的情况

若只有 w_1，最近点就是：

$$p = w_1.$$

下一搜索方向为：

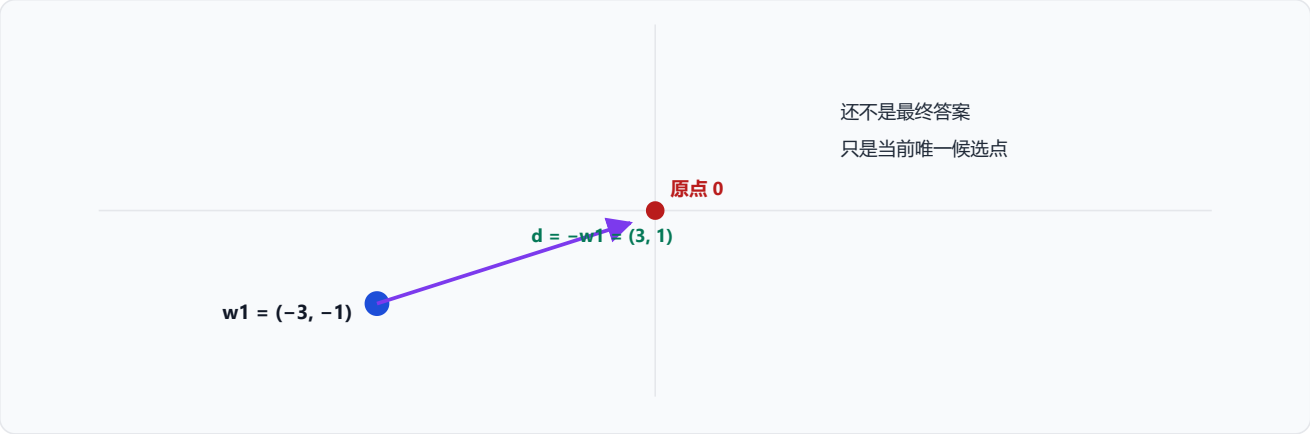
$$d = -w_1.$$

例如 $w_1 = (-3, -1)$，就朝 $(3, 1)$ 方向查询新的差集支撑点。

其目的不是直接走到原点，而是寻找能够让当前单纯形更靠近原点的极点。

图 7-3　一点单纯形：下一方向直接指向原点

w1=(−3,−1) 时，朝 d=(3,1) 再查一个新差点。目的是让单纯形更靠近原点。



看图： $w_1 = (-3, -1)$ 还不是答案。朝 $(3, 1)$ 再要一个支撑点，单纯形才能更靠近 0。

7. 两点单纯形：把点段距离中的查询点换成原点

设：

$$e = w_2 - w_1, \qquad p(t) = w_1 + te.$$

最小化 $\|p(t)\|^2$，得到：

$$t^* = -\frac{w_1 \cdot e}{e \cdot e}.$$

源码引入两个未归一化权重：

$$d_{12,2} = -w_1 \cdot e, \qquad d_{12,1} = w_2 \cdot e.$$

它们满足：

$$d_{12,1} + d_{12,2} = e \cdot e.$$

两个 d 的单位为 m^2。

7.1 三块区域

如果：

$$d_{12,2} \le 0,$$

则最近点是 w_1，保留权重 $(1, 0)$。

如果：

$$d_{12,1} \le 0,$$

则最近点是 w_2，保留权重 $(0, 1)$。

否则原点投影在线段内部：

$$\alpha_1 = \frac{d_{12,1}}{d_{12,1} + d_{12,2}},$$

$$\alpha_2 = \frac{d_{12,2}}{d_{12,1} + d_{12,2}}.$$

对应函数为 `b2SolveSimplex2`，范围 `274–306`。

7.2 端点区域例

取 $w_1 = (2, 1)$、$w_2 = (4, 1)$。

则：

$$e = (2, 0), \qquad d_{12,2} = -(2, 1) \cdot (2, 0) = -4.$$

最近点是 w_1，不能使用无限直线投影点 $(0, 1)$。

7.3 线段内部例

取：

$$w_1 = (-3, -1), \qquad w_2 = (-1, -3).$$

于是：

$$e = (2, -2), \quad d_{12,1} = 4, \quad d_{12,2} = 4.$$

所以：

$$\alpha_1 = \alpha_2 = 0.5, \qquad p = (-2, -2).$$

当前最近距离为：

$$\|p\| = \sqrt{8} \approx 2.828427.$$

教学上可以取下一方向 $-p = (2, 2)$。

源码使用叉积表达式得到同向的 $(32, 32)$。

两者正比例，因此支撑查询结果相同。

图 7-4　两点单纯形：原点可能落在端点外，也可能落在线段上

左：w1=(2,1)、w2=(4,1)，最近点是 w1，不能用无限直线上的 (0,1)。右：内部例 p=(−2,−2)。



看图对比： 左图若用无限直线，会得到虚假最近点 $(0, 1)$；右图权重各一半，$p = (-2, -2)$，距离 $\sqrt{8}$。

8. 三点单纯形：七块 Voronoi 区域

三角形有三块顶点区域、三块边区域和一块内部区域。

不是每次拿到三个点就已经包围原点。

需要检查最近特征到底落在哪里。

8.1 有向面积与内部权重

定义：

$$D = (w_2 - w_1) \times (w_3 - w_1).$$

再定义：

$$T_1 = D(w_2 \times w_3),$$
$$T_2 = D(w_3 \times w_1),$$
$$T_3 = D(w_1 \times w_2).$$

D 单位为 m^2，T_i 单位为 m^4。

额外乘上 D 可以统一顺时针、逆时针单纯形的符号判断。

内部权重为：

$$\alpha_i = \frac{T_i}{T_1 + T_2 + T_3}.$$

但必须先排除顶点和边区域，不能对任意三角形直接套这组内部权重。

8.2 当前区域判定的结构

`b2SolveSimplex3` 的范围是 `309–418`。

先对三条边分别构造和上一节类似的投影量。

然后检查：

区域	判断含义
顶点 1	对边 12、13 都投影到顶点 1 外侧
边 12	沿边投影在内部，且原点位于顶点 3 对侧
边 13	沿边投影在内部，且原点位于顶点 2 对侧
顶点 2	两条相邻边都支持顶点 2 最近
顶点 3	两条相邻边都支持顶点 3 最近
边 23	沿边投影在内部，且原点位于顶点 1 对侧
三角内部	上述外部区域均未命中

例如边 12 的源码条件为：

$$d_{12,1} > 0, \quad d_{12,2} > 0, \quad T_3 \le 0.$$

前两个条件确保投影不在线段端点之外，第三个条件检查三角形侧别。

8.3 三角形外的边区域例

设：

$$w_1 = (-1, 2), \quad w_2 = (1, 2), \quad w_3 = (0, 4).$$

有：

$$D = 4, \qquad d_{12,1} = d_{12,2} = 2,$$
$$T_3 = 4[(-1, 2) \times (1, 2)] = -16.$$

因此最近特征是边 12。

权重和最近点为：

$$(\alpha_1, \alpha_2, \alpha_3) = (0.5, 0.5, 0), \qquad p = (0, 2).$$

应删除第三个顶点，距离为 2 m。

8.4 包围原点的内部例

设：

$$w_1 = (1, 1), \quad w_2 = (-2, 1), \quad w_3 = (1, -2).$$

得到：

$$D = 9, \qquad T_1 = T_2 = T_3 = 27.$$

所以：

$$\alpha_1 = \alpha_2 = \alpha_3 = \frac{1}{3},$$

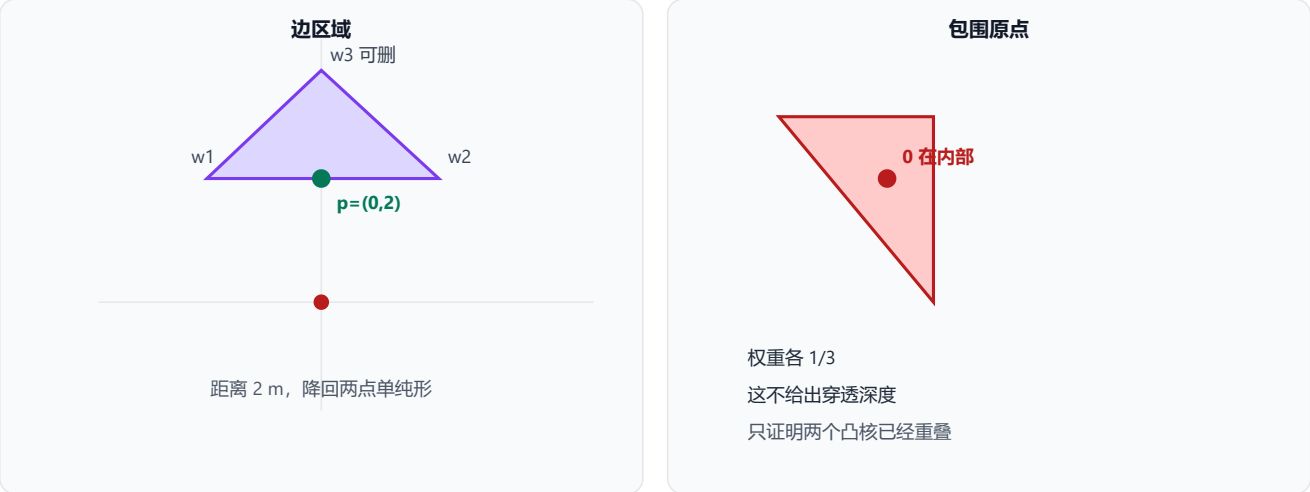
$$p = \frac{1}{3}(1, 1) + \frac{1}{3}(-2, 1) + \frac{1}{3}(1, -2) = (0, 0).$$

原点被单纯形包围，两个凸核重叠。

这不直接提供穿透深度，也不意味着两个形状只在一个点相切。

图 7-5　三点单纯形：不是拿到三角形就等于重叠

左：最近特征是边 12，删掉对面顶点。右：原点在内部，核心距离为零。



看图： 左边应删掉对面顶点，降回线段；右边三个权重都是 $1/3$，只证明重叠，不给出 SAT 那种穿透深度。

8.5 共线退化

三点共线时 $D = 0$，不能直接使用内部权重分母。

教学实现应明确降为最近的线段或点。

当前代码先尝试各 Voronoi 分支，主循环还检查过小搜索方向；不要宣称它依靠一个“万能三角形除法”处理全部病态输入。

9. 完整数值迭代表一：原点到三角形的距离

A 只有一个点：

$$a_0 = (0, 0).$$

B 的顶点按数组顺序为：

$$b_0 = (3, 1), \quad b_1 = (1, 3), \quad b_2 = (3, 3).$$

双方半径为零，相对变换为单位变换。

9.1 初始化

空缓存使用双方第 0 点：

$$w_1 = a_0 - b_0 = (-3, -1).$$

当前单点距离为 $\sqrt{10}$，还不是最终答案。

搜索方向：

$$d = (3, 1).$$

9.2 第一次支撑查询

B 要沿 $-d = (-3, -1)$ 查询。

B 顶点	与 $(-3, -1)$ 的点积
$b_0 = (3, 1)$	-10
$b_1 = (1, 3)$	-6
$b_2 = (3, 3)$	-12

最大值为 -6，选 b_1。

新差点为：

$$w_2 = a_0 - b_1 = (-1, -3).$$

9.3 解线段单纯形

由第 7 节的计算：

$$\alpha_1 = \alpha_2 = 0.5, \qquad p = (-2, -2).$$

源码下一方向为 $(32, 32)$。

9.4 第二次支撑查询并停止

B 沿 $(-32, -32)$ 的点积为：

顶点	点积
b_0	-128
b_1	-128

顶点	点积
b_2	-192

平局选较早的 b_0。

索引对 $(0,0)$ 已在当前保存的单纯形中，属于重复支撑点，停止。

9.5 输出重建

$$p_A = (0,0),$$
$$p_B = 0.5(3,1) + 0.5(1,3) = (2,2).$$

所以：

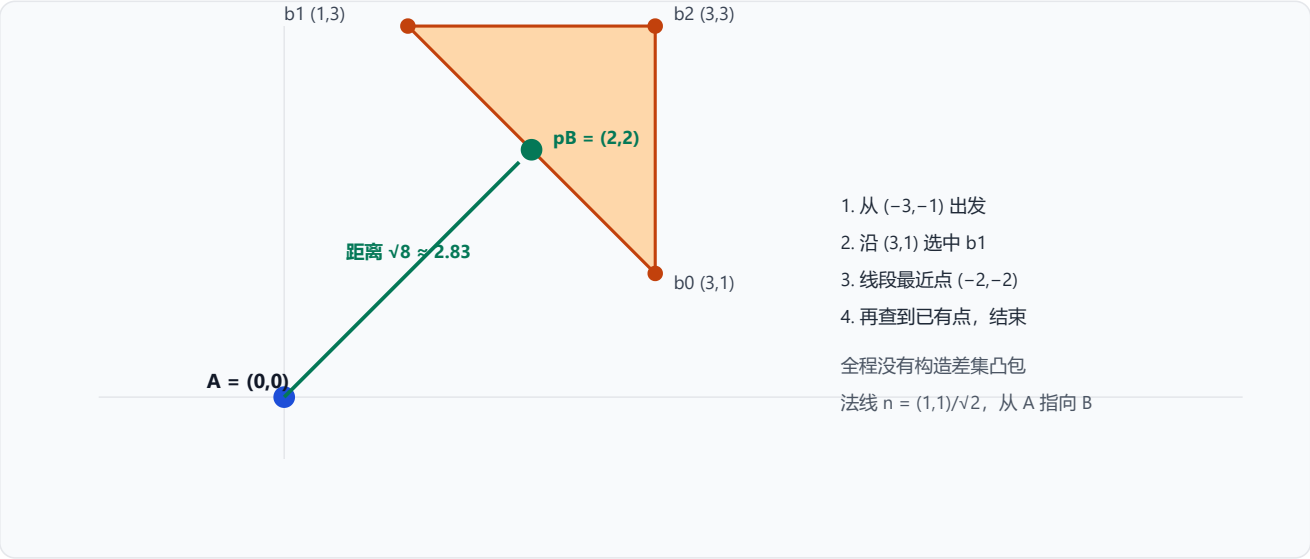
$$\text{distance} = \sqrt{8} \approx 2.828427,$$
$$n = \frac{(1,1)}{\sqrt{2}}.$$

共执行两次支撑查询，缓存保存边对应的两个索引对。

这套计算没有生成完整差集，也没有显式构造差集凸包。

图 7-6　第 9 节手算：A 是原点，B 是三角形，GJK 只走了两步

初始化 w1=−b0=(−3,−1) → 支撑到 b1 → 线段中点 (−2,−2) → 再支撑回到已有点，停止。



看图跟一次迭代： 从 b_0 出发 → 换到 b_1 → 最近点对着边中点 $(2,2)$ → 再查到已有点，结束。距离 $\sqrt{8}$，法线 $(1,1)/\sqrt{2}$。

10. 完整数值迭代表二：从线段扩展到包围原点

A 仍为原点，B 顶点改为：

$$b_0 = (-1,-1), \quad b_1 = (2,-1), \quad b_2 = (-1,2).$$

初始化得到：

$$w_1 = (1, 1), \qquad d = (-1, -1).$$

10.1 第一次支撑

B 沿 $(1, 1)$ 的点积依次为 $-2, 1, 1$。

平局选 b_1，得到：

$$w_2 = (-2, 1).$$

线段权重为：

$$\alpha_1 = \frac{2}{3}, \qquad \alpha_2 = \frac{1}{3}.$$

最近点是：

$$p = (0, 1).$$

源码搜索方向为 $(0, -18)$。

10.2 第二次支撑

B 沿 $(0, 18)$ 的点积依次为 $-18, -18, 36$。

选择 b_2，得到：

$$w_3 = (1, -2).$$

三角形恰好是第 8.4 节的内部例。

因此三个权重均为 $1/3$，原点在其中，核心距离为零。

算法不再请求第三个新支撑点。

这里几何过程执行了两次支撑查询。

但当前重叠提前返回分支没有经过普通尾部的 `iterations` 赋值，不能把教学计数硬当作所有路径的接口字段值。

11. 当前 GJK 的缓存、停止和数值保护

入口：`b2ShapeDistance`，范围 `424–610`。

11.1 先将 B 全部顶点放入 A 局部系

`435–444` 行预先转换 B。

主循环不必为每次支撑查询反复变换同一组顶点。

当前顶点数很小，一次预变换是合理优化。

11.2 缓存保存索引，不保存本帧答案

`b2MakeSimplexFromCache` 的范围是 `168–203`。

它读取缓存的顶点索引，用当前代理点重新构造：

$$w_A, \quad w_B, \quad w = w_A - w_B.$$

因此位置变化后仍需重新计算重心和距离。

如果上一例 B 整体平移 $(0.1, 0.1)$，缓存只是帮助重新找到同一条候选边，不会直接返回旧见证点 $(2, 2)$。

当前缓存恢复没有旧版本常见的 `metric` 比率有效性检查。

更换形状对、改变顶点数量或重排索引后，应清空缓存，不可假定接口会自动修复错误索引。

11.3 当前主要停止条件

- 最多 20 次支撑查询。
- 三角形包围原点：按重叠返回。
- 搜索方向平方过小：按无法产生可靠法线的重叠情况返回。
- 新索引对与本轮保存的旧单纯形重复：停止，避免循环。

重复检查比较的是 $(\text{indexA}, \text{indexB})$，不是只比较浮点坐标。

而且检查集合来自本轮简化之前的单纯形，可以阻止刚删除的特征立即反复返回。

不要把此版本描述成“只要两轮距离不再下降就停止”，源码主要终止判断并不是这个。

20 次上限限制工作量，不是对任意病态输入的无限精度数学保证。

12. `useRadii`：非负距离不等于任意两见证点的距离

GJK 首先计算凸核距离 ℓ。

普通输出尾部启用半径时：

$$\text{distance} = \max(0, \ell - r_A - r_B).$$

周界见证点更新为：

$$p'_A = p_A + r_A n, \qquad p'_B = p_B - r_B n.$$

源码范围是 `distance.c:598–608`。

12.1 分离例

核点分别为 $(0, 0)$、$(3, 0)$，半径为 0.5、0.75。

则：

$$\text{distance} = 3 - 0.5 - 0.75 = 1.75.$$

周界点为：

$$p'_A = (0.5, 0), \qquad p'_B = (2.25, 0).$$

二者距离也是 1.75 m。

12.2 半径重叠例

核点分别为 $(0,0)$、$(1,0)$，双方半径都为 0.75。

输出距离被夹到零：

$$\text{distance} = \max(0, 1 - 1.5) = 0.$$

但周界点仍分别偏移：

$$p'_A = (0.75, 0), \qquad p'_B = (0.25, 0).$$

两点之间仍相差 0.5 m。

当前源码刻意保留周界点的平滑移动，并没有把它们合并为中点。

因此不能在这种情况下用 $\|p'_B - p'_A\|$ 替代返回距离。

如果凸核本身已经重叠，算法会提前返回，甚至不经过这个半径尾部。

公开 `b2DistanceOutput` 明确说明零距离时法线无效。

GJK 距离并不是穿透深度算法；不要据此编造一个负距离或默认已经调用 EPA。

13. 凸包：当前是 QuickHull 式递归，不是 Andrew 单调链

GJK 可以查询凸代理，但形状输入仍需要合法几何。

`b2ComputeHull` 的范围是 `87–270`。

当前实现没有字典序排序，也没有上下链栈。

它采用最远点递归的 QuickHull 思路：

1. 检查输入点数。
2. 焊接过近的点。
3. 从 AABB 中心选最远点作为 p_1。
4. 选离 p_1 最远的点作为 p_2。
5. 将其余点按基线两侧分组。
6. 在一侧选最远点，递归处理两条新边。
7. 拼接两侧结果，保持 CCW。
8. 删除近共线中间点。

递归函数为 `b2RecurseHull`，范围 `13–80`。

13.1 基线距离符号

令：

$$e = \frac{p_2 - p_1}{\|p_2 - p_1\|}.$$

源码使用：

$$d = (p - p_1) \times e.$$

这个 d 单位为 m。

由于叉积顺序如此，正值对应有向基线右侧；不要和 $e \times (p - p_1)$ 的符号混淆。

13.2 完整数值例

输入点为：

$$(0,0), (2,0), (2,2), (0,2), (1,1), (0.001, 0.001).$$

默认 $\mathrm{slop} = 0.005$ m，焊接平方阈值为：

$$16\mathrm{slop}^2 = 0.0004.$$

对应距离阈值为 0.02 m。

最后一个点距第一个点约 0.001414 m，被焊接掉。

AABB 中心是 $(1,1)$，四角到中心同距，按顺序选择：

$$p_1 = (0,0), \qquad p_2 = (2,2).$$

基线单位方向为 $(1,1)/\sqrt{2}$。

- $(2,0)$ 的右侧距离为 $\sqrt{2}$。
- $(0,2)$ 的右侧距离为 $-\sqrt{2}$。
- $(1,1)$ 距离为零，不形成外边界。

最终凸包为：

$$(0,0), (2,0), (2,2), (0,2).$$

输入少于三个有效点、全部共线或点数超过接口上限时可能返回空包。

超过上限并不意味着自动从大量点中挑出若干代表点。

凸包会填掉凹缺口和孔洞；它不能替代保留凹轮廓的凸分解。

14. 射线查询：参数是平移比例，不是秒

射线段写成：

$$p(f) = o + f\Delta, \qquad 0 \le f \le f_{\max}.$$

其中 o 是起点，Δ 是完整平移向量，单位为 m。

比例 f 无量纲，不要求 Δ 是单位向量。

若要得到运动时间，需要另乘这段运动对应的持续时间。

14.1 圆射线的几何推导

将圆心移到原点，设相对起点 $s = o - c$。

将平移拆成长度与方向：

$$L = \|\Delta\|, \qquad d = \Delta / L.$$

无限直线上离原点最近的位置参数为：

$$t = -s \cdot d.$$

最近点是：

$$c_{\text{near}} = s + td.$$

若 $\|c_{\text{near}}\| > r$，整条直线都不碰圆。

否则由勾股定理，半弦长为：

$$h = \sqrt{r^2 - \|c_{\text{near}}\|^2}.$$

进入圆的线距离为 $t - h$，最终比例为：

$$f = \frac{t - h}{L}.$$

14.2 完整数值例

圆心为原点，半径 1。

射线输入：

$$o = (-3, 0.6), \qquad \Delta = (6, 0).$$

计算得到：

$$L = 6, \quad d = (1, 0), \quad t = 3, \quad c_{\text{near}} = (0, 0.6).$$

所以：

$$h = \sqrt{1 - 0.36} = 0.8, \qquad f = \frac{3 - 0.8}{6} = 0.3666667.$$

命中点和法线为：

$$p = (-0.8, 0.6), \qquad n = (-0.8, 0.6).$$

因为圆半径为 1，这里的点坐标恰好与单位法线数值相同。

若 $f_{\max} = 0.3$，则有限查询范围内未命中。

14.3 当前版本的初始重叠行为

入口：`b2RayCastCircle`，范围 `557–630`。

当前圆射线在起点严格位于圆内部时可以返回 `hit=true`、`fraction=0`。

零长度射线若起点在圆内部，也有这种返回路径。

该路径的法线可能保持零值。

无圆角多边形的初始内部路径也可以返回命中。

不要沿用“初始内部一定 miss”的旧版概括。

线段的平行、零长度和边界处理另有分支，不能将圆的规则扩展为所有形状的统一规则。

14.4 线段与多边形射线

`b2RayCastSegment`，范围 `769–847`：

1. 可选单面过滤。
2. 求射线与无限支撑线的交点比例。
3. 检查比例范围。
4. 检查交点是否在线段两个端点之间。

平行时分母为零，当前直接返回未命中。

`b2RayCastPolygon`，范围 `850–940`：

- 无圆角核心通过逐边半平面求进入和退出比例。
- 圆角多边形则改用点对形状的 `b2ShapeCast`。

15. 形状扫掠：B 相对 A 平移，姿态固定

`b2ShapeCastPairInput` 中：

- `proxyA` 是参考形状。
- `transform` 是 B 在 A 系中的相对姿态。
- `translationB` 是 B 在 A 系中的完整平移。
- `maxFraction` 是允许查询的最大比例。
- `canEncroach` 允许某些已接触情况轻微靠近。

此接口不在扫掠过程中改变 B 的旋转。

不能把它解释成“旋转物体的一般 TOI”。

15.1 从当前分离轴推导前进量

当前 GJK 核心距离为 d，单位法线 n 从 A 指向 B。

沿平移 Δ_B 的法向变化率为：

$$\nu = \Delta_B \cdot n.$$

它的单位是 m/单位比例。

若 $\nu \geq 0$，两形沿该分离轴没有接近，当前平移扫掠可拒绝。

若 $\nu < 0$，到目标距离 d_{target} 的比例增量为：

$$\Delta f = \frac{d_{\text{target}} - d}{\nu}.$$

分子、分母均为 m，比例无量纲。

每次推进后重新做距离查询，因为最近特征和法线可能变化。

15.2 当前目标与完整数值例

`b2ShapeCast`，范围 `614–705`，使用：

$$d_{\text{target}} = \max(\text{slop}, r_A + r_B - \text{slop}),$$

$$\varepsilon_d = 0.25\text{slop}.$$

取圆 A 核心在原点、圆 B 核心在 $(3, 0)$，半径各为 0.5，B 平移 $(-4, 0)$。

默认 $\text{slop} = 0.005$，则：

$$d_{\text{target}} = 0.995, \qquad \varepsilon_d = 0.00125.$$

初始 $d = 3$、$n = (1, 0)$、$\nu = -4$。

推进比例为：

$$f = \frac{0.995 - 3}{-4} = 0.50125.$$

此时核心距离为 0.995，满足目标。

严格几何相切比例是 0.5；差异来自有意设置的几何容差，不是公式算错。

15.3 初始重叠和 `canEncroach`

当前默认初始接近或重叠路径可以直接返回 `hit=true`、比例零。

这条初始命中分支没有给 `output.normal` 赋值，返回法线保持零向量；即使内部 GJK 算出了方向，该分支也不会把它复制到扫掠输出。因此不能直接把这个初始命中的法线当作单位向量，用于反弹或推出；需要另外处理初始重叠几何。

如果允许 `canEncroach` 且核心距离大于两倍 slop，则目标改为当前核心距离减一个 slop，允许轻微进一步靠近。

这为几何角色移动提供容忍度，不代表允许任意深穿透。

接口有最多 20 轮限制；达到或超过 `maxFraction` 的具体比较应以当前 `695–698` 行为准。

16. mover：平面位移修正不是刚体关节冲量

角色移动有两类不要混淆的机制：

- 几何 mover 查询和平面修正。
- 动力学 mover 关节的速度目标及冲量限制。

本节讨论第一类。

16.1 世界查询入口

`b2World_CastMover`，范围 `2868–2924`：

- 输入世界 `origin`、相对该原点的胶囊和目标平移。
- 设置 `canEncroach=true`。
- 查询三棵树，逐渐缩短可移动比例。
- 返回一个比例，不直接施加刚体冲量。

`b2World_CollideMover`，范围 `2970–3018`：

- 在当前位置查询附近重叠几何。
- 通过回调输出碰撞平面。
- 不执行普通接触的质量、摩擦和恢复求解。

16.2 平面约束的变量是什么

`b2CollideMoverAndPolygon` 用核心距离 d 和总半径 R 构造：

$$\text{offset} = R - d.$$

平面法线从障碍指向胶囊。

这里约束的变量是候选移动位移 Δ：

$$n \cdot \Delta - \text{offset} \geq 0.$$

不要把这个 `offset` 自动解释成世界空间平面高度。

16.3 完整位移修正例

设地面法线 $n = (0, 1)$，当前需要推出的距离为 0.1 m。

期望位移：

$$\Delta = (0.5, -0.2).$$

忽略容差时：

$$n \cdot \Delta - 0.1 = -0.3.$$

需要向上推 0.3 m，得到 $(0.5, 0.1)$。

当前 `b2SolvePlanes` 加入 0.005 m slop：

$$\text{separation} = -0.2 - 0.1 + 0.005 = -0.295.$$

因此推量为 0.295 m，最终：

$$\Delta' = (0.5, 0.095).$$

`push` 是位移，单位 m，不是 kg·m/s 的冲量。

源码还累计夹取 `pushLimit`，依次处理多个平面，最多迭代 20 轮。

16.4 裁掉进入平面的速度分量

`b2ClipVector`，范围 `56–71`，使用：

$$v' = v - \min(0, v \cdot n)n.$$

例如：

$$v = (2, -3), \qquad n = (0, 1) \quad \Longrightarrow \quad v' = (2, 0).$$

切向运动保留，进入地面的分量删除。

如果 $v = (2, 3)$，物体正在离开地面，不应删掉向上的分量。

当前代码只处理已有非零推量且开启 `clipVelocity` 的平面。

这种几何投影不会自动向另一刚体施加等大反向冲量，也不是库仑摩擦。

17. 边界练习与答案

题一：GJK 是否必须先显式生成整个 Minkowski 差?

答案：不需要。 差集支撑点可以通过双方支撑查询直接得到。

题二：搜索方向 $(2, 2)$ 和 $(32, 32)$ 哪个正确?

答案：都正确。 正比例缩放不改变支撑点排序。

题三：有三个差点就说明形状重叠吗?

答案：不是。 原点可能位于三角形外的顶点或边 Voronoi 区域。

题四：只保存差点，能否总能还原两形状见证点?

答案：不能。 还需要对应来源点、索引和重心权重。

题五：半径重叠时返回距离零，但两个周界点不同，是否矛盾?

答案：不矛盾。 当前距离被夹到零，周界点仍分别偏移以保持连续性。

题六：缓存可以复用于任意两个顶点数相同的形状吗?

答案：不能这样保证。 它依赖特定形状对及索引语义。

题七：当前凸包是 Andrew 排序单调链吗?

答案：不是。 它是最远点分组递归的 QuickHull 式实现。

题八：当前圆射线起点在圆内，一定返回未命中吗？

答案：不是。 可返回比例零的命中，且法线可能无效。

题九：形状扫掠会自动按角速度旋转 B 吗？

答案：不会。 当前 `b2ShapeCast` 只推进平移。

题十：mover 平面 `push=0.1` 表示施加了多大冲量？

答案：它不是冲量。 这里表示位移修正长度，应以 m 理解。

18. 源码阅读索引

主题	函数与范围
支撑点	`b2FindSupport`，`148–165`
缓存恢复	`b2MakeSimplexFromCache`，`168–203`
见证点	`b2ComputeWitnessPoints`，`220–246`
线段单纯形	`b2SolveSimplex2`，`274–306`
三角单纯形	`b2SolveSimplex3`，`309–418`
GJK 主循环	`b2ShapeDistance`，`424–610`
平移扫掠	`b2ShapeCast`，`614–705`
凸包	`b2ComputeHull`，`87–270`
圆射线	`b2RayCastCircle`，`557–630`
多边形射线	`b2RayCastPolygon`，`850–940`
平面位移求解	`b2SolvePlanes`，`7–53`
向量裁剪	`b2ClipVector`，`56–71`

建议把第 9、10 节的每个支撑索引和重心权重画出来。

如果还不能解释为什么删除某个顶点，先不要尝试记忆整个 GJK 主循环。

返回目录 · 下一章：TOI 与连续碰撞

08 • TOI 与连续碰撞

> 阅读目标：先解匀速圆圆扫掠，再理解带旋转的 sweep、目标距离、分离轴与根求解，最后准确对应世界求解器中的 CCD 调度、截断方式和限制。
>
> 基线：`dc67c40ba2ce1ad384b0c985dbc1b539cba9221b` 对应的 3.2.0 工作树。当前 CCD 不是对所有刚体对执行无限精度、剩余时间完整重算的事件模拟器。

返回目录 · 上一章：GJK 与形状查询

1. 连续碰撞首先是时间采样问题

离散碰撞只在有限时刻检查几何。

一个很小的圆可能从薄墙左侧移动到右侧，而起点、终点都不与墙重叠。

中间发生的穿越没有被采样到，这就是隧穿。

最朴素的改进是增加采样次数：

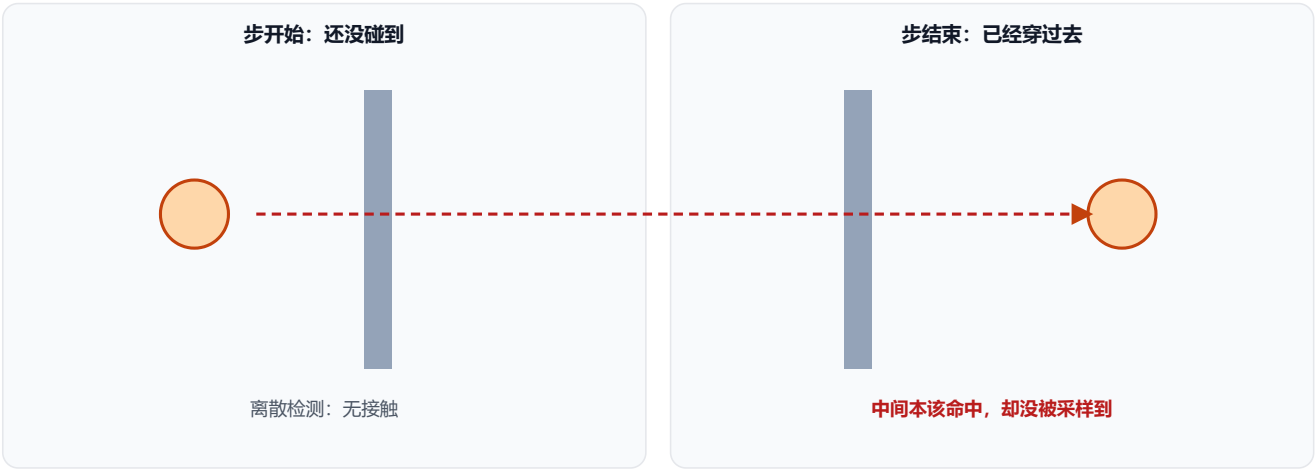
1. 将整步分成若干时间片。
2. 在每个采样时刻进行离散碰撞。
3. 找到第一次检测到重叠的位置。

但样本之间仍可能跨过更薄的障碍。

因此先从有解析解的连续问题入手，比直接增加任意采样次数更容易建立正确直觉。

图 8-1　离散碰撞只看有限时刻：中间穿越可能完全看不见

小圆这一步从墙左到墙右。起点、终点都不与墙重叠，于是被当成"没有撞到"。



看图： 这就是隧穿。加采样能减轻，但不能保证更薄的障碍一定被抓到。连续碰撞改问"这一段轨迹何时第一次碰到"。

1.1 必须分开的三个层次

- **数学轨迹相交**：理想连续运动是否相交？
- **TOI 几何函数**：当前算法如何近似定位接近时刻？
- **世界 CCD 调度**：哪些对象调用 TOI，结果如何影响位姿和速度？

独立 TOI 函数支持两形旋转，不代表世界会为所有形状对执行完整连续求解。

2. 匀速圆圆扫掠：从距离平方推导二次根

设两个世界圆心随物理时间 τ 变化：

$$a(\tau) = a_0 + v_A\tau, \qquad b(\tau) = b_0 + v_B\tau.$$

这里：

- τ：秒。
- a_0, b_0：起点位置，单位 m。
- v_A, v_B：线速度，单位 m/s。
- r_A, r_B：圆半径，单位 m。
- Δt：当前步时长，单位 s。

只考虑：

$$0 \le \tau \le \Delta t.$$

2.1 消去共同平移

定义初始相对位置、相对速度和半径和：

$$q = b_0 - a_0, \qquad u = v_B - v_A, \qquad R = r_A + r_B.$$

碰撞边界是：

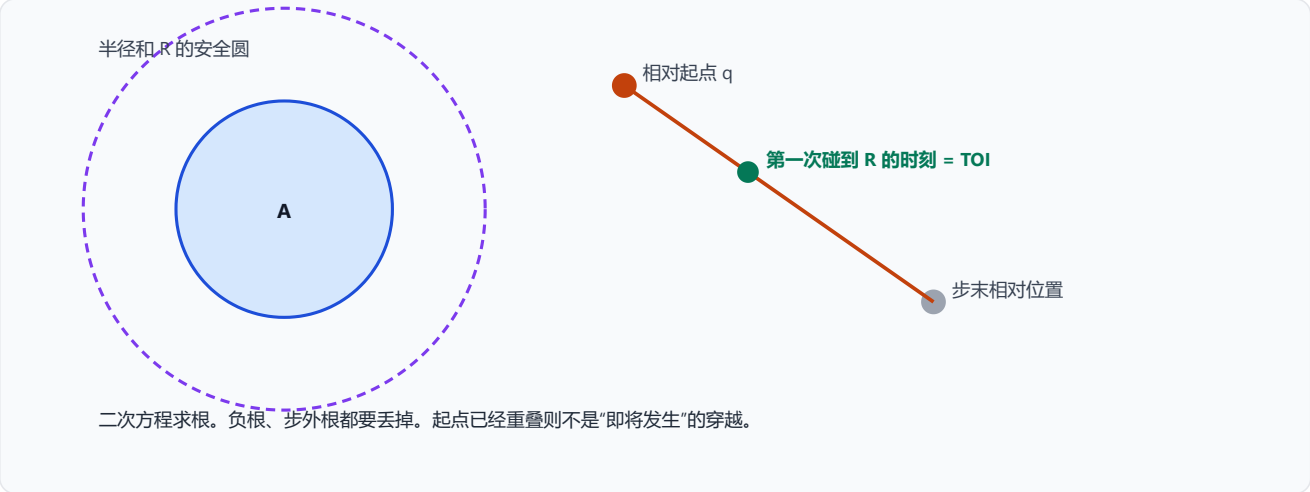
$$\|q + u\tau\| = R.$$

两个物体共同增加相同速度，不会改变相对距离。

但共同平移会改变世界命中点，所以计算时间和还原世界位置仍是不同步骤。

图 8-2　圆圆连续碰撞：看相对圆心会不会走进半径和圆

q = b0−a0，u = vB−vA。碰撞边界 ||q + uτ|| = R。共同平移不改相对距离。



看图： 紫色虚线圆半径是 $R = r_A + r_B$。相对起点沿相对速度走线段，第一次碰到虚线圆的时刻就是 TOI。共同平移可先消掉。

2.2 展开二次方程

两边平方：

$$(q + u\tau) \cdot (q + u\tau) = R^2.$$

展开：

$$a\tau^2 + b\tau + c = 0,$$

其中：

$$a = u \cdot u, \qquad b = 2q \cdot u, \qquad c = q \cdot q - R^2.$$

注意这里的标量 a, b 是二次系数，不再表示圆心。

单位分别为：

$$[a] = \mathrm{m}^2/\mathrm{s}^2, \quad [b] = \mathrm{m}^2/\mathrm{s}, \quad [c] = \mathrm{m}^2.$$

判别式和根为：

$$D = b^2 - 4ac,$$

$$\tau_{\pm} = \frac{-b \pm \sqrt{D}}{2a}.$$

根的单位为秒。

初始分离时，应选择位于当前步内的最小非负进入根。

较大的根通常是离开时刻，不能拿它代替第一次撞击。

3. 完整解析例：进入和离开不是同一个时刻

取 A 静止：

$$a_0 = (0,0), \qquad r_A = 0.5.$$

B 的输入为：

$$b_0 = (3,0), \qquad r_B = 0.5, \qquad v_B = (-4,0).$$

整步时长取 $\Delta t = 1$ s。

于是：

$$q = (3,0), \quad u = (-4,0), \quad R = 1.$$

二次系数为：

$$a = 16, \qquad b = -24, \qquad c = 8.$$

判别式：

$$D = (-24)^2 - 4 \times 16 \times 8 = 64.$$

因此：

$$\tau_{\text{enter}} = \frac{24 - 8}{32} = 0.5 \text{ s},$$
$$\tau_{\text{exit}} = \frac{24 + 8}{32} = 1 \text{ s}.$$

进入时 B 圆心为 $(1,0)$，两圆表面在 $(0.5,0)$ 相切。

A 到 B 法线为 $(1,0)$。

如果整步只有 0.4 s，进入根仍是 0.5 s，但它位于步外，所以本步没有命中。

3.1 两边都运动的校验

若改为：

$$v_A = (2,0), \qquad v_B = (-2,0),$$

相对速度仍为 $(-4,0)$，进入时刻仍为 0.5 s。

但此时 A、B 圆心分别为 $(1,0)$、$(2,0)$。

世界接触点变成 $(1.5,0)$，不是上一例的 $(0.5,0)$。

4. 解析根的退化和数值保护

4.1 没有相对运动

若 $a = u \cdot u = 0$，则两个圆心相对静止。

- $c > 0$：初始分离，未来仍分离。
- $c = 0$：初始相切。
- $c < 0$：初始重叠。

这时不能继续除以 $2a$。

4.2 初始重叠

若 $c < 0$，查询开始时已经重叠。

即使存在正的离开根，它也不代表未来第一次进入。

必须把初始重叠与正常进入事件分开处理。

4.3 擦切、背离和步外根

- $D < 0$：连无限直线轨迹都不接触半径和边界。
- $D = 0$：擦切双根。
- 初始分离且 $q \cdot u \geq 0$：匀速圆模型中正在背离，可提前拒绝。
- 根小于零：交点在过去。
- 根大于 Δt：交点在当前步之后。

浮点下很小的负判别式是否夹到零，应由有意义的数值容差决定。

不能把所有负判别式无条件改成零，否则会制造不存在的命中。

4.4 避免二次根的大数相减

标准公式在某些数值组合下可能发生消去误差。

一种教学上常用的稳定写法是：

$$Q = -\frac{1}{2}\left(b + \operatorname{sign}(b)\sqrt{D}\right),$$

再求：

$$\tau_1 = Q/a, \qquad \tau_2 = c/Q.$$

仍需单独处理 $a = 0$、$Q = 0$ 等退化。

这只是解析圆模型的数值改进，不是当前 `b2TimeOfImpact` 的实现方式。

5. 源码为什么使用时间比例

源码中的时间通常是归一化比例：

$$t = \frac{\tau}{\Delta t}.$$

于是整步相对平移为：

$$\Delta = (v_B - v_A)\Delta t.$$

圆碰撞方程可以写为：

$$\|q + t\Delta\|^2 = R^2, \qquad 0 \le t \le 1.$$

此时 q, Δ 都以 m 计，t 无量纲。

若返回 `fraction=0.4`，步长为 0.02 s，物理时刻就是：

$$\tau = 0.4 \times 0.02 = 0.008 \text{ s}.$$

不能把 `fraction` 直接当成秒。

6. 加入旋转后，为什么不再是一个二次方程

多边形运动过程中，最近特征会改变：

- 先是角对角。
- 随后变成角对边。
- 再变成面状接触。

旋转还让局部点沿弯曲轨迹运动。

沿某个法线的分离量不再对时间全局线性，最近距离也不再由一个固定二次式描述。

只扫质心会漏掉旋转杆端。

用一个很大的包围圆代替整个物体虽然简单，但对细长形状往往过于宽松。

TOI 的思路是反复建立临时分离函数，在这个较简单的一维函数上求根，再更新几何特征。

7. Sweep：插值质心，再恢复刚体原点

`sweep` 保存：

- `localCenter`：局部质心。
- `c1`、`c2`：起止质心位置。
- `q1`、`q2`：起止旋转。

质心线性插值为：

$$c(t) = (1 - t)c_1 + tc_2.$$

二维旋转用余弦、正弦对表示：

$$q_1 = (\cos\theta_1, \sin\theta_1), \quad q_2 = (\cos\theta_2, \sin\theta_2).$$

当前先做线性混合：

$$\tilde{q}(t) = (1 - t)q_1 + tq_2,$$

然后归一化：

$$q(t) = \frac{\tilde{q}(t)}{\|\tilde{q}(t)\|}.$$

这称为归一化线性插值，即 NLerp。

7.1 为什么要减去局部质心的旋转偏移

刚体原点是：

$$p(t) = c(t) - Q(t)c_L,$$

其中 c_L 为局部质心，$Q(t)$ 为旋转矩阵。

局部点 x_L 的世界位置为：

$$x(t) = p(t) + Q(t)x_L = c(t) + Q(t)(x_L - c_L).$$

这个公式确保旋转围绕质心进行。

若省略局部质心补偿，偏心形状会围绕错误的原点运动。

7.2 一个单姿态坐标校验

设：

$$c = (2, 3), \qquad c_L = (0.5, 0),$$

旋转为 90°。

于是：

$$Qc_L = (0, 0.5), \qquad p = (2, 2.5).$$

把局部质心变换回世界：

$$p + Qc_L = (2, 3),$$

与指定世界质心一致。

这只是单姿态变换例，不意味着一帧应该旋转 90 度。

7.3 对照源码与旋转限制

入口：`b2GetSweepTransform`，范围 `14–29`。

NLerp 不等于直接线性插值角度，也不保证严格恒角速度。

起止旋转若相差 180 度，混合向量甚至可能变成零。

两个端点旋转也无法表示中间转了多圈的信息。

当前 `B2_MAX_ROTATION` 为：

$$0.25\pi = 45^\circ.$$

`solver.c:123,148–152` 根据整步时间限制角速度，除非允许快速旋转。

常量附近还警告提高到 0.5π 或更大会破坏连续碰撞。

因此不能声称任意大角度旋转仍保持同样 CCD 保证。

8. TOI 的目标距离不是零，也不是表面分离量

入口：`b2TimeOfImpact`，范围 `1148–1440`。

它调用 GJK 时使用：

$$\texttt{useRadii=false}.$$

因此得到的是凸核之间的距离。

半径放到目标值中：

$$d_{\text{target}} = \max(\text{slop}, r_A + r_B - \text{slop}).$$

距离容差为：

$$\varepsilon_d = 0.25\text{slop}.$$

两者单位都是 m。

8.1 完整目标值例

默认：

$$\text{slop} = 0.005, \qquad r_A + r_B = 1.$$

所以：

$$d_{\text{target}} = 0.995, \qquad \varepsilon_d = 0.00125.$$

核心距离达到 0.995 时，表面分离大约是：

$$0.995 - 1 = -0.005.$$

也就是允许轻微进入，避免要求浮点算法精确卡在零间隙边界。

若半径和非常小，目标至少保留一个 slop，避免完全零距离的脆弱几何。

8.2 与解析圆根对照

前面的圆例，在整个 $t \in [0, 1]$ 范围内，非负核心距离是：

$$d(t) = |3 - 4t|.$$

首次相遇前的分支 $t \le 0.75$ 上，它才等于 $3 - 4t$。固定方向投影的有符号分离函数则可写成 $s(t) = 3 - 4t$，圆心交越后允许为负。距离与有符号分离不能混用。

首次严格几何相切发生在 $t = 0.5$。

TOI 目标对应：

$$3 - 4t = 0.995 \quad \Longrightarrow \quad t = 0.50125.$$

这点差异来自目标设定。

不要重复减去半径，也不要把距离容差误写成时间比例容差。

9. 三种分离函数

`b2MakeSeparationFunction` 的范围是 `953–1026`。

它利用当前 GJK 缓存构造更便宜的标量函数。

9.1 点点类型

缓存只有一个索引对时，建立世界方向：

$$n = \frac{p_B - p_A}{\|p_B - p_A\|}.$$

随后测量：

$$s(t) = n \cdot (p_B(t) - p_A(t)).$$

这里的轴在该轮分离函数中保持世界方向固定。

9.2 A 面类型

A 的两个缓存点形成参考面。

法线存储在 A 局部系，求值时随 A 旋转：

$$n_A(t) = Q_A(t) n_{A,L}.$$

分离量为：

$$s(t) = n_A(t) \cdot (p_B(t) - p_A(t)).$$

9.3 B 面类型

如果 A 缓存索引相同、B 有两个点，就构造 B 面。

其内部公式是：

$$s(t) = n_B(t) \cdot (p_A(t) - p_B(t)).$$

因为参考面换到了 B，点差顺序也改变。

不能因此随意改变最终公共输出的 A 到 B 法线约定。

9.4 找最深特征与固定特征求值要分开

`b2FindMinSeparation`，范围 `1029–1093`：

- 给定时刻。
- 用支撑查询找当前轴上最深的特征。
- 返回分离量并记下特征索引。

`b2EvaluateSeparation`，范围 `1097–1143`：

- 固定已选特征索引。
- 在不同时间只变换这些特征并求投影。

后者更适合高频根求解。

它不等于在每次试探时间都重新计算完整 GJK 距离。

10. TOI 的三层循环

10.1 外层：更新 GJK 和分离轴

在当前时间 t_1 计算核心距离。

- 距离不大于零：返回核心重叠状态。
- 距离已进入目标容差：返回命中。
- 否则构造新的分离函数。

外层允许最近特征和候选分离轴随推进改变。

10.2 中层：在较晚时间选择最深特征

先令 $t_2 = t_{\max}$，求：

$$s_2 = s(t_2).$$

若 $s_2 > d_{\text{target}} + \varepsilon_d$，当前分离模型返回分离结论。

若 s_2 已在目标附近，则把 t_1 推进到 t_2，回外层重新检查。

否则固定此时选到的特征，求其在 t_1 的分离 s_1。

只有 s_1 在目标外、s_2 在目标内时，才形成适合求根的区间。

如果内部状态已经不一致或无法继续进展，会返回失败状态，而不是伪装成正常未命中。

10.3 最内层：固定特征的一维根

求解：

$$s(t) - d_{\text{target}} = 0.$$

得到暂定 t_2 后，中层还会重新找最深特征。

如果新特征更早限制运动，还需要继续推进。

所以 TOI 不是“一次 GJK，加一次二分，然后完成”。

10.4 当前工作量上限

- 外层距离查询：最多 20 次。

- 一次固定特征求根：最多 50 次。
- 中层 push-back：最多 `B2_MAX_POLYGON_VERTICES` 次。

这些限制控制计算成本，不是对任意输入的无限精度保证。

11. 二分与假位法交替：为什么不用一种方法到底

设当前根区间为 $[a_1, a_2]$，满足：

$$s_1 > d_{\text{target}}, \qquad s_2 < d_{\text{target}}.$$

11.1 二分法

取：

$$t = \frac{a_1 + a_2}{2}.$$

它不关心曲线形状，能稳定缩小区间，但收敛速度有限。

11.2 假位法

用区间端点连线估计目标交点：

$$t = a_1 + \frac{d_{\text{target}} - s_1}{s_2 - s_1}(a_2 - a_1).$$

这是端点插值，不是牛顿法，因为没有计算导数。

当函数接近线性时，它可能非常快。

但曲率较大时某个端点可能长期不更新。

11.3 当前混合策略

源码 `distance.c:1346–1395` 从第零轮开始：

- 偶数轮使用二分。
- 奇数轮使用假位法。

计算分离量后，如果：

$$|s(t) - d_{\text{target}}| < \varepsilon_d,$$

则接受。

否则依据分离量在目标哪一侧更新对应端点，继续维持括住根的区间。

11.4 在线性圆例上手算两轮

取：

$$s(t) = 3 - 4t, \quad d_{\text{target}} = 0.995, \quad [a_1, a_2] = [0, 1].$$

第一轮二分：

$$t = 0.5, \qquad s = 1.$$

误差为 0.005，大于 0.00125，不能停止。

左端改为 0.5。

第二轮假位：

$$t = 0.5 + \frac{0.995 - 1}{-1 - 1}(1 - 0.5) = 0.50125.$$

此时 $s = 0.995$，达到目标。

12. 完整旋转几何例：四轮得到非线性根

这一例来自明确的几何与源码 NLerp，不是随手假设一个多项式。

12.1 形状与运动输入

A 是静止竖段：

$$(0, -10) \to (0, 10), \qquad r_A = 0.$$

取参考法线 $n = (1, 0)$。

B 代理有一个局部核心点：

$$x_L = (1, 0), \qquad r_B = 1.$$

B 的局部质心取原点，质心 sweep 为：

$$c_B(t) = (3 - 4t, 0).$$

旋转由 0° 变到 45°。

这个例子可理解为刚体上一个偏心圆形状；单个形状的圆心不必等于整个刚体质心。

总旋转量不超过默认每步 45 度限制。

12.2 写出 NLerp 后的点轨迹

定义：

$$u(t) = 1 - t + \frac{t}{\sqrt{2}}, \qquad v(t) = \frac{t}{\sqrt{2}}.$$

归一化旋转的余弦为：

$$\cos\theta(t) = \frac{u(t)}{\sqrt{u(t)^2 + v(t)^2}}.$$

因此 B 核心点的 x 坐标，即对 A 参考面的分离量，是：

$$s(t) = 3 - 4t + \frac{u(t)}{\sqrt{u(t)^2 + v(t)^2}}.$$

B 核心点的 y 坐标始终位于竖段端点之间，所以此处面参考特征合法。

12.3 目标与初始区间

半径和为 1 m，默认参数给出：

$$d_{\text{target}} = 0.995, \qquad \varepsilon_d = 0.00125.$$

端点分离量为：

$$s(0) = 4,$$

$$s(1) = -1 + \frac{1}{\sqrt{2}} \approx -0.2928932188.$$

目标被区间 $[0, 1]$ 括住。

12.4 实际算术核对的迭代表

轮次	方法	试探比例 t	分离量 $s(t)$	与目标之差
0	二分	0.5000000000	1.9238795325	+0.9288795325
1	假位	0.7095116723	1.0067815865	+0.0117815865
2	二分	0.8547558362	0.3594663339	−0.6355336661
3	假位	0.7121552168	0.9950684650	+0.0000684650

第一轮后区间是 $[0.5, 1]$。

第二轮后左端变为 0.7095116723。

第三轮后右端变为 0.8547558362。

第四轮误差满足：

$$0.0000684650 < 0.00125,$$

因此可以接受暂定根。

12.5 如何解释结果

此时表面分离约为：

$$s - R = 0.9950684650 - 1 = -0.0049315350 \text{ m}.$$

若整步为 $1/60$ s，对应物理时刻：

$$\tau = \frac{0.7121552168}{60} \approx 0.0118692536 \text{ s}.$$

随后中层与外层仍按算法检查最深特征和距离，不应把最内层暂定根描述成跳过所有后续验证。

该表使用独立双精度算术运行核对。

库内部采用 float，最后若干位允许不同；这里没有声称执行了 C 单元测试或逐位验证。

13. TOI 输出不能只读 `fraction`

`b2TOIOutput` 包含状态、点、法线和时间比例。

主要状态含义是：

- `Overlapped`：核心已重叠，比例通常为零。
- `Hit`：进入当前目标容差。
- `Separated`：当前查询给出分离结论，比例通常为查询上限。
- `Failed`：迭代未能正常继续，不能当成精确未命中。

`fraction=1` 必须与状态和 `maxFraction` 一起解释。

`fraction=0` 也不能保证存在适合直接施加冲量的有效单位法线。

头文件对 `b2TimeOfImpact` 的说明明确指出：扫掠分离轴可能漏掉某些中间、非隧穿型碰撞。

改变查询时间区间后应重新调用。

旋转使分离函数可能非单调，因此不能把当前近似算法包装成“求出了任意旋转轨迹的所有精确根”。

14. 世界实际时序：先离散求解，再决定哪些刚体补做 CCD

现在从几何函数回到 `src/solver.c`。

`b2Solve` 先执行普通约束求解和子步推进。

`1609–1610` 行在约束求解及岛分裂之后调用 `b2FinalizeBodiesTask`。

Finalize 把累计位移、累计旋转落实为预测终位姿，然后识别快物体。

不是先对所有刚体执行 TOI，再启动普通接触求解。

14.1 快运动度量

源码 `607–615` 行估计：

$$v_{\max} = \|v\| + |\omega| R_{\max}.$$

其中：

- v：质心线速度，m/s。
- ω：角速度，rad/s。
- $R_{\max}$：最远几何范围，m。

因此 $v_{\max}$ 单位为 m/s。

累计运动量还包含位置修正和旋转：

$$\Delta_{\max} = \|\Delta p\| + |\Delta q.s| R_{\max}.$$

接着取：

$$\text{maxMotion} = \max(\Delta_{\max}, v_{\max}\Delta t).$$

动态刚体满足连续检测等条件，且：

$$\text{maxMotion} > \text{safetyFactor} \times \text{minExtent},$$

就进入快物体路径。

`safetyFactor` 无量纲，`minExtent` 单位 m。

当前判定位于未进入睡眠处理的相应分支中，不要省略整个上下文后声称所有物体每步无条件检查。

14.2 平移快物体例

取：

$$\Delta t = 1/60 \text{ s}, \quad \|v\| = 30 \text{ m/s},$$
$$\text{minExtent} = 0.1 \text{ m}, \quad \text{safetyFactor} = 0.5.$$

速度运动量为：

$$30/60 = 0.5 \text{ m}.$$

门限为：

$$0.5 \times 0.1 = 0.05 \text{ m}.$$

因此需要快物体处理。

14.3 质心不动也可能很快

若 $v = 0$、$\omega = 20$ rad/s、$R_{\max} = 2$ m，则：

$$v_{\max} = 40 \text{ m/s}.$$

在 $1/60$ s 中对应约 0.6667 m 的端点运动估计。

所以不能只观察质心位移判断是否需要 CCD。

15. 普通 fast 和 bullet 的调度不同

入口：`b2FinalizeBodiesTask`，范围 `560–749`。

- 普通 fast dynamic 在 Finalize 中立即调用 `b2SolveContinuous`。
- fast bullet 先放入 `bulletBodies` 数组。

随后主流程更新非 bullet 的相关包围代理。

`1893–1902` 行再统一调用 bullet 任务。

`b2BulletBodyTask` 的 `1236–1248` 行仍调用同一个 `b2SolveContinuous`。

bullet 阶段完成后，主流程再串行扩大相应代理。

这种时序让 bullet 查询时能使用非 bullet 已更新的运动与包围信息。

它不是“bullet 使用另一套完全不同的 TOI 公式”。

16. 当前 CCD 到底查询哪些目标

入口：`b2SolveContinuous`，范围 `384–556`。

函数遍历 fast body 的形状，并构造扫掠查询框。

关键查询代码如下：

```
                    b2DynamicTree_Query( staticTree, sweptBox, B2_DEFAULT_MASK_BITS,
b2ContinuousQueryCallback, &context );

                    if ( isBullet )
                    {
                              b2DynamicTree_Query( kinematicTree, sweptBox, B2_DEFAULT_MASK_BITS,
b2ContinuousQueryCallback, &context );
                              b2DynamicTree_Query( dynamicTree, sweptBox, B2_DEFAULT_MASK_BITS,
b2ContinuousQueryCallback, &context );
                    }
```

因此：

快物体类别	CCD 查询树
普通 fast dynamic	static
fast bullet	static、kinematic、dynamic

不要把一般宽阶段动态物体查三棵树的规则，误写成普通 fast CCD 也查三棵树。

16.1 bullet 不对目标 bullet 做这条连续检测

`b2ContinuousQueryCallback` 在 `228–231` 行跳过目标 bullet。

所以 bullet 可以对非 bullet dynamic 做连续检查，但 bullet-bullet 被排除。

设置 bullet 不意味着对所有对象类型无条件提供同样检测。

16.2 回调还执行哪些过滤

- 跳过同一形状。
- 跳过同一刚体。
- 检查形状碰撞过滤。
- 检查刚体之间的碰撞许可。

- 检查自定义过滤。
- 对传感器检查相关事件标志。
- 对链段执行单面与近平行运动早退。

独立 TOI 函数只处理输入的凸代理与 sweep，世界上下文规则需要由外层补充。

17. 候选如何不断缩短最早比例

对每个候选，回调建立：

- A：目标形状及其 sweep。
- B：当前 fast shape 及 fast body 的 sweep。
- `maxFraction`：目前已知最早比例。

只要新结果满足：

$$0 < f_{\text{new}} < f_{\text{current}},$$

就缩短当前允许运动比例。

当前回调主要按比例比较，不是简单写成“只有 `state==Hit` 才接受”。

源码解读不能自行添加不存在的条件。

17.1 初始零比例的回退

若第一次 TOI 返回零比例，`347–359` 行会：

1. 取 fast shape 的几何质心。
2. 构造围绕该点的小圆代理。
3. 用与最小几何范围相关的核心半径重试 TOI。

这用于处理初始重叠等难以推进的情况。

它不是完整的穿透推出算法，也不是所有高速形状永远都被简化为圆。

若启用连续预处理回调，用户还可否决本次候选。

18. 命中后实际做的是位姿截断

假设最早比例为 $f < 1$。

当前源码计算：

$$c_f = (1 - f)c_1 + fc_2,$$

$$q_f = \mathrm{NLerp}(q_1, q_2, f).$$

再根据局部质心恢复刚体原点，更新最终变换和包围盒。

这发生在 `solver.c:454–506`。

18.1 没有完整重算剩余时间

这里并没有：

1. 在 TOI 时刻重新运行全部接触与关节求解。
2. 计算撞击反弹后的完整速度状态。
3. 再把剩余 $(1-f)\Delta t$ 完整模拟一遍。

也没有直接根据 TOI 法线将速度反弹或清零。

因此它是一种实用的时间损失式截断，不是理想事件驱动求解器。

18.2 损失时间只撤销重力贡献

定义：

$$\Delta t_{\text{loss}} = (1-f)\Delta t.$$

当前速度修正是：

$$v_{\text{new}} = v_{\text{old}} - \Delta t_{\text{loss}}\,\text{gravityScale}\,g.$$

源码 `468–473` 明确说明其他力与力矩暂时忽略。

减去重力贡献，是撤销已经多积分的那部分重力，不是再加一次重力。

18.3 完整数值例

设：

$$\Delta t = 1/60\ \text{s}, \qquad f = 0.4.$$

损失时间为：

$$\Delta t_{\text{loss}} = 0.6/60 = 0.01\ \text{s}.$$

取：

$$g = (0,-10)\ \text{m/s}^2, \quad \text{gravityScale} = 1,$$
$$v_{\text{old}} = (12,-5)\ \text{m/s}.$$

则：

$$v_{\text{new}} = (12,-5) - 0.01(0,-10) = (12,-4.9).$$

x 速度仍是 12 m/s，并未因 CCD 自动归零。

若预测质心从 $(0,0)$ 到 $(0.2,-0.0833333)$，截断后为：

$$c_f = (0.08,-0.0333333).$$

只接受原预测位移的 40%。

剩余 60% 时间没有在这个函数里按完整碰撞动力学恢复推进。

19. 包围框、传感器和旋转的工程限制

19.1 局部基准减少大坐标消去

`b2SolveContinuous` 使用旧质心作为相对基准，将 sweep 放到附近坐标系计算。

世界大位置先相减，再进入主要 float 几何，有利于保留局部细节。

这与直接在巨大世界坐标上反复计算差点不同。

19.2 两端 AABB 的并不是任意旋转轨迹的严格包络

源码使用旧包围框与预测终框的并作为候选查询范围。

对纯平移，它能覆盖中间形状位置。

对一般旋转，不能直接给出同样结论。

反例：细杆半长 2 m、半厚 0.1 m，从 -22.5° 转到 $+22.5^\circ$。

两端 AABB 半宽均为：

$$e_x = 2\cos 22.5^\circ + 0.1\sin 22.5^\circ \approx 1.8860274083.$$

中间水平姿态半宽为 2 m。

所以中途形状可能比两端框的并更宽。

这只证明包络性质不能普遍成立；具体场景是否漏碰，还与额外 margin、目标包围框及其他处理有关。

不要由这个反例反过来断言所有旋转杆都必然隧穿。

19.3 连续传感器不是无限事件收集器

`solver.c:164` 的连续传感器记录上限是 8。

当前 fast shape 若本身为传感器，不作为普通连续实体扫掠。

被扫描的传感器需要满足事件标志等条件。

最后还会丢弃位于实体截断之后的传感器命中记录。

因此不能承诺高速路径上任意数量、任意时序的传感器事件全部被保留。

重要游戏逻辑应使用与需求匹配的明确查询方案。

19.4 如何正确使用这些限制

- 采用合理的形状尺寸与长度尺度。
- 避免深度初始重叠。
- 对高速小动态物体按需要设置 bullet。
- 控制细长物体的大角度旋转。
- 必要时减小时间步。

- 对游戏命中判定使用专门射线或形状查询。

把所有物体都设成 bullet，既不是通用精度开关，也不能消除算法本身的限制。

20. 边界练习与答案

题一：半径各 0.5、中心距 3、相对速度为 −4 m/s，进入和离开时间是多少？

答案：0.5 s 和 1 s。 首次撞击取进入根。

题二：`fraction=0.4`、步长 0.02 s，物理时刻是多少？

答案：0.008 s。 比例本身没有秒单位。

题三：TOI 使用 `useRadii=false`，是否意味着完全忽略半径？

答案：不是。 半径进入目标距离，不能重复扣除。

题四：目标 0.995 m，当前分离 1 m，默认容差下可以停止吗？

答案：不行。 误差 0.005 m 大于 0.00125 m。

题五：假位法是否就是牛顿法？

答案：不是。 它使用区间端点插值，不计算导数。

题六：局部质心不在原点时，只插值 body origin 是否等价？

答案：不等价。 当前 sweep 插值质心，再减去旋转后的局部质心偏移。

题七：普通 fast dynamic 是否查询运动学和动态树？

答案：当前 CCD 路径不查询。 它只查 static；bullet 才追加另外两棵树。

题八：两个 bullet 之间是否使用这条 TOI 路径互相检测？

答案：当前回调显式跳过目标 bullet。

题九：CCD 命中后是否完整模拟剩余时间？

答案：没有。 当前截断末位姿，只撤销损失时间的重力速度贡献。

题十：旧 AABB 与新 AABB 的并是否严格覆盖任意中间旋转？

答案：不是。 细杆中途更宽的例子说明需要谨慎。

题十一：TOI 返回 `Failed` 是否说明确定没有碰撞？

答案：不能。 它表示算法没有正常获得所需结论。

题十二：连续传感器上限 8 是否意味着最多检查 8 个 TOI 候选？

答案：不是。 8 是相关传感器记录容量，不是所有 TOI 候选数量上限。

21. 源码阅读索引与验证范围

主题	当前入口与范围
sweep 变换	`b2GetSweepTransform`，`14–29`
分离函数构造	`b2MakeSeparationFunction`，`953–1026`
最深特征选择	`b2FindMinSeparation`，`1029–1093`
固定特征求值	`b2EvaluateSeparation`，`1097–1143`
TOI 主流程	`b2TimeOfImpact`，`1148–1440`
根求解	二分与假位法交替，`1346–1395`
快物体识别	`b2FinalizeBodiesTask`，核心判断 `607–665`
连续查询回调	`b2ContinuousQueryCallback`，`183–380`
CCD 截断	`b2SolveContinuous`，`384–556`
bullet 任务	`b2BulletBodyTask`，`1236–1248`
主调度位置	`b2Solve`，重点 `1610`、`1873`、`1902`、`1952`

本章旋转根表与细杆包围尺寸经过独立算术运行核验。

源码分支经过静态阅读核对；没有据此宣称运行过 C 构建、C 单元测试或完整世界场景仿真。

建议先用解析圆例作为简单基准，再观察旋转例的每次区间更新，最后对照世界 CCD 的对象选择和时间损失行为。

返回目录 · 上一章：GJK 与形状查询

09 • 碰撞冲量与约束方程

返回总目录 · 下一章：累计冲量、摩擦与热启动

> 本章先从动量定理推导单点碰撞，再把转动加入公式，最后认识多个接触点的耦合。 源码基准：Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b` 。 本章的理想碰撞公式不等于本版本完整求解流程；实际恢复时序见第12章。

插图按推导顺序出现：先认 A/B 与 u 的方向，再算对心冲量，然后加入转动、接触点速度、Jacobian、有效质量和两点耦合。

1. 先约定方向，避免所有公式一起出错

设接触涉及刚体 A、B，单位法线 $\mathbf{n}$ 从 A 指向 B。

符号	含义	SI单位
m_A, m_B	质量	kg
$\mathbf{v}_A, \mathbf{v}_B$	质心速度	m/s
I_A, I_B	绕质心的转动惯量	kg·m²
ω_A, ω_B	逆时针为正的角速度	rad/s
$\mathbf{r}_A, \mathbf{r}_B$	从各自质心指向接触点的杆臂	m
λ	本次标量冲量	N·s
Λ	当前累计标量冲量	N·s
$\Delta\Lambda$	对累计量的修正	N·s
e	恢复系数	无量纲

本章按通常力学约定将弧度视为无量纲。

暂不考虑转动时，相对法向速度是

$$u = (\mathbf{v}_B - \mathbf{v}_A) \cdot \mathbf{n}.$$

- $u < 0$：B相对A沿法线反方向运动，双方接近。
- $u = 0$：当前法向相对速度为零。
- $u > 0$：双方正在分离。

地面为A、箱子为B、法线向上时，下落箱子的 u 应为负。

这个小检查应在写第一行求解代码前完成。

 图 9-1　法向相对速度：对撞为负，分离为正

看图： 左图两球对撞，$u<0$。右图地面为 A、箱子下落，$u=v_{By}<0$。两种情形都用 $u=(\mathbf{v}_B-\mathbf{v}_A)\cdot\mathbf{n}$。法线或 A/B 任一写反，后面冲量符号全错。

2. 为什么求冲量，而不直接猜接触力

牛顿第二定律为

$$m\frac{d\mathbf{v}}{dt}=\mathbf{F}.$$

对短暂接触时间积分：

$$m(\mathbf{v}^+-\mathbf{v}^-)=\int \mathbf{F}\,dt=\mathbf{P}.$$

因此

$$\Delta\mathbf{v}=m^{-1}\mathbf{P}.$$

这里 $\mathbf{P}$ 是冲量，不是力。

其单位检查为

$$[\mathbf{P}]=\mathrm{N\,s}=\mathrm{kg\,m/s},\qquad [m^{-1}\mathbf{P}]=\mathrm{m/s}.$$

理想瞬时碰撞直接求接触力的时间积分，避免先猜一个巨大接触力。

这并不表示真实接触没有形变或有限接触时间，而是刚体离散模型做了简化。

3. 无转动单点碰撞的完整推导

令B受到 $\mathbf{P}=\lambda\mathbf{n}$，A受到反向冲量。

动量定理分别给出

$$\mathbf{v}_A^+=\mathbf{v}_A^--m_A^{-1}\lambda\mathbf{n},$$
$$\mathbf{v}_B^+=\mathbf{v}_B^-+m_B^{-1}\lambda\mathbf{n}.$$

相减后沿法线投影：

$$u^+=u^-+(m_A^{-1}+m_B^{-1})\lambda.$$

定义

$$K=m_A^{-1}+m_B^{-1}.$$

则 $K\lambda$ 就是这次冲量造成的相对法向速度变化。

恢复系数模型给出另一条条件：

$$u^+=-eu^-.$$

它是材料行为的近似模型，不是动量守恒自动提供的结论。

两式联立：

$$u^- + K\lambda = -eu^-,$$

$$\boxed{\lambda = -\frac{(1+e)u^-}{K}}.$$

3.1 代入两个有限质量物体

取

$$m_A = 2\ \mathrm{kg}, \quad m_B = 3\ \mathrm{kg}, \quad v_A^- = 3\ \mathrm{m/s}, \quad v_B^- = -1\ \mathrm{m/s}, \quad e = 0.5.$$

所有速度均指沿 $\mathbf{n}$ 的分量。

先计算接近速度和响应系数：

$$u^- = -1 - 3 = -4\ \mathrm{m/s}, \qquad K = \frac{1}{2} + \frac{1}{3} = \frac{5}{6}\ \mathrm{kg}^{-1}.$$

再计算冲量：

$$\lambda = -\frac{1.5(-4)}{5/6} = 7.2\ \mathrm{N\,s}.$$

碰后速度为

$$v_A^+ = 3 - \frac{7.2}{2} = -0.6\ \mathrm{m/s},$$

$$v_B^+ = -1 + \frac{7.2}{3} = 1.4\ \mathrm{m/s}.$$

两项校验：

$$v_B^+ - v_A^+ = 2 = -0.5(-4),$$

$$2(3) + 3(-1) = 3 = 2(-0.6) + 3(1.4).$$

恢复关系和总线动量同时满足。

图 9-2　对心碰撞：冲量同时改两边速度，总线动量不变

mA=2 kg，mB=3 kg，vA⁻=3，vB⁻=−1，e=0.5。λ=7.2 N·s。碰后 −0.6 与 1.4。



看图对公式： 先由碰前速度算出 $u^- = -4$，再用 $K = 5/6$ 换成冲量 $\lambda = 7.2$。碰后 $u^+ = 2 = -eu^-$，总线动量仍是 3。动能少掉的 7.2 J 就是恢复系数小于 1 吃掉的。三项必须同时成立。

3.2 边界不能省略

- $e = 0$：消除接近速度，不要求反弹。
- $e = 1$：理想孤立无摩擦单碰撞不损失该法向模式的动能。
- $u^- \geq 0$：已经分离，不能无条件套公式产生负接触冲量。
- $K = 0$：当前方向没有可改变的自由度，必须避免除零。
- 静态地面吸收反作用但不更新速度；移动刚体自身不是封闭系统。

4. 撞在边角上，为什么会转动

平面叉积定义为

$$\mathbf{a} \times \mathbf{b} = a_x b_y - a_y b_x.$$

接触点相对质心的杆臂是

$$\mathbf{r} = \mathbf{p} - \mathbf{c}.$$

角动量定理给出

$$I(\omega^+ - \omega^-) = \mathbf{r} \times \mathbf{P},$$

所以

$$\boxed{\Delta\omega = I^{-1}(\mathbf{r} \times \mathbf{P})}.$$

量纲为

$$[\mathbf{r} \times \mathbf{P}] = \mathrm{kg\,m^2/s}, \qquad [I^{-1}(\mathbf{r} \times \mathbf{P})] = \mathrm{s^{-1}}.$$

例：$\mathbf{r} = (0.5, -0.5)\ \mathrm{m}$、$\mathbf{P} = (0, 2)\ \mathrm{N\,s}$、$I = 0.5\ \mathrm{kg\,m^2}$。

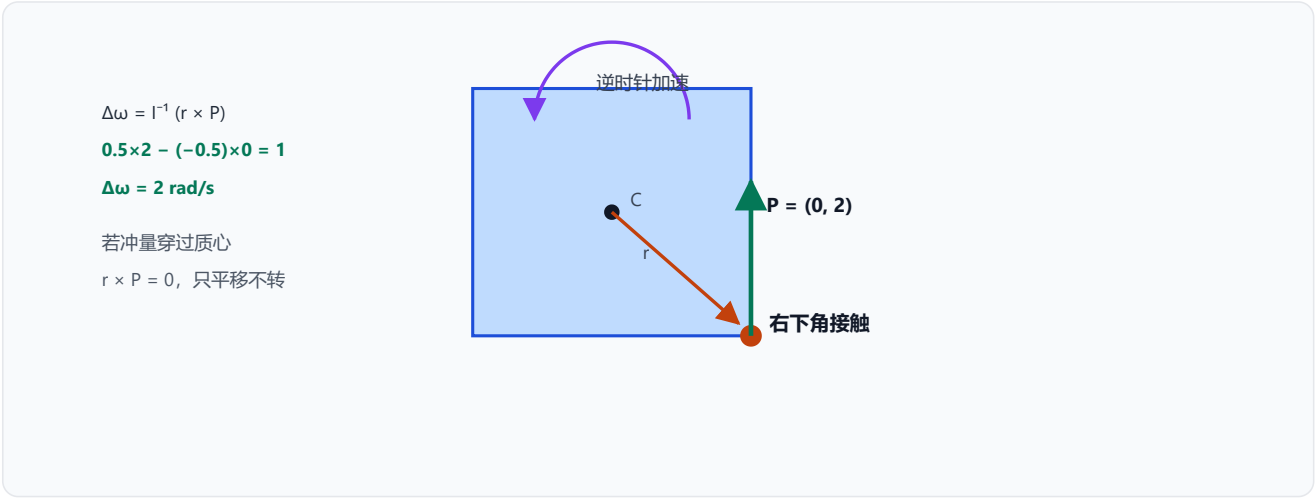
$$\mathbf{r} \times \mathbf{P} = 0.5(2) - (-0.5)(0) = 1\ \mathrm{kg\,m^2/s},$$
$$\Delta\omega = 2(1) = 2\ \mathrm{rad/s}.$$

右下角向上受冲量，物体逆时针加速。

若冲量通过质心，$\mathbf{r} \times \mathbf{P} = 0$，不会由这一冲量产生角速度变化。

图 9-3　撞在边角上：冲量不经过质心就会转

r=(0.5, −0.5) m，P=(0, 2) N·s，I=0.5。r×P=1，Δω=2 rad/s，逆时针。



看图： 杆臂 $\mathbf{r} = (0.5, -0.5)$ 与向上冲量叉乘得 $r \times P = 1$，于是 $\Delta\omega = 2$。正撞可以只改平动；边角碰撞必须带上这一项，否则“看起来撞偏了却不转”。

5. 接触点速度不是质心速度

刚体随体点的位置可写成

$$\mathbf{p} = \mathbf{c} + R(\theta)\mathbf{r}_{\text{local}}.$$

对时间求导，得到

$$\mathbf{v}_p = \mathbf{v} + \omega \times \mathbf{r}, \qquad \omega \times \mathbf{r} = (-\omega r_y, \omega r_x).$$

例如 $\omega = 2$、$\mathbf{r} = (0.5, -0.5)$，转动贡献为

$$\omega \times \mathbf{r} = (1, 1)\ \text{m/s}.$$

即使质心静止，这个点也在运动。

两刚体接触点的相对速度因此是

$$\mathbf{v}_{\text{rel}} = (\mathbf{v}_B + \omega_B \times \mathbf{r}_B) - (\mathbf{v}_A + \omega_A \times \mathbf{r}_A).$$

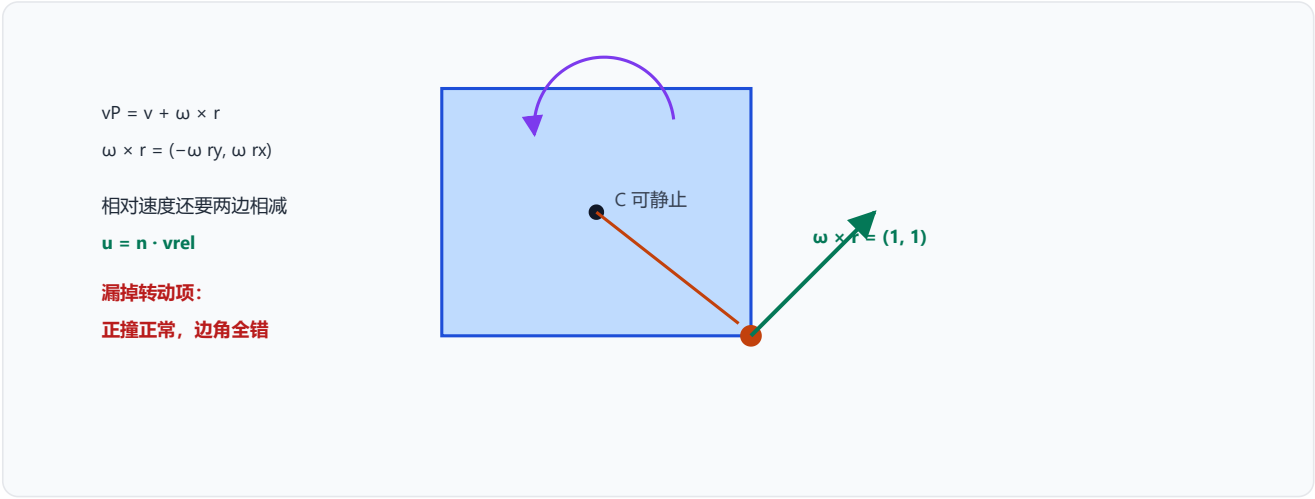
真正需要约束的法向速度为

$$u = \mathbf{n} \cdot \mathbf{v}_{\text{rel}}.$$

忽略转动项，是“正撞正常、边角碰撞异常”的常见原因。

图 9-4　接触点速度不是质心速度

ω=2，r=(0.5, −0.5)，ω×r=(1,1)。即使质心静止，这个角点也在动。



看图： 约束看的是角点 $\mathbf{v}_p = \mathbf{v} + \omega \times \mathbf{r}$。$\omega \times r = (1, 1)$。真正约束的是它在法线上的分量，不是质心速度。只改质心，边角碰撞会对不上。

6. Jacobian只是一行已经推出来的系数

利用恒等式

$$\mathbf{n} \cdot (\omega \times \mathbf{r}) = \omega(\mathbf{r} \times \mathbf{n}),$$

展开法向速度：

$$u = -\mathbf{n} \cdot \mathbf{v}_A - (\mathbf{r}_A \times \mathbf{n})\omega_A + \mathbf{n} \cdot \mathbf{v}_B + (\mathbf{r}_B \times \mathbf{n})\omega_B.$$

把六个速度排成列：

$$\mathbf{V} = (v_{Ax}, v_{Ay}, \omega_A, v_{Bx}, v_{By}, \omega_B)^T.$$

把相应系数排成一行：

$$J = (-n_x, -n_y, -\mathbf{r}_A \times \mathbf{n}, n_x, n_y, \mathbf{r}_B \times \mathbf{n}).$$

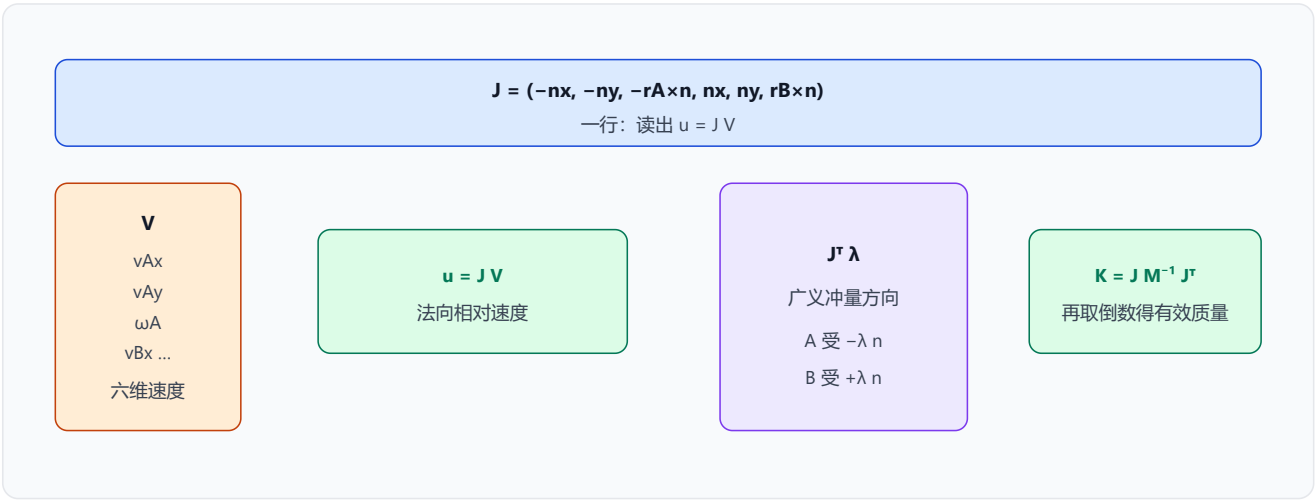
于是

$$\boxed{u = J\mathbf{V}}.$$

这就是接触法向的Jacobian。

图 9-5　Jacobian：一行系数读速度，转置后把冲量播回去

u = J V，ΔV = M⁻¹ Jᵀ λ。不要要求 J 的每一列单位相同：它们配对的是不同广义速度。



看图： 上一行用 J 读出 $u = J\mathbf{V}$；$J^T\lambda$ 把标量冲量播回两个刚体。合在一起就是 $K = JM^{-1}J^T$。

平动列无量纲，转动列具有长度，最终每项均为m/s。

不要要求Jacobian中的不同列具有相同单位：它们配对的是不同类型的广义速度。

定义广义逆质量矩阵：

$$M^{-1} = \mathrm{diag}(m_A^{-1}, m_A^{-1}, I_A^{-1}, m_B^{-1}, m_B^{-1}, I_B^{-1}).$$

作用反作用和角动量更新合并为

$$\boxed{\Delta\mathbf{V} = M^{-1}J^T\lambda}.$$

为什么出现转置?

若广义微位移 $\delta\mathbf{q}$ 产生约束位移 $\delta c = J\delta\mathbf{q}$，则

$$\lambda\delta c = \lambda J\delta\mathbf{q} = (J^T\lambda)^T\delta\mathbf{q}.$$

与广义坐标配对的冲量方向因此是 $J^T\lambda$。

这里是在辨认广义作用方向，不是在断言有恢复和摩擦的接触总功为零。

7. 有效质量：单位冲量能改多少速度

把速度更新左乘 J：

$$\Delta u = JM^{-1}J^T\lambda = K\lambda.$$

展开得到

$$K = m_A^{-1} + m_B^{-1} + I_A^{-1}(\mathbf{r}_A \times \mathbf{n})^2 + I_B^{-1}(\mathbf{r}_B \times \mathbf{n})^2.$$

因此

$$\boxed{m_{\text{eff}} = \frac{1}{K}}.$$

- K 的单位为kg^{-1}，表示冲量到相对速度的响应。
- m_{eff} 的单位为kg，才叫有效质量。
- 源码 `normalMass` 存的是 m_{eff}，不是其倒数。

实际表达式如下：

```
float rnA = b2Cross( rA, normal );
float rnB = b2Cross( rB, normal );
float kNormal = mA + mB + iA * rnA * rnA + iB * rnB * rnB;
cp->normalMass = kNormal > 0.0f ? 1.0f / kNormal : 0.0f;
```

这里局部变量 `mA`、`mB` 实际存逆质量。

7.1 偏心接触的数值例

A静态；B取 $m = 1\ \text{kg}$、$I = 0.5\ \text{kg}\,\text{m}^2$。

$$\mathbf{r}_B = (0.5, -0.5)\ \text{m}, \quad \mathbf{n} = (0, 1), \quad \mathbf{v}_B = (0, -4)\ \text{m/s}, \quad \omega_B = 0.$$

先计算

$$\mathbf{r}_B \times \mathbf{n} = 0.5\ \text{m}, \quad K = 1 + 2(0.5)^2 = 1.5\ \text{kg}^{-1}, \quad m_{\text{eff}} = \frac{2}{3}\ \text{kg}.$$

无恢复、无偏置时，消除当前接近速度需要

$$\lambda = -\frac{2}{3}(-4) = \frac{8}{3}\ \text{N}\,\text{s}.$$

结果是

$$v_y = -4 + \frac{8}{3} = -\frac{4}{3}, \qquad \omega = 2\left(0.5\frac{8}{3}\right) = \frac{8}{3}.$$

接触点法向速度为

$$u = v_y + 0.5\omega = -\frac{4}{3} + \frac{4}{3} = 0.$$

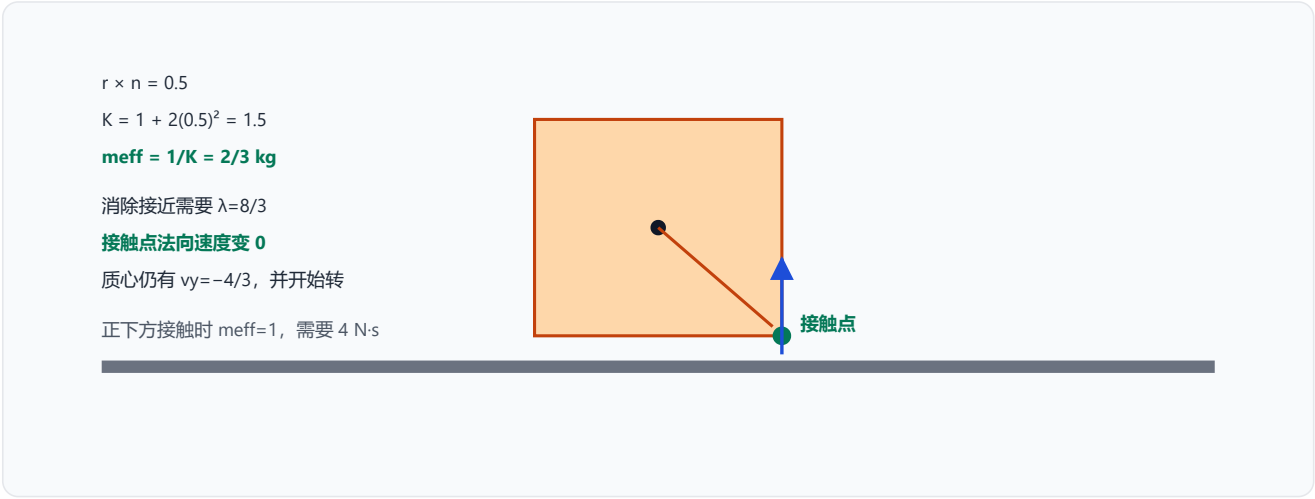
接触点停住，不代表质心停住：物体可以转着避让。

若改为质心正下方接触，$\mathbf{r} \times \mathbf{n} = 0$，则 $K = 1$、有效质量为1kg，需要4N·s。

杆臂使物体更容易转开，所以所需冲量变小。

图 9-6　有效质量：杆臂越大，同一冲量越容易把点转开

地面静态。B：m=1，I=0.5，r=(0.5,−0.5)，n=(0,1)，vy=−4。meff=2/3 kg，λ=8/3。



看图对比： 杆臂让 K 从 1 增到 1.5，所需冲量从 4 降到 $8/3$。正下方接触时 $m_{\text{eff}} = 1$ kg；撞在角上变成 $2/3$ kg。接触点停住后，质心仍可带着转动向下。

8. 硬约束bias的符号：目标速度是负bias

对硬约束的活动标量行，写法为

$$u + b = 0.$$

其中 b 的单位为m/s，目标速度是 $-b$。软约束还包含累计冲量柔度项，第 11 章会得到 $u^+ + \beta c + \gamma\Lambda^+ = 0$；不能把软式中已乘 `massScale` 的源码 `velocityBias` 直接代入这里的硬约束结论。

若恢复目标为 $u^+ = -eu_{\text{old}}$，应设置

$$\boxed{b = eu_{\text{old}}}.$$

碰前接近时 $u_{\text{old}} < 0$，所以恢复bias为负。

例：$u_{\text{old}} = -4\ \text{m/s}$、$e = 0.5$：

$$b = -2\ \text{m/s}, \qquad u^+ = 2\ \text{m/s}.$$

若误写 $b = -eu_{\text{old}} = 2$，目标反而是继续接近的 $-2\ \text{m/s}$。

本版本字段 `restitutionVelocity` 存正的恢复目标；Relax取其负值作为bias。

```
// Restitution in the relax phase only.
if ( useBias == false && cp->restitutionVelocity > 0.0f )
{
          velocityBias = b2MinFloat( velocityBias, -cp->restitutionVelocity );
          stopRestitution = stopRestitution && s > 0.0f;
```

这里的目标来自历史数据，第12章会追到它的真正生成位置。

9. 单向接触不是无条件等式

无黏附的法向接触只能推，不能拉。

对局部硬接触速度子问题，条件为

$$\Lambda \geq 0, \qquad w = u + b \geq 0, \qquad \Lambda w = 0.$$

简写为

$$0 \leq \Lambda \perp w \geq 0.$$

含义是：

1. 没有接触冲量时，可以自由分离。
2. 接触冲量为正时，活动约束的速度残差应为零。
3. 不允许用负法向冲量把已经离开的物体吸回来。

令自由速度为 $\mathbf{V}_{\text{free}}$，则

$$w = J\mathbf{V}_{\text{free}} + b + K\Lambda = q + K\Lambda.$$

单个点且 $K > 0$ 时：

$$\Lambda = \begin{cases} 0, & q \geq 0, \\ -q/K, & q < 0. \end{cases}$$

例：$q = -3\ \mathrm{m/s}$、$K = 2\ \mathrm{kg}^{-1}$，得到 $\Lambda = 1.5\ \mathrm{N\,s}$。

若 $q = +3$，则应取零，而不是负冲量。

这并不保证已有几何穿透瞬间消失；位置误差、软度和推测模型将在第11章加入。

图 9-7　无黏附接触是互补条件，不是无条件等式

0 ≤ Λ ⊥ w ≥ 0。能自由离开；只有还在接近时才施加正冲量。



看图： 左图还在砸向地面，施加正冲量；右图已经跳开，冲量必须是 0。负的累计法向冲量等于把物体吸回去。

10. 两个接触点为什么互相影响

取静态地面上的动态物体：

$$m = 1, \quad I = 0.5, \quad \mathbf{r}_1 = (-0.5, -0.5), \quad \mathbf{r}_2 = (0.5, -0.5), \quad \mathbf{n} = (0, 1).$$

两个法向约束的耦合系数为

$$K_{ij} = m^{-1} + I^{-1}(\mathbf{r}_i \times \mathbf{n})(\mathbf{r}_j \times \mathbf{n}).$$

代入：

$$K = \begin{pmatrix} 1.5 & 0.5 \\ 0.5 & 1.5 \end{pmatrix}.$$

非对角的0.5表示：另一个点施加冲量，也会改变本点速度。

初始 $v_y = -4, \omega = 0$，若两个点都活动，联立解为

$$K \begin{pmatrix} \Lambda_1 \\ \Lambda_2 \end{pmatrix} = \begin{pmatrix} 4 \\ 4 \end{pmatrix}, \qquad \Lambda_1 = \Lambda_2 = 2.$$

于是 $v_y = -4 + 2 + 2 = 0$，$\omega = -2 + 2 = 0$。

若候选解有负分量，还必须改查单点活动或全不活动；不能把负解直接取绝对值。

10.1 顺序求解的两次扫描

从零累计开始，第一点先得到 $\Delta\Lambda_1 = 8/3$。

此时 $v_y = -4/3$、$\omega = -8/3$，第二点法向速度为 $-8/3$。

第二点随后得到 $\Delta\Lambda_2 = 16/9$。

扫描结束	Λ_1	Λ_2	v_y	ω	u_1	u_2
第1扫	$8/3$	$16/9$	$4/9$	$-8/9$	$8/9$	0
第2扫	$56/27$	$160/81$	$4/81$	$-8/81$	$8/81$	0
联立解	2	2	0	0	0	0

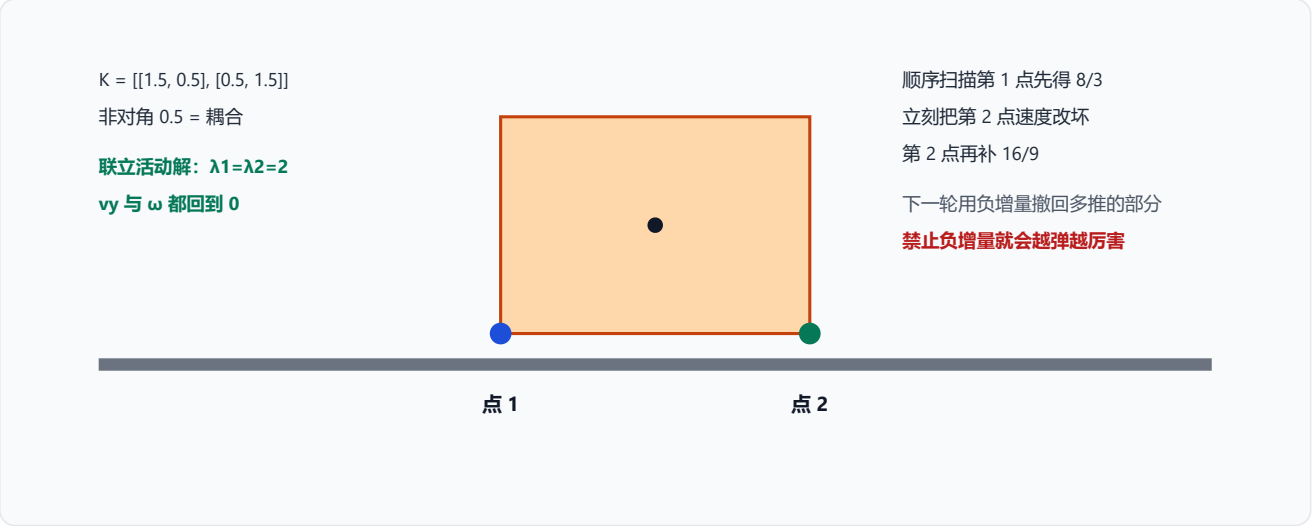
第2扫第一点需要修正 $-16/27$，第二点需要修正 $+16/81$。

负修正是撤回先前推多的部分，不是最终接触变成吸力。

这个固定几何线性例每扫残差缩小到原来的 $1/9$；它不是任意摩擦多体场景的普遍收敛倍率。

图 9-8　两个接触点为什么不能各算各的

m=1，I=0.5，左右底角杆臂 (±0.5, −0.5)。推左边也会改右边的法向速度。



看图： 点 2 的冲量会转动箱子，从而改变点 1 的法向速度，所以 K 有非对角元 0.5。顺序扫描先把第一点推多，下一轮用负增量撤回；禁止负增量就会越弹越厉害。

11. 最朴素的实现顺序

依次完成以下版本，每一步都先跑可手算用例：

1. 无旋转单点，验证作用反作用和恢复符号。
2. 加点速度与角冲量，验证边角碰撞。
3. 加累计投影，允许撤回过量冲量。
4. 多点顺序扫描，后一约束立即读取前一约束改过的速度。
5. 再加热启动、摩擦、软度及子步。

教学伪代码，非项目源码：

```
for each contact point:
    if K <= 0: continue
    residual = normalVelocity() + bias
    candidateDelta = -residual / K
    next = max(0, accumulated + candidateDelta)
    actualDelta = next - accumulated
    accumulated = next
    applyImpulse(actualDelta * normal)
```

若所有点都读同一份旧速度，最后再一起写回，就不再是这里描述的顺序Gauss-Seidel。

本版本标量路径在 `contact_solver.c:267-329` 逐点处理，并未在这里联立精确求解上述2×2法向矩阵。

12. 习题、诊断与源码导航

习题	答案
两个1kg物体相对接近6m/s，$e = 0.25$，求冲量	$K = 2$，$\lambda = 3.75\ \mathrm{N\,s}$
第7节锁旋转后有效质量是多少	1kg，而非原来的 $2/3$kg
$u_{old} = -3, e = 0.4$，恢复bias是多少	-1.2m/s，目标为 $+1.2$m/s
第二接触为何破坏第一接触的零残差	因为 $K_{12} \neq 0$，共同刚体传播了速度变化
$K = 0$应怎样处理	当前方向无响应，跳过，不能除零

能量校验也可以用来抓符号错误。理想孤立单碰撞有

$$\Delta T = \lambda u^- + \frac{1}{2} K \lambda^2 = -\frac{1 - e^2}{2K} (u^-)^2.$$

第3节代入得到 $\Delta T = -7.2\ \mathrm{J}$，与动能从10.5J降到3.3J一致。

有摩擦、位置偏置、外力或多个耦合接触时，不要把这条单模式结果当作整体求解器保能承诺。

现象	优先检查
落地后继续加速向下	法线方向、B减A、A减B加的符号
正撞正确，边角错误	是否漏掉 $\omega \times \mathbf{r}$ 或写反叉积
杆臂越大，需要冲量反而越大	是否把 K 当作有效质量
多点支撑越迭代越弹	是否禁止负增量，或重复施加累计
主题	源码入口
法向/切向有效质量	`contact_solver.c:123-131`
点速度、恢复偏置	`contact_solver.c:296-305`
累计投影、冲量应用	`contact_solver.c:308-328`
恢复历史来源	`contact.c:627-645`

最后才需要关心优化：图着色和宽向量组织这些相同的局部计算，但不能代替对冲量方程的理解。

返回总目录 · 继续第10章

10 • 累计冲量、摩擦与热启动

返回总目录 · 上一章 · 下一章

> 本章解决三个实现问题：为什么夹累计而不是夹增量，如何真正施加热启动，以及摩擦怎样与法向支撑耦合。 源码基准：Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b`。

1. 三个冲量符号不能混用

符号	含义	单位
Λ_{old}	当前已有的累计冲量估计	N·s
d	根据当前残差算出的未经限制修正	N·s
Λ_{new}	投影后的新累计	N·s
$\Delta\Lambda$	真正施加到速度上的增量	N·s
u	当前相对约束速度	m/s
b	速度偏置	m/s
K	冲量到相对速度的响应	kg^{-1}

第9章已经得到

$$d = -\frac{u + b}{K}.$$

但这个候选还没有满足接触只能推、不能拉的限制。

限制作用于累计量：

$$\Lambda_{\text{new}} = \max(0, \Lambda_{\text{old}} + d).$$

真正施加量必须重新计算：

$$\boxed{\Delta\Lambda = \Lambda_{\text{new}} - \Lambda_{\text{old}}}.$$

随后才更新缓存并应用冲量。

2. 为什么不能把每次增量都夹成非负

“接触不能吸引”要求 $\Lambda \geq 0$，不要求每次 $\Delta\Lambda \geq 0$。

先前迭代推多了，就必须能撤回一部分。

旧累计	候选修正	新累计	真正施加量	含义
3	−1	2	−1	撤回部分过量冲量

旧累计	候选修正	新累计	真正施加量	含义
3	−5	0	−3	最多撤回已有的3
3	+1	4	+1	只补上缺少的1

表中所有数值单位均为N·s。

第一行若错误地执行 $\max(d, 0)$，修正变成零，永远无法减少过大支撑。

第二行若直接施加候选−5，最终累计变成−2，产生不允许的吸力。

第三行若将新累计 4 整笔再施加，实际施加了 4 N·s，而正确增量只有 1 N·s，因此额外多施加 3 N·s。速度误差还需乘以相应逆质量与角惯量响应，不能把冲量数值直接当作速度。

正确源码把“投影”和“重新算差额”明确分开：

```
// clamp the accumulated impulse
float newImpulse = b2MaxFloat( cp->normalImpulse + impulse, 0.0f );
impulse = newImpulse - cp->normalImpulse;
cp->normalImpulse = newImpulse;
cp->totalNormalImpulse += impulse;
```

这里最后一行加的是带符号增量，负修正不会被丢掉。

2.1 投影在数学上做了什么

记非负半轴投影为

$$\Pi_{[0,\infty)}(x) = \max(x, 0).$$

顺序求解的一行更新为

$$\Lambda_i \leftarrow \Pi_{[0,\infty)}\left(\Lambda_i - \frac{w_i}{K_{ii}}\right), \qquad w_i = J_i\mathbf{V} + b_i.$$

量纲检查：w_i/K_{ii} 为 $(\mathrm{m/s})/(\mathrm{kg}^{-1}) = \mathrm{N\,s}$。

投影将候选解返回单向接触允许的集合；它不是随意添加的防抖经验值。

一个容易忽略的区别：

- 零累计且正在分离：负候选全部丢弃，不施加吸力。
- 正累计且正在分离：允许减少旧估计，但不能降到零以下。

3. 热启动必须让冲量账和速度账一致

仅保存上次 Λ，并不等于已经完成热启动。

若把初始累计猜测设为 Λ_0，速度也必须初始化为

$$\mathbf{V}_0 = \mathbf{V}_{\text{free}} + M^{-1}J^T\Lambda_0.$$

之后施加累计变化 $\Lambda - \Lambda_0$，得到

$$\mathbf{V} = \mathbf{V}_0 + M^{-1}J^T(\Lambda - \Lambda_0) = \mathbf{V}_{\text{free}} + M^{-1}J^T\Lambda.$$

这条恒等式就是热启动的正确性账本。

若缓存说已经施加了2N·s，速度里却没有这2N·s的作用，后续投影便建立在错误前提上。

3.1 静止支撑的最小算例

A为静态地面，B质量1kg，无转动影响。

本子步外力积分后

$$v_y = -0.1\ \mathrm{m/s}, \qquad K = 1\ \mathrm{kg}^{-1}.$$

历史支撑估计为

$$\Lambda_0 = 0.1\ \mathrm{N\,s}.$$

真正施加缓存后

$$v_y = -0.1 + 1(0.1) = 0.$$

本次求解只需

$$d = -u/K = 0.$$

累计仍为0.1N·s，本子步实际得到的支撑冲量也是0.1N·s。

下一子步重力又使速度下降，再施加上一子步的支撑估计，是为新的时间区间预测支撑，不是重复同一历史事件。

3.2 两种不完整热启动

错误一：保留缓存，却不施加。

速度仍为−0.1，求解又增加0.1，缓存变0.2，但实际只施加0.1。

错误二：施加缓存，随后清空累计。

速度已包含缓存作用，求解器却认为累计为零，不能正确撤回这部分估计。

两者都会使缓存账和速度账分离。

3.3 过期缓存必须能撤回

B本来以 $+0.2\ \mathrm{m/s}$ 离开，旧缓存为0.1N·s。

WarmStart后速度为0.3，候选修正为−0.3。

$$\Lambda_{\text{new}} = \max(0, 0.1 - 0.3) = 0,$$

$$\Delta\Lambda = 0 - 0.1 = -0.1.$$

最终速度回到 $+0.2\ \mathrm{m/s}$，而非强行变成零。

这正是“缓存预测”和“单向投影”配合的意义。

4. Box2D真正施加缓存的位置

Prepare先将跨大步缓存复制到约束副本。

```
cp->normalImpulse = warmStartScale * mp->normalImpulse;
cp->tangentImpulse = warmStartScale * mp->tangentImpulse;
cp->totalNormalImpulse = 0.0f;
```

之后，每个子步WarmStart都将法向和切向缓存组成冲量并真正作用到速度：

```
b2Vec2 P = b2Add( b2MulSV( cp->normalImpulse, normal ), b2MulSV( cp->tangentImpulse, tangent ) );

cp->totalNormalImpulse += cp->normalImpulse;

wA -= iA * b2Cross( rA, P );
vA = b2MulAdd( vA, -mA, P );
wB += iB * b2Cross( rB, P );
vB = b2MulAdd( vB, mB, P );
```

本版本有两个需要如实说明的细节：

1. Prepare中的 `warmStartScale` 为0或1；此处没有按新旧时间步比值缩放。
2. 关闭热启动会清掉跨大步种子，但子步WarmStart函数仍被调用，函数内部没有再次跳过开关。

因此不能把“关闭热启动”描述成删除整个子步WarmStart阶段。

5. 切向摩擦的方向与速度

本版本使用右垂线：

$$\mathbf{t} = \mathrm{RightPerp}(\mathbf{n}) = (n_y, -n_x).$$

法线向上 $(0,1)$ 时，切线就是向右 $(1,0)$。

切向相对速度定义为

$$u_t = \mathbf{t} \cdot \mathbf{v}_{\mathrm{rel}} - v_{\mathrm{belt}}.$$

其中：

量	含义	单位
$\mathbf{v}_{\mathrm{rel}}$	含旋转的两接触点速度差	m/s
v_{belt}	传送带表面相对目标速度	m/s

量	含义	单位
u_t	相对目标的剩余滑移	m/s
μ	库仑摩擦系数	无量纲
Λ_t	累计切向冲量	N·s

将第9章法向推导中的 $\mathbf{n}$ 换成 $\mathbf{t}$，得到

$$K_t = m_A^{-1} + m_B^{-1} + I_A^{-1}(\mathbf{r}_A \times \mathbf{t})^2 + I_B^{-1}(\mathbf{r}_B \times \mathbf{t})^2.$$

有效切向质量及无界候选为

$$m_t = K_t^{-1}, \qquad d_t = -m_t u_t.$$

这一步只回答“停住滑移需要多少冲量”，还没有回答“表面能提供多少冲量”。

6. 从库仑力限制推到冲量限制

库仑模型规定

$$|F_t| \le \mu F_n.$$

假设本子步 μ 不变，对时间积分并使用三角不等式：

$$\left|\int F_t dt\right| \le \int |F_t| dt \le \mu \int F_n dt.$$

于是

$$\boxed{|\Lambda_t| \le \mu \Lambda_n}.$$

两侧均为N·s。

正确切向累计投影为

$$\Lambda_{t,\text{new}} = \text{clamp}(\Lambda_{t,\text{old}} - m_t u_t, -\mu\Lambda_n, +\mu\Lambda_n).$$

真正应用的仍是差额：

$$\Delta\Lambda_t = \Lambda_{t,\text{new}} - \Lambda_{t,\text{old}}.$$

```
float vt = b2Dot( b2Sub( vrB, vrA ), tangent ) - constraint->tangentSpeed;

// incremental tangent impulse
float impulse = cp->tangentMass * ( -vt );

// clamp the accumulated force
float maxFriction = friction * cp->normalImpulse;
float newImpulse = b2ClampFloat( cp->tangentImpulse + impulse, -maxFriction, maxFriction );
impulse = newImpulse - cp->tangentImpulse;
cp->tangentImpulse = newImpulse;
```

源码注释写 `force`，但此处变量实际是冲量，不能据此误写单位。

摩擦界使用当前累计法向冲量，不是刚才法向求解的增量。

稳定支撑时法向增量可能接近零，累计支撑却不为零；若用增量限幅，接触会错误地失去摩擦。

7. 摩擦算例：需要多少与能给多少

取

$$m_t = 0.5\ \mathrm{kg},\quad K_t = 2\ \mathrm{kg}^{-1},\quad u_t = 3\ \mathrm{m/s},\quad \Lambda_n = 2\ \mathrm{N\,s},\quad \Lambda_{t,old} = 0.$$

无界需求是

$$d_t = -0.5(3) = -1.5\ \mathrm{N\,s}.$$

7.1 摩擦不足

若 $\mu = 0.4$，允许范围为

$$[-\mu\Lambda_n, +\mu\Lambda_n] = [-0.8, +0.8]\ \mathrm{N\,s}.$$

实际施加−0.8，末滑移为

$$u_t^+ = 3 + 2(-0.8) = 1.4\ \mathrm{m/s}.$$

滑移没有归零，是摩擦能力不足，不是求解器忘记刹车。

7.2 摩擦足够

若 $\mu = 1$，允许范围为±2N·s，候选−1.5合法。

$$u_t^+ = 3 + 2(-1.5) = 0.$$

局部切向约束可以进入黏着状态。

本求解器使用一个摩擦系数与投影，并没有因此实现两套独立的静摩擦系数和动摩擦系数。

7.3 法向支撑变小时，旧摩擦也要缩回

旧切向累计为0.6N·s，新法向累计降到0.5N·s，$\mu = 0.4$。

新摩擦界只有±0.2N·s。

即便当前滑移为零，投影仍需把0.6改为0.2，施加−0.4N·s。

不能仅在 $u_t \neq 0$ 时才检查摩擦限幅。

8. 摩擦为什么会重新改变法向速度

法向与切向的交叉响应为

$$K_{nt} = J_n M^{-1} J_t^T.$$

因为 $\mathbf{n}\cdot\mathbf{t}=0$，平动交叉项消失，但转动项一般不消失：

$$K_{nt}=I_A^{-1}(\mathbf{r}_A\times\mathbf{n})(\mathbf{r}_A\times\mathbf{t})+I_B^{-1}(\mathbf{r}_B\times\mathbf{n})(\mathbf{r}_B\times\mathbf{t}).$$

沿用右下角接触：

$$I_B^{-1}=2,\quad \mathbf{r}_B=(0.5,-0.5),\quad \mathbf{n}=(0,1),\quad \mathbf{t}=(1,0).$$

得到

$$K_{nt}=2(0.5)(0.5)=0.5\ \mathrm{kg}^{-1}.$$

若法向刚解到零，随后切向施加 $-8/9\ \mathrm{N\,s}$：

$$\Delta u_n=K_{nt}\Delta\Lambda_t=0.5\left(-\frac{8}{9}\right)=-\frac{4}{9}\ \mathrm{m/s}.$$

水平摩擦改变角速度，角速度再改变接触点的竖直速度。

因此“法向先满足，摩擦再满足，所以一遍后两者同时满足”是错误推断。

后续迭代或子步会继续修正这个耦合残差。

9. 传送带是在改变表面目标速度

令切线向右，传送带目标为

$$v_{\mathrm{belt}}=2\ \mathrm{m/s}.$$

物体接触点当前水平相对速度为零时：

$$u_t=0-2=-2\ \mathrm{m/s}.$$

摩擦候选为正，向右推动物体。

若物体接触点已向右运动5m/s：

$$u_t=5-2=3\ \mathrm{m/s}.$$

摩擦转而向左，试图回到目标速度2，而不是永远向带子方向加速。

目标能否达到仍受 $\mu\Lambda_n$ 限制。

本版本将两形状材料的切向速度相加：

```
contactSim->tangentSpeed = shapeA->material.tangentSpeed + shapeB->material.tangentSpeed;
```

加号来自两侧表面方向约定；不能脱离A/B及切向约定直接改成“世界速度相减”。

10. 滚动阻力是一条角速度约束

切向摩擦限制接触点滑动；滚动阻力直接限制相对角速度。

令

$$u_r = \omega_B - \omega_A.$$

滚动角冲量 Λ_r 的单位是kg·m²/s，而非N·s。

作用关系为

$$\Delta\omega_A = -I_A^{-1}\Lambda_r, \qquad \Delta\omega_B = I_B^{-1}\Lambda_r.$$

所以

$$\Delta u_r = (I_A^{-1} + I_B^{-1})\Lambda_r.$$

定义有效转动惯量

$$I_{\text{eff}} = \frac{1}{I_A^{-1} + I_B^{-1}},$$

无界候选为

$$d_r = -I_{\text{eff}}(\omega_B - \omega_A).$$

源码 `rollingMass` 实际具有kg·m²单位；名字里的 `Mass` 不应让人误以为它是线性质量。

10.1 滚阻限幅为什么需要长度尺度

本版本限制为

$$|\Lambda_r| \le \rho \sum_j \Lambda_{n,j}.$$

右侧法向冲量单位为kg·m/s，左侧为kg·m²/s，因此

$$[\rho] = \mathrm{m}.$$

材料属性的滚阻比例通常无量纲，接触更新先乘形状半径，得到求解器需要的长度参数。

```
float radiusA = b2GetShapeRadius( shapeA );
float radiusB = b2GetShapeRadius( shapeB );
float maxRadius = b2MaxFloat( radiusA, radiusB );
contactSim->rollingResistance =
          b2MaxFloat( shapeA->material.rollingResistance, shapeB->material.rollingResistance ) *
maxRadius;
```

这里半径不是任意包围半径：

- 圆和胶囊返回自己的半径。
- 多边形返回圆角半径 `polygon.radius`。
- 线段等默认返回零。

因此两个零圆角多边形可能即使设置非零材料滚阻，最终接触滚阻仍为零。

10.2 滚阻数值例

取 $I_A = I_B = 1\ \mathrm{kg\,m^2}$、$\omega_A = 0$、$\omega_B = 2\ \mathrm{rad/s}$。

$$I_{\text{eff}} = 0.5\ \mathrm{kg\,m^2}, \qquad d_r = -1\ \mathrm{kg\,m^2/s}.$$

材料比例0.1，最大形状半径0.5m，总法向累计4N·s：

$$\rho = 0.1(0.5) = 0.05\ \mathrm{m}, \qquad \rho \sum \Lambda_n = 0.05(4) = 0.2\ \mathrm{kg\,m^2/s}.$$

从零累计开始，实际角冲量只能取−0.2。

$$\omega_A^+ = 0.2, \quad \omega_B^+ = 1.8, \quad u_r^+ = 1.6\ \mathrm{rad/s}.$$

滚阻能力不足以本次完全消除相对转动。

11. 三种缓存必须分别理解

机制	缓存什么	解决什么问题
接触点ID匹配	旧点的法向/切向冲量	哪个历史冲量属于新点
冲量热启动	当前约束初始累计估计	如何把历史猜测放进当前速度状态
接触几何回收	锚点、法线、相对位姿等	是否可跳过本次窄相重算

11.1 ID匹配不是按数组下标复制

`contact.c:603-655` 先清零新点缓存，再按16位特征ID匹配旧点。

两点数组顺序对调但ID不变时，应按ID迁移。

没有匹配历史的新点，法向和切向冲量从零开始。

11.2 几何回收不是另一种冲量投影

`physics_world.c:507-585` 检查相对位姿变化等条件，允许保留旧锚点和法线，只更新分离估计。

这是跳过窄相的几何优化，不是只存下一个冲量数。

回收分支提前 `continue`，还会跳过 `b2UpdateContact` 中的材质刷新及相应PreSolve调用路径。

因此不能承诺修改材料后每个被回收接触都立即刷新；具体回收策略见优化章节。

12. Box2D顺序与朴素实现

最朴素实现可以先固定几何，多次扫描法向与摩擦，验证累计投影和动量更新。

教学伪代码，非项目源码：

```
match cached impulses by feature id
apply cached normal, tangent and rolling impulses
for each contact point:
    solve projected accumulated normal impulse
for each contact point:
    solve projected accumulated tangent impulse
```

实际版本则每子步执行：

1. 积分外力速度。
2. WarmStart真正施加缓存。
3. Solve处理法向推测或软穿透修正。
4. 积分位置。
5. Relax先再解法向，然后滚阻，最后摩擦。

摩擦使用Relax法向更新后的累计量，但摩擦之后仍可能留下法向耦合残差。

默认材料混合为 $\mu = \sqrt{\mu_A \mu_B}$、$e = \max(e_A, e_B)$，可被回调替换，不是动量定理。

例如 $\mu_A = 0.25, \mu_B = 1$ 得 $\mu = 0.5$；任意一侧为零时默认混合摩擦为零。

13. 习题、诊断与源码导航

习题	答案
旧法向累计1，候选−1.5，实际施加多少	−1，新累计0
缓存0.2、自由速度−0.2、$K = 1$，真warm后速度	0，后续增量可为0
$\mu = 0.5, \Lambda_n = 3, \Lambda_{t,old} = -1, d_t = -1$	新切向累计−1.5，实际增量−0.5
相对速度+1、带速+3，摩擦倾向哪边	剩余滑移−2，正切向
材料滚阻0.2、半径0.25m、总法向6N·s	$\rho = 0.05$m，角冲量界0.3N·m·s
点数组顺序改变，如何保留缓存	按特征ID而不是下标
现象	优先检查
越迭代推得越大	是否夹增量，或重复施加新累计
静止支撑有力但没有摩擦	是否错误使用法向增量作摩擦界
只在旋转时摩擦方向异常	点速度旋转项、叉积、右垂线方向
改单位后滚阻失真	是否漏掉接触滚阻的长度尺度
材料修改后暂时未刷新	是否命中几何回收提前返回
主题	源码入口

习题	答案
缓存准备	`contact_solver.c:112-133`
真正WarmStart	`contact_solver.c:180-199`
法向累计投影	`contact_solver.c:308-328`
滚阻与摩擦	`contact_solver.c:340-384`
材料尺度与带速	`contact.c:553-572`
ID缓存迁移	`contact.c:603-655`
几何回收	`physics_world.c:507-585`
形状半径含义	`shape.h:107-119`

优化应放在正确实现之后：缓存迁移减少冷启动误差，几何回收减少窄相工作，图着色与宽向量改善执行效率；它们不是同一种优化。

返回总目录 · 继续第11章

11 • 软约束、子步进与Relax

返回总目录 · 上一章 · 下一章

> 本章从弹簧阻尼微分方程推到 `b2MakeSoft` 三系数，再解释子步与Relax为何按当前顺序执行。 源码基准：Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b` 。

1. 为什么只消除接近速度还不够

物体已经穿入地面2cm时，只令接触点法向速度为零，它可能永远停在穿透位置。

另一种极端是要求一个子步纠正全部穿透：

$$u_{\text{target}} = -\frac{s}{h}.$$

取 $s = -0.02\ \text{m}$、$h = 1/240\ \text{s}$，得到

$$u_{\text{target}} = 4.8\ \text{m/s}.$$

对于本来应该安静支撑的接触，这可能制造明显弹跳。

因此要解决两个问题：

1. 位置误差应逐渐减小，而非永远保留。
2. 纠偏速度不应全部成为长期保留的反弹动能。

先构造可控制的弹簧阻尼响应，再讨论怎样安排求解阶段。

2. 一个约束方向上的弹簧阻尼模型

以下推导针对冻结几何、J, K, k, d 的单个双侧线性约束模式，假设 $K > 0, h > 0, f > 0, \zeta \geq 0$。齐次 ODE 暂不计外部驱动和其他约束；实际增量求解时，它们对速度的影响已反映在当前 u 中。零频率另按第 7 节的源码特殊分支处理；$K = 0$ 时不能定义 $m_{eff} = 1/K$。

符号	含义	单位
c	有符号位置误差	m
$u = \dot{c}$	约束方向速度	m/s
$K = JM^{-1}J^T$	冲量响应系数	kg⁻¹
$m_{eff} = 1/K$	有效质量	kg
k	弹簧刚度	N/m
d	线性阻尼系数	N·s/m
f	自然频率	Hz
$\omega = 2\pi f$	自然角频率	s⁻¹

符号	含义	单位
ζ	阻尼比	无量纲
h	子步长度	s

弹簧阻尼力为

$$F = -kc - du.$$

结合牛顿第二定律：

$$m_{eff}\ddot{c} + d\dot{c} + kc = 0.$$

定义

$$k = m_{eff}\omega^2, \qquad d = 2\zeta m_{eff}\omega,$$

便得到标准形式

$$\boxed{\ddot{c} + 2\zeta\omega\dot{c} + \omega^2 c = 0}.$$

连续单自由度系统中，$0 \le \zeta < 1$ 欠阻尼（其中零阻尼不衰减），$\zeta = 1$ 临界阻尼，$\zeta > 1$ 过阻尼。

接触还会单向投影、限幅并与别的刚体耦合，不能据此保证整个场景没有任何振荡。

2.1 为什么用频率而不是固定刚度

若不同物体采用相同 f, ζ，则其局部 k, d 随有效质量一起变化。

这让响应的时间尺度更容易控制，而非质量变大后弹簧突然显著变慢。

例如 $f = 10\ \mathrm{Hz}$、$\zeta = 1$、$m_{eff} = 2\ \mathrm{kg}$：

$$\omega = 20\pi \approx 62.8319\ \mathrm{s}^{-1},$$

$$k = 2(20\pi)^2 \approx 7895.6835\ \mathrm{N/m}, \quad d = 4(20\pi) \approx 251.3274\ \mathrm{N\,s/m}.$$

有效质量翻倍，维持同频率需要刚度和阻尼一起翻倍。

3. 隐式离散：先推目标方程

记子步初误差为 c，步末速度为 u^+。

采用隐式位置近似：

$$c^+ = c + hu^+.$$

用步末弹簧阻尼力近似本步冲量：

$$\Lambda = hF^+ = -h(kc^+ + du^+).$$

代入位置：

$$\Lambda = -hk(c + hu^+) - hdu^+ = -hkc - h(d + hk)u^+.$$

移项并除以 $h(d+hk)$：

$$\boxed{u^{+}+\beta c+\gamma\Lambda=0},$$

其中

$$\beta=\frac{k}{d+hk},\qquad \gamma=\frac{1}{h(d+hk)}.$$

量纲检查：

$$[\beta]=\mathrm{s}^{-1},\quad [\gamma]=\mathrm{kg}^{-1},\quad [\beta c]=[\gamma\Lambda]=\mathrm{m/s}.$$

β 将位置误差转成速度偏置；γ 将累计冲量转成柔度项。

不要把 γ 与无量纲阻尼比 ζ 混淆。

4. 再推“当前累计基础上的增量”

当前累计猜测为 Λ，当前速度 u 已包含该猜测的作用。

再施加 $\Delta\Lambda$ 后：

$$u^{+}=u+K\Delta\Lambda,\qquad \Lambda^{+}=\Lambda+\Delta\Lambda.$$

代入软约束目标：

$$u+K\Delta\Lambda+\beta c+\gamma(\Lambda+\Delta\Lambda)=0.$$

整理：

$$(K+\gamma)\Delta\Lambda=-(u+\beta c+\gamma\Lambda).$$

因此

$$\boxed{\Delta\Lambda=-\frac{u+\beta c+\gamma\Lambda}{K+\gamma}}.$$

这才是软约束冲量修正的来源。

它不是对硬碰撞公式随意乘一个“软度百分比”。

5. 化成 `b2MakeSoft` 的三系数

代入 $k=m_{eff}\omega^2$、$d=2\zeta m_{eff}\omega$：

$$\beta=\frac{\omega}{2\zeta+h\omega},$$

$$\gamma=\frac{K}{h\omega(2\zeta+h\omega)}.$$

定义三个中间量：

$$a_1 = 2\zeta + h\omega, \quad a_2 = h\omega a_1, \quad a_3 = \frac{1}{1 + a_2}.$$

于是

$$\frac{1}{K + \gamma} = m_{eff}\frac{a_2}{1 + a_2}, \qquad \frac{\gamma}{K + \gamma} = \frac{1}{1 + a_2}.$$

定义

$$\mathrm{biasRate} = \frac{\omega}{a_1}, \quad \mathrm{massScale} = a_2 a_3, \quad \mathrm{impulseScale} = a_3.$$

最终增量公式为

$$\boxed{\Delta\Lambda = -m_{eff}\,\mathrm{massScale}(u + \mathrm{biasRate}\,c) - \mathrm{impulseScale}\,\Lambda}.$$

三系数分别具有s^{-1}、无量纲、无量纲单位。

非零频率常规分支满足

$$\mathrm{massScale} + \mathrm{impulseScale} = 1.$$

但不要把它简单说成“本帧和上帧冲量按比例混合”。

这里 Λ 是当前累计估计，可能已经在本子步多次更新；最后一项来自 $\gamma\Lambda$ 柔度项。

实际计算中间量的位置：

```
float omega = 2.0f * B2_PI * hertz;
float a1 = 2.0f * zeta + h * omega;
float a2 = h * omega * a1;
float a3 = 1.0f / ( 1.0f + a2 );
```

返回值为：

```
return (b2Softness){
          .biasRate = omega / a1,
          .massScale = a2 * a3,
          .impulseScale = a3,
};
```

6. 一组真实参数的完整代入

取静态接触实际使用的

$$f = 60\ \mathrm{Hz}, \quad \zeta = 10, \quad h = 1/240\ \mathrm{s}, \quad m_{eff} = 2/3\ \mathrm{kg}.$$

为什么静态接触使用60Hz，将在第8节说明。

先计算

$$\omega = 120\pi \approx 376.9911184308,$$

$$a_1 = 20 + \frac{376.9911184308}{240} \approx 21.5707963268,$$

$$a_2 = \frac{376.9911184308}{240}(21.5707963268) \approx 33.8833276362.$$

于是

系数	数值
`biasRate`	17.4769217009612 s^{-1}
`massScale`	0.971333010129369
`impulseScale`	0.0286669898706311
γ	0.0442695598291461 kg^{-1}

取 $c = -0.02\ \mathrm{m}$、$u = -4\ \mathrm{m/s}$、$\Lambda = 0$。

代码使用的速度偏置是

$$b = \mathrm{massScale}\,\mathrm{biasRate}\,c \approx -0.339518219271798\ \mathrm{m/s}.$$

冲量为

$$\Delta\Lambda = -\frac{2}{3}[0.971333010129369(-4) - 0.339518219271798] \approx 2.816566839859516\ \mathrm{N\,s}.$$

新法向速度为

$$u^+ = -4 + 1.5(2.816566839859516) \approx 0.224850259789273\ \mathrm{m/s}.$$

它没有变成零，因为此处求的是软约束方程。

将结果代回完整残差：

$$u^+ + \beta c + \gamma\Lambda^+ \approx 0.224850259789273 - 0.349538434019223 + 0.124688174229950 \approx 0.$$

这是比“系数看起来合理”更有力的验证。

注意两种偏置记号：$b_{ODE} = \beta c$，而未触发夹限时 $b_{code} = \mathrm{massScale}\,b_{ODE}$。因此源码 `velocityBias` 为 −0.3395182193 m/s，Solve 后法向速度为 +0.2248502598 m/s，并不矛盾；软式并不要求 $u^+ + b_{code} = 0$。

该残差方程针对尚未投影、未夹限偏置的局部 Solve 更新，不是整个 Solve—积分—Relax 流程最终状态必须满足的方程。

接触最后仍要投影累计非负；弹簧双侧方程本身并不禁止吸引。

7. 零频率是特殊分支，不是连续极限

源码明确规定：

```
if ( hertz == 0.0f )
{
          return (b2Softness){
                    .biasRate = 0.0f,
                    .massScale = 0.0f,
                    .impulseScale = 0.0f,
          };
}
```

因此精确零频时两个scale之和为零。

固定 $\zeta > 0$、$h > 0$，令数学上的 $\omega \to 0^+$，却得到

$$\mathrm{biasRate} \to 0, \quad \mathrm{massScale} \to 0, \quad \mathrm{impulseScale} \to 1.$$

两者不同，不能把程序分支称为普通公式的连续极限。

若 $\zeta = 0$ 且 $\omega > 0$，则

$$\mathrm{biasRate} = 1/h, \quad \mathrm{massScale} = \frac{(h\omega)^2}{1 + (h\omega)^2}, \quad \mathrm{impulseScale} = \frac{1}{1 + (h\omega)^2}.$$

这也说明无阻尼时不能使用“频率趋零则biasRate必趋零”的结论。

对接触调用路径：

- $s \le 0$ 的Solve软支路采用三零系数时，本次增量为零。
- 已有缓存不会仅因为这个分支自动清空。
- Relax仍使用硬基线系数，继续解法向。
- $s > 0$ 的推测支路也仍然存在。

所以“零频让整个接触关闭”和“零频自动变无限硬”都不准确。

关节还可能有独立启用开关；特别是pogo先通过 `b2SpringDamper` 构造速度目标，不应冒充所有关节都逐字采用本章增量式。

8. 接触频率先夹限，再给静态接触加倍

世界步长 Δt、子步数 N、子步长度的关系为

$$h = \frac{\Delta t}{N}, \qquad h^{-1} = \frac{N}{\Delta t}.$$

本版本实际有效频率为

$$f_c = \min\left(f_{\mathrm{requested}}, \frac{1}{8h}\right).$$

普通接触使用 f_c，静态软度使用 $2f_c$。

```
// Hertz values get reduced for large time steps
float contactHertz = b2MinFloat( world->contactHertz, 0.125f * context.inv_h );
context.contactSoftness = b2MakeSoft( contactHertz, world->contactDampingRatio, context.h );
context.staticSoftness = b2MakeSoft( 2.0f * contactHertz, world->contactDampingRatio, context.h );
```

默认请求30Hz时：

Δt	N	h	**普通有效频率**	**静态有效频率**
1/60s	4	1/240s	30Hz	60Hz
1/60s	1	1/60s	7.5Hz	15Hz
1/30s	2	1/60s	7.5Hz	15Hz

因此频率设置值不能不经夹限直接代入每个算例。

普通接触满足 $h\omega \le \pi/4$，静态二倍频率后允许到 $\pi/2$。

Overflow按任一侧状态索引是否为null选择静态软度；运动学体可能逆质量为零但仍有状态，不能只检查逆质量。

9. 推测bias：正间隙允许有限接近速度

若当前分离 $s > 0$，局部线性预测为

$$s^+ = s + hu.$$

希望下一子步不越过接触边界：

$$s + hu \ge 0 \quad \Longleftrightarrow \quad u + \frac{s}{h} \ge 0.$$

所以

$$\boxed{b_{spec} = s/h > 0}.$$

正bias不是“向外推的速度”，而是允许仍以一定负速度接近。

例：$s = 0.03\ \mathrm{m}$、$h = 0.01\ \mathrm{s}$，则 $b = 3\ \mathrm{m/s}$。

当前 u	**线性预测间隙**	**零缓存新接触的结果**
−2m/s	0.01m	不需要新法向冲量
−5m/s	−0.02m	需要减慢接近

第二行若 $m_{eff} = 1\ \mathrm{kg}$，则

$$\Delta\Lambda = -1(-5 + 3) = 2\ \mathrm{N\,s}, \qquad u^+ = -3\ \mathrm{m/s}.$$

这只保证当前局部线性预测到达边界，不等于精确连续碰撞检测。

10. 穿透bias：软纠偏并夹住极端偏置

当 $s \le 0$ 且 `useBias=true`：

$$b = \max(\text{massScale}\,\text{biasRate}\,s, -v_{\text{push,max}}).$$

其中 $v_{\text{push,max}}$ 对应 `contactSpeed`，默认SI下为3m/s。

未经夹限为−0.3395时保持原值；为−20时取−3。

必须用max：目标是避免偏置过度负向，而不是挑选更猛烈的负值。

源码增量写成

$$\Delta\Lambda = -m_{eff}(\text{massScale}\,u + b) - \text{impulseScale}\,\Lambda.$$

注意 b 已经含 `massScale`，不能再把整个括号额外乘一次它。

夹限后不再严格等价于无限线性弹簧的隐式离散，这是额外的鲁棒性措施。

严谨说法是“夹住穿透速度偏置项”，不是承诺总接触速度、恢复速度或所有约束作用后的速度都不超过3m/s。

11. 子步不等于固定几何多迭代

两者在解决不同误差：

方法	每次重复改变什么	主要作用
固定几何多次迭代	冲量、速度	降低同一代数系统残差
子步推进	外力速度、冲量、位置、分离估计	降低时间推进及几何滞后误差

本版本每个子步的执行顺序是

1. 积分速度。
2. WarmStart。
3. Solve，`useBias=true`。
4. 积分位置。
5. Relax，`useBias=false`。

Prepare在循环前一次执行，Store在全部子步后执行。

`solver.c:29-30` 设置Solve和Relax各一遍；增加子步不等于在完全相同几何上重复相同方程。

`b2SolverTask` 的旧阶段注释仍列出独立Restitution，但实际执行代码没有该独立阶段，恢复已经位于Relax中。

12. 不重做完整窄相，怎样更新分离

保存大步开始的杆臂 $\mathbf{r}_A, \mathbf{r}_B$、法线 $\mathbf{n}$ 和分离 s_0。

准备阶段定义

$$s_{base} = s_0 - \mathbf{n} \cdot (\mathbf{r}_B - \mathbf{r}_A).$$

子步内用累计位姿增量计算

$$s = s_{base} + \mathbf{n} \cdot [\Delta\mathbf{p}_B - \Delta\mathbf{p}_A + R(\Delta q_B)\mathbf{r}_B - R(\Delta q_A)\mathbf{r}_A].$$

初始 $\Delta\mathbf{p} = 0$、$\Delta q = I$，代入立刻回到 $s = s_0$。

这是检验是否重复加了初始杆臂的简单方法。

一个无旋转数值例：$s_0 = -0.02$m，B相对A向上平移0.005m，则新分离为−0.015m。

实际表达式：

```
// compute current separation
// this is subject to round-off error if the anchor is far from the body center of mass
b2Vec2 ds = b2Add( dp, b2Sub( b2RotateVector( dqB, rB ), b2RotateVector( dqA, rA ) ) );
float s = cp->baseSeparation + b2Dot( ds, normal );
```

必须区分两种用途：

- 估算分离时，使用增量旋转后的杆臂。
- 计算点速度和冲量力矩时，本版本仍使用固定杆臂。

法线和有效质量没有每子步完整重建；这是效率和稳定性的折中，不保证任意大旋转仍精确。

13. Relax为什么在位置积分之后

带穿透bias的Solve先产生纠偏速度，用它积分位置，穿透才会减少。

随后关闭重叠软偏置，再解法向速度，使纠偏带来的分离速度不必一直保留。

若在积分位置前就完全去掉纠偏速度，刚产生的位置修正作用也会被抵消。

教学例Solve后 $u_n = 0.18\ \mathrm{m/s}$，$m_{eff} = 2/3\ \mathrm{kg}$：

$$\Delta\Lambda_{relax} = -\frac{2}{3}(0.18) = -0.12\ \mathrm{N\,s}.$$

先前累计2.7866667减少到2.6666667，但已经完成的位置推进不会回滚。

Relax也可能增加冲量。

例如30Hz普通软度算例Solve后 $u_n \approx -0.0599786$，则

$$\Delta\Lambda_{relax} = -\frac{2}{3}(-0.0599786) \approx +0.0399858\ \mathrm{N\,s}.$$

因此它不是“永远扣掉一笔能量”的固定操作。

13.1 `useBias=false` 并不等于没有任何bias

情况	Relax中的行为
$s \le 0$、无恢复目标	不用重叠软偏置，硬基线解法向
$s > 0$	仍保留推测 s/h
恢复目标为正	使用更负的恢复bias

法向处理完成后，Relax还解滚阻和摩擦。

摩擦可以再次改变法向速度；有限遍之后可能仍有耦合残差。

Relax不直接改位置，也不以最终速度重新积分同一子步。

14. 朴素实现、习题与失败诊断

先实现单自由度弹簧，验证软残差；再加累计投影；最后把它放入子步时序。

教学伪代码，非项目源码：

```
prepare fixed geometry and softness once
for each substep:
    integrate external-force velocities(h)
    apply cached impulses
    solve contact normals(useOverlapBias = true)
    integrate positions(h)
    solve contact normals(useOverlapBias = false)
    solve rolling resistance and friction
store impulses
```

习题	答案
$s = 0.01, h = 0.005, u = -1$，新接触需推测冲量吗	不需，$u + s/h = 1 \ge 0$
未夹bias为−8，`contactSpeed=3`	取−3
精确零频时两个scale之和	0，而非1
默认30Hz，$\Delta t = 1/30, N = 2$，普通/静态频率	7.5Hz、15Hz
每子步是否完整重跑窄相	否，使用缓存几何与位姿增量估算分离
Relax放位置积分前的风险	提前消除纠偏速度，使位置修正作用消失
现象	优先检查
增加子步后软度变化异常	是否用了整步 Δt 代替 h
推开过猛	bias符号、max夹限、是否重复乘scale

习题	答案
静态接触数值对不上	是否遗漏频率夹限后的静态二倍
法向算完又接近	是否出现摩擦或其他约束的耦合反馈
速度稳定但位置不修正	是否在积分位置前就去掉纠偏速度
主题	源码入口
软度三系数和零频	`solver.h:238-280`
子步长度与频率夹限	`physics_world.c:956-980`
实际子步阶段	`solver.c:1082-1169`
分离、bias与软法向	`contact_solver.c:271-328`
Relax后半部分	`contact_solver.c:331-385`
位置积分	`solver.c:114-158`

最后再看优化：固定准备数据、缓存几何和子步复用减少重复工作；宽向量与图着色属于执行组织，不改变本章每个公式的量纲和符号。

返回总目录 · 继续第12章

12 • 接触求解器逐行导读与完整算例

返回总目录 · 上一章：软约束、子步进与Relax

> 本章将前面三章的公式逐项对应到标量接触求解器，并完整计算一个子步。 源码基准：Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b`。 数值使用独立双精度计算复核，不代表已经运行C仿真测试；源码float及FMA可能造成末位差异。

1. 先画清楚数据的生命周期

数据	保存什么	何时变化
`manifold`	接触几何、特征ID、缓存冲量、速度历史	窄阶段更新，整步末提交冲量
`constraint`	本大步求解副本、有效质量、当前累计冲量	Prepare一次，子步反复更新
`bodyState`	当前速度、相对大步开始的位姿增量	积分与约束求解
`context`	子步长度、软度、索引、阶段参数	大步组织时准备

读每一条赋值，先判断它属于以下哪类：

1. 从历史复制初始猜测。
2. 初始化本大步统计。
3. 真正改变物理速度或位置。
4. 向下一大步提交结果。

“保留冲量就是热启动” “恢复在Prepare现算” “总冲量就是最后累计”都源于混淆这些生命周期。

2. Prepare：把接触整理成求解数据

入口：`b2PrepareContacts_Overflow`。

2.1 24–77行：选择接触与复制公共属性

行区间	表达式或对象	应怎样理解
24–33	world、graph、Overflow数组	选择待求解接触，不在这里检测碰撞
35–65	验证bodySim索引	检查索引是否仍对应awake状态
39–43	普通/静态软度、warmStartScale	缓存种子是否乘0或1
69–71	`indexA + 1`、`indexB + 1`	存储索引以0代表null
72–77	normal、friction、rolling、tangentSpeed	接触共有数据，而不是每点单独一份

滚动冲量是整个manifold共用的角冲量，不应为每个点重复存一份同样滚阻。

2.2 79–105行：逆质量与方向

局部变量 `mA`、`mB` 存的是逆质量，`iA`、`iB` 存的是逆惯量。

$$[m_A^{-1}] = \mathrm{kg}^{-1}, \qquad [I_A^{-1}] = (\mathrm{kg\,m^2})^{-1}.$$

84–91行按任意一侧状态索引是否null选择静态软度。

运动学体也可能逆质量为零但拥有状态，不能只按逆质量零来复刻这个判断。

100–101行计算

$$I_{eff,r} = \frac{1}{I_A^{-1} + I_B^{-1}}.$$

这就是 `rollingMass`，单位kg·m²；分母为零时取零。

104–105行保留法线并构造右垂线作为切线。

2.3 107–133行：每点准备

```
cp->normalImpulse = warmStartScale * mp->normalImpulse;
cp->tangentImpulse = warmStartScale * mp->tangentImpulse;
cp->totalNormalImpulse = 0.0f;

b2Vec2 rA = mp->anchorA;
b2Vec2 rB = mp->anchorB;

cp->anchorA = rA;
cp->anchorB = rB;
cp->baseSeparation = mp->separation - b2Dot( b2Sub( rB, rA ), normal );
```

这十行分别完成：

1. 载入跨大步法向、切向初始猜测。
2. 清零本大步实际冲量统计，而不是把两个累计猜测一起清空。
3. 保留大步开始的世界方向质心杆臂。
4. 为位姿增量分离公式构造基准。

这里

$$s_{base} = s_0 - \mathbf{n} \cdot (\mathbf{r}_B - \mathbf{r}_A).$$

它不同于几何回收路径中的 `mp.baseSeparation`。

`physics_world.c:619` 给manifold基准直接保存当时分离；同名字段不能跨结构体互换公式。

123–131行计算法向和切向有效质量：

$$K_n = m_A^{-1} + m_B^{-1} + I_A^{-1}(\mathbf{r}_A \times \mathbf{n})^2 + I_B^{-1}(\mathbf{r}_B \times \mathbf{n})^2, \qquad m_n = 1/K_n.$$

切向仅将 $\mathbf{n}$ 换成 $\mathbf{t}$。

最后一项只是复制：

```
cp->restitutionVelocity = mp->restitutionVelocity;
```

这里没有读取当前刚体速度、没有乘恢复系数，也没有检查恢复阈值。

3. WarmStart：真正把缓存打进速度

入口：`b2WarmStartContacts_Overflow`。

149–163行读取awake状态数组，静态侧使用单位dummy状态。

这个状态给出零速度、零位移增量和单位旋转。

165–178行将速度、逆质量、法线、切线读到局部变量。

180–195行逐点构造并施加

$$\mathbf{P} = \Lambda_n \mathbf{n} + \Lambda_t \mathbf{t}.$$

法向与切向都是线冲量，能够合成二维向量。

```
b2Vec2 P = b2Add( b2MulSV( cp->normalImpulse, normal ), b2MulSV( cp->tangentImpulse, tangent ) );

cp->totalNormalImpulse += cp->normalImpulse;

wA -= iA * b2Cross( rA, P );
vA = b2MulAdd( vA, -mA, P );
wB += iB * b2Cross( rB, P );
vB = b2MulAdd( vB, mB, P );
```

190行将本次实际施加的法向缓存计入大步统计。

每点都立即改变局部速度，下一点会看到变化。

198–199行另外施加滚动角冲量，只改角速度，不应把它混进二维 $\mathbf{P}$。

201–211行仅将动态刚体的局部速度写回。

运动学体可以贡献给定速度，但不由接触冲量加速；静态dummy也不写回为真实动态物体。

4. Solve与Relax共用同一个函数

入口：`b2SolveContacts_Overflow`。

4.1 229–265行：读取当前子步状态

数据	含义
`inv_h`	子步长度倒数，不是整步倒数
`contactSpeed`	穿透速度偏置的夹限尺度
`vA,wA,vB,wB`	当前已经经历外力、缓存及前序约束的速度
`dqA,dqB`	相对大步开始的累计旋转
`dp`	双方累计位移增量之差
局部 `totalNormalImpulse`	本次各点最终法向累计之和，仅供滚阻限幅

最后一项不同于每点字段 `cp.totalNormalImpulse` 的跨子步统计。

4.2 271–294行：当前分离与三种偏置情况

几何分离采用

$$s = s_{base} + \mathbf{n} \cdot [\Delta\mathbf{p}_B - \Delta\mathbf{p}_A + R(\Delta q_B)\mathbf{r}_B - R(\Delta q_A)\mathbf{r}_A].$$

这里旋转杆臂用于估计几何分离；之后点速度和力矩仍使用固定杆臂。

分支	条件	bias	scale
推测	$s > 0$	s/h	硬基线1、0
穿透软修正	$s \le 0$，Solve	夹限后的软偏置	MakeSoft系数
无重叠偏置	$s \le 0$，Relax	初始为0	硬基线1、0

因此 `useBias=false` 并不会关闭正间隙的推测bias。

4.3 296–305行：点速度和恢复目标

点速度为

$$\mathbf{v}_{rA} = \mathbf{v}_A + \omega_A \times \mathbf{r}_A, \quad \mathbf{v}_{rB} = \mathbf{v}_B + \omega_B \times \mathbf{r}_B,$$

$$u_n = (\mathbf{v}_{rB} - \mathbf{v}_{rA}) \cdot \mathbf{n}.$$

仅Relax且恢复目标为正时，执行

$$b \leftarrow \min(b, -v_{rest}).$$

正恢复目标通过负bias实现。

`stopRestitution` 只对具有正恢复目标的点更新AND。

严谨说法是：所有仍有正恢复目标的点均已分离时，当前约束副本的恢复目标会清零。

没有恢复目标的点，不会阻止这次清除。

4.4 308–328行：法向增量、投影、应用

```
// incremental normal impulse
float impulse = -cp->normalMass * ( massScale * vn + velocityBias ) - impulseScale * cp-
>normalImpulse;

// clamp the accumulated impulse
float newImpulse = b2MaxFloat( cp->normalImpulse + impulse, 0.0f );
impulse = newImpulse - cp->normalImpulse;
cp->normalImpulse = newImpulse;
cp->totalNormalImpulse += impulse;
```

逐项翻译：

1. 用当前速度、偏置和旧累计算候选增量。
2. 将累计限制到非负。
3. 重算真正能施加的差额。
4. 更新本子步累计。
5. 将有符号差额计入大步统计。

319–320行把新累计加入滚阻的本次法向汇总。

322–328行应用 $\Delta\mathbf{P} = \Delta\Lambda_n\mathbf{n}$，A减B加，并改变角速度。

4.5 331–384行：只在Relax执行

先处理恢复终止，再处理滚阻，最后处理所有点摩擦。

滚阻为

$$d_r = -I_{eff,r}(\omega_B - \omega_A), \quad |\Lambda_r| \le \rho \sum_j \Lambda_{n,j}.$$

摩擦读取滚阻已经修改后的速度：

$$u_t = (\mathbf{v}_{rB} - \mathbf{v}_{rA}) \cdot \mathbf{t} - v_{belt}, \quad d_t = -m_t u_t, \quad |\Lambda_t| \le \mu\Lambda_n.$$

两者都投影累计并仅施加差额。

摩擦会通过角速度改变法向速度；这一函数不会在每点摩擦之后自动再解一次法向。

388–398行将最终局部速度写回动态状态。

5. Store：不要在433行停止阅读

完整入口：`b2StoreImpulses_Overflow`。

426–431行提交三种每点冲量：

```
b2ManifoldPoint* mp = manifold->points + j;
mp->normalImpulse = constraint->points[j].normalImpulse;
mp->tangentImpulse = constraint->points[j].tangentImpulse;
mp->totalNormalImpulse = constraint->points[j].totalNormalImpulse;
```

434–447行检查hit事件，要求启用事件，并同时满足

$$u_{recorded} < -v_{hit,threshold}, \qquad T_n > 0.$$

有manifold点不等于真的发生了撞击；推测接触可能从未产生实际净法向作用。

450行另外写回整个manifold的滚动冲量。

Store没有写回 `normalVelocity`，也没有写回约束副本的 `restitutionVelocity`。

6. 完整数值算例：先列清所有输入

本例先使用便于手算的教学bias，随后替换为真实MakeSoft参数。

项目	数值或条件
A	静态，逆质量与逆惯量均零
B质量	$m = 1$kg
B惯量	$I = 0.5$kg·m²
法线、切线	$\mathbf{n} = (0, 1), \mathbf{t} = (1, 0)$
杆臂	$\mathbf{r}_A = (0, 0), \mathbf{r}_B = (0.5, -0.5)$m
初始速度	$\mathbf{v}_B = (0, -4)$m/s，$\omega_B = 0$
初始分离	$s_0 = -0.02$m
子步长度	$h = 1/240$s
摩擦系数	$\mu = 0.6$
滚阻、恢复目标	均为零
缓存冲量	均为零
本子步外力、重力、阻尼	均为零，用于隔离接触计算
运动锁与限速	没有锁定，不触发限速
初始位姿增量	位移零，旋转单位

惯量是独立给定的教学数据，不声称它等于边长1m均匀方盒的真实惯量。

教学软度取 `massScale=1`、`impulseScale=0`、`biasRate=9`，`contactSpeed=3`。

这个组合用于方便手算，不是假称默认 `b2MakeSoft` 精确返回这些值。

6.1 Prepare

$$\mathbf{r}_B \times \mathbf{n} = \mathbf{r}_B \times \mathbf{t} = 0.5\ \mathrm{m}.$$

$$K_n = K_t = 1 + 2(0.5)^2 = 1.5\ \mathrm{kg}^{-1}, \quad m_n = m_t = \frac{2}{3}\ \mathrm{kg}.$$

$$s_{base} = -0.02 - (-0.5) = 0.48\ \mathrm{m}.$$

滚动有效惯量为0.5kg·m²，但滚阻上限为零，因此不会施加滚阻。

6.2 积分速度与WarmStart

本例外力为零，积分速度后仍为 $(0, -4)$、$\omega = 0$。

缓存也为零，WarmStart不改速度，本步总冲量统计仍为零。

6.3 Solve法向

初始位姿增量为单位状态，所以

$$s = 0.48 - 0.5 = -0.02\ \mathrm{m}.$$

教学bias为

$$b = \max(9(-0.02), -3) = -0.18\ \mathrm{m/s}.$$

初始 $u_n = -4$，因此

$$\Delta\Lambda_n = -\frac{2}{3}(-4 - 0.18) = \frac{209}{75} \approx 2.7866666667\ \mathrm{N\,s}.$$

投影不截断，应用后

$$v_y = -4 + \frac{209}{75} = -\frac{91}{75} \approx -1.2133333333,$$

$$\omega = 2\left(0.5\frac{209}{75}\right) = \frac{209}{75} \approx 2.7866666667.$$

校验接触点法向速度：

$$u_n = v_y + 0.5\omega = 0.18\ \mathrm{m/s}.$$

此时切向速度为 $u_t = 0.5\omega = 1.3933333333\ \mathrm{m/s}$。

6.4 积分位置：不能跳过这一步

$$\Delta y = h v_y = \frac{1}{240}\left(-\frac{91}{75}\right) = -0.005055555555556\ \mathrm{m}.$$

传入旋转积分的参数为

$$\delta = h\omega = 0.0116111111111111.$$

本版旋转积分不是直接精确求 $\sin\delta, \cos\delta$，而是归一化Euler结果。

```
b2Rot q2 = { q1.c - deltaAngle * q1.s, q1.s + deltaAngle * q1.c };
float mag = sqrtf( q2.s * q2.s + q2.c * q2.c );
float invMag = mag > 0.0f ? 1.0f / mag : 0.0f;
b2Rot qn = { q2.c * invMag, q2.s * invMag };
return qn;
```

由初始单位旋转得到

$$\Delta q = \frac{(1,\delta)}{\sqrt{1+\delta^2}} \approx (0.999932597864567, 0.0116103284974275).$$

旋转后的杆臂为

$$R(\Delta q)\mathbf{r}_B \approx (0.505771463180997, -0.494161134683570)\ \mathrm{m}.$$

所以更新分离为

$$s = 0.48 - 0.005055555555556 - 0.494161134683570 \approx -0.0192166902391253\ \mathrm{m}.$$

它仍为负，因此接下来的Relax不会走正间隙推测分支。

6.5 Relax必须先重新解法向

点速度使用固定杆臂，当前法向速度仍为0.18m/s。

无恢复目标、仍穿透时，Relax采用 $b = 0$、两个scale为1和0。

$$\Delta\Lambda_n = -\frac{2}{3}(0.18) = -0.12\ \mathrm{N\,s}.$$

新累计为

$$\Lambda_n = \frac{209}{75} - 0.12 = \frac{8}{3}\ \mathrm{N\,s}.$$

速度变成

$$v_y = -\frac{4}{3}, \qquad \omega = \frac{8}{3}, \qquad u_n = 0, \qquad u_t = \frac{4}{3}.$$

负修正也从总冲量统计中扣除。

这一步不能省略，否则后面的摩擦数值会错成约−0.929。

6.6 Relax滚阻

滚阻系数为零，累计角冲量从零投影到零。

虽然物体仍在旋转，但本例不施加任何滚阻角冲量。

6.7 Relax摩擦

$$d_t = -m_t u_t = -\frac{2}{3}\frac{4}{3} = -\frac{8}{9}\ \mathrm{N\,s}.$$

允许的摩擦界为

$$\mu\Lambda_n = 0.6\frac{8}{3} = 1.6\ \mathrm{N\,s}.$$

候选没有超界，因此切向累计为 $-8/9$。

平动与转动更新：

$$v_x = -\frac{8}{9}, \qquad \Delta\omega = 2\left[0.5(0) - (-0.5)\left(-\frac{8}{9}\right)\right] = -\frac{8}{9}.$$

最终

$$\boxed{\mathbf{v}_B = (-8/9, -4/3)\ \mathrm{m/s}, \qquad \omega_B = 16/9\ \mathrm{rad/s}}.$$

检查两个方向：

$$u_t = -\frac{8}{9} + 0.5\frac{16}{9} = 0,$$

$$\boxed{u_n = -\frac{4}{3} + 0.5\frac{16}{9} = -\frac{4}{9}\ \mathrm{m/s}}.$$

切向满足后，法向又产生接近残差。

因为 $K_{nt} = 0.5\ \mathrm{kg}^{-1}$，摩擦使法向速度改变 $0.5(-8/9) = -4/9$。

这是真实耦合，不应谎称一遍后两个方向都严格满足。

7. 完整状态表与Store结果

下表速度单位为m/s，角速度单位rad/s；方向速度按求解器固定杆臂计算。

阶段	v_x	v_y	ω	u_n	u_t
初始/积分速度	0	−4	0	−4	0
WarmStart	0	−4	0	−4	0
Solve	0	−1.2133333333	2.7866666667	0.18	1.3933333333
积分位置	0	−1.2133333333	2.7866666667	0.18	1.3933333333
Relax法向	0	−1.3333333333	2.6666666667	0	1.3333333333
Relax滚阻	0	−1.3333333333	2.6666666667	0	1.3333333333
Relax摩擦/Store	−0.8888888889	−1.3333333333	1.7777777778	−0.4444444444	0
阶段	Λ_n，N·s	Λ_t，N·s	总法向冲量，N·s	Δy，m	
---	---	---	---	---	
WarmStart后	0	0	0	0	
Solve后	2.7866666667	0	2.7866666667	0	
积分位置后	2.7866666667	0	2.7866666667	−0.0050555556	

阶段	v_x	v_y	ω	u_n	u_t
Relax法向后	2.6666666667	0	2.6666666667	−0.0050555556	
Relax摩擦/Store	2.6666666667	−0.8888888889	2.6666666667	−0.0050555556	

Relax后没有重新积分位置或旋转，积分位置阶段得到的 $\Delta y, \Delta q$ 保持不变。

如果整个大步只有这一个子步，Store写出的最终法向累计和总量恰好相同；多子步时不应由此认定二者总相同。

字段也必须分清：若 −4 m/s 已在本次窄阶段采样，持久 `mp->normalVelocity` 仍是 −4 m/s，不是 Relax 后的 −4/9 m/s。算出的 −0.0192166902 m 是求解器局部的**当前分离估计**；Store 不将它写回 `mp->separation`，本例原 manifold 字段仍为 −0.02 m。真实软度例中的 −0.0190290999 m 也同样是分离估计。

固定几何、硬约束并反复迭代的对照解为

$$\begin{pmatrix} 1.5 & 0.5 \\ 0.5 & 1.5 \end{pmatrix} \begin{pmatrix} \Lambda_n \\ \Lambda_t \end{pmatrix} = \begin{pmatrix} 4 \\ 0 \end{pmatrix}, \qquad \Lambda_n = 3, \quad \Lambda_t = -1.$$

此时 $\mathbf{v} = (-1, -1), \omega = 2$，两个点速度分量都为零，且 $|\Lambda_t| \le 0.6\Lambda_n$。

这是耦合联立解标尺，不是声称本版本一遍Solve/Relax已经达到它。

8. 再用真实MakeSoft参数走一次

默认世界请求30Hz、阻尼比10，$\Delta t = 1/60, N = 4$，因此 $h = 1/240$。

普通频率夹限为30Hz；A静态，所以本例实际使用60Hz静态软度。

量	真实静态接触数值
`biasRate`	17.4769217009612 s^{-1}
`massScale`	0.971333010129369，无量纲
`impulseScale`	0.0286669898706311，无量纲
初始bias	−0.339518219271798 m/s
Solve法向累计	2.816566839859516 N·s
Solve后 v_y	−1.183433160140484 m/s
Solve后 ω	2.816566839859516 rad/s
Solve后 u_n	0.224850259789273 m/s
位置增量 Δy	−0.00493097150058535 m
旋转增量	(0.999931143841893, 0.0117348870911995)
更新后分离	−0.0190290998759322 m

冲量计算为

$$\Delta\Lambda_n = -\frac{2}{3}[0.971333010129369(-4) - 0.339518219271798] = 2.816566839859516.$$

Relax法向修正为

$$\Delta\Lambda_{relax} = -\frac{2}{3}(0.224850259789273) = -0.149900173192849.$$

累计再次到达 $8/3$，随后摩擦仍为 $-8/9$，**第一个子步 Relax 结束时**的速度与教学例相同。这里没有继续算余下三个子步，不能把它当成四子步大步的最终结果。

但本子步位置推进和分离估计不同；同一阶段速度相同不等于全过程相同。

30Hz普通软度只能作为非静态软度对照：

$$b \approx -0.170904080033821, \quad \Lambda_{solve} \approx 2.626680909548488, \quad u_{solve} \approx -0.0599786356772674.$$

这时Relax反而补上 $+0.0399857571181782$N·s。

9. 恢复目标来自上一大步，而非Prepare现算

普通更新在 `contact.c:529` 保存旧manifold，再生成新几何。

新点按ID匹配历史，只有匹配点才有相应旧速度和冲量证据。

```
if ( mp1->totalNormalImpulse > 0.0f && mp1->normalVelocity < -world->restitutionThreshold )
{
          mp2->restitutionVelocity = -contactSim->restitution * mp1->normalVelocity;
```

时间顺序为：

1. 用旧 `totalNormalImpulse` 和旧 `normalVelocity` 决定本次恢复目标。
2. 在 `physics_world.c:638` 记录本次窄阶段的接近速度。
3. 之后才进行本步积分速度、WarmStart和约束求解。
4. Store只提交冲量，不重写这次采样的 `normalVelocity`。

回收路径在 `physics_world.c:551-576` 保持同样“先用旧历史，再记录当前速度”的顺序。

大步	几何与历史	恢复行为
k	新ID，无匹配旧点；记录速度−4m/s	初始恢复目标为0
k求解后	产生正总法向冲量	留下实际作用证据
$k+1$	ID持续，$e=0.5$，阈值1m/s	由旧速度−4生成目标+2m/s
$k+1$Relax	使用已复制目标	bias取−2以求恢复
$k+2$	再检查 $k+1$ 新历史	不应无限复用最早的−4

因此本版本不是“首次发现接触时在Prepare用当前速度立刻计算同帧反弹”的最简教科书实现。

源码也明确说明恢复取上一时间步接近速度，并非高精度恢复系数实验模拟器。

10. `totalNormalImpulse` 的准确含义

Prepare令 $T_n = 0$。

每子步WarmStart加初始累计；每次Solve/Relax再加有符号增量。

该子步贡献为

$$\Lambda_{in} + \sum \Delta\Lambda = \Lambda_{out}.$$

全部子步完成后

$$\boxed{T_n = \sum_{k=1}^{N} \Lambda_{out,k}}.$$

所以它不是：最后增量、最后累计、历史最大值，或只累加正增量的统计。

它包含WarmStart和恢复的作用，也包含撤回冲量的负增量。

例：水平地面上 1 kg 物体稳定静止，全部重量由一个无转动耦合的接触点承担，$g = 10\text{m/s}^2$、$h = 1/240\text{s}$、四子步。若由多个点共同支撑，下式的冲量应解释为这些支撑点的合计：

$$\Lambda_{out,k} \approx mgh = \frac{1}{24}\,\text{N s}, \quad T_n \approx 4\frac{1}{24} = \frac{1}{6}\,\text{N s}.$$

大步平均法向力是

$$\boxed{\bar{F}_n = T_n/\Delta t = (1/6)/(1/60) = 10\ \text{N}}.$$

整步总量不能除以子步 h，否则得到40N。

末子步累计 $\Lambda_{out,N}/h$ 可以估计末子步平均法向力，但它与整步统计是不同用法。

11. 习题、失败诊断与源码导航

习题	答案
教学例摩擦前法向累计	$8/3$，已经在Relax撤回0.12
摩擦后法向速度为何不为0	角速度耦合，得到 $-4/9$m/s
Store是否写normalVelocity或restitutionVelocity	此标量函数都不写
旧速度−0.5、阈值1、正总冲量，会生成恢复吗	不会，未超过接近阈值
四子步末累计均0.25N·s，总量多少	1N·s
默认请求30Hz的本地面例实际使用几Hz	60Hz静态软度
结果异常	优先检查

习题	答案
摩擦约为−0.929而非−0.888889	是否漏掉Relax法向修正
位置推进对不上	是否错误用最终Relax速度再积分同一子步
更新分离末位不同	是否将归一Euler旋转换成精确sin/cos
恢复提前一个大步出现	是否在Prepare用当前速度现算
推测点频繁误报hit	是否没有检查正总法向冲量
滚阻过大	是否拿跨大步总冲量代替当前各点法向累计之和
主题	源码入口
求解字段	`contact_solver.h:10-38`
Prepare	`contact_solver.c:24-138`
WarmStart	`contact_solver.c:140-215`
Solve/Relax	`contact_solver.c:217-402`
Store与hit	`contact_solver.c:404-456`
子步实际顺序	`solver.c:1082-1169`
位置积分	`solver.c:114-158`
归一化旋转	`math_functions.h:388-399`
ID迁移及恢复历史	`contact.c:603-655`
回收路径及当前速度采样	`physics_world.c:507-638`

先确认这些标量步骤与账本正确，再去阅读宽向量、图着色及任务调度；后者组织计算，不能替代上述物理顺序。

返回总目录 · 回看第11章

13 • 距离关节与统一关节框架

返回目录

> 基线：Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b` 的当前工作树。 本章把“两个锚点之间放一根杆”拆成几何、求导、质量响应和冲量四个问题。 先写最朴素的求解器，再解释本版为何缓存、热启动、旋转锚点、分支限幅。

1. 先确定我们究竟要约束什么

没有关节的二维刚体有两个平移自由度和一个转动自由度。 距离关节并不是把两个刚体整体焊死。 它只规定两个指定锚点之间的距离。 两物体仍可绕锚点连线的二维运动关系摆动。

令两刚体的质心为 c_A, c_B，单位为米。 令从各自质心指向锚点的世界空间向量为 r_A, r_B，单位也为米。 锚点位置为

$$p_A = c_A + r_A, \qquad p_B = c_B + r_B.$$

注意：局部锚点通常相对于刚体局部原点，不一定相对于质心。 所以不能只旋转 `localFrameA.p` 就当作力臂。 还需要先减去 `localCenter`。

锚点连线、长度和方向定义为

$$d = p_B - p_A, \qquad L = \|d\|, \qquad u = d/L.$$

d 的单位为米，L 为米，u 无量纲。 u 从 A 指向 B。 在正常的非零长度情况下，$u \cdot u = 1$。

目标长度记作 L_0。 硬杆的约束误差为

$$C = L - L_0 = 0.$$

$C > 0$ 表示太长，应把两锚点拉近。 $C < 0$ 表示太短，应把两锚点推开。 这个符号约定会一直传到冲量的正负号。

2. 从位置约束求导，而不是直接背 Jacobian

质心线速度是 v，单位为米每秒。 角速度是 ω，单位为弧度每秒。 二维中约定正角速度表示逆时针。

标量与向量的叉乘定义为

$$\omega \times r = (-\omega r_y, \omega r_x).$$

锚点不是质心，锚点速度因此是

$$v_p = v + \omega \times r.$$

令

$$v_r = v_B + \omega_B \times r_B - v_A - \omega_A \times r_A.$$

这才是 B 锚点相对于 A 锚点的速度。 如果只写 $v_B - v_A$，偏心连接时就漏掉旋转贡献。

对 $L = \sqrt{d \cdot d}$ 求导:

$$\dot{L} = \frac{d \cdot \dot{d}}{\|d\|} = u \cdot v_r.$$

因为 L_0 在这一约束求解期间固定，得到

$$\dot{C} = u \cdot v_r.$$

标量叉乘定义为 $a \times b = a_x b_y - a_y b_x$。 利用 $u \cdot (\omega \times r) = \omega(r \times u)$，可写成

$$\dot{C} = -u \cdot v_A - (r_A \times u)\omega_A + u \cdot v_B + (r_B \times u)\omega_B.$$

把速度拼为

$$V = (v_{Ax}, v_{Ay}, \omega_A, v_{Bx}, v_{By}, \omega_B)^T,$$

Jacobian 就是

$$J = [-u_x, -u_y, -r_A \times u,\ u_x, u_y, r_B \times u].$$

于是 $JV = \dot{C}$。 Jacobian 是“各个速度对距离变化率的贡献系数”，不是额外的神秘物理量。 平移列无量纲，转动列带米的量纲。

3. 冲量如何改变距离变化率

令 m_A, m_B 表示真实质量，单位为千克。 令 I_A, I_B 表示绕质心的转动惯量，单位为千克·平方米。 本章用 $\mu_A = 1/m_A$、$\mu_B = 1/m_B$ 表示逆质量。 用 $\iota_A = 1/I_A$、$\iota_B = 1/I_B$ 表示逆惯量。 源码中的局部变量 `mA`、`mB` 实际上是逆质量，不是本章的真实质量。

标量冲量 λ 的单位为牛顿·秒。 对应的世界向量冲量为

$$P = \lambda u.$$

把 $+P$ 施加给 B，把 $-P$ 施加给 A:

$$\Delta v_A = -\mu_A P, \qquad \Delta\omega_A = -\iota_A(r_A \times P),$$
$$\Delta v_B = \mu_B P, \qquad \Delta\omega_B = \iota_B(r_B \times P).$$

正冲量倾向于增大距离，负冲量倾向于缩短距离。 两边的线冲量大小相等方向相反，因此动态刚体对的总线动量守恒。 静态体不更新速度，它相当于连接了外部支撑。

代回距离速度，有

$$\Delta\dot{C} = K\lambda,$$

其中

$$K = \mu_A + \mu_B + \iota_A(r_A \times u)^2 + \iota_B(r_B \times u)^2.$$

K 的单位为每千克。 它是“每单位冲量产生多少约束速度变化”。 有效质量为

$$M_{\text{eff}} = 1/K.$$

它的单位为千克。 有效质量不是简单把两个物体质量相加。 偏心力臂越长，旋转越容易帮助消除锚点速度误差，K 越大，有效质量越小。

如果 $K = 0$，不能相除，代码使用有效质量零。 这表示没有可响应的质量自由度，不表示无限大的修正冲量。

4. 最朴素的硬杆实现

先不做热启动、子步、弹簧和马达。 给每条杆按以下顺序处理：

1. 从当前刚体变换计算两个世界锚点。
2. 求 d, L, u。
3. 求锚点相对速度 v_r。
4. 求 $\dot{C} = u \cdot v_r$。
5. 求 K 和 M_{eff}。
6. 解一个标量冲量。
7. 将等大反向的冲量施加到两边线速度和角速度。
8. 对全部约束重复若干轮，再积分位置。

无位置修正时，希望求解后 $\dot{C}' = 0$：

$$\dot{C} + K\Delta\lambda = 0, \qquad \Delta\lambda = -M_{\text{eff}}\dot{C}.$$

这只阻止继续伸长或缩短。 已经出现的长度误差不会自动消失。

教学上可以引入偏置速度

$$b = \frac{\beta}{h}C, \qquad \Delta\lambda = -M_{\text{eff}}(\dot{C} + b).$$

h 是这一子步的秒数，β 是无量纲修正比例。 b 的单位为米每秒。 太长时 $C > 0$，得到负冲量，方向正确。 这里是教学用的简单偏置模型，并非声称源码硬杆直接使用固定 β。 本版用后文的 `constraintSoftness` 控制修正。

5. 完整数值例：偏心锚点不能只看质心

设

$$m_A = 2,\ m_B = 1,\ I_A = 1,\ I_B = 2,$$

单位分别为千克、千克和千克·平方米。 因此

$$\mu_A = 0.5, \quad \mu_B = 1, \quad \iota_A = 1, \quad \iota_B = 0.5.$$

取 $u = (1, 0)$，$r_A = (0, 1)$，$r_B = (0, -1)$ 米。 取初始 $v_A = (0, 0)$、$v_B = (3, 0)$ 米每秒，两角速度均为零。 关闭位置偏置，并假设本次几何非退化。

叉乘为

$$r_A \times u = -1, \qquad r_B \times u = 1.$$

所以

$$K = 0.5 + 1 + 1 \times 1 + 0.5 \times 1 = 3, \qquad M_{\text{eff}} = 1/3.$$

$\dot{C} = 3$ 米每秒，冲量为

$$\Delta\lambda = -\frac{1}{3} \times 3 = -1\ \text{N s}.$$

向量冲量 $P = (-1, 0)$ 牛顿·秒。 更新结果为

$$v'_A = (0.5, 0), \quad \omega'_A = -1, \qquad v'_B = (2, 0), \quad \omega'_B = -0.5.$$

重新计算锚点速度：

$$v'_{pA} = (0.5, 0) + (-1) \times (0, 1) = (1.5, 0),$$
$$v'_{pB} = (2, 0) + (-0.5) \times (0, -1) = (1.5, 0).$$

两质心速度并不相同，但两锚点沿约束方向的速度已经相同。 这就是偏心有效质量中必须出现转动项的原因。

还可核对线动量：初始总动量为 3，末态为 $2 \times 0.5 + 1 \times 2 = 3$。

6. 本版的统一骨架：Prepare、WarmStart、Solve

入口位于 `src/joint.c`。 `b2PrepareJoint` 为各类型生成共用约束软度，再分发到各类型。 `b2WarmStartJoint` 应用缓存的冲量。 `b2SolveJoint` 根据当前速度求本轮冲量增量。

基础配置 `b2JointDef` 包含两刚体、两局部关节坐标系、约束调参和碰撞开关。 距离配置 `b2DistanceJointDef` 额外包含长度、弹簧、限位、马达。 类型名一样不表示所有开关都能同时生效，必须读求解分支。

6.1 Prepare 做可复用的工作

`b2PrepareDistanceJoint`， `src/distance_joint.c:250-313`：

- 找到刚体对应的 solver set 与速度数组索引。
- 保存逆质量和逆惯量。
- 把局部锚点减去局部质心后旋转到世界方向。
- 保存起始质心差 `deltaCenter`。
- 计算初始轴与 `axialMass`。
- 根据用户弹簧参数生成 `distanceSoftness`。
- 若关闭热启动，清除所有累积冲量。

本版并不是在每次距离求解中重算有效质量。 `axialMass` 在 Prepare 缓存，Solve 更新轴和力臂但复用该标量。 这是计算成本与子步几何变化之间的折中。

6.2 为什么要增量位置与旋转

Solve 中当前力臂为

$$r_A = \Delta R_A\, r_{A0}, \qquad r_B = \Delta R_B\, r_{B0}.$$

当前连线为

$$d = (c_{B0} - c_{A0}) + (\Delta c_B - \Delta c_A) + (r_B - r_A).$$

r_{A0}, r_{B0} 是 Prepare 保存的力臂。 ΔR、Δc 是求解中的累计旋转与位移。

每个子步旋转锚点，能避免把一根已经转弯的杆仍沿旧方向拉。 源码在 `distance_joint.c:375-384` 更新几何。

6.3 WarmStart 是好的初值，不是第二份外力

缓存的总轴向冲量为

$$\lambda_{\text{total}} = \lambda_{\text{spring/rigid}} + \lambda_{\text{lower}} - \lambda_{\text{upper}} + \lambda_{\text{motor}}.$$

`b2WarmStartDistanceJoint`， `distance_joint.c:315-352`，沿当前轴应用它。 下限的冲量增加长度，上限的冲量减小长度，所以必须相减。 后续 Solve 是在这个初值上继续修正，不是每轮重新施加完整累计量。

7. 从弹簧方程到本版软约束

理想线性弹簧阻尼模型为

$$F = -kC - d_s\dot{C}.$$

k 的单位为牛顿每米，d_s 的单位为牛顿·秒每米。 频率 f 用 Hz，角频率 $\Omega = 2\pi f$ 用每秒。 阻尼比 ζ 无量纲。 按约束有效质量参数化：

$$k = M_{\text{eff}}\Omega^2, \qquad d_s = 2\zeta M_{\text{eff}}\Omega.$$

隐式一步把末态位置近似为 $C + h\dot{C}'$，冲量为

$$\lambda = h[-k(C + h\dot{C}') - d_s\dot{C}'].$$

整理得到速度层面的软约束：

$$\dot{C}' + b + \gamma\lambda = 0, \quad \gamma = \frac{1}{h(d_s + hk)}, \quad b = \frac{k}{d_s + hk}C.$$

γ 与 K 同为每千克。 用 $\dot{C}' = \dot{C} + K\Delta\lambda$，并考虑已有累计冲量 λ_{old}：

$$\Delta\lambda = -\frac{\dot{C} + b + \gamma\lambda_{old}}{K + \gamma}.$$

令

$$a_1 = 2\zeta + h\Omega, \quad a_2 = h\Omega a_1,$$

就能整理为本版形式：

$$B = \frac{\Omega}{a_1}, \quad S = \frac{a_2}{1 + a_2}, \quad T = \frac{1}{1 + a_2},$$

$$\Delta\lambda = -SM_{\text{eff}}(\dot{C} + BC) - T\lambda_{old}.$$

`biasRate` 对应 B，单位为每秒。 `massScale` 对应 S，`impulseScale` 对应 T，二者无量纲。 非零频率时 $S + T = 1$。 `b2MakeSoft` 的零频率特判返回三个零，因此不能把这一恒等式无条件延伸到零频率。

7.1 软弹簧与拉压力上限数值例

取 $h = 0.1$ 秒、$\Omega = 10$ 每秒，即 $f = 5/\pi \approx 1.59155$ Hz。 取 $\zeta = 0.5$，则

$$a_1 = 2, \quad a_2 = 2, \quad B = 5, \quad S = 2/3, \quad T = 1/3.$$

继续取 $M_{\text{eff}} = 1/3$ 千克。 假设 $C = 0.2$ 米、$\dot{C} = 0$、$\lambda_{old} = 0$。 未限幅的冲量为

$$\Delta\lambda = -\frac{2}{3}\frac{1}{3}(0 + 5 \times 0.2) = -\frac{2}{9} \approx -0.222222.$$

令 `lowerSpringForce=-1` 牛顿、`upperSpringForce=5` 牛顿。 一子步允许的累计冲量范围为 $[-0.1, 0.5]$ 牛顿·秒。 因此实际累计冲量是 -0.1，本轮增量也是 -0.1。 本轮沿轴速度变化为 $K\Delta\lambda = 3 \times (-0.1) = -0.3$ 米每秒。

负下界控制能承受多大拉力，正上界控制能提供多大推力。 不能把两个参数都当成“正数大小”而忽略方向。

8. 开关关系：硬杆与弹簧家族不是全部并列

下面这条判断比记忆功能列表更重要：

```
          // joint is soft if
          // - spring is enabled
          // - and (joint limit is disabled or limits are not equal)
          if ( joint->enableSpring && ( joint->minLength < joint->maxLength || joint->enableLimit
== false ) )
          {
                    // spring
                    if ( joint->hertz > 0.0f )
```

配置	实际求解
`enableSpring=false`	硬杆分支；不求马达和上下限
`enableSpring=true`，且未启用限位	进入弹簧家族；有正频率才求弹簧；马达可求
`enableSpring=true`，限位启用且 `minLength<maxLength`	弹簧、马达、两端限位按开关顺序求
`enableSpring=true`，限位启用且有效上下限相等	硬杆分支

因此，`hertz=0` 不等于“距离关节自动变成硬杆”。 例如开启弹簧、关闭限位、频率零、关闭马达，会进入弹簧家族但跳过弹簧求解。 它不是目标长度的刚性约束。

硬杆分支 `distance_joint.c:503-529` 使用

$$C = L - \texttt{joint->length}.$$

特别注意：上下限相等触发该分支时，目标仍来自 `length`，不是临时从 `minLength` 读取。 要表达同一刚性长度，配置时应使这些参数一致。 不要拿旧版本的直觉替代当前条件判断。

9. 两个限位为何都能把累计冲量限定为非负

下限定义

$$C_L = L - L_{min} \geq 0.$$

正冲量沿 $+u$ 推开两端，因此有 $\lambda_L \geq 0$。 上限改写成

$$C_U = L_{max} - L \geq 0.$$

其速度为 $\dot{C}_U = -\dot{L}$。 为了保持 $\lambda_U \geq 0$，对 B 施加的冲量必须为 $-\lambda_U u$。

统一累计投影形式为

$$\lambda_{new} = \max(0, \lambda_{old} + \Delta\lambda_{trial}), \qquad \Delta\lambda_{apply} = \lambda_{new} - \lambda_{old}.$$

即使试算增量为负，也可能只是撤回上一次过大的正冲量。 把“每轮增量必须非负”当成规则会使关节卡死。

9.1 尚未越界，也可以预防越界

当 $C > 0$ 时，源码使用 $b = C/h$。 这是预测式限位：不必把安全区中的速度立刻减到零，只需保证这一小步不要越过边界。

例如下限安全余量 $C = 0.05$ 米，$h = 0.1$ 秒，$\dot{C} = -2$ 米每秒。 取 $M_{\text{eff}} = 1/3$，旧冲量零：

$$b = 0.5, \qquad \lambda = -\frac{1}{3}(-2 + 0.5) = 0.5 \ \text{N s}.$$

施加后 $\dot{C}' = -2 + 3 \times 0.5 = -0.5$。 预测余量 $C + h\dot{C}' = 0.05 - 0.05 = 0$，刚好落到边界。 若误用零偏置，则会过早完全刹停。

已越界时，`useBias=true` 使用基础约束软度修复误差。 `useBias=false` 不继续注入这种位置修复偏置，但仍约束相关速度。

10. 马达：目标是速度，能力是力

距离马达希望

$$\dot{L} = v_{target}.$$

v_{target} 单位为米每秒，正值主动伸长，负值主动缩短。 试算冲量是

$$\Delta\lambda_{trial} = M_{\text{eff}}(v_{target} - \dot{L}).$$

累计冲量限制为

$$-hF_{max} \le \lambda_{motor} \le hF_{max}.$$

为什么乘的是子步时长 h？ 因为 $\lambda = \int F dt$，恒力近似下就是 Fh。 若每次迭代都额外施加 hF_{max}，迭代次数越多马达越强，物理语义就错了。 本版限制累计量，而不是仅限制本轮试算量。

数值例：$h = 0.1$，$F_{max} = 6$ 牛顿，$M_{\text{eff}} = 1/3$。 当前轴速度零，目标为 3 米每秒，旧马达冲量零。 理想冲量 1，但上限只有 0.6。 实际轴速度增加 $3 \times 0.6 = 1.8$ 米每秒。 未达到目标不是求解失败，而是明确受到力上限限制。

本版软分支先弹簧，再马达，再下限与上限。 每个分支读取前一分支已经更新的速度。 它们不能全部拿同一个旧速度各自计算后简单相加。

11. 退化、安全和调试

11.1 零长度不是随便选一个方向

`b2Normalize` 在长度平方极小时返回零向量，见 `math_functions.h` 的 `340-350` 行。 这是避免除零与 NaN 的数值防护。 它不会发明“应该向右拉”的任意方向。

源码为目标长度和最小长度设置稳定下限，见 `joint.c:414-418`。 但目标长度合法不保证两个当前锚点永远不重合。 若当前连线为零，$P = \lambda u$ 也是零，距离求解无法从这个不存在的方向产生几何修复。

还要注意，源码的 `K` 简式预设正常单位轴。 轴变成零时，公式里逆质量和仍可能非零。 不能据此声称代码一定把 `axialMass` 变为零；真正的防护结果是零方向不会产生非零向量冲量。

11.2 基础硬约束调参与用户弹簧不是一组参数

`b2PrepareJoint` 把基础 `constraintHertz` 限制到 $0.25/h$。 此举减少过硬修正造成的抖动。 它生成的是 `constraintSoftness`。 距离用户弹簧则另外生成 `distanceSoftness`。 不要把这两者混称为“关节频率”。

11.3 应记录什么

建议逐子步记录 $L, C, \dot{C}, K$ 和四组累计冲量。 同时记录每个开关及实际进入的分支。 有力上限时记录 λ/h，检查是否在设定力范围内。 弹簧启用且频率零时，优先看是否只是跳过了弹簧，不要先增加迭代数。

12. 源码阅读索引

内容	函数或范围
基础关节配置	`types.h`， `b2JointDef:596-631`

内容	函数或范围
距离参数与刚性覆盖说明	`b2DistanceJointDef:637-681`
创建与长度安全下限	`joint.c`， `b2CreateDistanceJoint:393-436`
基础软度与 Prepare 分发	`b2PrepareJoint:1406-1452`
WarmStart 分发	`b2WarmStartJoint:1454-1497`
Solve 分发	`b2SolveJoint:1498-1541`
准备距离几何	`distance_joint.c`， `b2PrepareDistanceJoint:250-313`
热启动	`b2WarmStartDistanceJoint:315-352`
求解与分支	`b2SolveDistanceJoint:354-541`
软度生成	`solver.h`， `b2MakeSoft:238-279`
归一化退化防护	`math_functions.h`， `b2Normalize:340-350`

13. 习题与答案

题 1：轴穿过两个质心

两质量分别为 2 和 3 千克，力臂均为零。 求有效质量。

答案：$K = 1/2 + 1/3 = 5/6$，所以 $M_{\text{eff}} = 6/5 = 1.2$ 千克。 不是 5 千克，也不是算术平均质量。

题 2：为什么要先累计再限幅

旧冲量为 0.4，试算增量为 0.5，上限为 0.6 牛顿·秒。 实际施加多少？

答案：新累计量为 $\min(0.4 + 0.5, 0.6) = 0.6$，只施加 0.2。 若直接把增量限到 0.6，会施加 0.5，累计超出上限。

题 3：关掉 spring，但开启 motor，能否驱动长度变化

答案：本版进入硬杆分支，马达分支不执行。 要使用距离马达，应按当前分支条件启用弹簧家族，再独立决定是否用正频率弹簧。

题 4：角速度已经变化，为什么还要检查锚点速度

答案：约束的是锚点距离变化率，不是质心速度，也不是角速度单独相等。 必须把 $v + \omega \times r$ 重算后投影到 u。

题 5：相同力上限，子步时长减半会怎样

答案：每子步允许的冲量上限减半。 相同总模拟时长内子步数加倍，恒定饱和力的总冲量仍应保持同一量级。

下一章：旋转、棱柱、轮与焊接 · 返回目录

14 • 旋转、棱柱、轮与焊接关节

返回目录 · 上一章

> 基线：Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b` 的当前工作树。 本章不把四种关节压缩成名字表，而是逐个写出目标、导数、Jacobian、有效质量和冲量。 读完应能从“允许怎么动”反推出应该写哪几行约束。

1. 公共记号与自由度

两刚体各有三维速度：二维线速度与一维角速度。 本章仍用

$$V = (v_{Ax}, v_{Ay}, \omega_A, v_{Bx}, v_{By}, \omega_B)^T.$$

v 的单位为米每秒，ω 为弧度每秒。 正角速度表示逆时针。 μ_A, μ_B 是逆质量，单位为每千克。 ι_A, ι_B 是逆转动惯量，单位为每千克·平方米。 源码中的 `mA/mB/iA/iB` 对应这四个逆量。

r_A, r_B 是从质心到锚点的世界向量，单位为米。 锚点间向量为

$$d = c_B + r_B - c_A - r_A.$$

世界关节坐标系的方向分别记为 Q_A, Q_B。 它们包含刚体旋转和 `localFrameA.q/localFrameB.q`。 因此关节相对角不是永远等于“刚体 B 角度减刚体 A 角度”。 局部框架的参考方向也参与其中。

设

$$\theta = \operatorname{angle}(Q_A^{-1} Q_B).$$

θ 单位为弧度。 在选定的局部角度分支内，$\dot{\theta} = \omega_B - \omega_A$。

关节	基础约束	不加额外功能时允许的相对运动
revolute	两锚点重合	相对转动
prismatic	横向位移为零、相对角为零	沿滑轴移动
wheel	横向位移为零	沿悬挂轴移动且可相对转动
weld	两锚点重合且相对角为零	无理想相对自由度；可配置弹性

这里说的是两物体之间的相对自由度，不是整个系统的绝对运动。 两物体一起平移、一起旋转不应被无故锁死。

2. Revolute：两个锚点重合，为什么需要二维冲量

2.1 从几何目标开始

铰链点必须重合：

$$C_p = d = \begin{bmatrix} d_x \\ d_y \end{bmatrix} = 0.$$

这是两条约束，不是一条距离为零的普通杆。 零长度杆没有稳定的连线方向，而点对点约束明确要求 x、y 两分量都一致。

锚点相对速度为

$$\dot{C}_p = v_B + \omega_B \times r_B - v_A - \omega_A \times r_A.$$

令 $r^{\perp} = (-r_y, r_x)^T$。 则二维 Jacobian 为

$$J_p = \begin{bmatrix} -1 & 0 & r_{Ay} & 1 & 0 & -r_{By} \\ 0 & -1 & -r_{Ax} & 0 & 1 & r_{Bx} \end{bmatrix}.$$

第一行控制世界 x 方向锚点速度，第二行控制世界 y 方向。 由于一个方向的偏心冲量会产生旋转，旋转又影响另一个方向，两行通常耦合。

2.2 从质量响应推出完整矩阵

线冲量记为 $P = (P_x, P_y)^T$，单位为牛顿·秒。 施加后

$$\Delta\dot{C}_p = KP, \qquad K = J_p M^{-1} J_p^T.$$

展开：

$$K = \begin{bmatrix} \mu_A + \mu_B + \iota_A r_{Ay}^2 + \iota_B r_{By}^2 & -\iota_A r_{Ax} r_{Ay} - \iota_B r_{Bx} r_{By} \\ -\iota_A r_{Ax} r_{Ay} - \iota_B r_{Bx} r_{By} & \mu_A + \mu_B + \iota_A r_{Ax}^2 + \iota_B r_{Bx}^2 \end{bmatrix}.$$

每个矩阵元素的单位都是每千克。 非对角项描述“一个方向的冲量，通过旋转影响另一个方向”。 如果只用对角项和同一份旧残差做一次更新，这一轮就不能准确处理耦合。但若逐轴求解并立即更新刚体速度，再反复计算残差，耦合仍能通过迭代传递；这仍是在逼近同一约束系统。

理想无偏置的解为

$$P = -K^{-1}\dot{C}_p.$$

带本版软度时，先解

$$z = K^{-1}(\dot{C}_p + b),$$

再求

$$\Delta P = -Sz - TP_{old}.$$

$b = BC_p$ 是米每秒量纲的偏置。 B 是每秒，S, T 无量纲。 这三个量对应基础 `constraintSoftness`。

2.3 朴素实现与本版实现的分界

教学最小版可以先把两个标量约束顺序求多次。 这样能工作，但耦合误差要靠迭代慢慢传递。 下一步应实现真正的二维块求解。

当前源码已经实现二维块，而不是“也许逐轴近似”：

```
b2Mat22 K;
K.cx.x = mA + mB + rA.y * rA.y * iA + rB.y * rB.y * iB;
K.cy.x = -rA.y * rA.x * iA - rB.y * rB.x * iB;
K.cx.y = K.cy.x;
K.cy.y = mA + mB + rA.x * rA.x * iA + rB.x * rB.x * iB;
b2Vec2 b = b2Solve22( K, b2Add( Cdot, bias ) );

b2Vec2 impulse;
impulse.x = -massScale * b.x - impulseScale * joint->linearImpulse.x;
impulse.y = -massScale * b.y - impulseScale * joint->linearImpulse.y;
```

`b2Mat22` 以列向量保存矩阵。 不要看见 `cx/cy` 就误把数学中的行列交换。 上式矩阵恰好对称，但写一般矩阵时这种误读会出错。

2.4 完整二维数值解

取 A 静态，因此 $\mu_A = \iota_A = 0$。 取 B 质量 1 千克，惯量 1 千克·平方米。 所以 $\mu_B = \iota_B = 1$。 令 B 的锚点力臂为 $r_B = (1, 1)$ 米。 取初始 $v_B = (3, 0)$ 米每秒，$\omega_B = 0$。 本例只看无偏置、无额外角度约束的一轮。

得到

$$K = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}, \qquad \det K = 4 - 1 = 3,$$

$$K^{-1} = \frac{1}{3}\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

于是

$$P = -\frac{1}{3}\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}\begin{bmatrix} 3 \\ 0 \end{bmatrix} = \begin{bmatrix} -2 \\ -1 \end{bmatrix}\ \mathrm{N\,s}.$$

为什么 y 初速度为零，却要一个 y 冲量？ 因为 x 冲量在偏心锚点产生角速度，角速度会让锚点向 y 方向运动。 y 冲量必须同时抵消这部分耦合。

更新 B：

$$v_B' = (3, 0) + (-2, -1) = (1, -1),$$

$$\omega_B' = 0 + (1, 1) \times (-2, -1) = 1.$$

检查锚点速度：

$$v_{pB}' = (1, -1) + 1 \times (1, 1) = (1, -1) + (-1, 1) = (0, 0).$$

也可直接核对

$$\dot{C}_p + KP = (3, 0) + (-3, 0) = (0, 0).$$

如果两轴都使用同一份旧残差、只用矩阵对角做一次更新，会得到 $(-1.5, 0)$ N·s，不是正确块解。

若采用“先 x、立即更新速度、再 y”的顺序标量方法，则 x 冲量为 −1.5，更新后角速度为 +1.5，y 方向点速度为 +1.5，故 y 冲量为 $-1.5/2 = -0.75$。这一轮合计 $(-1.5, -0.75)$ N·s，也还没有达到块解 $(-2, -1)$，但多轮迭代可以进一步逼近。

以上两种近似更新不可混称；当前该处源码直接解二维块。

3. Revolute 的角弹簧、马达与角限位

3.1 角约束的质量与单位

相对角速度为

$$\dot{\theta} = \omega_B - \omega_A.$$

角度行的 Jacobian 为

$$J_\theta = [0, 0, -1, 0, 0, 1].$$

对应的逆有效惯量与有效惯量为

$$K_\theta = \iota_A + \iota_B, \qquad M_\theta = 1/K_\theta.$$

M_θ 单位为千克·平方米。 角冲量 λ_θ 单位为牛顿·米·秒。 它更新角速度，而不是直接加到角度上：

$$\omega'_A = \omega_A - \iota_A \lambda_\theta, \qquad \omega'_B = \omega_B + \iota_B \lambda_\theta.$$

两边都固定旋转时 $K_\theta = 0$。 `fixedRotation` 分支会跳过角弹簧、角马达和角限位。 平动锚点约束仍须按可响应的自由度处理。

3.2 角弹簧的目标是关节相对角

令目标角为 θ_0，角误差为

$$C_\theta = \mathrm{unwind}(\theta - \theta_0).$$

`b2UnwindAngle` 避免把跨角度分支的微小转动误写成接近整圈的大误差。 本版角度来自旋转对象的相对角，并非无限累计的多圈编码器。

弹簧增量为

$$\Delta\lambda_s = -S M_\theta (\dot{\theta} + B C_\theta) - T \lambda_{s,old}.$$

例如 $M_\theta = 0.5$、$B = 5$、$S = 2/3$、$T = 1/3$。 取角误差 0.2 弧度、相对角速度零、旧冲量零。 则 $\Delta\lambda_s = -1/3$ 牛顿·米·秒。 $K_\theta = 2$，相对角速度变化为 $-2/3$ 弧度每秒，朝减小角误差方向转动。

3.3 角马达的限幅是力矩乘时间

目标相对角速度为 ω_t：

$$\Delta\lambda_{trial} = M_\theta(\omega_t - \dot{\theta}),$$

$$\lambda_{new} = \mathrm{clamp}(\lambda_{old} + \Delta\lambda_{trial}, -h\tau_{max}, h\tau_{max}).$$

τ_{max} 单位为牛顿·米。 正目标使 B 相对于 A 逆时针转动。 负目标相反。

数值例：$M_\theta = 0.5$，目标 4 弧度每秒，当前零。 取 $h = 0.1$ 秒、$\tau_{max} = 2$ 牛顿·米。 试算冲量 2，累计上限仅 0.2。 实际相对角速度增加 $2 \times 0.2 = 0.4$ 弧度每秒。 马达不是瞬间覆盖刚体角速度。

3.4 上下限的三处符号必须一起翻转

下限与上限写成

$$C_L = \theta - \theta_L \geq 0, \qquad C_U = \theta_U - \theta \geq 0.$$

下限使用 $\dot{C}_L = \omega_B - \omega_A$，正角冲量加给 B。 上限使用 $\dot{C}_U = \omega_A - \omega_B$，正累计量对应负角冲量加给 B。

因此上限同时翻转：

1. 位置误差的定义。
2. 相对角速度的定义。
3. 最终施加冲量的方向。

只翻其中一处会造成“越过上限后越转越快”。 两种限位的累计标量都投影到 $[0, \infty)$。 热启动的角总量为

$$\lambda_{axial} = \lambda_s + \lambda_m + \lambda_L - \lambda_U.$$

3.5 上限完整代入

A 静态，B 逆惯量 $\iota_B = 2$，故 $M_\theta = 0.5$。 当前角 0.45，上限 0.5 弧度。 当前 B 角速度 3 弧度每秒，$h = 0.1$ 秒。 旧上限冲量零。

$$C_U = 0.05, \quad b = C_U / h = 0.5, \quad \dot{C}_U = -3.$$

$$\lambda_U = -0.5(-3 + 0.5) = 1.25 \ \mathrm{N\,m\,s}.$$

施加方向是负角冲量：

$$\omega_B' = 3 - 2 \times 1.25 = 0.5.$$

预测下一角度 $0.45 + 0.1 \times 0.5 = 0.5$，恰好到上限。 将整个例子镜像成下限 -0.5、当前 -0.45、角速度 -3，会得到 $+1.25$ 的下限冲量与末态 -0.5 弧度每秒。

4. Prismatic：沿导轨滑动，同时锁住相对转角

4.1 导轨跟着 A 转，不是固定世界向量

定义 A 关节框架的世界 x 轴为

$$t = Q_A(1, 0), \qquad n = \mathrm{leftPerp}(t) = (-t_y, t_x).$$

t 是滑动方向，n 是横向，两者无量纲且正交。 当前轴向位移为

$$x = t \cdot d.$$

基础棱柱约束为

$$C_1 = n \cdot d = 0, \qquad C_2 = \theta = 0.$$

C_1 单位为米，C_2 为弧度。 允许沿 t 平移，但不允许横向脱轨或相对转动。

4.2 必须对轴方向本身求导

这是最容易漏掉的一步。 因为 n 随 A 转动，

$$\dot{C}_1 = \dot{n} \cdot d + n \cdot \dot{d}.$$

且 $\dot{n} = \omega_A \times n$。 展开后

$$\dot{C}_1 = n \cdot (v_B - v_A) + (r_B \times n)\omega_B - [(r_A + d) \times n]\omega_A.$$

定义

$$s_1 = (r_A + d) \times n, \qquad s_2 = r_B \times n.$$

于是

$$\dot{C}_1 = n \cdot (v_B - v_A) + s_2\omega_B - s_1\omega_A.$$

s_1, s_2 单位为米。 `d+rA` 并不是为了凑代码风格，而是导轨方向求导产生的真实项。 若漏掉 $\dot{n} \cdot d$，会错误地把导轨当成世界固定方向。

两行 Jacobian 为

$$J = \begin{bmatrix} -n_x & -n_y & -s_1 & n_x & n_y & s_2 \\ 0 & 0 & -1 & 0 & 0 & 1 \end{bmatrix}.$$

4.3 横向和角度也需要块求解

有效矩阵为

$$K = \begin{bmatrix} \mu_A + \mu_B + \iota_A s_1^2 + \iota_B s_2^2 & \iota_A s_1 + \iota_B s_2 \\ \iota_A s_1 + \iota_B s_2 & \iota_A + \iota_B \end{bmatrix}.$$

这一矩阵的两行不是同一种单位。 K_{11} 单位为每千克；K_{12} 为每千克·米；K_{22} 为每千克·平方米。 冲量向量的第一分量是横向线冲量，单位为牛顿·秒。 第二分量是角冲量，单位为牛顿·米·秒。 它们装进同一个 `b2Vec2`，不表示它们有相同物理量纲。

令块冲量为 $(\lambda_\perp, \lambda_\theta)^T$。 对应线冲量与角冲量臂为

$$P = \lambda_\perp n,$$

$$L_A = s_1\lambda_\perp + \lambda_\theta, \qquad L_B = s_2\lambda_\perp + \lambda_\theta.$$

更新

$$v_A' = v_A - \mu_A P, \quad \omega_A' = \omega_A - \iota_A L_A,$$

$$v_B' = v_B + \mu_B P, \quad \omega_B' = \omega_B + \iota_B L_B.$$

注意不能直接用普通锚点力臂 $r_A \times P$ 替代 $s_1\lambda_\perp$。 导轨约束的 Jacobian 已经包含导轨转向产生的贡献。

4.4 完整棱柱二维解

取 $\mu_A = \mu_B = \iota_A = \iota_B = 1$。 取 $t = (1, 0)$、$n = (0, 1)$。 取 $r_A = r_B = (0, 0)$，而 $d = (1, 0)$ 米。

于是

$$s_1 = (1, 0) \times (0, 1) = 1, \qquad s_2 = 0.$$

即使两个锚点都在各自质心，s_1 仍不为零。 这正好突出导轨转向的影响。

初始 $v_A = 0$、$v_B = (0, 2)$、$\omega_A = 0$、$\omega_B = 1$。 则块速度为 $(2, 1)^T$。 不加偏置时

$$K = \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix}, \quad \det K = 5, \quad K^{-1} = \frac{1}{5}\begin{bmatrix} 2 & -1 \\ -1 & 3 \end{bmatrix}.$$

$$\begin{bmatrix} \lambda_\perp \\ \lambda_\theta \end{bmatrix} = -\frac{1}{5}\begin{bmatrix} 2 & -1 \\ -1 & 3 \end{bmatrix}\begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.6 \\ -0.2 \end{bmatrix}.$$

因此

$$P = (0, -0.6), \quad L_A = -0.8, \quad L_B = -0.2.$$

末态

$$v'_A = (0, 0.6), \quad v'_B = (0, 1.4), \quad \omega'_A = 0.8, \quad \omega'_B = 0.8.$$

核对角约束：$0.8 - 0.8 = 0$。 核对横向约束：

$$n \cdot (v'_B - v'_A) + s_2\omega'_B - s_1\omega'_A = 0.8 + 0 - 0.8 = 0.$$

不能因为两质心 y 速度仍不同就断言没有解好。 导轨自己在转，横向速度条件包含导轨方向的导数。

4.5 沿轴的弹簧、马达和限位

同理，对 $x = t \cdot d$ 求导，定义

$$a_1 = (r_A + d) \times t, \qquad a_2 = r_B \times t,$$

$$\dot{x} = t \cdot (v_B - v_A) + a_2\omega_B - a_1\omega_A.$$

轴向逆有效质量为

$$K_a = \mu_A + \mu_B + \iota_A a_1^2 + \iota_B a_2^2.$$

轴向冲量 λ_a 的作用为

$$P = \lambda_a t, \quad L_A = a_1\lambda_a, \quad L_B = a_2\lambda_a.$$

弹簧位置误差是 $x - \texttt{targetTranslation}$，单位为米。 马达误差是 $\dot{x} - \texttt{motorSpeed}$，单位为米每秒。 马达累计量限制到 $\pm h\,\texttt{maxMotorForce}$。 限位使用 $x - x_L \geq 0$ 和 $x_U - x \geq 0$，上限依旧翻转方向。

本版先解弹簧、马达、上下限，再解横向与角度块。 和距离关节不同，这里基础横向与角度约束一直存在；关闭轴向 spring 不会把整个滑轴锁成硬杆。

4.6 为什么限位附近还要一个距离门槛

`prismatic_joint.c:497-590` 使用

$$d_{spec} = 0.25(x_U - x_L).$$

仅在相应安全余量 $C < d_{spec}$ 时求该端限位，否则清除该端累计量。 预测偏置的距离还限到一米对应的当前长度单位。 这减少在远离边界时不必要的预测式限制与过大的偏置。

不要把距离关节的限位实现逐行复制到这里。 物理思想相同，但本版棱柱有额外范围判断。

4.7 朴素实现到本版优化

第一版先实现两行硬约束，固定步长反复求解。 第二版增加轴向弹簧或马达，验证目标的单位。 第三版才加上下限、热启动与子步几何更新。

本版在每次 Solve 中重算 `axialMass` 和二维 `K`。 这是与距离关节缓存标量质量不同的具体选择。 两边都固定旋转时，源码把块的 `k22` 改为 `1` 防止该部分退化。 这不是凭空赋予刚体可旋转能力，因为真正的角速度更新仍乘逆惯量零。

5. Wheel：有悬挂行程，但不要锁住轮子转动

5.1 轮关节不是棱柱加一个名字

两者共有导轨几何：t、n、d、a_1, a_2, s_1, s_2。 但 wheel 不加入 $\theta = 0$ 那一行。 如果把棱柱的角度锁定直接搬过来，轮子就不能相对车身自由旋转。

wheel 基础约束只有

$$C_\perp = n \cdot d = 0.$$

悬挂方向允许运动，随后可用弹簧和行程限位控制。 轮子的旋转由单独的角马达控制。

5.2 横向防脱轨

横向速度与逆有效质量为

$$\dot{C}_\perp = n \cdot (v_B - v_A) + s_2\omega_B - s_1\omega_A,$$
$$K_\perp = \mu_A + \mu_B + \iota_A s_1^2 + \iota_B s_2^2.$$

无偏置的冲量为 $-\dot{C}_\perp / K_\perp$。 加偏置时应用基础软度，与前章同样区分累计量和增量。

它只有一个标量，不需要棱柱的横向—角度二维块。 这正是相对转动没有被锁死的数学表现。

5.3 悬挂弹簧

本版悬挂弹簧误差直接为

$$C_s = t \cdot d.$$

这里不像棱柱暴露单独的 `targetTranslation`。 希望改变悬挂的参考位置，应正确设置局部锚点与关节框架。

轴向质量仍为

$$M_a = 1/(\mu_A + \mu_B + \iota_A a_1^2 + \iota_B a_2^2).$$

弹簧求解

$$\Delta\lambda_s = -SM_a(\dot{x} + BC_s) - T\lambda_{s,old}.$$

下限和上限约束的是悬挂行程，不是轮子的旋转角度。 相同关节内可以同时存在悬挂力、限位力和轮轴驱动力矩。

5.4 轮马达是角马达

目标为 $\omega_B - \omega_A = \texttt{motorSpeed}$。 最大能力是 `maxMotorTorque`，单位为牛顿·米。 不是棱柱的 `maxMotorForce`，也不是沿悬挂方向的目标速度。

角有效惯量为

$$M_m = 1/(\iota_A + \iota_B).$$

轮子对车身也产生等大反向的驱动力矩。 静态车身不响应不代表动态车身也没有反作用。

5.5 完整数值例：三种作用各管一件事

取 A 静态，B 质量 2 千克、惯量 1 千克·平方米。 令 B 锚点在质心，A 的锚点力臂也取零。 取悬挂轴 $t = (0, 1)$，横向 $n = (-1, 0)$，两锚点差 $d = (0, 0.2)$ 米。

由于 A 静态，涉及 A 逆惯量的项全为零。 所以

$$M_a = M_\perp = 2\ \mathrm{kg}, \qquad M_m = 1\ \mathrm{kg\,m^2}.$$

初始 B 速度 $(3, 0)$ 米每秒，角速度零。 取 $h = 0.1$ 秒，弹簧参数 $B = 5, S = 2/3, T = 1/3$，旧冲量全零。 关闭行程限位；横向约束本例按无偏置模式计算。

第一步，角马达目标 5 弧度每秒，最大力矩 2 牛顿·米。 理想冲量为 5，限幅后只有 0.2。 B 的角速度成为 0.2 弧度每秒。

第二步，弹簧误差 $C_s = 0.2$ 米，轴速度仍零。

$$\lambda_s = -\frac{2}{3} \times 2 \times (5 \times 0.2) = -\frac{4}{3}\ \mathrm{N\,s}.$$

B 的 y 速度变化为 $0.5 \times (-4/3) = -2/3$ 米每秒。 它向缩短悬挂位移的方向运动。

第三步，横向速度

$$\dot{C}_\perp = (-1, 0) \cdot (3, -2/3) = -3.$$

横向冲量 $\lambda_\perp = -2(-3) = 6$ 牛顿·秒。 世界冲量是 $6(-1, 0) = (-6, 0)$，所以 B 的 x 速度减为零。

末态为

$$v_B' = (0, -2/3), \qquad \omega_B' = 0.2.$$

它既没有横向脱轨，又在悬挂方向回弹，还能转动。 三个作用不是在争夺同一个目标。 但一般偏心几何下它们会通过角速度互相影响，仍需按当前速度顺序求解。

5.6 本版有哪些具体优化

`b2PrepareWheelJoint` 缓存 `perpMass`、`axialMass`、`motorMass`。 Solve 更新当前锚点与方向，却复用 Prepare 的三个标量质量。 这与棱柱每次重算轴向质量的策略不同。

求解顺序为角马达、悬挂弹簧、行程限位、横向约束。 真实弹簧在 relax 中仍求解，不能把 `useBias=false` 理解成“所有弹簧都关闭”。

6. Weld：平动和角度都要锁，为什么本版不用活动的三维块

6.1 理想焊接的目标

理想焊接有三条目标：

$$C_p = d = 0, \qquad C_\theta = \theta = 0.$$

平动两行使用 revolute 的 J_p。 角度一行使用 J_θ。 因此理论上可以拼成一个 3×6 Jacobian 和一个 3×3 有效质量矩阵。

其平动子块就是前面的 K_p，角度元素为 $\iota_A + \iota_B$。 平动—角度耦合列为

$$k_{p\theta} = \begin{bmatrix} -\iota_A r_{Ay} - \iota_B r_{By} \\ \iota_A r_{Ax} + \iota_B r_{Bx} \end{bmatrix}.$$

整体矩阵为

$$K_3 = \begin{bmatrix} K_p & k_{p\theta} \\ k_{p\theta}^T & \iota_A + \iota_B \end{bmatrix}.$$

这解释了完整耦合从哪里来。 但推得出三维块，不表示当前版本就在运行该块。

6.2 当前真正运行的是分开的角度与二维平动

`src/weld_joint.c:15` 定义 `B2_WELD_BLOCK_SOLVE 0`。 源码保留了三维块的实验分支，但默认没有编译执行。 `weld_joint.c:275` 明确说明不同刚度混合时该块求解不能正确工作。

活动分支 `367-434` 的顺序是：

1. 解一个角度约束。
2. 用已经更新的角速度重算锚点相对速度。
3. 解一个二维平动约束。

角度增量为

$$\Delta\lambda_\theta = -S_\theta M_\theta(\dot\theta + B_\theta\theta) - T_\theta\lambda_{\theta,old}.$$

平动增量为

$$\Delta P = -S_p K_p^{-1}(\dot C_p + B_p C_p) - T_p P_{old}.$$

角度与平动允许使用不同频率和阻尼。 顺序求解在局部并不保证一次消除所有交叉耦合，整体迭代与子步继续改善误差。 本版选择的是兼容混合刚度的实际实现，而不是为了追求矩阵更大。

6.3 Weld 的零 Hz 特判与 Distance 不同

`b2PrepareWeldJoint` 的逻辑是：

- `linearHertz==0`：平动使用基础 `constraintSoftness`。
- `angularHertz==0`：角度使用基础 `constraintSoftness`。
- 正频率：分别调用 `b2MakeSoft` 生成对应弹簧软度。

因此 weld API 的零频率代表相应方向采用刚性约束模式。 不是直接使用 `b2MakeSoft(0)` 返回的全零系数。 这正是为什么不能将“Hz 为零”的含义跨关节照抄。

Solve 使用条件

$$\texttt{useBias} \ \vee \ (\text{该方向}\texttt{Hertz} > 0).$$

硬模式在 relax 中退回零位置偏置、单位质量缩放。 真实弹簧模式即使 relax 也仍包含弹簧作用。

6.4 混合软硬的完整数值例

取 A 静态，B 质量与惯量均为 1 的 SI 数值。 两锚点都在质心，因此平动与转动在本例中没有力臂耦合。 初始线速度为 $(1,-1)$ 米每秒，角速度为 2 弧度每秒。 位置误差 $d=(0.1,-0.2)$ 米，相对角误差 0.1 弧度。

设平动 `linearHertz=0`，角度取 $\Omega=10$、$\zeta=0.5$、$h=0.1$。 角度软度因此为 $B_\theta=5, S_\theta=2/3, T_\theta=1/3$。 旧冲量零，本轮 `useBias=false`。

角度频率为正，因此即使本轮 relax，仍求弹簧：

$$\Delta\lambda_\theta=-\frac{2}{3}(2+5\times 0.1)=-\frac{5}{3}.$$

角速度变为

$$\omega_B'=2-\frac{5}{3}=\frac{1}{3}\ \mathrm{rad/s}.$$

平动频率零且 `useBias=false`，所以不加位置偏置。 K_p 为单位矩阵，得到

$$P=-(1,-1)=(-1,1)\ \mathrm{N\,s}.$$

最终线速度为零，角速度为 $1/3$。 平动误差没有在这一轮被直接改写；后续有偏置阶段与积分负责处理。 角弹簧保留非零速度也不是失败，它按弹性动力学响应，而不是无条件把角速度设零。

若角度也设为硬模式，并仍考察无偏置轮，角冲量就是 -2，角速度成为零。 这能直接对比“刚性速度约束”和“柔性弹簧响应”。

7. 从零实现时建议的顺序

先只实现 revolute 点对点约束，并用非零矩阵交叉项的例子测试。 接着做单独角约束，确认角冲量单位和正方向。 然后实现 prismatic，必须用带导轨转动的例子验证 $d+r_A$。 在棱柱基础上移除角度锁定，再加旋转马达，就能理解 wheel 的结构。 最后实现 weld 的角度与平动软硬选择。

每种关节都先只开一个可选功能。 确认单独正确后再叠加弹簧、马达和限位。 如果一开始全部启用，正负号错了也可能被另一约束暂时压住，反而更难定位。

8. 源码地图

类型	定义	Prepare / WarmStart / Solve
revolute	`types.h:863-901`	`revolute_joint.c`：`217` / `283` / `317`
prismatic	`types.h:816-854`	`prismatic_joint.c`：`295` / `353` / `407`
wheel	`types.h:941-975`	`wheel_joint.c`：`197` / `280` / `329`
weld	`types.h:912-931`	`weld_joint.c`：`148` / `222` / `253`

特别应对照：

- Revolute 点对点完整块：`revolute_joint.c:443-486`。
- Revolute 上限三处符号：`410-439`。
- Prismatic 当前轴和轴向质量：`prismatic_joint.c:429-448`。
- Prismatic 横向—角度块：`595-647`。
- Prismatic 范围限制：`497-590`。
- Wheel 缓存三种质量：`wheel_joint.c:247-268`。
- Wheel 实际四阶段：`365-509`。
- Weld 宏开关：`weld_joint.c:15`。
- Weld 零 Hz 改用基础软度：`197-212`。
- Weld 当前活动分支：`367-434`。

9. 习题与答案

题 1：Revolute 为什么不使用一条零长度距离约束

答案：距离约束只沿一条单位连线作用，零长度时方向不定义。 点对点约束是两个确定的坐标分量，Jacobian 有两行，结构不同。

题 2：什么时候 revolute 的二维矩阵交叉项为零

答案：当 $\iota_A r_{Ax} r_{Ay} + \iota_B r_{Bx} r_{By} = 0$。 两锚点都在质心是充分条件，但不是唯一条件；两贡献也可能抵消。

题 3：棱柱两个锚点都在质心，能否把所有转动系数删掉

答案：不能。$s_1 = d \times n$、$a_1 = d \times t$ 仍可能非零。 原因是导轨方向随 A 转动，而不是锚点自身偏心。

题 4：Wheel 的 motorSpeed 为 10，单位是什么

答案：弧度每秒，它是相对角速度，不是悬挂伸缩速度。 Prismatic 的同名参数才是沿轴米每秒。

题 5：为什么 wheel 不能多加一条相对角为零

答案：那会把车轮与车身的相对转动锁住，变成更接近棱柱的结构。 旋转马达会与角度锁定产生冲突。

题 6：Weld 两方向都零 Hz，是否没有约束

答案：不是。Prepare 为零频率方向改用基础硬约束软度。 不要把 `b2MakeSoft(0)` 的实现直接套在 weld 的用户参数上。

题 7：看到 `b2Solve33` 就能说本版 weld 使用三维块吗

答案：不能。必须检查预处理开关。 本版 `B2_WELD_BLOCK_SOLVE=0`，活动路径是角度标量加平动二维块。

下一章：马达、mover 与 pogo · 返回目录

15 • 马达、mover 与 pogo 关节

返回目录 · 上一章

> 基线：Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b` 的当前工作树。 本章回答“如何驱动物体，但仍允许力不够、仍接受碰撞与反作用”。 也会分清名字相近的 mover 关节与角色碰撞查询，不把二者误讲成同一求解器。

1. 先分清四种目的

对象	主要目标	是否主动生成约束冲量
motor joint	用有限力/力矩追踪关节位姿及相对速度	是
mover joint	用每个世界轴独立的力上限追踪质心相对线速度	是
pogo joint	用弹簧内部状态生成恢复速度，再沿指定法线施力	是
filter joint	管理一对刚体是否互相碰撞	否

“驱动”不等于每帧直接写位置。 直接改位置没有自动计算推动所需的冲量，也不会自然给另一物体反作用。 这些关节则仍使用刚体质量、角惯量和成对冲量更新。

所有力上限都应先乘子步时长 h 再参与冲量限幅。 线冲量单位为牛顿·秒。 角冲量单位为牛顿·米·秒。 本章沿用 $\mu = 1/m$、$\iota = 1/I$，以免把源码局部变量 `mA` 误看成真实质量。

2. Motor：位姿控制与速度控制是四个独立通道

2.1 当前 API 不是旧版 motor joint 参数表

`b2MotorJointDef` 的主要字段是：

- `linearVelocity`：目标相对线速度向量。
- `maxVelocityForce`：线速度通道最大力。
- `angularVelocity`：目标相对角速度。
- `maxVelocityTorque`：角速度通道最大力矩。
- `linearHertz/linearDampingRatio/maxSpringForce`：平动位姿弹簧。
- `angularHertz/angularDampingRatio/maxSpringTorque`：角度位姿弹簧。

因此应想成四个可单独启用的执行器：

1. 平动弹簧。
2. 角度弹簧。
3. 线速度马达。

4. 角速度马达。

没有足够力时，目标只是希望达到的状态，不是硬性覆盖。 `linearVelocity` 与 `angularVelocity` 是目标速度，不是“最高允许速度”阈值。 当前这些字段并没有定义一个通用的绝对刚体速度裁剪器。

默认构造 `b2DefaultMotorJointDef` 将这些驱动量置零。 只指定两个刚体而不设置相应最大力/力矩，不会自动得到很强的跟随器。

2.2 目标位姿藏在两个局部关节框架里

设刚体变换为 X_A, X_B，局部关节框架为 F_A, F_B。 对应的世界关节框架为

$$G_A = X_A F_A, \qquad G_B = X_B F_B.$$

位置控制的目标是让两个框架重合：

$$G_B = G_A.$$

等价地，目标 B 变换为

$$X_{B,target} = X_A F_A F_B^{-1}.$$

这只是表达目标关系，不表示源码直接把该矩阵赋给 B。 实际求解仍根据当前误差产生有限冲量。

框架的平移误差为两锚点差

$$C_p = p_B - p_A = c_B + r_B - c_A - r_A.$$

角误差为

$$C_\theta = \mathrm{angle}(Q_A^{-1} Q_B).$$

C_p 单位为米，C_θ 为弧度。 若希望 B 相对 A 保持偏移，就应设置适当的局部框架，而不是误以为目标永远是两质心重合。

2.3 朴素实现：先只追一个速度

最小练习可以让 A 为静态、B 的锚点位于质心。 此时暂时没有转动耦合。 按以下步骤处理：

1. 求速度误差 $e_v = v_B - v_A - v_t$。
2. 求标量有效质量 $M_v = 1/(\mu_A + \mu_B)$。
3. 求理想冲量 $\Delta P = -M_v e_v$。
4. 将新累计冲量投影到允许的最大冲量圆盘。
5. 只施加新旧累计量之差。
6. 将相反冲量施加给 A。

这个最小版先验证“目标速度”“最大力”“质量大小”的关系。 然后才把锚点离开质心，加入下面的二维质量矩阵。

3. Motor 线速度通道：不是只有质心速度

3.1 几何速度与 Jacobian

一般锚点速度为

$$v_{pA} = v_A + \omega_A \times r_A, \qquad v_{pB} = v_B + \omega_B \times r_B.$$

线速度通道误差为

$$e_v = v_{pB} - v_{pA} - v_t.$$

v_t 是 `linearVelocity`，单位为米每秒。 本版直接从世界空间锚点相对速度减去该向量，没有在 Solve 中再按 A 框架旋转它。 若应用希望“角色面向方向的前进速度”，应先转换为所需世界向量。

Jacobian 与点对点关节相同：

$$J_p = \begin{bmatrix} -1 & 0 & r_{Ay} & 1 & 0 & -r_{By} \\ 0 & -1 & -r_{Ax} & 0 & 1 & r_{Bx} \end{bmatrix}.$$

3.2 完整线有效质量

令

$$K_p = \begin{bmatrix} \mu_A + \mu_B + \iota_A r_{Ay}^2 + \iota_B r_{By}^2 & -\iota_A r_{Ax} r_{Ay} - \iota_B r_{Bx} r_{By} \\ -\iota_A r_{Ax} r_{Ay} - \iota_B r_{Bx} r_{By} & \mu_A + \mu_B + \iota_A r_{Ax}^2 + \iota_B r_{Bx}^2 \end{bmatrix}.$$

K_p 单位为每千克，$M_p = K_p^{-1}$ 单位为千克。 理想线速度冲量为

$$\Delta P_{trial} = -M_p e_v.$$

如果两个锚点偏心，线速度驱动可以产生角速度。 所以 motor 的线驱动和角驱动可能通过力臂彼此影响。 不能把它永远当成两个互不相关的质心力。

3.3 圆盘限幅，而不是逐轴限幅

设 $P_{trial} = P_{old} + \Delta P_{trial}$。 允许的累计线冲量满足

$$\|P\| \le h F_{max}.$$

投影公式为

$$P_{new} = \begin{cases} P_{trial}, & \|P_{trial}\| \le h F_{max}, \\ \dfrac{h F_{max}}{\|P_{trial}\|} P_{trial}, & \text{否则}. \end{cases}$$

实际施加 $\Delta P = P_{new} - P_{old}$。 投影保留方向，只限制合力大小。 若分别把 x、y 都限制到 $h F_{max}$，对角线最大合力会变成原来的 $\sqrt{2}$ 倍，这不是当前 motor 语义。

3.4 完整数值例：目标 3–4，冲量 6–8，再整体限幅

A 静态，B 质量 2 千克，锚点在质心。 所以 $\mu_B = 0.5$，$M_p = 2I_2$。 初始线速度零，目标 $v_t = (3, 4)$ 米每秒。 取 $h = 0.1$ 秒、最大速度驱动力 10 牛顿，旧冲量零。

$$e_v = (-3, -4), \qquad \Delta P_{trial} = (6, 8)\ \mathrm{N\,s}.$$

其长度为 10 牛顿·秒，但上限只有 $hF = 1$。 所以

$$P_{new} = \frac{1}{10}(6, 8) = (0.6, 0.8).$$

B 的速度成为

$$v'_B = 0 + 0.5(0.6, 0.8) = (0.3, 0.4)\ \mathrm{m/s}.$$

检查平均力：$P/h = (6, 8)$ 牛顿，合力大小为 10 牛顿。 目标速度的方向被保留，但受限于这一子步只能提供的冲量。

4. Motor 角速度与位姿弹簧

4.1 角速度通道

角速度误差为

$$e_\omega = \omega_B - \omega_A - \omega_t.$$

Jacobian 为 $[0, 0, -1, 0, 0, 1]$。 有效惯量

$$M_\theta = 1/(\iota_A + \iota_B).$$

试算增量与累计限幅为

$$\Delta\lambda_{trial} = -M_\theta e_\omega,$$

$$\lambda_{new} = \mathrm{clamp}(\lambda_{old} + \Delta\lambda_{trial}, -h\tau_{max}, h\tau_{max}).$$

在前例中再取 B 惯量 4 千克·平方米，角目标 2 弧度每秒。 $M_\theta = 4$，理想冲量 8。 若最大力矩 5 牛顿·米，$h = 0.1$，只能施加 0.5 牛顿·米·秒。 角速度增加 $0.25 \times 0.5 = 0.125$ 弧度每秒。

4.2 平动弹簧

第 13 章推导的软约束形式现在变成向量：

$$\Delta P_s = -S_p M_p(\dot{C}_p + B_p C_p) - T_p P_{s,old}.$$

$\dot{C}_p = v_{pB} - v_{pA}$，这里不减线速度通道目标。 位置弹簧和速度马达分别表达各自的需求。

平动弹簧的累计量独立限制到

$$\|P_s\| \le h\,\mathtt{maxSpringForce}.$$

只有 `maxSpringForce>0` 且 `linearHertz>0` 才执行该通道。 频率零在这里表示不执行位置弹簧，不是 weld 那样的刚性替代模式。

4.3 角度弹簧

角度弹簧为

$$\Delta\lambda_s = -S_\theta M_\theta(\omega_B - \omega_A + B_\theta C_\theta) - T_\theta \lambda_{s,old}.$$

独立限制到 $\pm h\,\texttt{maxSpringTorque}$。 只有 `maxSpringTorque>0` 且 `angularHertz>0` 才执行。 正角误差对应负恢复角冲量，促使 B 相对 A 向回转。

4.4 位姿弹簧完整限幅例

本例独立于前面的速度驱动例，初始速度重新取零。 A 静态，B 质量 2、惯量 4，锚点在质心。 取 $h = 0.1$、$\Omega = 10$、$\zeta = 0.5$。 所以 $B = 5, S = 2/3, T = 1/3$。

位置误差 $C_p = (0.2, -0.1)$ 米，旧弹簧冲量零：

$$\Delta P_{trial} = -\frac{2}{3} \times 2 \times 5(0.2, -0.1) = (-4/3, 2/3).$$

长度为 $\sqrt{20}/3 \approx 1.490712$ 牛顿·秒。 若最大弹簧力为 5 牛顿，则冲量圆盘半径为 0.5。 因此

$$P_s = (-1/\sqrt{5}, 1/(2\sqrt{5})) \approx (-0.447214, 0.223607).$$

线速度变化为

$$\Delta v_B = 0.5 P_s \approx (-0.223607, 0.111803)\ \mathrm{m/s}.$$

角误差再取 0.2 弧度，角弹簧理想冲量为

$$\Delta\lambda_{trial} = -\frac{2}{3} \times 4 \times (5 \times 0.2) = -8/3.$$

若最大弹簧力矩为 2 牛顿·米，冲量下限是 -0.2。 实际角速度变化为 $0.25 \times (-0.2) = -0.05$ 弧度每秒。 两个通道都朝恢复位姿方向作用，又都没有超出自己的能力上限。

4.5 四通道的上限不是一个共享预算

本版分别保存线速度冲量、角速度冲量、线弹簧冲量、角弹簧冲量。 例如线速度力上限为 10 牛顿、线弹簧力上限为 5 牛顿。 不能据此断言总 motor 线力仍永远不超过 10 牛顿。 两通道同向饱和时，总量可以接近 15 牛顿。 反向时则可能互相抵消。

若业务需要一个严格共享的总力预算，需要在设计层面另行处理，不能假装当前两个独立限幅已经实现该需求。

5. Motor 的实际求解顺序与优化

`b2PrepareMotorJoint`， `motor_joint.c:179-249`，准备两框架、质量矩阵及两组软度。

`b2WarmStartMotorJoint`， `251-285`，先把两组线冲量相加、两组角冲量相加，再施加到速度。

`b2SolveMotorJoint`， `287-436`，顺序为：

1. 角弹簧。
2. 角速度马达。
3. 平动弹簧。
4. 线速度马达。

角度通道的结果会影响后面的锚点速度。 因此不能让四通道都读取同一份旧速度，最后才统一应用。

Prepare 计算初始 `linearMass`。 正频率线弹簧执行时，Solve 根据当前力臂再次更新这个逆矩阵。 后面的线速度通道使用当前存储的 `linearMass`。 若线弹簧关闭，线速度通道使用先前 Prepare 的矩阵。 这是源码的具体缓存策略，不应概括为“所有分支每次都重算矩阵”。

`b2SolveMotorJoint` 不接收 `useBias`。 统一分发调用它时，不会因为 relax 而自动关闭这些驱动通道。

6. Mover joint：两个世界轴上的有限力速度追踪

6.1 先纠正同名概念

`src/mover_joint.c` 不收集碰撞平面。 它不扫描胶囊，不沿墙滑动，也不在这个函数里处理台阶。 它的物理目标很明确：

$$v_B - v_A = v_t.$$

这里是质心线速度，不是偏心锚点速度。 没有 $\omega \times r$ 项。

Prepare 显式将这条关节缓存中的 `invIA/invIB` 设为零。 这不修改刚体真实惯量，只表示此关节不通过角自由度求响应。

6.2 Jacobian 和标量有效质量

二维 Jacobian 为

$$J_m = \begin{bmatrix} -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 \end{bmatrix}.$$

因此

$$K_m = (\mu_A + \mu_B) I_2, \qquad M_m = \frac{1}{\mu_A + \mu_B}.$$

M_m 单位为千克。 同一个标量可同时用于 x、y，不需要偏心矩阵求逆。

误差与试算冲量为

$$e = v_B - v_A - v_t, \qquad \Delta P_{trial} = -M_m e.$$

冲量更新

$$v'_A = v_A - \mu_A \Delta P, \qquad v'_B = v_B + \mu_B \Delta P.$$

角速度完全不在这段更新中。 若希望避免角色因驱动力自转，这是很有用的控制方式。 但碰撞或其他关节仍可能改变它的角速度。

6.3 真正的区别：累计冲量可行域是矩形

Mover 的 `maxVelocityForce` 是二维向量，不是 motor 的标量。 分别记 x、y 最大力为 F_x, F_y。 约束为

$$-hF_x \le P_x \le hF_x, \qquad -hF_y \le P_y \le hF_y.$$

所以必须逐分量投影：

$$P_{new,x} = \mathrm{clamp}(P_{old,x} + \Delta P_x, -hF_x, hF_x),$$
$$P_{new,y} = \mathrm{clamp}(P_{old,y} + \Delta P_y, -hF_y, hF_y).$$

这是一块轴对齐矩形，不是圆盘。 方向可以因为 x、y 不同程度饱和而改变。 当 $F_y = 0$ 时，这个关节不能提供竖直驱动力，即使目标竖直速度不为零。

```
	// linear velocity
	if ( joint->maxVelocityForce.x > 0.0f || joint->maxVelocityForce.y > 0.0f )
	{
		b2Vec2 cdot = b2Sub( vB, vA );
		cdot = b2Sub( cdot, joint->linearVelocity );
		b2Vec2 b = b2MulSV( joint->linearMass, cdot );
		b2Vec2 impulse = { -b.x, -b.y };

		b2Vec2 oldImpulse = joint->linearVelocityImpulse;
		joint->linearVelocityImpulse = b2Add( joint->linearVelocityImpulse, impulse );
		b2Vec2 maxImpulse = b2MulSV( context->h, joint->maxVelocityForce );

		// clamp x and y separately
		joint->linearVelocityImpulse = b2Clamp( joint->linearVelocityImpulse, b2Neg(
maxImpulse ), maxImpulse );

		impulse = b2Sub( joint->linearVelocityImpulse, oldImpulse );
		vA = b2MulSub( vA, mA, impulse );
		vB = b2MulAdd( vB, mB, impulse );
```

6.4 两边都动态的完整数值例

取 A 质量 2、B 质量 1 千克。 两边初始线速度零，目标相对速度为 $(3, 4)$ 米每秒。

$$\mu_A = 0.5, \quad \mu_B = 1, \quad M_m = 2/3.$$

取 $h = 0.1$ 秒，$(F_x, F_y) = (6, 2)$ 牛顿。 理想冲量

$$\Delta P_{trial} = \frac{2}{3}(3, 4) = (2, 8/3).$$

允许矩形为 $[-0.6, 0.6] \times [-0.2, 0.2]$。 因此实际累计冲量为 $(0.6, 0.2)$ 牛顿·秒。 旧冲量零，所以全部作为本次增量施加。

$$v'_A = (-0.3, -0.1), \qquad v'_B = (0.6, 0.2).$$

相对速度为

$$v'_B - v'_A = (0.9, 0.3)\ \mathrm{m/s}.$$

它没有保持目标向量 $(3, 4)$ 的方向，因为 y 轴的力预算更小。 两边线动量核查：$2(-0.3, -0.1) + (0.6, 0.2) = (0, 0)$。 两边角速度不变。

平均合力大小为 $\sqrt{6^2 + 2^2} \approx 6.3249$ 牛顿。 它可以大于 x 分量上限 6，因为限制的是分量，不是合力模长。

6.5 朴素实现与本版优化

Mover 是很适合第一个实现的“有力上限速度执行器”。 只需两刚体索引、一个标量质量、一组二维累计冲量。 不必计算锚点方向、角度或二维矩阵。

本版仍使用统一的 Prepare/WarmStart/Solve 调度。 两个力上限都不为正时，Solve 清除累计冲量。 同 motor 一样，它的 Solve 不接收 `useBias`。 不要因为它可用于角色，就把它写成世界碰撞查询的一部分。

7. Pogo：恢复长度的方向，与施力方向可以不同

7.1 本版比“沿一根轴的普通弹簧”多一步

Pogo 常用来表达角色与支撑表面之间的弹性恢复。 但这份源码不是直接复制距离弹簧的软约束增量公式。 它先推进一个内部弹簧速度，再把这个速度当成法向速度目标。

必须区分两个方向：

- a：测量腿长的 pogo 轴。
- n：施加冲量的法线 `normal`。

在 `b2SolvePogoJoint` 中

$$a = \texttt{frameB.q}\,(0, 1).$$

而 n 在创建时直接从 `def.normal` 保存。 它不是由两锚点连线归一化得到，也不是从 A 的局部 x 轴自动推导。 创建代码没有在这里代替调用者重新归一化。 使用者应提供方向正确的单位法线，才能满足下面有效质量简式的单位轴假设。

7.2 长度不是欧氏距离

Pogo 测量

$$\ell = a \cdot d, \qquad C_\ell = \ell - \ell_0.$$

ℓ_0 是 `restLength`，单位为米。 $C_\ell < 0$ 表示短于静止长度，通常需要恢复向上运动。 这不等于 $\|d\| - \ell_0$。

Solve 中的测长轴使用 Prepare 保存的 `frameB.q`，没有额外乘子步的 `deltaRotation`。 锚点力臂则仍按子步旋转更新。 所以要把它理解为本版具体采用的步内近似，而不是泛化成“所有方向每次都重算”。 公开的 `b2PogoJoint_GetLength` 从当前刚体变换和局部框架计算投影长度。 阅读 getter 与求解缓存时，应分别看清使用的是哪个阶段的变换。

7.3 从连续弹簧到内部恢复速度

设内部弹簧速度为 u，单位为米每秒。 以有效质量归一化后，弹簧阻尼方程为

$$\dot{C}_\ell = u, \qquad \dot{u} = -\Omega^2 C_\ell - 2\zeta\Omega u.$$

$\Omega = 2\pi f$，单位为每秒。 ζ 是无量纲阻尼比。

使用隐式一步：

$$u_{new} = u_{old} - h\Omega^2(C_\ell + h u_{new}) - 2h\zeta\Omega u_{new}.$$

整理得到

$$u_{new} = \frac{u_{old} - h\Omega^2 C_\ell}{1 + 2h\zeta\Omega + h^2\Omega^2}.$$

这就是 `b2SpringDamper` 的返回式。 分母无量纲，分子是米每秒。 压缩误差为负时，$-h\Omega^2 C_\ell$ 为正，推动内部速度朝伸长方向发展。

实际 pogo 每次从当前几何重新测量 C_ℓ，内部 u 则作为状态保留。 它不是独立于刚体运动、完全自行积分一根抽象弹簧长度。

7.4 第二阶段：沿法线追踪这个恢复速度

法线上的锚点相对速度为

$$v_n = n \cdot (v_B + \omega_B \times r_B - v_A - \omega_A \times r_A).$$

本阶段目标为

$$v_n = u_{new}.$$

固定当前法线时，Jacobian 为

$$J_n = [-n_x, -n_y, -r_A \times n,\ n_x, n_y, r_B \times n].$$

逆有效质量

$$K_n = \mu_A + \mu_B + \iota_A (r_A \times n)^2 + \iota_B (r_B \times n)^2, \qquad M_n = 1/K_n.$$

上式说明有效质量的来源。本版在 Prepare 用初始力臂计算并缓存 `linearMass`（`pogo_joint.c` 第 145–152 行）；Solve 更新当前力臂，但仍使用这份缓存质量，并不重新求它的倒数。

源码将偏置设为 $b = -u_{new}$，因此

$$\Delta\lambda_{trial} = -M_n(v_n + b) = -M_n(v_n - u_{new}).$$

最后按拉力与压力两个上限投影累计量：

$$-hF_{tension} \le \lambda \le hF_{compression}.$$

两上限参数本身是非负的力大小。 负冲量是沿 $-n$ 拉，正冲量是沿 $+n$ 推。 这与 distance 的 `lowerSpringForce` 直接保存有符号下界的 API 形式不同。

实际线冲量 $P = \Delta\lambda n$。 两刚体仍接受等大反向线冲量及相应偏心角冲量。

7.5 为什么不能把整个 pogo 写成一个长度 Jacobian

测量误差用 a，最终施力速度用 n。 当 $a \neq n$ 时，这不是对同一个 $a \cdot d$ 约束直接使用其完整梯度求解。 它是“弹簧恢复目标生成器 + 法向速度执行器”的组合。

源码注释提到：若要补偿方向夹角，可以考虑除以 $n \cdot a$。 但当法线近乎垂直于测长轴时，分母很小，恢复速度会被放大得很大。 当前实现宁可恢复慢一些，也不做这种危险放大。

这属于明确的控制与稳定性选择，不应包装成一个没有近似的各向同性弹簧。

8. Pogo 有偏置求解阶段的数值代入

以下三个例子各自独立，均重置旧内部弹簧速度和旧累计冲量为零，只考察一次 `useBias=true` 求解，不把其速度误当成包含 Relax 后的子步末速度。第 9 节再补完整阶段对照。

8.1 平地压缩，恢复力不够时只能先刹住下落

A 静态地面，B 质量 2 千克。 锚点在质心，$n = a = (0, 1)$。 当前投影长度 0.8 米，静止长度 1 米。 所以 $C_\ell = -0.2$ 米。

取 $h = 0.1$ 秒、$\Omega = 10$、$\zeta = 0.5$、旧内部速度零。

$$u_{new} = \frac{0 - 0.1 \times 100 \times (-0.2)}{1 + 2 \times 0.1 \times 0.5 \times 10 + 0.1^2 \times 100} = \frac{2}{3}\ \mathrm{m/s}.$$

B 当前竖直速度为 -1 米每秒。 因此 $v_n = -1$，$M_n = 2$。

$$\Delta\lambda_{trial} = -2(-1 - 2/3) = 10/3\ \mathrm{N\,s}.$$

若最大压缩推力为 20 牛顿，一子步最多提供 2 牛顿·秒。 旧冲量零，所以实际 $\lambda = 2$。

$$v'_{By} = -1 + 0.5 \times 2 = 0.$$

本轮只刹住下落，尚未达到内部希望的 $2/3$ 米每秒上升速度。 这是力预算不足，不是把恢复速度的正负号写反。 若推力预算至少允许 $10/3$ 冲量，才可在这一独立阶段达到目标。

8.2 斜面：同样恢复目标，竖直恢复会变慢

重新取初始 B 速度零，其余参数相同。 令测长轴仍为 $a = (0, 1)$，法线改为 $n = (0.6, 0.8)$。 它已经是单位向量，$n \cdot a = 0.8$。 内部恢复速度仍为 $u = 2/3$。

$$\lambda = 2 \times \frac{2}{3} = \frac{4}{3}\ \mathrm{N\,s}.$$

在足够的力预算下，B 速度变化为

$$\Delta v_B = 0.5 \times \frac{4}{3}(0.6, 0.8) = (0.4, 8/15).$$

检查法线目标：

$$n \cdot \Delta v_B = 0.6 \times 0.4 + 0.8 \times \frac{8}{15} = \frac{2}{3}.$$

但竖直速度只有 $8/15 \approx 0.533333$ 米每秒。 它等于 $0.8u$，不是 u。 这正好展示未除以 $n \cdot a$ 的实际效果。

8.3 拉力上限控制贴地强度

改成平地 $n = a = (0, 1)$，长度误差 $+0.2$ 米、当前刚体速度零。 内部目标变成 $-2/3$ 米每秒。 理想冲量为 $-4/3$ 牛顿·秒。

若最大拉力为 3 牛顿，$h = 0.1$，下界为 -0.3。 实际竖直速度变化为 $0.5(-0.3) = -0.15$ 米每秒。 若最大拉力为零，下界就是零，Pogo 不会主动把 B 向下拉。

因此拉力不是压力参数的另一个名字。 它决定“允许多强地贴向支撑表面”。

9. Pogo 的真实分支与状态保存

`b2SolvePogoJoint` 只有在 `useBias=true` 时更新内部弹簧速度，并令 $b = -u$。 当 `useBias=false` 时，本轮 $b = 0$，不再次推进内部弹簧速度。 它仍执行法线速度冲量步骤，并应用相同的拉压累计限幅。 不要把 relax 说成“这个关节完全不工作”。

以第 8.2 节为例，保持质心锚点、无其他外力或约束、足够力预算，补全子步的阶段表：

阶段	B 速度（m/s）	累计法线冲量（N•s）	累计位置变化（m）
开始	`(0,0)`	0	`(0,0)`
有偏置 Solve	`(0.4,8/15)`	`4/3`	`(0,0)`
以 0.1 s 积分位置	同上	`4/3`	`(0.04,4/75)`
Relax	`(0,0)`	0	保留 `(0.04,4/75)`

最后一行来自 $\Delta\lambda = -2 \times (2/3) = -4/3$ N·s：把本轮为修位置而施加的冲量撤回，速度回到零；之前已经积分的位置不倒退。实际有重力或其他接触时，应把它们也纳入阶段状态，不能套用这一孤立例子的最终零速度。

当 `hertz==0`，WarmStart 与 Solve 都清除 `impulse` 并返回。 这里零 Hz 表示这条 Pogo 不施加弹簧冲量，不表示刚性腿。

`b2PogoJointDef` 有初始 `impulse` 和 `velocity` 字段。 头文件说明 Pogo 常每帧重建，因此允许调用者把内部状态传递到下一条新关节。 若每帧重建时总把两者清零，响应会失去原有连续性。

`b2PogoJoint_GetImpulse` 与 `b2PogoJoint_GetVelocity` 可获取这些状态。 旧支撑完全改变时，是否继续沿用状态应由控制器评估。 不要把任意旧法线方向上的冲量无条件当成新支撑的正确初值。

这里描述的是当前实现及其约束方向。 不能因为源码注释出现“位置修正”，就声称当前函数直接执行了胶囊扫掠或完整的防穿透位置求解；实际函数更新的是内部速度与刚体速度。

10. Filter joint：没有冲量，但仍有逻辑用途

10.1 它不是零刚度弹簧

Filter 不要求两物体靠近，也不限制相对速度。 因此没有要写入求解器的 Jacobian 行，没有有效质量，也没有可施加的 λ。 把它当成“弹簧刚度设零”会混淆对象职责。

`b2PrepareJoint`、`b2WarmStartJoint`、`b2SolveJoint` 的 filter 分支都不调用力学求解。
`b2GetJointConstraintForce` 的 filter 分支返回零向量。 `b2GetJointConstraintTorque` 对 filter 返回零。

10.2 禁止碰撞来自 collideConnected

默认 `b2JointDef` 的 `collideConnected` 为 false。 Filter 默认因此能表达“这两个刚体不要相互产生普通碰撞接触”。

实际判断在 `b2ShouldBodiesCollide`： 只要连接这两个刚体的某条关节满足 `collideConnected==false`，就返回不碰撞。 判断依据是这项设置，不是看到 `filter` 类型就无条件永久禁碰。

所以更准确的表述是： Filter 是不施力的关节关系，通常利用默认关闭的相连体碰撞来做成对过滤。 若改成允许碰撞，它不会因为类型名而偷偷继续施加力学约束或必然禁止接触。 其他过滤条件仍可能阻止碰撞。

10.3 改开关为什么还要操作宽阶段

`b2Joint_SetCollideConnected`：

- 改为 false 时，销毁两刚体间已有接触。
- 改为 true 时，将一边的形状代理重新加入宽阶段移动缓冲，促使重新寻找可能接触。

这解释了为什么“只改一个布尔值”并不足以完整维护已存在的接触状态。 Filter 虽然不算力，仍参与对象关系与碰撞生命周期。

数值上，无论两体原先质量和速度如何，Filter 自身这一轮的 Δv、$\Delta \omega$ 都为零。 碰撞被禁用后，原本可能产生的接触冲量也不会由 Filter 补出来。

11. 与角色移动查询明确分开

名字相近，但下面这些函数属于另一条工作流：

接口	输入/输出核心	是否直接求成对刚体关节冲量
`b2World_CastMover`	扫描胶囊位移，返回运动比例 fraction	否
`b2World_CollideMover`	查询胶囊附近碰撞，通过回调返回平面结果	否
`b2SolvePlanes`	根据碰撞平面修正目标位移，返回 `delta`	否
`b2ClipVector`	按已生效的平面裁剪向量的入射分量	否

接口	输入/输出核心	是否直接求成对刚体关节冲量
`b2SolveMoverJoint`	对两刚体的线速度施加有反作用的有限冲量	是

`b2World_CastMover` 位于 `physics_world.c:2868-2924`。 它读取世界形状，进行胶囊扫描，返回可用比例。 返回一个数，不等于已经把刚体运动到那个位置。

`b2World_CollideMover` 位于 `physics_world.c:2970-3019`。 它用 AABB 与树查询寻找相关形状，并把碰撞平面交给调用者。

`b2SolvePlanes` 位于 `mover.c:7-53`。 它迭代修正几何位移，`push` 是位移修正量，不是牛顿·秒的刚体冲量。 不要因为也使用“累计并限幅”，就把两者当成同一个物理量。

`b2ClipVector` 位于 `mover.c:56-71`。 它裁掉指向已生效约束平面内部的分量。 这不是调用 `b2SolveMoverJoint` 的某个隐藏步骤。

应用可以把查询结果用于选择 Pogo 支撑或驱动目标。 但这种组合是应用层控制逻辑，不是同名函数自动互相调用的保证。

12. 从零搭建时的调试顺序

第一步只做 mover 的有限力相对速度控制。 检查一对动态体线动量是否守恒，角速度是否没有被该关节改变。

第二步实现 motor 的二维圆盘限幅。 用 $(3, 4)$ 目标验证合力大小没有因对角方向增大。

第三步把锚点移离质心，加入角惯量与二维质量矩阵。 检查线力引起的角速度是否满足 $\Delta\omega = \iota(r \times P)$。

第四步加入独立角速度驱动和位姿弹簧。 确认四通道各自的力上限，不误当成共享预算。

第五步单独测试 Pogo 的内部标量弹簧。 先用平地，后用斜面，最后考虑每帧重建与状态传递。

最后再把碰撞查询、支撑选择和应用层角色逻辑连接起来。 如果一开始把几何投影、马达驱动、弹簧恢复混写在一个函数里，很难分辨是控制目标错、法线错还是碰撞结果错。

13. 源码索引

内容	文件与准确入口
Motor 参数	`types.h`，`b2MotorJointDef:706-743`
Mover 参数	`b2MoverJointDef:752-765`
Pogo 参数与传递状态	`b2PogoJointDef:773-806`
Motor 三阶段	`motor_joint.c`，`179` / `251` / `287`
Motor 线速度圆盘限幅	`motor_joint.c:398-422`
Mover 三阶段	`mover_joint.c`，`50` / `94` / `120`
Mover 逐轴限幅	`mover_joint.c:137-158`

内容	文件与准确入口
Pogo 法线直接保存	`joint.c`，`b2CreatePogoJoint:521-561`，特别是 `549`
Pogo 三阶段	`pogo_joint.c`，`98` / `160` / `200`
Pogo 测长与内部速度	`pogo_joint.c:233-252`
Pogo 法线冲量与拉压上限	`pogo_joint.c:254-268`
隐式弹簧标量式	`math_functions.h`，`b2SpringDamper:832-837`
Filter 返回零力	`joint.c:1089-1090`
Filter Solve 空分支	`joint.c:1514-1515`
成对关节碰撞判断	`body.c`，`b2ShouldBodiesCollide:2076-2111`
碰撞开关维护	`joint.c:907-951`
角色扫描与平面查询	`physics_world.c`，`2868` / `2970`
几何平面求解与裁剪	`mover.c`，`7` / `56`

14. 习题与答案

题 1：Motor 与 mover 的线冲量可行域有什么不同

答案：Motor 是半径 hF_{max} 的圆盘；mover 是半宽分别为 hF_x, hF_y 的轴对齐矩形。 前者限制合力大小，后者限制每个世界分量。

题 2：Mover 的竖直最大力为零，目标竖直速度不为零，会怎样

答案：该关节无法提供竖直驱动冲量。 它不会为了满足速度目标而绕过竖直力预算，也不会自动改为碰撞平面解算。

题 3：Motor 的线速度目标是否自动随 A 的朝向旋转

答案：本版 Solve 直接从世界锚点相对速度减 `linearVelocity`，没有在该处进行局部到世界的旋转。 需要局部前进方向时，应用应提供转换后的目标向量。

题 4：Pogo 的 normal 是否一定等于测长轴

答案：不一定。 测长轴来自 B 关节框架局部 y 轴，normal 是创建时提供的方向。 两者不一致时，本版不会除以它们的点积来强行补偿恢复速度。

题 5：Pogo 的频率零是否构成刚性悬挂

答案：不是。WarmStart 与 Solve 都清除冲量并返回。 Weld 的零频率刚性替代语义不能套在 Pogo 上。

题 6：Filter 是否通过“反向接触冲量”抵消碰撞

答案：不是。它不产生约束冲量。 通常通过 `collideConnected=false` 使这对刚体不产生对应普通接触，而不是先碰撞再抵消。

题 7：`b2SolvePlanes` 的 push 能直接当作关节反作用冲量吗

答案：不能。它是几何位移修正量，没有使用成对刚体质量和惯量。 单位、目标、状态更新路径都不同。

题 8：两个 motor 线通道最大力分别 10 N 与 5 N，总力上限一定 10 N 吗

答案：不是。这是独立预算；同向可相加，反向可抵消。 要共享总预算，需要另行设计，而不能从当前参数名自行推断。

返回目录

16 • 世界步进与数据生命周期：跟踪一次 Step 到底改了什么

返回总目录

> 前置：第 03、09～12 章。第一次读本章只追踪单线程等价顺序，暂时忽略 SIMD 和原子变量。 目标：沿一个刚体、一个接触、一份冲量缓存，走完当前源码的完整大步。

1. 先建立朴素方案，不先研究调度器

一个最简单的物理世界只需要：

- 刚体数组：位置、旋转、速度、质量、外力。
- 形状数组：圆、盒子等几何。
- 接触数组：哪两个形状、接触点、法线、分离量、累计冲量。
- 可选关节数组。

单线程实现的主干可以用文字写成：

1. 找可能接触的形状对。
2. 更新接触几何和上步冲量缓存。
3. 把外力积分进速度。
4. 施加缓存冲量作为初值。
5. 求解接触和关节。
6. 用求解后的速度积分位置。
7. 保存下次要复用的数据。
8. 对外报告移动与接触变化。

这已经具备物理意义。当前 Box2D 的复杂性主要来自三件事：

- 一个大步被细分为多个子步。
- 数据按照活动状态、颜色和访问频度拆分。
- 几何计算、求解、事件汇总之间存在严格依赖。

接下来把上述朴素顺序逐项映射到源码，而不是把整个 `solver.c` 当作一个必须一次看懂的大函数。

2. 先分清三个数据层

2.1 持久身份：`b2Body`、`b2Contact`

这些对象告诉世界“这是谁、连接谁、现在存在哪里”。

例如一个接触包含两个形状 ID、两端刚体连接边、所属集合、颜色和局部数组位置。它不是接触方程本身。

一个刚体的公开 ID 不因睡眠、唤醒、数组搬迁而改变。数组的局部下标却可能改变。

2.2 持久模拟数据：`b2BodySim`、`b2ContactSim`

- `b2BodySim` 保存质心、变换、力、逆质量和 CCD 所需几何尺度。
- `b2ContactSim` 保存 manifold、材料混合结果、缓存等。

接触从“未 touching”变成“touching”时，其 Sim 数据可能从求解集数组搬进约束图颜色数组。**这只是存放地点变了，不是创建了另一次物理碰撞。**

2.3 热状态与临时约束

`b2BodyState` 是子步中反复改写的数据：

字段	含义	字节数
`linearVelocity`	质心线速度	8
`angularVelocity`	角速度	4
`flags`	动态、锁轴等标记	4
`deltaPosition`	相对大步开始的位移	8
`deltaRotation`	相对大步开始的旋转	8
合计	每个热状态	32

来源：`src/body.h` 第 153–168 行。

Prepare 还会建立临时的接触约束，其中包括有效质量、基础分离量和求解中的累计冲量。它们不是新的世界对象；大步结束后临时内存会释放，必要结果先写回 manifold。

3. `b2World_Step` 的实际顺序

入口：`src/physics_world.c`，`b2World_Step` 第 894–1054 行。

顺序	源码区间	做什么	此时尚未做什么
1	896–929	参数断言、清理若干事件数组、锁定世界、重置任务信息	未推进时间
2	949–954	`b2UpdateBroadPhasePairs`，创建新候选接触	未计算所有精细接触点
3	956–984	建立步进上下文、时长与软度	未施加冲量
4	986–991	`b2Collide` 更新接触及状态迁移	未积分本次大步位置
5	993–999	正时间步时调用 `b2Solve`	传感器最终重叠尚未更新
6	1001–1008	等待剩余树任务	保证后续查询使用可用的树

顺序	源码区间	做什么	此时尚未做什么
7	1010–1015	`b2OverlapSensors`	不因此给物体施加接触冲量
8	1019–1051	临时内存检查、事件缓冲交换、可选记录 hash、解锁	用户之后可读取本步结果

3.1 锁定世界不是什么“大锁保护一切”

`world->locked` 主要阻止用户在内部模拟过程中执行不允许的世界操作。它不是“任何用户回调都可以随便访问世界”的线程安全承诺。

例如 pre-solve 回调可能在并行窄阶段中调用，其用户逻辑必须满足相应线程安全要求。不要在这里创建、销毁刚体，也不要保存内部临时指针供以后使用。

3.2 `timeStep == 0` 不是完全不做事

上下文时间及倒数被置零，跳过 `b2Solve`，但仍执行配对、接触更新、传感器处理和事件缓冲操作。

所以它不是一个纯读取操作，也不等于“任何状态都不变”。由于没有执行 Finalize，本次不会按正步进路径清除外力。

3.3 不要把绘制帧数当成这里的步数

代码只知道调用者给出的物理时间和子步数。固定物理步长累加器、渲染插值属于应用层。第 04 章先讲朴素固定步长方案。

4. 配对阶段：先建记录，不立即产生冲量

入口：`src/broad_phase.c` 的 `b2UpdateBroadPhasePairs`；接触创建见 `src/contact.c` 的 `b2CreateContact` 第 220–369 行。

4.1 一对形状经历的步骤

1. 移动代理查询包围盒树，得到可能相遇的形状。
2. 过滤同刚体、掩码、关节禁止碰撞等不允许的对。
3. 用形状对的 key 排除已有接触。
4. 如果该类型组合有 manifold 函数，创建 Contact。
5. 在 A、B 两个刚体的接触边链中登记它。
6. 初始化缓存和空 manifold，等待窄阶段。

没有对应几何求解函数的组合不会凭空产生接触，例如代码明确指出线段与线段没有该接触实现。

4.2 A/B 次序可能调整

`b2CreateContact` 第 234–238 行会根据注册表的 primary 顺序交换两个形状。

因此调试法线时，先看接触实际保存的 `shapeIdA/B`。不能仅凭你先创建哪个刚体，就认定它永远是 A。

4.3 未 touching 接触放在哪里

- 至少一端醒着：初始进入 awake 集的非 touching 接触数组。
- 两端都不醒着：初始非 touching 接触可以放在 disabled 集。

这里 disabled 集不仅仅是“用户主动禁用刚体的容器”。判断一个接触是否参与求解，要看完整状态，不要只按集合名称猜测。

源码依据：`b2CreateContact` 第 247–258、335–354 行。

5. 窄阶段：计算几何和提交状态变化分开

入口：`b2Collide` 第 687–892 行。

5.1 为什么先收集一张接触指针表

要更新的接触分散在各颜色数组以及 awake 非 touching 数组。当前实现先把它们的地址收集为临时数组，方便统一并行遍历。

此时数组结构保持稳定。每个任务处理自己负责的接触几何，不在任务中任意搬迁整个接触数组。

5.2 每个接触需要计算什么

大体包括：

- 检查 AABB 是否已分离。
- 检查能否采用接触几何回收路径。
- 否则调用具体形状对的 manifold 算法。
- 将局部几何变换为求解器所需锚点和法线。
- 更新材料混合、可选 pre-solve 结果。
- 按接触特征 ID 匹配旧点，复制可用冲量。
- 标记开始、停止 touching 或完全分离。

几何回收不是“重复做一遍 manifold 再热启动”，而是可能跳过重新构造几何，见第 17 章。

5.3 为什么状态变化用 ID 位图记录

因为稍后的数组搬迁会让旧指针失效。每个 worker 记录“哪个 contactId 发生变化”，而不是“哪个地址发生变化”。

任务结束后，主流程对位图按位或，再按确定顺序遍历 contactId。由当前 Contact 的集合、颜色、局部索引重新找到 Sim。

这既避免并行搬迁冲突，也固定了后续修改世界结构的顺序。

5.4 三条状态分支

分支	物理/几何含义	数据操作
`b2_simDisjoint`	包围盒已不再满足保留条件	销毁 Contact
`b2_simStartedTouching`	现在存在可用 manifold 点	连接岛、必要时唤醒、搬进约束图
`b2_simStoppedTouching`	本次没有 manifold 点	断开岛连接、从图搬回非 touching 数组

相应代码在 `b2Collide` 第 815–879 行。

这里 touching 与 `pointCount > 0` 对应，**不必等到分离量变负**。推测接触也可能进入求解图。因此 begin 事件不应被无条件解释为“这一瞬间发生了剧烈撞击”。

5.5 为什么先 Link，再重新取指针，再 AddToGraph

开始 touching 分支的顺序尤其值得阅读：

1. `b2LinkContact` 可能唤醒另一端的睡眠集合。
2. 唤醒可能增长 awake 数组，使原先的 `contactSim` 指针失效。
3. 因此重新从 `awakeSet->contactSims` 取地址。
4. 先复制进约束图。
5. 再从非 touching 数组移除旧 Sim。

这是为了保证“源数据仍然有效”和“索引已经更新”，与冲量公式无关，但缺一步就可能出现内存错误。

6. 时长和软度上下文：只算一次，后续共享

设总步长为 Δt，子步数为 N：

$$h = \Delta t / N, \qquad h^{-1} = N / \Delta t.$$

作用： 给每次子步积分和误差修正使用统一的时间尺度。

例：$\Delta t = 1/60$ s，$N = 4$，得到 $h = 1/240$ s，`inv_h` 为 240 s^{-1}。

本版接触频率实际取：

$$f_c = \min(f_{world}, 0.125/h).$$

变量： f_{world} 是世界设置的接触频率，f_c 是本次真正使用的频率，单位均为 Hz。

作用： 步长较大时，限制求解器试图表达的恢复频率。

代入默认 $f_{world} = 30$ Hz：

子步数	子步长	频率上限	实际 f_c
1	1/60 s	7.5 Hz	7.5 Hz

子步数	子步长	频率上限	实际 f_c
2	1/120 s	15 Hz	15 Hz
4	1/240 s	30 Hz	30 Hz
8	1/480 s	60 Hz	30 Hz

静态接触的软度用两倍该频率构造，并不是再次拿两倍频率与原上限比较。数学推导见第 11 章。

源码：`b2World_Step` 第 977–980 行。

7. 求解器：Prepare 一次，五个阶段重复，Store 一次

入口：`src/solver.c`，`b2Solve` 第 1255–2057 行，核心阶段执行在 `b2SolverTask` 第 1008–1234 行。

7.1 先准备，不先改速度

Prepare 分别处理关节和接触：

- 找醒着的刚体状态索引。
- 拷贝逆质量等反复使用的数据。
- 计算有效质量和基础分离量。
- 根据热启动开关初始化累计冲量。
- 准备静态端 dummy state 所需约定。

这部分每个大步一次。没有必要在每一次约束增量中都重新查对象池、重新计算所有乘积。

7.2 一个子步的输入输出表

阶段	读入	改写	为什么放这里
IntegrateVelocities	当前速度、力、重力、阻尼、h	线速度、角速度	先得到外力驱动的速度
WarmStart	缓存累计冲量、质量、杆臂	两端动态刚体速度	让求解初值接近持续接触的解
Solve / Push	当前速度、分离估计、软度	累计冲量与速度	处理不穿透与位置修正
IntegratePositions	已求解速度、h	大步内累计位移与旋转	把修正速度转化为位置变化
Relax	更新后分离、当前速度、累计冲量	法向、恢复、滚阻、摩擦响应	去掉重叠修正 bias 后再校正速度

注意：Relax 不是只有摩擦，它先重新解法向。漏掉这一项会导致完整算例错误，见第 12 章。

7.3 子步内哪些几何更新，哪些不更新

- 不在每个子步重新运行完整宽阶段和所有 manifold 算法。
- 利用位置增量、旋转增量和初始锚点估计当前分离。

- 冲量计算所用的接触杆臂和有效质量采用准备阶段的数据。
- 更新分离时的旋转锚点，与接触速度 Jacobian 使用固定杆臂，不能混为一个变量策略。

这是“成本、稳定性、几何变化误差”之间的选择，而非精确连续刚体几何的恒等重写。

7.4 Store 的生命周期

子步全部结束后，把接触求解结果写回持久 manifold。下一大步的接触更新可以据此进行 ID 匹配与恢复目标计算。

临时约束缓冲随后被释放。**保存了缓存不等于保存了指向临时缓冲的指针。**

8. 无接触算例：走完一个大步，而不只算最后一条公式

为了只检查时序，设：

- 一个醒着的动态刚体，质量 2 kg。
- 初始世界质心 $c_0 = (5, 10)$ m。
- 初始速度 $v_0 = (1, 0)$ m/s，角速度零。
- 外力 $F = (4, 0)$ N，重力 $g = (0, -10)$ m/s²。
- 无阻尼、无接触、无关节，不触发速度上限。
- 总时间 $\Delta t = 1/60$ s，子步数 2。

8.1 先把力转成加速度

$$a = F/m + g = (4/2, 0) + (0, -10) = (2, -10)\ \mathrm{m/s^2}.$$

作用： 用相同外力和重力更新每个子步的速度。

8.2 每子步更新

无阻尼情况下：

$$v_{k+1} = v_k + ha, \qquad \Delta c_{k+1} = \Delta c_k + h v_{k+1}.$$

变量： Δc 是相对大步开始的累计位移，初始为零；这里使用新速度积分位置。

取 $h = 1/120$ s，则每次速度变化为 $(1/60, -1/12)$ m/s。

时刻	速度 x	速度 y	累计位移 x	累计位移 y
大步开始	1	0	0	0
子步 1 积分速度后	1.016666667	−0.083333333	0	0
子步 1 积分位置后	同上	同上	0.008472222	−0.000694444
子步 2 积分速度后	1.033333333	−0.166666667	0.008472222	−0.000694444
子步 2 积分位置后	同上	同上	0.017083333	−0.002083333

无约束时 WarmStart、Solve、Relax 不增加额外冲量，因此表格没有隐藏省略的接触修正。

8.3 Finalize 才把增量写回持久位置

$$c_{end} = c_0 + \Delta c = (5.017083333, 9.997916667)\ \mathrm{m}.$$

若局部质心为 $(0.2, 0)$ m，且旋转仍为单位旋转，则世界原点为：

$$p_{end} = c_{end} - (0.2, 0) = (4.817083333, 9.997916667)\ \mathrm{m}.$$

作用： 将质心运动转换成用户/形状使用的刚体原点变换。不能把质心位置直接当成 `transform.p`。

8.4 外力为什么不能在第一个子步后清掉

如果第一个子步后清力，第二个子步将失去水平 2 m/s² 的加速度，只推进了一半的施力时间。

本版在 Finalize 清除 `force`、`torque`，使本大步内各子步都能读取同一份外力累计。下一大步若还需持续施力，应由应用按其输入策略继续提供。

源码：`b2FinalizeBodiesTask` 第 631–633 行。

9. Finalize：不是只做一次位置加法

`b2FinalizeBodiesTask` 第 560–750 行承担多个收尾工作。

9.1 先写位姿，再计算状态指标

1. 持久质心加累计位移。
2. 大步旋转与累计旋转相乘并归一化。
3. 计算线速度与转动影响下的运动尺度。
4. 用真实速度和位置修正量共同评估睡眠。
5. 清空增量，反推出刚体原点。
6. 填移动事件。
7. 清外力和力矩。
8. 判断 fast、调度普通 CCD 或登记 bullet。
9. 更新形状包围盒、标记需要更新的树代理。

这里的“写回位置”可能随后被 CCD 修正。因此只在 Finalize 最前面打断点，不一定看到最终对外位置。

9.2 为什么睡眠还看位置变化

Relax 可能把修正速度去掉，但物体这一步仍然被位置修正移动了不少。如果只看最终速度，可能把仍在修正的堆叠错误地判为稳定。

具体速度指标推导和数字例在第 18 章。

9.3 普通 fast 与 bullet 分开

- 普通高速动态刚体在 Finalize 中调用连续求解，查询静态树。
- 高速 bullet 先登记，等相关树状态更新后统一处理，还会查询运动学和动态树。
- 目标为 bullet 的候选被当前连续路径跳过。

这些限制来自本版执行代码，不是说“所有运动物体两两都有完整 CCD”。详见第 08 章。

10. `b2Solve` 后半段为何按这个次序收尾

顺序	源码区间	原因
1	1583–1610	等岛拆分完成后 Finalize，避免同时改岛结构
2	1624–1795	收集关节事件和 hit 事件，使用已保存的冲量结果
3	1798–1890	等树任务、汇总需要扩大的 AABB，建立确定顺序的移动代理记录
4	1893–1960	处理高速 bullet 并扩大相关代理
5	1967–1998	汇总连续路径发现的 sensor hits
6	2000–2055	最后把符合条件的岛移到睡眠集合

为什么不能先睡眠再处理树更新

睡眠会把 awake 稠密数组中的对象搬走，并可能用末尾元素补洞。前面按 bodySimIndex 记录的 enlarged 位图就可能指向另一个刚体。

因此代码明确把岛睡眠放到最后。这是一条数据依赖，不是可随意调换的性能偏好。

11. 接触跨大步：四种数据不要混淆

数据	表示什么	产生与使用
`normalImpulse`	当前接触点保留的累计法向冲量	Store 写回，下一次作为热启动缓存
`tangentImpulse`	累计切向冲量	同上，用于持续摩擦
`totalNormalImpulse`	本大步各子步实际施加的法向冲量的代数累计	包含 WarmStart 与 Solve/Relax 的有符号增量，不通常等于最终 `normalImpulse`
`normalVelocity`	本步窄阶段、积分速度和热启动之前采样的法向相对速度	直接写入持久 manifold，供下一步恢复和当前 hit 使用；不是求解结束速度

`totalNormalImpulse` 在 Prepare 清零，在 WarmStart 加缓存冲量，在 Solve/Relax 加投影后的增量，包括负增量。它不是仅正增量之和，也不是历史最大冲量。对应 `contact_solver.c` 第 114、190、309–315 行。若要换算本大步的等效平均法向力，应除以大步时长，而不是子步时长。

`normalVelocity` 的正常更新位于 `b2CollideTask` 第 621–638 行，几何回收路径更新位于第 560–576 行。Store 写回冲量，不写回这个速度。

`b2UpdateContact` 第 603–650 行先匹配特征 ID；匹配成功时复制缓存。若旧点的 `totalNormalImpulse > 0`，且旧的接近速度超过阈值，则生成恢复目标；“中途曾经施加正增量”本身不足以代替这个最终判据。

一个只检查字段生命周期的例子

设旧点记录：

- 旧法向速度 −4 m/s。
- 旧 `totalNormalImpulse` 大于零。
- 新旧特征 ID 相同。
- 混合恢复系数 0.5，阈值 1 m/s。

本步接触更新得到恢复速度：

$$u_r = -e v_n^{old} = -0.5 \times (-4) = 2\ \mathrm{m/s}.$$

作用： 存一个正的目标分离速度。Prepare 只把它复制进临时约束，Relax 再把它转为法向速度目标。

若写为速度方程中的 bias，则为 −2 m/s，因为目标方程是“当前法向速度减去 2 等于零”。不能把正的 `restitutionVelocity` 直接当成正的 `velocityBias`。

这个例子只说明字段和符号，不能保证任意复杂接触最终恰好弹回 2 m/s；还受投影、其他点和恢复清除分支影响。

12. 事件不是一个统一的“碰撞回调”

12.1 三种接触信号分别意味着什么

- begin：从没有可用接触点变成有点，可能是推测接触。
- end：停止该接触关系或接触被销毁等。
- hit：接近速度超过阈值，并且该点整步 `totalNormalImpulse > 0`；不是只要中途出现过一个正增量就一定触发。

本版 hit 汇总在 `b2Solve` 第 1734–1746 行检查接近速度和 `totalNormalImpulse`。仅凭 AABB 重叠，不会自动得到 hit。

两个点都满足条件时，当前代码选择更大接近速度的点作为该接触的事件代表，而不是对每个点无条件发一个事件。

12.2 传感器为什么放到位姿求解之后

传感器不需要施加阻止运动的冲量，主要回答“现在有哪些 visitor 在里面”。放在求解和 CCD 收尾之后，可以按更新后的世界状态计算重叠，并融合相应连续路径记录。

入口：`src/sensor.c`，`b2OverlapSensors` 第 239–369 行。

12.3 新旧集合怎样生成 begin/end

例：旧 visitor 集合为 `{2,5}`，新集合为 `{5,9}`，且相同 ID 的 generation 没变化。

- 旧有新无：2，生成 end。
- 新有旧无：9，生成 begin。
- 新旧都有：5，保持持续重叠，不重复生成 begin。

实际实现比较已整理的 visitor 列表，同时比较 generation。若一个槽位销毁后被新形状复用，不能把相同 shapeId 当成同一个 visitor。

12.4 双缓冲的“previous”不等于上一物理帧

假设当前写入 end 事件的数组索引是 0：

1. 本次步进把结束事件写入数组 0。
2. Step 结尾把写入索引切到 1，并清数组 1。
3. 读取接口使用 `1 - world->endEventArrayIndex`，因此读到数组 0，也就是**刚完成这次步进的事件。**

不要把源码里的“previous buffer”译成“用户总是读到上一帧事件”。它指交换后的另一侧缓冲。

读取函数位于 `physics_world.c`：`b2World_GetSensorEvents` 第 1502–1523 行，`b2World_GetContactEvents` 第 1526–1551 行。

数组内容由世界管理。应按 API 生命周期及时消费，需要跨步保留时复制业务所需数据，并检查对象句柄是否仍有效。

13. 到这里才看优化：同一物理工作为何出现 Stage、Block、Color

13.1 Stage 是阶段，不是一条物理约束

Prepare、积分速度、WarmStart 等各是一种工作阶段。某些阶段按颜色拆成多个 stage 条目。

本版正常求解每个子步有一次 Solve 和一次 Relax。若活跃颜色数为 C，阶段表条目数为：

$$S = 5 + 3C.$$

推导： 两个 Prepare、一个积分速度、一个积分位置、一个 Store，共 5；WarmStart、Solve、Relax 各有 C 个颜色条目，共 $3C$。

代入： 3 个活跃颜色时，阶段表有 14 个条目。

这张表被子步循环重用，不为每个子步重新分配一份。

若有 N 个子步，阶段表执行次数为：

$$E = 3 + N(2 + 3C).$$

解释： 两个 Prepare 加 Store 只执行一次；每子步两个积分阶段，加三类颜色阶段。

代入： $C = 3, N = 4$ 时是 $3 + 4 \times 11 = 47$ 次。这里统计阶段表执行，不包含单独调用的 overflow 工作，也不是物理约束数量。

代码依据：`b2Solve` 第 1428–1445 行，`b2SolverTask` 第 1058–1169 行。

13.2 Block 是分工单位

一个阶段可以包含多个工作块，由 worker 领取。两个工作块能否同时执行，取决于它们有没有写同一份动态刚体状态。

颜色解决这种依赖；不是因为“多线程写同一个 float 也许没问题”。细节见第 19 章。

13.3 为什么 Prepare/Store 可以跨颜色铺平

它们主要处理各自约束数据，而不是像 Solve 那样同时修改两端动态刚体速度，所以可采用统一的平铺区间，减少小颜色带来的调度碎片。

源码第 1350–1354 行明确说明该设计。不能据此推导“所有阶段都可以去掉颜色”。

13.4 源码注释也可能落后于实现

`b2SolverTask` 第 1040–1050 行的阶段序列注释还提到 `b2_stageRestitution`。但 `src/solver.h` 第 75–85 行枚举和实际执行路径没有独立恢复阶段。

本书采用当前执行代码：恢复包含在 Relax 接触求解中。

14. 调试时应在哪些点观察什么

断点	看什么	不应期待什么
`b2CreateContact` 结束	对象关系、空 manifold、缓存初值	不期待立刻有冲量
`b2UpdateContact` 结束	法线、锚点、分离、ID、恢复来源	不期待本步已改速度
Prepare 结束	`normalMass`、`baseSeparation`、缓存累计值	不期待热启动已经施加
WarmStart 结束	刚体速度是否收到缓存响应	不期待所有约束已满足
Solve 结束	带修正目标的速度和累计冲量	不期待摩擦一定已更新
积分位置结束	`deltaPosition`、`deltaRotation`	不期待持久 `transform` 已写回
Relax 结束	法向、恢复、摩擦之后的残差	不假设一遍解完所有耦合
Store 结束	manifold 的冲量和事件所需字段	不保留临时约束指针
Step 返回	最终位姿、事件、睡眠与 CCD 标志	不继续使用上一阶段内部地址

15. 练习与答案

题 1：接触刚变成 touching，为什么还要重新读取 `contactSim`？

连接接触可能唤醒睡眠物体，使 awake 数组扩容。旧地址可能已失效，必须按当前索引重新获取。

题 2：本步没有移动的刚体一定没有参与求解吗？

不一定。它可能一直醒着，通过接触冲量平衡重力，最终位移很小。是否参与由求解集合和约束状态决定，不由画面是否移动决定。

题 3：缓存冲量、恢复速度都在 Prepare 里重新从零算吗？

不是。冲量来自持久 manifold 的缓存；恢复目标在接触更新或回收路径中利用旧步数据生成，Prepare 主要搬运并计算本步求解准备量。

题 4：为何不能在睡眠迁移以后继续使用旧 enlarged 位图？

它按原 awake Sim 下标编码；RemoveSwap 搬迁之后，同一位可能对应另一刚体。

题 5：碰撞 begin 一定意味着声音系统应该播放撞击音吗？

不一定。begin 可以来自推测接触。需要撞击强度时，应考虑 hit 事件及其阈值和业务条件。

继续：17 · 宽阶段动态树与几何缓存 · 返回总目录

17 宽阶段、动态树与几何缓存

> 目标：从不会漏对的双重循环出发，理解 Box2D 为什么维护三棵树、为什么允许包围盒变胖，以及三种“缓存”究竟省掉了什么。
>
> 本章对应 Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b`。 下列次数和几何数字是理论推导或手算，不是本次运行得到的性能测量。

17.1 先保留一份朴素正确的答案

暂时不考虑动态树。 对所有不同形状对执行过滤和 AABB 相交测试，通过后进入窄阶段。

教学伪代码，非源码：

```
for i in [0, shapeCount):
    for j in (i, shapeCount):
        if allowedPair(i, j) and overlaps(aabb[i], aabb[j]):
            narrowPhase(i, j)
```

第二层从 `i + 1` 开始，同时避免自己碰自己和重复检查。 它很慢，却非常适合充当后续优化的正确性参考。

令 n 为形状数量，N 为候选对检查次数，两者都是无量纲计数：

$$N = \frac{n(n-1)}{2}.$$

形状数 n	每帧候选对数 N
100	4,950
1,000	499,500
10,000	49,995,000

若每秒推进 60 帧，最后一行意味着每秒约 29.997 亿次候选检查。 这还没有计算多边形距离、接触点和约束求解。

第一项优化目标因此不是“把一次碰撞公式写得更短”，而是减少不可能碰撞的形状对。 但必须牢记输出规模的下界：如果所有包围盒真的互相重叠，候选结果本身仍可能达到 $O(n^2)$。 动态树不能把必须输出的五千万个结果变成十万个。

17.2 过滤规则首先是语义

在优化过滤之前，先写对过滤。 `b2ShouldShapesCollide` 的规则不是把所有条件做一次逻辑与。 它先处理组号，再处理双向掩码。

```
static inline bool b2ShouldShapesCollide( b2Filter filterA, b2Filter filterB )
{
	if ( filterA.groupIndex == filterB.groupIndex && filterA.groupIndex != 0 )
	{
		return filterA.groupIndex > 0;
	}

	return ( filterA.maskBits & filterB.categoryBits ) != 0 && ( filterA.categoryBits &
filterB.maskBits ) != 0;
}
```

按下面的顺序判断：

1. 两个 `groupIndex` 相等且非零：正值允许，负值禁止。
2. 其余情况：A 的掩码接受 B 的类别，而且 B 的掩码也接受 A 的类别。

A	B	这个过滤函数的结果
`category=1, mask=2, group=0`	`category=2, mask=1, group=0`	允许
`category=1, mask=2, group=0`	`category=2, mask=4, group=0`	禁止
`mask=0, group=3`	`mask=0, group=3`	允许
`mask=全部, group=-3`	`mask=全部, group=-3`	禁止
`group=3`	`group=4`	回到双向掩码判断

第三行容易被“提前按 mask 剪枝”的优化误杀。 因此当前宽阶段查询树时使用 `B2_DEFAULT_MASK_BITS`，把这项形状过滤留给回调。 不能直接拿查询形状的 `maskBits` 替换它。

正组号也不是绝对的“无条件碰撞通行证”。 它只覆盖这个函数中的掩码规则。 同一刚体上的形状、传感器处理路径、没有支持算法的形状组合、关节禁止碰撞和自定义过滤，仍有各自的判断。

`b2PairQueryCallback` 的主要顺序是：

1. 排除自身代理与重复的移动代理对。
2. 查询 `pairSet`，已有接触对不重复创建。
3. 排除同一刚体和传感器。
4. 执行形状过滤和形状类型支持检查。
5. 检查刚体之间的关节过滤。
6. 必要时执行用户自定义过滤。
7. 把新候选写入该查询自己的结果链表。

实验：用表中的五组输入测试过滤；尤其检查“同正组号、双方 mask 为零”。 优化前后应得到完全相同的允许集合，而不是仅比较接触总数。

17.3 为什么是三棵树

朴素方案可以把所有形状放在同一棵树里，查询后再根据刚体类型过滤。 这样正确，但会访问不少永远不产生实体接触的组合。

当前宽阶段按 `static`、`kinematic`、`dynamic` 分开存放代理。 查询策略由被移动代理的类型决定：

被查询的移动代理	查询哪些树
动态	运动学、静态、动态
运动学	动态
静态	动态

所以“移动动态物体查询另外两棵树”是不完整的。 它还要查询动态树，因为动态物体之间也会碰撞。

“移动集合”也不等于“速度不为零的动态刚体”。 静态形状修改、创建策略等操作，也可能让静态代理进入移动集合。

重复对有两种不同来源：

- A 和 B 都移动，分别查询时可能都发现对方。
- A 和 B 已经有持久接触，本帧查询又发现一次。

前一种使用移动位图与代理键规则去重。 后一种使用 `pairSet` 检查已有形状对。 不要把这两个用途合并成一句“用哈希缓存所有碰撞”。

代价：三棵树增加维护结构，但让类型规则更早减少无效查询。 是否值得，取决于场景中三类代理的数量和移动比例。

实验：分别创建大量静态地形、运动学平台和动态箱子。 固定同一移动序列，记录各类查询访问的树及候选集合，与朴素类型过滤后的全配对结果比较。 这与 CCD 的目标树选择不是同一套流程，不要混用两者规则。

17.4 胖 AABB：允许小移动暂时不维护树

如果每移动 1 mm 就删除并重新插入代理，缓慢滑动的物体也会持续支付树维护成本。 胖 AABB 的思想是预留一点空间：只要新包围盒仍在旧预留范围内，就不调整树代理。

这里要区分两层范围：

- `shape->aabb` 已包含 speculative distance，不一定是紧贴几何的盒。
- `shape->fatAABB` 在它外面再加用于减少树维护的 margin。

在默认米制单位下：

量	默认值
`B2_LINEAR_SLOP`	0.005 m

量	默认值
`B2_SPECULATIVE_DISTANCE`	0.020 m
`B2_MAX_AABB_MARGIN`	0.050 m
`B2_AABB_MARGIN_FRACTION`	0.125

`b2ComputeShapeMargin` 根据形状尺度计算 margin，再限制最大值。 若圆半径为 r，单位 m，则圆的 margin 为：

$$m = \min(0.05, 0.125r).$$

例如 $r = 0.1\ \mathrm{m}$，则 $m = 0.0125\ \mathrm{m}$，不是固定 5 cm。 其他形状使用代码中对应的尺度计算，不能统一把它替换为刚体半径。

再看一个 margin 已达到 0.05 m 的盒子。 假设几何范围为 $[0,1] \times [0,1]\ \mathrm{m}$：

1. 加 speculative distance 后，范围为 $[-0.02, 1.02]^2\ \mathrm{m}$。
2. 再加树维护 margin 后，范围为 $[-0.07, 1.07]^2\ \mathrm{m}$。
3. 向右移动 0.04 m，新带 speculative 范围的右边界为 1.06 m，仍被包含。
4. 向右累计移动 0.051 m，右边界变成 1.071 m，越过原 fat AABB。

当前 `b2FinalizeBodiesTask` 对非 fast 路径重新计算 `aabb`。 只有 `b2AABB_Contains(shape->fatAABB, aabb)` 为假，才生成新 fat AABB 并设置 enlarged 标记。 静态代理初始化还有自己的额外 margin 分支，不能把上述动态例子当成所有类型统一规则。

代价：盒子更胖，会产生更多“粗测重叠、细测并未接触”的候选。 margin 太小使树维护频繁；太大则让候选和持久接触数量增加。 这是一项时间成本交换，不是越大越快。

实验：让盒子每帧右移 1 mm，记录 containment 失败次数。 再插入一次 51 mm 跳跃，检查代理确实更新。 同时使用朴素几何相交检查确认没有遗漏真实相交对。

17.5 动态树如何排除整片空间

动态树的叶节点对应代理。 内部节点的 AABB 包住两个子节点，类别位则是两个子节点类别位的按位或。

如果查询盒与某个内部节点都不相交，其全部后代都可跳过。 与逐叶测试相比，一次判断可能排除几百个代理。

`b2DynamicTree_Query` 使用显式栈遍历：

1. 根节点入栈。
2. 弹出一个节点，增加 `nodeVisits`。
3. AABB 和类别位都通过才继续。
4. 遇到叶节点调用回调，增加 `leafVisits`。
5. 内部节点把两个孩子入栈。

当前先压 `child1`，再压 `child2`，所以 LIFO 弹出顺序是 `child2` 先。 这个顺序会影响候选链表顺序，不能认为只要树集合相同，任意遍历次序都无关紧要。

理想平衡的 1,024 叶树深度约为 10。 但“深度 10”不代表每次查询只访问 10 个节点。 查询可能同时进入多个分支，实际成本应观察 `nodeVisits` 和 `leafVisits`。

实验：对固定查询集合，同时记录树高和访问数。 比较均匀分布与全部挤在一处的场景，观察树高相近时查询工作量也可能相差很大。

17.6 插入位置为什么比较周长

最简单的插入方案是任意选一个叶子作兄弟。 它可以保持树结构合法，却可能迅速制造很大的内部包围盒。

当前 `b2FindBestSibling` 用周长代价选择兄弟。 源码中有些变量叫 `area`，实际调用的是 `b2Perimeter`。 在二维这里比较的是长度，单位 m，而不是面积 m²。

令 $P(X)$ 为盒 X 的周长：

$$P(X) = 2(\mathrm{width}(X) + \mathrm{height}(X)).$$

令 D 为待插入叶盒、N 为当前节点盒、I 为祖先累计增长代价，三种代价的单位均为 m：

$$C_{\mathrm{current}} = P(N \cup D) + I,$$

$$I' = I + P(N \cup D) - P(N).$$

若孩子 C 是叶子，候选代价为 $P(C \cup D) + I'$。 若孩子是内部节点，代码使用下界：

$$L_C = I' + P(C \cup D) + \min(P(D) - P(C), 0).$$

这个下界用于判断继续下降是否还有希望改进结果。 算法沿一条贪心路径下降，不是遍历所有叶子求全局最优。 内部孩子下界相同时，还会使用中心距离平方打破平局。

手算一次插入

设两个旧叶子为：

- $A = [0, 2] \times [0, 2]\ \mathrm{m}$。
- $B = [8, 10] \times [0, 2]\ \mathrm{m}$。
- 根盒为 $[0, 10] \times [0, 2]\ \mathrm{m}$，周长 24 m。

插入 $D = [2.5, 3.5] \times [0, 1]\ \mathrm{m}$：

候选	并集周长	继承代价	总代价
根	24	0	24
A	11	0	11
B	19	0	19

因此选择 A 作兄弟。 新建父节点包住 A 和 D，随后向上更新盒、类别位和高度。

如果把 D 改成 $[11, 12] \times [0, 1]\ \mathrm{m}$，根并集周长变成 28 m。 孩子继承的增长代价为 $28 - 24 = 4\ \mathrm{m}$。 此时 A 候选为 $28 + 4 = 32\ \mathrm{m}$，B 候选为 $12 + 4 = 16\ \mathrm{m}$，所以选择 B。

`b2InsertLeaf` 还必须处理数组扩容后节点指针失效的问题。 稳定节点索引不意味着先前保存的 `nodes` 指针仍然有效。

17.7 旋转与重建分别修复什么

插入启发式不能保证树永远优良。 物体移动之后，原本相近的一组叶子可能已经相距很远。

局部旋转尝试交换孩子与孙子，让内部盒的周长更小。 这不是只比较高度差的 AVL 旋转。

考虑 `b2RotateNodes` 中 B 是叶、C 是内部节点的分支。 C 的孩子为 F 和 G，所有盒的 y 范围均为 $[0, 1]\ \mathrm{m}$：

盒子	x 范围
B	[0,1]
F	[10,11]
G	[2,3]

原 C 包住 F 和 G，周长为 $2(9 + 1) = 20\ \mathrm{m}$。 交换 B 与 F 后，C 包住 B 和 G，周长为 $2(3 + 1) = 8\ \mathrm{m}$。 交换 B 与 G 后，C 包住 B 和 F，周长为 $2(11 + 1) = 24\ \mathrm{m}$。 所以第一种交换把局部代价从 20 m 降到 8 m，根范围并未改变。

边界细节也值得读：一叶分支只有旧代价严格小于两个候选时才直接返回，因此等成本时仍可能交换。 两子均为内部节点的分支，则用严格小于当前最优代价来选交换。

当前代理操作并不都旋转：

操作	主要行为
`b2DynamicTree_CreateProxy`	插入时允许旋转
`b2DynamicTree_MoveProxy`	删除再插入，但 `shouldRotate=false`
`b2DynamicTree_EnlargeProxy`	扩大祖先盒并标记，不立即重插

重建处理的是更大范围的结构质量。 `b2DynamicTree_Rebuild(tree, false)` 把未扩大子树整体保留为重建单元；扩大部分则拆开重组。 `fullBuild=true` 才展开全部内部结构。 返回的 `leafCount` 是本次重建单元数量，局部重建时不必等于代理总数。

当前启用的 `B2_TREE_HEURISTIC` 为 0。 `b2PartitionMid` 按中心范围最长轴的**空间中点**划分，不是对象数量中位数，也不是全局 SAH 排序。 例如中心 x 为 0、1、8、9，y 相同，中点 4.5 把它们分成两组。 如果中心全部相同而划分失败，则退回 `count / 2`。

宽阶段后台重建任务只重建动态树和运动学树，且可与窄阶段重叠执行。 修改树的 refit 阶段之前必须等待这个任务完成。

代价：重建需要扫描、临时数组和可能的分配。 树高降低也不等于整帧耗时必然降低，应同时测查询访问数和重建开销。

实验：复用 `TreeRebuildAndValidateTest` 的固定代理布局。 保存相同查询的结果集合、访问数和树高，分别在重建前后比较。 正确性要求集合相同，性能结论则来自实测，不能预填一个加速倍数。

17.8 三种缓存不能混为一谈

缓存	保存什么	主要省掉什么
接触点 ID 热启动	点特征 ID 对应的旧冲量	求解器从零建立支撑冲量的工作
GJK simplex cache	支持点索引组成的单纯形	距离搜索的部分迭代
Contact recycling	相对姿态及已有 manifold	满足条件时整次接触更新

接触点 ID 热启动

朴素求解每帧把冲量从零开始。 稳定堆叠却往往需要重新找回与上一帧相近的支撑冲量。

当前 `b2UpdateContact` 仍重新计算 manifold，再按新旧点的特征 ID 匹配冲量。 假设旧点为 `ID7: 1.2 N·s`、`ID9: 0.8 N·s`，新点为 `ID9`、`ID12`。 新初值应为 0.8 N·s 和 0，而不是按数组位置复制 1.2 和 0.8。

未匹配点从零开始。 源码中把未匹配旧冲量重新分配的尝试位于 `#if 0`，当前并不执行。 这项优化不等于跳过窄阶段。

实验：记录点 ID 与新旧冲量，制造接触点交换顺序和点消失的情形，检查匹配的是身份而非槽位。

GJK 单纯形缓存

朴素 GJK 每次从空单纯形开始。 连续小移动时，上次靠近最近点的支持点索引可能仍是好的起点。

`b2MakeSimplexFromCache` 根据缓存的 `indexA/indexB` 从**当前**代理中重新取点。 它并不是把旧世界空间顶点原封不动拿来继续算。 空缓存则从索引 0 的支持点对开始。

缓存依赖对应的几何索引仍有效；几何拓扑变化不能盲目复用。 也不是所有形状对的 manifold 算法都使用 GJK。

实验：固定两个凸形状的小位移序列，比较每次清空 cache 与连续使用 cache。 检查距离、最近点及迭代总数，不能事先声称每次必少三轮。

17.9 Contact recycling 的容差与副作用

朴素方案每帧对每个有效接触重新计算 manifold 和材料。 如果相对姿态变化很小，重做全部工作可能浪费。 当前回收优化允许在保护条件满足时复用已有 manifold。

进入回收判断之前，仍会更新两个刚体的求解索引和逆质量。 这是必要的：睡眠迁移和稠密数组交换可能改变索引。

主要条件为：

1. 两端都不是本轮标记的 fast 物体。
2. 世界回收距离大于零。
3. 缓存相对姿态有效。
4. 接触具有回收标志；创建时要求两刚体都开启回收。
5. 两刚体相对缓存的世界旋转余弦都超过阈值。
6. 相对平移与相对转动的合成指标严格小于容差。

令 d 为相对姿态平移变化长度，单位 m；R 为两端非静态最大 extent 的最大值，单位 m。 令 $q_r.s$ 为相对旋转变化的正弦分量，无量纲。 代码比较：

$$d + R|q_r.s| < \tau.$$

默认 touching 接触 $\tau = 0.05\ \mathrm{m}$。 non-touching 接触使用 $\min(0.05, 0.02) = 0.02\ \mathrm{m}$，避免错过刚开始的接触。

取 $d = 0.01\ \mathrm{m}$、$R = 0.5\ \mathrm{m}$、$|q_r.s| = 0.02$：

$$d + R|q_r.s| = 0.01 + 0.5 \times 0.02 = 0.02\ \mathrm{m}.$$

它通过 touching 的距离条件，却不通过 non-touching 条件，因为比较是严格小于。 世界旋转条件另要求 `minCos > 0.98`，不能仅检查两物体相对姿态不变。 例如两个物体一起转过 20°，相对姿态可不变，但世界旋转条件会拒绝复用。

成功时，代码用旋转后的旧锚点临时计算新的 separation，并更新法向速度、恢复相关速度和 `persisted`。 不能说它重新生成了法线或把新锚点写回了 manifold；这不是实际代码步骤。 缓存姿态只在真正重新计算前刷新，所以容差约束的是自上次刷新以来的累计变化。

必须公开说明副作用：

- 成功回收直接 `continue`，跳过 `b2UpdateContact`。
- 因而材料参数刷新也被跳过。
- 位于该函数里的 `PreSolve` 回调同样会被跳过。
- 当前条件没有“开启 PreSolve 自动禁止回收”的分支。

如果业务依赖逐次更新材料或逐帧执行回调，必须理解并管理这条路径。 它是受阈值控制的近似优化，不应承诺与关闭回收时逐位相同。

实验：同一输入分别使用回收距离 0 和默认值。 记录 recycled contact 数、窄阶段耗时、最大穿透和事件差异。 还应安排材料修改和 PreSolve 计数实验，而不只观察画面是否相似。

17.10 自测与参考答案

1. 1,000 个形状的全配对检查次数是多少？

2. 同正组号且双方 mask 为零时，形状过滤是否允许？
3. 插入例中靠近 A 的新叶，候选最小代价是多少？
4. 旋转例的旧、新局部周长分别是多少？
5. non-touching 回收指标等于 0.02 m，默认条件能通过吗？
6. 新 `ID12` 没有旧匹配点时，热启动冲量是多少？
7. 树查询访问数减少，是否就证明整帧更快？

参考答案：

1. $1000 \times 999/2 = 499500$ 次。
2. 这个形状过滤函数允许，但其他碰撞限制仍可拒绝。
3. 11 m；把新叶移到另一手算位置后，B 候选为 16 m。
4. 20 m 与 8 m。
5. 不能，源码使用严格小于。
6. 0。
7. 不能，还要计算维护、重建、候选处理及其他阶段成本。

17.11 源码导航

主题	相对源码链接与入口行号
分组与掩码	`shape.h`，`b2ShouldShapesCollide`：122
形状 margin	`shape.c`，`b2ComputeShapeMargin`：36
配对过滤与三树查询	`broad_phase.c`，172、333
树后台重建	`broad_phase.c`，`b2UpdateTreesTask`：401
插入代价与旋转	`dynamic_tree.c`，164、315、602
查询与重建	`dynamic_tree.c`，1085、1433、1879
GJK 缓存	`distance.c`，168、206、424
点 ID 匹配	`contact.c`，`b2UpdateContact`：525
回收条件	`physics_world.c`，`b2CollideTask`：449，重点 507–585
树实验入口	`test_dynamic_tree.c`，329、398、449

返回 README · 下一章：岛、睡眠与数据布局

18 岛、睡眠与数据布局

> 目标：解释为什么物体会“搬家”、为什么睡眠必须考虑连通关系，以及这些机制如何减少真正进入热循环的数据。
>
> 本章对应 Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b`。 字节量与操作次数是明确假设下的理论估算，不代表本次执行过性能测试。

18.1 从一份简单的 Body 数组开始

第一版引擎完全可以让一个 `Body` 结构包含全部信息：

- 用户数据、名字和形状列表。
- 世界位置、旋转和质量。
- 速度、外力和本步增量。
- 接触、关节和睡眠标志。

每帧遍历数组，对醒着的物体积分，对所有约束顺序求解。 这容易实现，也容易验证。

问题出现在“世界很大，但大多数物体不动”的场景。 假设世界有 100,000 个刚体，只有 1,000 个醒着。 如果每次热循环都扫描整个世界并判断 `isAwake`，仍要读取 100,000 个标志。 增加子步和阶段以后，这项扫描会重复出现。

教学伪代码，非源码：

```
for body in allBodies:
    if body.awake:
        integrate(body)
```

这里并不是判断语句写错了，而是数据组织迫使求解器不断经过不需要的数据。 优化目标应是让热循环天然只看见活跃物体。

18.2 冷热拆分到底节省什么

当前实现把刚体信息拆成三层。

结构	主要用途	典型内容
`b2Body`	身份、API、连接关系	ID、代数、用户数据、邻接索引、睡眠计时
`b2BodySim`	碰撞准备与积分参数	世界变换、质心、力、逆质量、阻尼、尺度
`b2BodyState`	约束热循环反复读写	速度、增量位移、增量旋转、标志

`b2BodySim` 没有保存线速度和角速度。 速度在 `b2BodyState` 中；这直接影响后面的睡眠与唤醒语义。

`b2BodyState` 的布局为：

字段	字节
`linearVelocity`	8
`angularVelocity`	4
`flags`	4
`deltaPosition`	8
`deltaRotation`	8
总计	32

32 B 不只是注释中的愿望，接触求解器使用静态断言检查这一布局。 其他结构的实际大小受 ABI、编译选项和双精度世界模式影响，不应随意写死。

考虑一个明确标注为假设的成本模型。 假设未拆分的一体结构占 256 B，但约束循环实际只需要 32 B。

$$100000 \times 256 = 25{,}600{,}000\ \mathrm{B},$$

$$100000 \times 32 = 3{,}200{,}000\ \mathrm{B}.$$

若只让 1,000 个醒体进入热数组，则有效状态范围变为：

$$1000 \times 32 = 32{,}000\ \mathrm{B}.$$

这些是数据跨度与有效工作集的比较，不是自动得到八倍或八百倍加速。 实际时间还受缓存命中、访问模式、计算量和预取影响。

若假设缓存行为 64 B，一行可容纳两个连续的 `b2BodyState`。 这改善局部性，但也意味着两个线程写不同刚体时仍可能发生伪共享。 数据拆分不会自动解决所有并发访存问题。

实验：记录热循环迭代数、`bodyStates.count` 与 `capacity`。 再测实际时间，检查主要收益来自少扫描物体，还是更紧凑的访存。

18.3 稳定 ID 不等于稳定位置

让用户直接保存数组元素地址，会使扩容和搬迁变得危险。 让用户保存醒体数组下标，也会使睡眠和删除改变对象身份。

当前设计把身份与存储位置分开：

1. 用户持有 `b2BodyId`。
2. `index1 - 1` 对应世界中的刚体身份槽。
3. `generation` 用于识别槽位重新使用。
4. `b2Body.setIndex` 指向当前 solver set。
5. `b2Body.localIndex` 指向该集合里的稠密位置。

因此一个刚体可以换集合、换稠密下标，公开 ID 仍然不变。 稳定的是身份，不是热数据地址。

手算 RemoveSwap

设某稠密数组为：

下标	对象
0	A，ID 10
1	B，ID 20
2	C，ID 30
3	D，ID 40

删除 B 时，不把 C、D 全部向前移动。 而是把尾部 D 复制到下标 1，再把有效数量从 4 改成 3：

下标	删除后对象
0	A，ID 10
1	D，ID 40
2	C，ID 30

必须同时把 D 的 `localIndex` 从 3 改成 1。 D 的公开 ID 40 没有变。

`b2RemoveBodySim` 专门处理这项映射修复。 一般数组的 `b2Array_RemoveSwap` 返回被交换元素原来的尾下标，供调用者更新其反向索引。 如果删除的本来就是尾元素，则返回 `B2_NULL_INDEX`。

重要不变量：醒体的 `bodySims[i]` 和 `bodyStates[i]` 必须代表同一个刚体。 因此删除醒体时，两份数组必须在同一位置执行相同的尾交换。 只搬 Sim 不搬 State，会把一个刚体的质量和另一个刚体的速度配在一起。

代价：RemoveSwap 不保持原遍历顺序。 所有依赖该顺序的算法，都必须把确定性和索引修复一起考虑。 数组增长也仍会使旧指针失效。

实验：用四个物体重复删除中间和末尾元素。 保留公开 ID 检查身份，用内部断言检查 Sim、State 和 `localIndex` 的一致性。

18.4 SolverSet 把“不参与热循环”落实到布局

仅有一个 `sleeping` 布尔值，不能阻止求解器扫到睡眠物体。 当前 solver set 将它们从热数组中分离。

集合编号	含义
0	`b2_staticSet`
1	`b2_disabledSet`
2	`b2_awakeSet`
3 及以上	睡眠集合

其中有两个不能省略的区别。

第一，`disabledSet` 不只存用户禁用的刚体和关节。 它还承接某些未触碰接触，例如初次创建接触时两端都不在 awake 集合。

第二，awake 集合中的 `contactSims` 是未触碰接触。 正在触碰的接触和醒着的关节放在约束图对应颜色中。 不能说“awakeSet 包含所有醒体约束”。

`bodyStates` 只存在于 awake 集合，用于醒着的动态和运动学刚体。 静态体没有这份可写状态。

睡眠集合通常只有一个岛，但这不是永久一一对应关系。 在睡眠岛之间创建关节可能合并睡眠集合，使一个集合暂时拥有多个岛。 `solver_set.h` 和唤醒函数明确保留了这种情况。

收益：积分与约束热循环处理的是醒体工作集，不必反复过滤全世界。 **代价：**进入和离开集合要复制数据、修映射，有时还要扩容。 因此一次迁移不是常数时间修改标志。

18.5 岛是睡眠连通分量，不是 SIMD 工作块

如果每个刚体独立睡眠，可能出现一个箱子刚被暂停，支撑它的另一个箱子却还在移动。 接触约束连接的状态不能随意只停一半。

岛把通过触碰接触和关节连接的非静态刚体组织起来，作为睡眠判断单位。 一个成员还不满足睡眠条件，就可以让整岛保持醒着。

静态地面不把所有落在地面的箱子合并成一个大岛。 否则世界里一个箱子移动，就可能阻止全部堆叠睡眠。

运动学体与静态体在这里不能混称。 当前拆岛逻辑连接非静态端，运动学体可以参与岛连接。 这与下一章“图着色只给动态写端占位”的判据不同。

Box2D 的求解并行主要按约束图颜色组织。 岛在本章首先回答“谁一起睡、谁一起醒”，不应套用旧版本按岛划分求解工作的解释。

18.6 新连接合并，删除连接延后检查

最朴素的连通分量维护方式，是每帧重新遍历全部刚体和约束。 这容易理解，却会重复检查大量没有变化的连接。

当前合并路径在出现连接时增量处理。 `b2MergeIslands` 按刚体数量保留大岛，把小岛成员迁入大岛。

具体步骤为：

1. 两端本来同岛则返回。
2. 静态端没有岛时，保留另一端岛。
3. 比较两个岛的刚体数量，选择大岛。
4. 迁移小岛的 body、contact、joint 列表。
5. 修正每个成员的 `islandId` 和 `islandIndex`。
6. 累加待检查的约束删除计数，销毁小岛。

例如 1 体岛连接 10,000 体岛，改小岛一侧通常比搬大岛便宜。 但连接列表也需要迁移，因此不能只按刚体数宣称总成本恰好为一次赋值。

删除连接更麻烦：删掉一条边，不一定真的切断了图。

- A—B—C—D 删除 B—C，会得到两块。
- 若还存在 A—D，删除 B—C 后仍然连通。

所以 `b2UnlinkContact`、`b2UnlinkJoint` 增加 `constraintRemoveCount`，先保守保留原岛。 等岛中出现足够困倦的成员，再考虑拆分。

收益：避免每次接触短暂消失都立即做完整连通分析。 **代价：**已经断开的部分可能暂时还共享睡眠单位，延迟进入睡眠。

18.7 当前拆岛用临时并查集

不要把当前 `b2SplitIsland` 描述成沿刚体接触链表做一次旧式 DFS。 它对该岛现有成员建立临时并查集。

主要过程为：

1. 为每个原岛刚体建立 `parent` 与 `rank`。
2. 遍历仍存在的接触，把两个非静态端 union。
3. 遍历关节，执行同样的连接合并。
4. 对根节点统计组件数量和各组件约束数量。
5. 若仍只有一组件，清除删除计数后返回。
6. 否则预先计算新岛容量，再创建新岛。
7. 分配 body、contact、joint 到各新岛，修复索引。
8. 销毁旧岛并逆序释放临时数组。

对 A—B 与 C—D 两条剩余边，union 后得到 $\{A, B\}$ 和 $\{C, D\}$。 若加上 A—D，所有 parent 最终归于一个组件，不需要真正搬家。

令 V 为岛内刚体数，E 为接触与关节数量。 并查集部分的常见摊还复杂度写作：

$$O((V + E)\alpha(V)),$$

其中 α 为反阿克曼函数，在实际规模中增长极慢。 另外还有成员计数、数组分配和迁移成本，不能忽略。

拆岛任务不能与 `FinalizeBodies` 并发修改岛索引。 当前调度在安全区间发出一个拆岛任务，并在 Finalize 前等待完成。 每个 worker 推荐最困的候选；睡眠时间相等时，较大的岛 ID 优先，以保持确定性。 跨 worker 归约也使用同一平局规则。

实验：构造链条，删除桥边与非桥边各一次。 记录组件数量、删除计数和何时真正拆岛，验证“删除边”与“出现新岛”不是同一事件。

18.8 睡眠不能只检查线速度

一个细长物体质心几乎不动，但可能还在快速旋转。 一个堆叠速度接近零，也可能持续依赖位置修正。 只看 $|v|$ 会过早宣布它稳定。

当前 Finalize 使用以下变量：

符号	含义	单位
v	质心线速度	m/s
ω	角速度	rad/s
R	`maxExtent`	m
Δp	大步累计增量位移	m
$\Delta q.s$	增量旋转的正弦分量	无量纲
Δt	整个大步时间	s

代码先估计最远点速度和大步运动量：

$$u = |v| + |\omega| R,$$

$$d = |\Delta p| + |\Delta q.s| R.$$

再定义睡眠指标：

$$s = \max\left(u, \frac{0.5 d}{\Delta t}\right).$$

s 的单位是 m/s。 这里取最大值，不是把两项直接相加。 分母是大步 Δt，不是某个子步的 h。 $\Delta q.s$ 是正弦分量，不是直接保存的旋转角。

数值例：速度很小，仍然不能睡

设：

- $|v| = 0.01\ \mathrm{m/s}$。
- $|\omega| = 0.02\ \mathrm{rad/s}$，$R = 0.5\ \mathrm{m}$。
- $|\Delta p| = 0.001\ \mathrm{m}$，$|\Delta q.s| = 0.002$。
- $\Delta t = 1/60\ \mathrm{s}$。

于是：

$$u = 0.01 + 0.02 \times 0.5 = 0.02\ \mathrm{m/s},$$

$$d = 0.001 + 0.002 \times 0.5 = 0.002\ \mathrm{m},$$

$$s = \max(0.02, 0.5 \times 60 \times 0.002) = 0.06\ \mathrm{m/s}.$$

默认睡眠阈值为 0.05 m/s，因此该物体的睡眠计时清零。 真实速度虽然不大，本步运动指标却仍然过大。

若改成 $|\Delta p| = 0.0002\ \mathrm{m}$、$|\Delta q.s| = 0.0004$，则 $d = 0.0004\ \mathrm{m}$。 位置项为 0.012 m/s，最终 $s = 0.02\ \mathrm{m/s}$，可以累计睡眠时间。

边界条件

`b2FinalizeBodiesTask` 在以下任一条件满足时清零 `sleepTime`：

- 世界禁用睡眠。
- 刚体禁用睡眠。
- `sleepVelocity > sleepThreshold`。

注意比较为严格大于，所以等于阈值时可以累计。 满足安静条件则 `sleepTime += timeStep`。 默认 `B2_TIME_TO_SLEEP` 为 0.5 s。

理想算术下，60 Hz 的 30 帧正好是 0.5 s。 实际精确睡眠步号还受浮点累加、运动变化和拆岛等待影响，不能保证任何场景都在第 30 帧睡眠。

若岛内任意体的 `sleepTime < 0.5`，该 worker 就设置岛的 awake 位。 所有 worker 的位图按位或以后，只要还有一个成员要求清醒，整岛就不能入睡。

18.9 入睡时真正发生的数据迁移

`b2TrySleepIsland` 首先拒绝“仍待拆分且成员数大于一”的岛。 单体岛是这个条件的例外，不应省略后半个判断。

之后不是只设置一个布尔标志，而是执行搬迁：

1. 分配睡眠集合，并按岛大小预留数组容量。
2. 将 body Sim 从 awake 复制到 sleep。
3. 从 awake Sim 数组 RemoveSwap，并修正被交换体索引。
4. 从 awake State 数组同位置 RemoveSwap；不复制睡前 State。
5. 更新原体的 `setIndex/localIndex` 和睡眠事件。
6. 把触碰 contact Sim 从约束图移到睡眠集合。
7. 清除正常颜色中的占位 bit，并修复交换接触索引。
8. 迁移关节与岛描述。

未触碰接触有额外所有权判断。 如果另一端仍然 awake，这个接触继续保留在 awake 集合。 只有另一端也不 awake 时，才把它移入 disabled 集合。

原因是未触碰接触可能跨越不同的睡眠岛，没有明确的单岛归属。 不能简单说“睡眠岛的全部接触都进入它的睡眠集合”。

睡眠阶段放在最后，还因为它会打乱 awake 稠密索引。 如果在 enlarged body 位图消费之前搬家，位图中的下标就可能指向别的体。 当前倒序枚举待睡岛，也是在配合 RemoveSwap 的行为。

18.10 唤醒不是恢复一份隐藏速度

唤醒反向迁移 Sim，但并不恢复睡前 State。 源码非常明确：

```
b2BodySim* simDst = b2Array_Emplace( awakeSet->bodySims );
memcpy( simDst, simSrc, sizeof( b2BodySim ) );

b2BodyState* state = b2Array_Emplace( awakeSet->bodyStates );
*state = b2_identityBodyState;
state->flags = body->flags;
```

新状态的线速度、角速度和增量位移为零。 增量旋转为 $(1, 0)$，即单位旋转，而不是全零。

因此“从 BodySim 取出速度重建 State”是错误解释：BodySim 本来就没有这些速度字段。 唤醒后再施加冲量或设置速度，是另一项操作。

接下来恢复未触碰接触、重新把触碰接触与关节加入约束图，并迁回岛描述。 接触需要重新染色，因此唤醒不是免费操作。

内存收益的精确说法

睡眠减少 `bodyStates.count`，缩小有效热工作集。 但 RemoveSwap 不会自动缩减数组 `capacity`。 不能说每睡一个体，就立刻向系统归还 32 B。

这里主要避免的是反复读取和求解睡眠体。 频繁醒睡反而可能增加复制、分配和重新染色成本。

实验：让两个不相连箱堆落在同一静态地面上。 都睡眠后只推动其中一堆，检查另一堆是否保持睡眠，并记录 awake 数量和集合迁移。

18.11 临时栈分配也有真实代价

宽约束、工作块和临时索引数组通常只活过一个步骤或阶段。 朴素方式每次调用通用分配器；稳定重复的高峰则适合集中管理。

当前 `b2StackAlloc` 对正请求大小按 32 B 向上对齐。 容量内使用 bump index，容量不足则回退到堆分配。 每一笔分配仍会记录一项 entry。

设容量为 128 B，依次请求 1、33、64 B：

请求	对齐后大小	处理
1	32	栈内，累计索引 32
33	64	栈内，累计索引 96
64	64	超容量，堆回退

总分配高峰为 $32 + 64 + 64 = 160$ B。 释放必须按 64、33、1 的逆序进行。 堆回退的那笔由通用释放处理，其他笔回退索引。

所有分配释放后，`b2GrowStack` 才允许增长。 此例下一次容量可增长到：

$$160 + 160/2 = 240 \text{ B}.$$

下一轮同样请求就不需要那次堆回退。 但这不等于“每帧一个 O(1) reset，任何时候都零分配”。 entry 数组可能增长，每笔仍有登记和释放，首次高峰也可能走堆。

实验：区分首帧与稳定期，记录 capacity、max allocation 和回退次数。 不要删除首帧成本后，又把稳定期数字解释为整个生命周期从不分配。

18.12 自测与参考答案

1. 删除稠密数组第二项时，最后一项的公开 ID 会改变吗?
2. 1,000 个有效 `b2BodyState` 占多少有效数据字节?
3. 上述第一组睡眠例中最终指标是多少?
4. 两个箱堆共享静态地面，是否因此必然同岛?
5. 唤醒的初始速度从哪里恢复?
6. 一次 RemoveSwap 是否保证释放底层容量?
7. 128 B 栈容量下请求 1、33、64 B，有几次堆回退?

参考答案：

1. 不变，改变的是 `localIndex`，并需修复映射。
2. 32,000 B，不包括保留容量、其他数组和分配器元数据。
3. 0.06 m/s，大于默认 0.05 m/s，计时清零。
4. 不必；静态地面不桥接它们的睡眠连接。
5. 不恢复睡前速度，新建 identity State；后续输入另行改变速度。
6. 不保证，它只减少有效数量。
7. 一次；对齐后的总请求高峰为 160 B。

18.13 源码导航

主题	相对源码链接与入口行号
Body、State、Sim	`body.h`，68、153、175
Sim 删除修索引	`body.c`，`b2RemoveBodySim`：34
通用 RemoveSwap	`container.h`，136
集合定义及多岛情况	`solver_set.h`，14、54
唤醒与入睡迁移	`solver_set.c`，35、155

主题	相对源码链接与入口行号
初始接触集合选择	`contact.c`，247–258
岛合并、删除与拆分	`island.c`，82、235、311、389
睡眠指标	`solver.c`，`b2FinalizeBodiesTask`：560
拆岛调度与最终睡眠	`solver.c`，1454、2000
栈分配与增长	`arena_allocator.c`，30、65、83

返回 README · 下一章：图着色、SIMD 与多线程

19 图着色、SIMD 与多线程

> 目标：从正确的单线程顺序冲量出发，逐步回答哪些约束可以同时计算、SIMD 的一条 lane 代表什么，以及线程之间在哪里等待。
>
> 本章对应 Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b`。 操作次数和加速上界均为理论模型，不是本次测量结果。

19.1 为什么不能直接给接触循环加线程

朴素顺序冲量按照固定顺序处理约束。 处理 A—B 后，B 的速度立刻更新；处理 B—C 时读到的是更新后的 B。 这正是 Gauss-Seidel 式迭代的依赖关系。

教学伪代码，非源码：

```
for constraint in constraints:
    read current body velocities
    compute impulse
    write updated body velocities
```

如果让共享 B 的两条约束同时运行，会发生读写冲突。 例如 B 初始速度为零，两线程分别计算需要增加 1 m/s 和 2 m/s。 若它们都先读到零，再分别写回 1 和 2，最终可能只留下其中一次更新。 这不是可接受的“浮点最后一位不同”，而是丢失了物理作用。

给每个刚体加锁能够避免部分数据竞争，却引入另一个问题。 先拿到锁的是哪条约束，可能随调度变化；约束处理顺序也随之变化。 高连接度刚体还会成为锁争用热点。

因此当前实现首先解决的是依赖组织，而不是先增加线程。

这条热路径为什么值得优化

假设有 100,000 条接触，4 个子步，每个子步分别进行 WarmStart、Solve、Relax。 仅按每条接触一次访问粗略计数，便有：

$$100000 \times 4 \times 3 = 1{,}200{,}000$$

次接触处理。 这个模型尚未计入 Prepare、Store、多接触点、关节等工作。 它说明优化热循环有潜力，却没有说明实际 CPU 时间一定由哪一项主导。

19.2 图着色选择互不冲突的写入集合

把刚体看作图的顶点，把接触和关节看作边。 两条边如果共享会被约束写回的动态刚体，就不能同色。

同色边因此可以并行求解。 不同颜色仍按固定次序处理，让后一个颜色读到前一个颜色完成后的速度。

注意，这里是**边着色**，不是给每个刚体涂一种颜色。 一个刚体连接的多条边可以处于不同颜色。

当前接触与关节使用一致的基本规则：

边类型	搜索颜色	哪些端点占位
动态—动态	0 到 19	两端动态体
动态—静态	22 到 1	只有动态端
动态—运动学	22 到 1	只有动态端
正常颜色找不到	23	进入 overflow

总颜色数为 24，其中最后一色是 overflow。 动态—动态正常只用前 20 色。 单动态端约束从后往前搜索，而且不搜索颜色 0。

它是一种增量贪心着色，不保证最少颜色。 20、21、22 三色不会被动态—动态约束占用，但单动态端约束也可以退到较低颜色。

19.3 逐条手算五条边

设 A、B、C、D 为动态刚体，K 为运动学刚体。 按下表顺序加入五条边，开始时所有颜色均为空。

次序	边	搜索过程	结果
1	A—B	色 0 没有 A、B	0
2	B—C	色 0 已有 B；色 1 可用	1
3	C—D	色 0 只有 A、B，不冲突	0
4	A—K	从色 22 开始，只检查 A	22
5	D—K	色 22 已有 A，但没有 D	22

最终为：

颜色	边集合	动态端占位集合
0	A—B、C—D	A、B、C、D
1	B—C	B、C
22	A—K、D—K	A、D

色 0 的两条边不共享动态端，可以同时求解。 色 22 的两条边共享 K，也仍然允许同色，因为 K 不是约束写回对象。

这正是一个重要版本区别。 旧教程以及 `constraint_graph.c` 文件头旧注释中存在“运动学也占位”的说法。 当前 `b2AddContactToGraph` 和 `b2AssignJointColor` 实际只给动态端设置 bit，应以执行代码为准。

如果把 K 换成静态地面 S，A—S 和 D—S 同样可以同色。 但 static 与 kinematic 的状态来源不同，不能仅用“质量无穷大”把实现细节混为一谈。

19.4 静态与运动学为什么不会写冲突

静态刚体没有 awake `b2BodyState`。 求解时使用局部的 identity 状态提供零速度和单位增量旋转。

运动学刚体有状态，必须提供由用户设置的速度。 否则动态刚体无法正确响应移动平台。 但接触约束不应反过来修改运动学刚体的速度。

当前 SIMD scatter 明确检查 `b2_dynamicFlag`：

```
#if 1
		// I don't use any dummy body in the body array because this will lead to multithreaded
cache coherence problems.
		if ( i1 != B2_NULL_INDEX && ( states[i1].flags & b2_dynamicFlag ) != 0 )
		{
			// [t1.x t1.y t3.x t3.y]
			b2StoreW( (float*)( states + i1 ), _mm_shuffle_ps( t1, t3, _MM_SHUFFLE( 1, 0,
1, 0 ) ) );
		}
```

即使计算得到的运动学速度没有改变，也不能让多个线程“把同样的值再写一次”。 并发写同一地址仍会造成数据竞争和缓存一致性成本。 正确做法是根本不写回这个端点。

同样，不应在状态数组里放一个全线程共用的可写 dummy body，让所有静态 lane 都写它。 本来没有物理依赖的约束，会因此争用一个地址。

图着色消除的是语义上的共享写端。 它不能保证不同动态体不落在同一缓存行，因此伪共享仍可能存在。

实验：让多个动态箱子接触同一运动学平台。 记录着色结果并检查 scatter 条件，确认平台状态仅在适当积分阶段更新，而非由接触线程反复写回。

19.5 为什么必须保留 overflow

固定颜色数不是所有图都能正常着色的保证。 正确实现必须在颜色不足时继续求解，而不是丢掉约束。

设中心动态体 X 连接 21 个不同动态体。 每条边都共享 X，所以不能共色。 前 20 条边可以分别占用 0 到 19。 第 21 条边进入颜色 23 的 overflow，即使 20、21、22 仍空着。

原因是动态—动态边不能侵入为单动态端约束保留的颜色。 这不是颜色查找遗漏。

如果 X 连接的是 23 个静态端，正常可用色为 22 到 1，共 22 色。 第 23 条单动态端约束同样进入 overflow。

overflow 使用单线程标量路径保底。 在 WarmStart、Solve、Relax 中，当前先处理 overflow，再按正常颜色顺序处理。

后颜色最后修改速度，单动态端约束靠后有利于减轻堆叠把物体推入静态地面的趋势。 这是一种求解次序设计，不是“绝对优先级”或“保证无穿透”的证明。

代价：高连接度场景可能有较多串行 overflow，增加线程无法消除这部分成本。

实验：比较大量互不相关的约束与中心星图。 约束数相近时，它们的可并行性可能完全不同，应同时报告颜色分布和 overflow 数量。

19.6 SIMD 的 lane 是不同约束，不是空间维度

二维向量只有 x、y，为什么还可以用四条或八条 lane？ 因为 lane 对应不同接触，而不是一个物体的 x、y、z、w。

先看一个不含角项和 bias 的标量法向求解例。

变量	含义	单位
K	有效逆质量	kg⁻¹
$m_e = 1/K$	有效质量	kg
v_n	相对法向速度	m/s
λ_o	已累计法向冲量	N·s

每条约束执行：

$$\Delta\lambda = -m_e v_n,$$
$$\lambda_n = \max(\lambda_o + \Delta\lambda, 0),$$
$$\Delta\lambda_{\text{used}} = \lambda_n - \lambda_o.$$

四条互不冲突的约束可以一起代入：

lane	K	m_e	v_n	λ_o
0	2	0.5	-2	0
1	1	1	-1	0.2
2	0.5	2	0.5	0.3
3	4	0.25	-4	0

得到：

$$\Delta\lambda = [1, 1, -1, 1]\ \text{N·s},$$
$$\lambda_n = [1, 1.2, 0, 1]\ \text{N·s},$$
$$\Delta\lambda_{\text{used}} = [1, 1, -0.3, 1]\ \text{N·s}.$$

lane 2 正在分离，只能撤回已有的 0.3 N·s，不能产生负累计法向冲量把物体拉回。 向量化没有改变这条物理规则，只把四份相同算式并列执行。

先让标量版本和这个表一致，再谈 SIMD，会比直接读向量 intrinsic 容易得多。

19.7 AoS 状态怎样变成宽寄存器

持久的 `b2BodyState` 是每体一份的 AoS 布局。 SSE2 gather 读取四个状态，再做转置。

输入可以理解成：

物体	第一组字段	第二组字段
1	vx_1、vy_1、ω_1、$flags_1$	Δx_1、Δy_1、$\Delta q.c_1$、$\Delta q.s_1$
2	vx_2、vy_2、ω_2、$flags_2$	Δx_2、Δy_2、$\Delta q.c_2$、$\Delta q.s_2$
3	vx_3、vy_3、ω_3、$flags_3$	Δx_3、Δy_3、$\Delta q.c_3$、$\Delta q.s_3$
4	vx_4、vy_4、ω_4、$flags_4$	Δx_4、Δy_4、$\Delta q.c_4$、$\Delta q.s_4$

转置后的寄存器则分别保存：

- `v.X = [vx₁,vx₂,vx₃,vx₄]`。
- `v.Y = [vy₁,vy₂,vy₃,vy₄]`。
- `w = [ω₁,ω₂,ω₃,ω₄]`。
- 其他增量和标志也按相同方式跨物体排列。

宽约束计算完成，再转置并 scatter 回动态体速度。 所以不能说“Box2D 全部持久数据都是 SoA”。 更准确的说法是：持久状态保持紧凑 AoS，计算阶段使用跨约束的宽布局。

当前平台选择为：

配置	宽度
SSE2	4
NEON	4
当前 WASM SIMD 路径	4
AVX2	8
`BOX2D_DISABLE_SIMD` 的 NONE 路径	4 个标量模拟 lane

关闭 SIMD 不会自动把 `B2_SIMD_WIDTH` 改为 1，也不会关闭图着色。 关节工作块仍分发到各关节求解函数，不能泛称所有关节都已经 SIMD 化。

代价：gather、scatter、转置、非连续读取和分支都消耗时间。 SIMD 宽度是吞吐潜力，不是整个求解器的加速倍数。

19.8 尾 lane 必须显式无害化

某颜色只有 5 条接触，四宽向量需要几组？ 令 N_c 为该色接触数，W 为 lane 数，两者无量纲：

$$N_w = \left\lceil \frac{N_c}{W} \right\rceil .$$

当 $N_c = 5, W = 4$，需要 2 个 wide 槽，共 8 个 lane。 有效利用率为：

$$\eta = \frac{5}{2 \times 4} = 62.5\%.$$

当 $N_c = 10$ 时，四宽利用率为 $10/12 \approx 83.3\%$，八宽为 $10/16 = 62.5\%$。 因此更宽向量在小颜色集合中可能浪费更多尾 lane。

不能为了填满向量而随意把不同颜色混在一次 Solve 中。 这些颜色之间可能共享动态端。

当前处理分为两步：

1. `b2Solve` 在有余数时把最后整个 wide 结构清零。
2. Prepare 对有效 lane 填入接触；空 lane 使用零接触输入，索引保持零。

宽约束的索引使用 `bodySimIndex + 1` 编码。

原含义	wide 索引
真正的 dense index 0	1
真正的 dense index 7	8
`B2_NULL_INDEX=-1`	0
空 lane	0

gather 减 1 后，零索引得到 `B2_NULL_INDEX`，使用 identity 状态而不访问真实 `states[0]`。 scatter 也不会向这个空端写回。

这与公开 ID 的加一编码形式类似，但不是同一个数组、同一种身份。 不能把 wide 索引当作用户的 `b2BodyId`。

实验：分别安排每色 1、3、4、5、7、8、9 条接触。 检查尾槽初始化、索引与越界保护，并比较标量模拟和 SIMD 路径的回归结果。

19.9 stage、block、worker、task 是四个层次

名称	含义
stage	一次算法阶段，例如某颜色的 Solve
block	该阶段的一段工作范围
worker	参与处理的逻辑工人编号
task	交给调度系统执行的回调

一个 worker 可以处理多个 block。 一个 task 可以活过整个约束求解过程。 同一 block 描述还能在后续子步被复用。 不要画成“一个接触等于一个线程”。

当前目标是每个 worker 至多安排约四个块，给工作窃取留余量。 设 worker 数为 P，最大块数目标为 $B = 4P$。 给定工作项数 M 和最小块大小 m：

$$b = \begin{cases} m, & M \le mB, \\ \lceil M/B \rceil, & M > mB, \end{cases}$$

$$N_b = \lceil M/b \rceil.$$

这里 b 为块大小，N_b 为块数，都是工作项计数。 body 的最小块大小为 32；contact 的最小块大小为 4 个 **wide 接触槽**；joint 为 4 条关节。

四个 worker、1,024 个 body 时：

$$B = 16, \quad b = \lceil 1024/16 \rceil = 64, \quad N_b = 16.$$

若有 200 个 wide 接触槽，则块大小为 13，块数为 16，最后一块只有 5 项。 如果只有 5 个 wide 槽，按最小块大小 4 只能产生两块，增加到八个 worker 也造不出八份独立工作。

19.10 当前求解阶段表如何复用

这里只讨论并行调度所需的阶段结构，完整世界步进见第 16 章。

当前 Prepare 在大步开始执行一次，Store 在结束时执行一次。 每个子步重复：

1. 积分速度。
2. 按颜色 WarmStart。
3. 按颜色 Solve。
4. 积分位置。
5. 按颜色 Relax。

Prepare 与 Store 不要求按照颜色串行更新共享速度，可以使用跨颜色的平坦工作范围。 WarmStart、Solve、Relax 则必须遵守颜色边界。

设有效正常颜色数为 C。 当前 `ITERATIONS=1`、`RELAX_ITERATIONS=1` 时，阶段描述数为：

$$N_{\text{stage}} = 5 + 3C.$$

若 $C = 3$，只需要 14 个阶段描述。 4 个子步不会因此分配 56 个描述，而是复用阶段表并增加同步代数。

按执行次数计，开头两次 Prepare、最后一次 Store，再加每子步的 $2 + 3C$ 次阶段调用：

$$3 + 4(2 + 3 \times 3) = 47.$$

47 不是实际屏障次数，因为空阶段直接返回、单块阶段也有快速路径。 `b2SolverTask` 的旧注释仍列出 Restitution，但当前构建和执行阶段表没有独立 Restitution 阶段。

19.11 谁领取块，谁宣布这一色结束

`b2ExecuteStage` 让每个 worker 从分散的起始块开始扫描。 块上带有同步代数 `syncIndex`。

领取过程可以这样理解：

1. 尝试 CAS：只有块仍处于上一代，才能改成本代。

2. CAS 成功者独占执行这个块。
3. 其他 worker 发现它已被领取，就继续寻找其他块。
4. worker 把自己完成的块数原子加到阶段完成计数。

主调度路径 `b2ExecuteMainStage` 分三种情况：

块数	行为
0	直接返回
1	主调度者直接执行
大于 1	发布阶段、参与抢块、等待全部完成

发布信息 `atomicSyncBits` 的低 16 位是阶段索引，高 16 位是同步代数。 body、contact、joint、graph 共享块组各自维护递增代数。 不能简单断言整个 `syncBits` 在所有分组切换中始终数值单调。

代数的目的，是让复用的块区分“这一轮工作”和“上一轮已经完成的工作”。 完成计数达到块总数以后，才能让下一颜色读取更新后的速度。

worker 空等时会使用 pause，并定期 yield。 这些等待都是并行开销的一部分，不是免费同步。

19.12 为什么调用 Step 的线程还要参加

逻辑 worker 0 通过 `mainClaimed` 的 CAS 争夺 orchestrator 身份。 排队 worker 0 和调用 `b2World_Step` 的线程都可能进入这个竞争。 只有获胜者推进阶段，失败者返回。

这样，即使外部任务系统没有及时调度排队的 worker 0，调用线程仍能推动求解。 orchestrator 自己也处理块，因此其他工人无法及时运行时，仍有串行完成路径。

结束时发布 `UINT_MAX`，通知其他 worker 退出。 随后等待非空 task handle 完成，才回收有关临时数据。

通用 `b2ParallelFor` 是另一种实现。 它通过共享的 `nextBlock` 计数领取块，不是复用求解器的每块同步代数协议。 内置 scheduler 又在 task 层使用 Pending→Claimed→Complete 状态，等待任务时调用线程也会帮忙执行可用工作。

层次清楚以后，就不会把“任务系统的任务完成”和“同一求解颜色的屏障完成”混成一回事。

19.13 代价、测量与实验

这组优化的收益来自减少冲突并提高吞吐，但增加了：

- 着色和颜色数组维护。
- SIMD gather、scatter 与尾 lane 浪费。
- 工作块分配和原子领取。
- 颜色之间的等待。
- overflow 的串行保底路径。

可用 Amdahl 模型检查不合理预期。 若假设总时间中 20% 必须串行，80% 可以理想分给 8 个工人，则加速上界为：

$$S \le \frac{1}{0.2 + 0.8/8} = 3.33.$$

这是假设模型，不是 Box2D 的实测倍数。 实际还有屏障、负载不均、内存带宽和缓存开销。

建议测四组场景：

1. 无接触或极少接触，观察调度开销。
2. 大量互不相关的小堆，观察可并行规模。
3. 高连接度星图，观察 overflow 和色间串行。
4. 大量共享运动学平台接触，检查只读端的作用。

每组固定创建顺序、时间步和子步数，只改变 worker 数或 SIMD 配置。 同时记录颜色分布、overflow、wide 槽数量、有效 lane 比例、setup 和 constraints 耗时。 先通过确定性与线程回归测试，再解释速度结果。

19.14 自测与参考答案

1. 五边演算的颜色依次是什么？
2. 共享运动学平台的两条边为什么能同色？
3. 动态星图的第 21 条边去哪里？
4. 5 条接触、四宽向量的利用率是多少？
5. wide 索引 0 是否代表真实 `states[0]`？
6. 三个有效颜色对应多少个当前阶段描述？
7. 四个 worker 处理 1,024 个 body，块大小和块数是多少？

参考答案：

1. 0、1、0、22、22。
2. 只要动态端不冲突，平台状态只读且 scatter 不写回它。
3. overflow 23，不能使用为单动态端保留的正常颜色。
4. 62.5%。
5. 不是；0 是空端编码，真实 dense index 0 编码为 1。
6. $5 + 3 \times 3 = 14$。
7. 每块 64 个 body，共 16 块。

19.15 源码导航

主题	相对源码链接与入口行号
颜色分配	`constraint_graph.c`，67、217

主题	相对源码链接与入口行号
SIMD 平台与宽度	`core.h`，50
SSE2 gather/scatter	`contact_solver.c`，1415、1474
宽接触 Prepare/Solve	`contact_solver.c`，1654、2161
分块计算	`solver.c`，`b2ComputeBlockCount`：769
抢块与阶段等待	`solver.c`，934、973
orchestrator 与 worker	`solver.c`，`b2SolverTask`：1008
阶段表和尾槽初始化	`solver.c`，1255，重点 1333–1514
通用并行循环	`parallel_for.c`，33、64
内置任务调度	`scheduler.c`，55、176

返回 README · 下一章：确定性、精度与性能验证

20 确定性、精度与性能验证

> 目标：学会验证优化有没有改变不该改变的结果，并区分“快了”“更准了”“能重放了”这三种不同结论。
>
> 本章对应 Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b`。 本章的源码预期值来自静态核查；数值例为理论推导，不表示本次已经运行 C 测试或性能测量。

20.1 先定义成功，才能讨论优化

最朴素、也最容易比较的基线是：

- 固定创建与销毁顺序。
- 固定每帧输入。
- 固定时间步与子步数。
- 单线程执行。
- 保持同一编译配置。

以后每增加一个优化，先问它应保持什么不变。

验收目标	问题
物理正确性	接触方向、冲量符号、约束限制是否正确?
数值精度	与参考值或可接受误差相比偏差多大?
确定性	同条件相同输入是否逐位得到同结果?
性能	相同任务和质量要求下消耗多少时间、内存?

一个总是稳定算错的求解器可以非常确定。 一个更接近数学真值的 FMA 路径，也可能与另一平台产生不同末位。 只观察两个画面相似，不能证明其中任何一项。

还有一些优化本来就是受控近似。 例如 Contact recycling 改变了 manifold 刷新的时机。 比较开启与关闭回收的结果，应检查误差与业务行为，而不是预设两者必须拥有完全相同 hash。

20.2 并行计算，串行按稳定键提交

朴素并行方案常让 worker 算完一个接触就立即追加结果。 追加时加锁，可以保护容器不被写坏，却不能固定哪个 worker 先拿到锁。

这会沿着如下链条影响物理结果：

1. 完成顺序改变接触创建顺序。
2. 创建顺序可能改变接触 ID 和数组布局。
3. 布局与插入顺序影响图着色。

4. 颜色和约束顺序改变浮点迭代顺序。
5. 后续位置、接触和事件逐渐分叉。

所以当前实现并不是“所有数组最后统一排序”。 它在关键状态提交位置使用稳定索引或位图，将计算完成次序与提交次序分开。

接触状态位图

窄阶段每个 worker 拥有自己的 `contactStateBitSet`。 发生开始触碰、停止触碰或完全分离时，设置对应 `contactId` 的位。 这里使用持久 contact ID，而不是会迁移的 contact Sim 指针。

worker 完成以后：

1. 将所有位图按位或到一个集合。
2. 按 64 位字块索引递增遍历。
3. 每次通过 `ctz` 找最低置位。
4. 计算 `contactId = 64 * block + bit`。
5. 串行执行接触迁移、岛连接、着色和事件提交。

例如 worker 0 得到 $\{70, 2\}$，worker 1 得到 $\{9, 2\}$。 最终提交顺序是 2、9、70，重复的 2 只处理一次。 这个顺序不依赖谁先完成。

```
// Process contact state changes. Iterate over set bits
for ( uint32_t k = 0; k < bitSet->blockCount; ++k )
{
	uint64_t bits = bitSet->bits[k];
	while ( bits != 0 )
	{
		uint32_t ctz = b2CTZ64( bits );
		int contactId = (int)( 64 * k + ctz );
```

处理后使用 `bits & (bits - 1)` 清除最低置位，继续下一个 ID。 这是一项明确的稳定枚举协议，不是抽象地“排一下序就确定了”。

位图不是零成本

令 ID 容量 $N = 100000$，每块 64 位，每块 8 B。 每个 worker 的位图大小为：

$$\left\lceil \frac{100000}{64} \right\rceil \times 8 = 1563 \times 8 = 12504 \text{ B}.$$

8 个 worker 合计为 100,032 B，不包含结构体和分配器开销。 归约还需要约 7×1563 次 64 位 OR，以及清零和枚举成本。 当 ID 容量大而变化很少时，这些扫描也值得测量。

实验：固定输入，把 worker 数改为 1、2、4、8，逐帧比较 hash。 若首先发现接触创建或着色顺序不同，应先检查提交协议，而不是只调浮点容差。

20.3 宽阶段的确定性不是按原子分配顺序

并行配对会为每个 moved proxy 保留自己的 `moveResults` 槽。 候选 pair 的内存空间可以通过原子索引分配，但创建接触时不按这些分配地址遍历。

`b2UpdateBroadPhasePairs` 按 `moveArray` 顺序，依次消费每个查询自己的结果链表。 因此原子分配先后不直接变成接触创建先后。 前提是移动数组和每个查询的树遍历本身保持相应稳定性。

`FinalizeBodies` 对 enlarged body 的处理又用了另一种位图。 它按 `bodySimIndex` 标记，而非 contact ID。 这些稠密下标在该阶段内有效，所有 worker 位图 OR 后按索引顺序建立移动代理数组。

为什么睡眠要在后面？ 因为睡眠会 RemoveSwap 醒体数组，使旧 `bodySimIndex` 失效。 确定性不仅是排序问题，也包括索引在哪个生命周期里仍然有效。

fast bullet 的移动代理在非确定顺序 CCD 工作之前先进入确定的移动数组。 后续某些 bullet 列表遍历允许非确定顺序，是因为该操作被设计为不让顺序影响结果。 不能反过来推导“所有并行临时数组都必须排序”。

岛拆分候选还有显式平局规则：睡眠时间相等时选择较大的岛 ID。 同色约束只写互不共享的动态端，色间屏障则固定跨色依赖顺序。 这些协议共同维护确定性，没有一个单独的“确定性开关”能替代它们。

20.4 浮点数的间距可以手算

对于二进制 32 位浮点正常数，若数值位于 $[2^e, 2^{e+1})$，相邻表示值间距为：

$$\mathrm{ULP}(x) = 2^{e-23}.$$

它表示这个数量级附近的分辨率，不是一个复杂算法的完整误差上界。

位置量级	所在二进制区间	单精度间距
$10^5\ \mathrm{m}$	$[2^{16}, 2^{17})$	0.0078125 m
$10^6\ \mathrm{m}$	$[2^{19}, 2^{20})$	0.0625 m
$10^7\ \mathrm{m}$	$[2^{23}, 2^{24})$	1 m

在 $10^6\ \mathrm{m}$ 附近给一个可表示坐标增加 1 cm，增量可能在舍入时完全消失。 问题不是积分公式没有执行，而是结果落回了同一个浮点表示值。

这种误差会进入接触间距、关节锚点差和世界位置更新。 不能只把最终渲染坐标改成 double，就认为物理阶段已经恢复丢失的信息。

20.5 算式相同，不代表计算路径相同

浮点加法一般不满足结合律。 用无量纲标量 $a = 10^8$、$b = -10^8$、$c = 1$ 举例：

$$\mathrm{fl}(\mathrm{fl}(a + b) + c) = 1,$$

$$\mathrm{fl}(a + \mathrm{fl}(b + c)) = 0.$$

第二条路径中，$b + c$ 的 1 先被舍入丢失。 这说明即使没有数据竞争，改变约束计算次序也可能改变结果。

FMA 为什么又准又可能不同

设三个无量纲、可由 float 表示的数：

$$a = 1 + 2^{-23}, \quad b = 1 - 2^{-23}, \quad c = -1.$$

精确乘积为：

$$ab = 1 - 2^{-46}.$$

分开的乘法先把乘积舍入成 1，再加上 -1，结果为 0。 FMA 将乘加作为一次舍入，得到 -2^{-46}。

FMA 在这个例子中更接近精确值，却不等于另一条路径的结果。 所以跨平台确定性有时要求约束编译器的运算收缩行为。

当前顶层 CMake 对 GNU、Clang 使用 `-ffp-contract=off`。 clang-cl 则通过 `/clang:-ffp-contract=off` 传递到前端。 这是按编译器判断，不是只在某个操作系统才启用。 不要擅自加入 fast-math 后继续沿用原确定性承诺。

数学函数也属于配置合同

当前 `b2Atan2` 使用自定义近似表达式。 `b2ComputeCosSin` 使用角度归约、有理式近似和归一化。 它们不是简单调用系统 `atan2f/sinf/cosf` 后假定所有库相同。

这不意味着“所有浮点问题都被消灭”。 例如实现仍使用 `sqrtf`，实际支持范围还需要编译配置和测试来验证。 数值精度测试与确定性回归测试应分别保留。

20.6 双精度世界坐标，不等于整个求解器 double

朴素全 float 容易在远离原点时丢失小距离。 朴素全 double 则可能把大量局部向量、约束与热数据一并扩大。 当前采用混合精度边界。

数据	大世界模式下的精度
`b2Pos` 世界平移	double
`b2WorldTransform.p`	double
`b2WorldTransform.q`	float
`b2Vec2`、局部几何	float
线速度、角速度	float
求解增量	float
`b2AABB`	float

启用宏为 `BOX2D_DOUBLE_PRECISION`。 该宏通过 CMake 作为 PUBLIC 定义传递，因为库和调用者必须看到一致的头文件布局。 只改应用一侧宏会造成 ABI 不匹配。

关键是先相减，再降精度

令：

$$x_A = 10000000.0\ \mathrm{m}, \quad x_B = 10000000.25\ \mathrm{m}.$$

若先分别转成 float，再相减，两个数可能都变成 10000000，差为零。 若先用 double 相减，再转为 float，则得到可精确表示的 0.25 m。

```
/// a - b, demoted to float. The primary precision boundary operation.
B2_INLINE b2Vec2 b2SubPos( b2Pos a, b2Pos b )
{
	return B2_LITERAL( b2Vec2 ){ (float)( a.x - b.x ), (float)( a.y - b.y ) };
}
```

`b2InvMulWorldTransforms` 同样先计算世界平移差，再进入 A 的局部坐标系。 `b2UpdateContact` 在 A frame 中调用 manifold 算法，随后恢复世界朝向与相对锚点。

求解器还使用大步内的 `deltaPosition` 和 `deltaRotation`，避免每个子步直接在巨大绝对位置上累计微小变化。 大步结束再通过 `b2OffsetPos` 更新世界质心。

例如 0.001 m 附近 float 间距约为 $2^{-33} \approx 1.16 \times 10^{-10}\ \mathrm{m}$。 这比在 100 km 附近直接修改世界 float 坐标更适合表示小增量。 但局部浮点计算仍有误差，相对坐标不是精确算术。

20.7 为什么大世界的宽阶段仍可能变慢

世界位置可以 double，树的 AABB 仍是 float。 把 double 边界转换成 float 时必须保守向外舍入。 否则包围盒可能向内缩，导致真实候选被漏掉。

当前辅助函数分别向负无穷和正无穷取边界。 必要时使用 `nextafterf` 移到相邻 float。 几何 AABB 先在 double 位置上加入额外范围，再执行向外转换。

例如真实 x 区间为：

$$[10000000.1, 10000000.2]\ \mathrm{m}.$$

保守 float 区间可以变成：

$$[10000000, 10000001]\ \mathrm{m}.$$

实际宽度 0.1 m，被扩大成了 1 m。 这种转换避免漏碰撞，却增加了假阳性候选。 所以大世界模式不是“无限远、无限小物体、零性能代价”的承诺。

`b2ToVec2` 明确是有损转换。 如果业务先把远世界坐标降成 float 再交给引擎，后面无法恢复已丢失的信息。

实验：在原点和 x=10^7 m 处建立相同相对布局。 比较相对位置、命中数量和查询 fraction，同时记录候选访问数。 精度通过不代表树查询成本不变。

20.8 哪些测试使用容差，哪些必须精确相等

当前大世界金字塔测试在 double 模式下比较原点与 10^7 m 场景。 它要求睡眠步号相等，相对位置差使用 0.001 m 容差。 大世界查询测试对命中数量使用精确比较，对 fraction 和位置分别使用容差。

这些是在验证平移后的几何精度，使用 epsilon 是有意义的。 但不能据此说“Box2D 确定性测试只检查 epsilon”。

`test/test_determinism.c` 对睡眠步号和最终 hash 使用精确相等。 当前写在源码中的预期值为：

编译精度	预期 sleepStep	预期 hash
float 世界位置	245	`0x9F027FDF`
double 世界位置	356	`0x448C8E48`

这些值是当前提交的回归基准，不是本章作者本次运行得出的结果。 两种精度的预期不同，说明不能要求 float 与 double 使用同一个 golden hash。

`MultithreadingTest` 重复不同 worker 数，`BuiltInSchedulerTest` 检查内置调度，`CrossPlatformTest` 使用固定预期回归。 改变算法后若 hash 变化，应先分析原因，再决定是否有依据更新基准。 不能把更新预期值当作“修复确定性测试”。

20.9 输入录制、快照和状态 hash 各有职责

输入录制回答“用户按什么次序调用了哪些影响仿真的操作”。 快照回答“怎样从一个中途状态继续仿真”。 状态 hash 回答“两个执行过程在某个检查点是否出现可检测差异”。

只保存刚体位置和速度通常不足以无缝回放。 持久接触冲量、睡眠集合、ID 池、约束布局和树状态都可能影响下一步。 两份画面相同的世界，内部状态可以不同。

当前存在三类容易混淆的 hash：

入口	主要覆盖内容
Falling Hinges 测试中的 hash	按创建 body ID 顺序混入最终 `b2WorldTransform` 字节
`b2HashWorldState`	按活跃身份槽顺序混入位姿，以及存在 State 时的速度
`b2HashWorldStateDeep`	再加入部分索引、接触和关节冲量、ID 池统计等

浅 hash 不包含整个世界。 Deep hash 也不等于逐字段覆盖全部世界，例如它没有混入所有树节点和所有 manifold 几何字段。 公共 `b2World_GetStateHash` 使用 deep 路径。

hash 不同可以确定观察到的状态不一致。 hash 相同则仍存在覆盖范围和哈希碰撞的局限，不能当数学证明。

录制每步的 StateHash 时，当前世界仍保持 locked。 这样可以维持录制缓冲的写入协议，不在解锁后与其他操作竞争追加。

实验：逐帧比较而不是只比较最后一帧。 找到第一次分叉，再比较该帧前后的接触 ID、集合索引、冲量、位姿和速度。 这比看两段最终动画哪里不同更容易定位原因。

20.10 快照是续算工具，不是任意版本通用档案

当前快照头包含 magic、version、layout hash 和精度标志。 加载时拒绝不兼容精度与布局。 layout hash 混入多个关键结构大小和常量，也包括指针大小。

但同尺寸字段交换或含义重解释，未必改变结构大小。 因此格式版本仍需要人为维护，不能认为 layout hash 自动解决所有兼容性问题。

快照 API 使用两步缓冲模式：

1. `b2World_Snapshot(world, NULL, 0)` 查询需要大小。
2. 分配足够缓冲，再调用填写。

实际提供缓冲但容量不足时，函数返回所需大小，不复制超容量数据。 不能把返回值解释成“永远等于成功写入字节数”。

快照和恢复必须发生在世界未锁定时。 新建世界加载快照后，不携带原来的主机回调和用户指针，应由业务重新接线。 原地 Restore 保留世界身份与主机连接，但需要注意失败边界。

头部检查失败可以在不修改世界的情况下返回。 然而通过头部检查后，截断或损坏的主体仍可能在开始重建后失败；当前注释要求调用者销毁已经不可用的世界。 不要把所有失败都描述成事务式“原世界完全不变”。

已有 `SnapshotTest` 不只比较两份克隆是否互相一致。 它还从运动中的真实世界做快照，保存真实世界后续 90 步 hash，再让加载世界逐步复现。 这比“两份克隆以相同方式算错”更有检验价值。

20.11 现有测试入口怎样使用

测试可执行文件的第一个参数是精确测试函数名。 它不是子串过滤，也不是 `--filter=...`。 不传参数则运行全部注册测试。

下列为供读者执行的命令示例，非源码；本任务没有执行这些命令。 假设使用 Windows 多配置构建，输出位于 `build-perf/bin/Release`：

```
& .\build-perf\bin\Release\test.exe DeterminismTest
& .\build-perf\bin\Release\test.exe SnapshotTest
& .\build-perf\bin\Release\test.exe LargeWorldTest
& .\build-perf\bin\Release\test.exe DynamicTreeTest
& .\build-perf\bin\Release\test.exe ThreadTest
```

对应功能为：

测试	重点
`DeterminismTest`	worker、内置调度和固定精确预期
`SnapshotTest`	睡眠集合、恢复、持有 ID、续算和录制
`LargeWorldTest`	远坐标几何、查询与回放
`DynamicTreeTest`	查询、移动、重建及树结构
`ThreadTest`	线程相关行为

还有 `RecordingTest`、`RecordingKeyframeTest`、`RecordingScrubTest`、`ReStepRaceTest` 等独立入口。传参拼错时不能只看到程序结束就认为目标测试通过，应检查实际输出的测试名。

当前单元测试需要静态链接。 顶层 CMake 在 `BUILD_SHARED_LIBS=ON` 时会跳过该测试目标。 这里的测试目标名为 `test`，不是为每个函数单独创建一个可执行文件。 当前所查 CMake 文件也没有把这些函数注册成对应的 CTest 测试名。

20.12 建立可复现的性能基线

先记录以下条件，再看耗时数字：

- 提交 ID、编译器和版本。
- CPU、操作系统、架构与功耗环境。
- Release、Debug 或 RelWithDebInfo。
- SIMD、AVX2、double 和验证配置。
- worker 数、时间步、子步数。
- 场景创建顺序和随机输入。
- 睡眠、CCD、热启动、回收等功能开关。

60 Hz 的一帧预算为：

$$1000/60 \approx 16.667\ \text{ms}.$$

如果假设测得物理 10 ms、其他工作 6 ms，总体已接近预算。 接着应看物理内部是配对、窄阶段、求解还是维护占主导，而不是凭术语选择 SIMD。 这个数字例只是预算演算，不是仓库实测报告。

`b2Profile` 含嵌套和不同范围的计时字段。 不能把 `solve`、`constraints` 等重叠区间全部加起来当总时间。 先读计时位置，再解释字段。

构建命令示例，非源码，未在本任务执行：

```
cmake -S . -B build-perf -DBOX2D_SAMPLES=OFF -DBOX2D_UNIT_TESTS=ON -DBOX2D_BENCHMARKS=ON -
DBUILD_SHARED_LIBS=OFF
cmake --build build-perf --config Release --target test benchmark
```

单配置生成器的可执行文件路径通常不带 `Release` 子目录，应以自己的构建输出为准。 不同配置建议使用不同构建目录，避免混淆实际启用的宏。

20.13 读懂现有 benchmark 的统计口径

当前 benchmark 支持按名称选择场景，例如：

- `many_pyramids`、`large_pyramid`。
- `joint_grid`、`compounds`。
- `rain`、`smash`、`spinner`、`tumbler`、`washer`、`junkyard`。

命令参数示例为：

```
--benchmark=many_pyramids --threads=4 --workers=4 --repeats=5 --record-steps
```

这是命令参数示例，不是源码。 `--threads` 是最大线程数上限，实际还会受核数和最大 worker 常量限制。 `--workers` 同样会被限制，应读取输出确认实际运行数。

程序在计时循环之前先执行一个初始 Step，避免初始成本直接进入后面的总 run 时间。 Debug 模式只运行 10 步；Release 使用各场景配置的步数。 因此 Debug 的 10 步不能冒充正式场景成绩。

CSV 不是平均值

当前 `${name}.csv` 写的是同一 worker 多次 run 总耗时的最小值。 如果只测一个 worker，其他未运行 worker 的行可能为零。 这些零不是“这个线程数计算不花时间”。

`--record-steps` 也不是原始逐次样本

`${name}_tN.dat` 的七列依次为：

列	内容
1	`step`
2	`pairs`
3	`collide`
4	`constraints`
5	`transforms`
6	`refit`
7	`sleepIslands`

这些字段通过 `MinProfile` 分别取最小值。 不同列可能来自不同 run，不能把一行解释成同一真实帧的完整分解。 也不能用这些最小值样本计算 p95。

当前 profile 数组只在程序开始初始化一次，多个 worker 或场景循环之间没有重新清空。 为避免混合这项统计，严谨比较时每个进程只选择一个场景和一个 worker 数。 若需要中位数或尾延迟，必须保存每次独立运行的原始耗时或另建明确采样的测试入口。

benchmark 在当前目录写入结果，并覆盖同名文件。 应由读者选择独立结果目录，保留配置和原始日志，不要直接覆盖历史基线。

20.14 优化前后的比较矩阵

一次只改变一项条件，能减少解释歧义。

比较	保持不变	主要检查
1 worker 与多 worker	源码、输入、物理参数	精确回归与耗时
SIMD 与 NONE	算法、场景、worker	结果与吞吐
float 与 double	相对场景与输入	各自确定性、远坐标精度
睡眠开关	场景与输入	醒体数量、误差、维护成本
回收 0 与默认值	场景与输入	更新次数、几何误差、回调行为
CCD 开关	其余配置	质量变化，不只时间变化

关闭 CCD 后变快，并不代表同等碰撞质量下的性能优化。 增大睡眠阈值也可能用“更早停止仿真”换时间，应公开其质量变化。

加速比定义为：

$$S = T_{\mathrm{base}} / T_{\mathrm{new}}.$$

若未来实际测到 8 ms 与 6 ms，可报告 $S = 1.333$。 没有测量时，应留下待测表格，而不是写“数组快一个数量级”或“SIMD 必然四倍”。

推荐验收顺序：

1. 编译与内存安全检查。
2. 物理和几何测试。
3. 确定性、回放与快照测试。
4. 再运行统一条件的性能基准。
5. 最后分析收益是否值得新增复杂度。

20.15 自测与参考答案

1. 100 万米附近 float 的 ULP 是多少？
2. 远坐标差应先转 float 再相减，还是先相减再降精度？
3. 两 worker 分别标记 $\{70, 2\}$、$\{9, 2\}$，提交顺序是什么？

4. 浅状态 hash 是否覆盖全部世界?
5. `--record-steps` 当前输出能直接计算真实 p95 吗?
6. `DeterminismTest` 是否只做 epsilon 比较?
7. 为什么两份克隆互相一致还不够?

参考答案:

1. $2^{19-23} = 0.0625\ \mathrm{m}$。
2. 先以世界坐标精度相减，再转换局部差值。
3. 2、9、70，每个 ID 只处理一次。
4. 不能；deep hash 也有覆盖范围限制。
5. 不能，它按字段保存最小值，不是原始时间样本。
6. 不是，它对当前 golden sleepStep 与 hash 使用精确相等。
7. 两份克隆可能遗漏相同状态，应再与真实原世界未来轨迹比较。

20.16 源码导航

主题	相对源码链接与入口行号
FMA 与构建配置	`CMakeLists.txt`，50、65、116
混合精度与相对差	`math_functions.h`，50、627、649、683
自定义数学函数	`math_functions.c`，96、142
保守世界 AABB	`geometry.c`，405、430、485
接触确定提交	`physics_world.c`，740–883
录制帧末 hash	`physics_world.c`，1034
移动位图稳定消费	`solver.c`，1813
配对串行创建	`broad_phase.c`，469
浅状态 hash	`recording.c`，896
快照与 deep hash	`world_snapshot.c`，43、848、939、966、1021
精确确定性测试	`test_determinism.c`，18、163
大世界与快照测试	`test_large_world.c`，99、308；`test_snapshot.c`，227
测试过滤入口	`main.c`，50；`test_macros.h`，72
benchmark 统计口径	`benchmark/main.c`，40、143、327、375、405

返回 README · 上一章：图着色、SIMD 与多线程

21 • 源码勘误与实验手册：怎样证明自己真的理解了

返回总目录

> 本章既是 `readmegrok/` 的核对记录，也是新教程的验收手册。 先区分：**数学算术已复核、源码已核对、完整引擎实验待运行。**三者不能相互替代。

1. 本次验证边界

本书基线为 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b`，工作树声明版本 3.2.0。

本次文档工作完成的验证包括：

- 阅读并核对 `readmegrok/` 的全部主题。
- 对关键说明定位当前函数、分支和字段。
- 用独立 Python 算术复核自由落体、偏心冲量、Relax、摩擦、软度、二维关节点约束和简单 TOI 等数字。
- 区分精确分数计算、双精度计算与源码中的单精度结果；打印小数只作容差比较。

本章不声称执行了全部 C 测试，不声称实测了性能倍率，也不把教学伪代码称为完整 Box2D 的替代实现。

实际源码中包含速度上限、锁轴、子步几何近似、投影、恢复阈值、睡眠等条件。复现一个教学例子时，必须固定这些条件。

2. 原文修正清单：按证据而不是凭印象

2.1 数学、积分与质量

原文位置	需要修正的说法	正确理解与证据
`02-物理定律与数值积分.md`，自由落体段	显式欧拉走 60 步仍停在初始高度	显式也会下落；本书例子是 5.083333 m，半隐式是 4.916667 m，解析是 5 m
同文，积分稳定性	半隐式欧拉可被理解为任意步长都稳定	无阻尼振子仍受步长稳定条件限制；数值方法不是无条件保证
同文，圆角多边形质量	视为核心多边形加精确扇形积分	当前 `geometry.c` 第 323–335 行先向外推顶点，形成近似多边形再积分
`00-学习路线图.md`，逆质量段	浮点数没有无穷大	IEEE 浮点有无穷大；零逆质量是约束响应与实现约定
`README.md`，凸包源码地图	Andrew 单调链	`hull.c` 第 12–83 行的 `b2RecurseHull` 是 QuickHull 路线

相关源码：`solver.c`、`geometry.c`、`hull.c`。

2.2 接触与约束

原文位置	需要修正的说法	正确理解与证据
`04-约束与接触求解.md`，有效质量段	把 `1/K` 叫作有效质量的倒数	`K` 是逆有效质量，`normalMass=1/K` 是有效质量；`contact_solver.c` 第 123–131 行
同文，教材恢复对应段	`bias=-e*vn_old`	在接近速度为负、方程为 `vn+bias=0` 的约定下，应为 `bias=e*vn_old`
同文，完整手算段	Solve 后直接按原法向冲量算摩擦	Relax 先再解法向，然后才解滚阻和摩擦；第 267–385 行
同文，最小实现段	热启动只需保留 `accPn`	还必须把缓存冲量施加到新一步速度；第 140–215 行
`02`、`04` 的 Prepare 段	恢复速度在 Prepare 根据本帧接近速度算好	Prepare 只复制；旧步接近速度和冲量在 `contact.c` 第 632–645 行参与生成目标
`03-碰撞检测.md`，缓存段	将 ID 匹配与位姿阈值写成同一个热启动条件	几何回收和重建后 ID 匹配是不同路径，另有 GJK simplex cache
`03` 中 manifold 概括	一对刚体最多两点	单个形状对的 manifold 最多两点；复合刚体可以有多个形状对
`06-从零打造2D物理引擎.md`，盒子验收	斜盒子应恒有两个点并保持倾斜	点数由几何决定；在重力矩作用下自然转平可能完全正确

相关源码：`contact_solver.c`、`contact.c`、`physics_world.c`。

2.3 碰撞范围、关节和优化

原文位置	需要修正的说法	正确理解与证据
`03-碰撞检测.md`，CCD 段	普通 fast 物体查询全部三类树	普通 fast 查询静态树；bullet 额外查询运动学、动态树；目标 bullet 被跳过；`solver.c` 第 225–231、443–449 行
`01-物理世界框架.md`，配对段	移动代理只查询另外两棵树	移动 dynamic 也查 dynamic 树；见 `broad_phase.c` 第 370–393 行
`03`，过滤段	groupIndex 按同号/异号关系处理	必须两值相等且非零；正值允许，负值禁止，否则双向 category/mask；`shape.h` 第 122–129 行
`05-关节与并行.md`，距离关节	Hertz 为零就等于硬杆	要看 `enableSpring`、限位和长度分支；`distance_joint.c` 第 386–392、503–529 行
同文，mover	mover joint 是角色平面约束	当前是逐轴限力的相对线速度驱动；`mover_joint.c` 第 120–169 行
`01`、`05`，图着色	运动学必须占用颜色中的刚体位	当前只占用动态端；速度散射只写 dynamic，旧注释已落后于实现

原文位置	需要修正的说法	正确理解与证据
`05`，睡眠唤醒	从 BodySim 恢复原速度	唤醒创建 identity BodyState 并复制 flags；`solver_set.c` 第 57–62 行
`01`，求解集合	每个睡眠集合永远恰好一个岛	通常如此，但连接睡眠岛时可暂存多个岛；`solver_set.h` 第 54–56 行
`README.md`，分配器	每帧 O(1) 回收概括所有情况	实际逐次 LIFO 释放，空间不足有堆回退；`arena_allocator.c` 第 30–64 行

相关源码：`shape.h`、`constraint_graph.c`、`solver_set.c`、`arena_allocator.c`。

2.4 连源码注释也应验证的两个例子

1. `constraint_graph.c` 开头仍可见运动学着色的历史解释；当前判断和写回路径才是依据。
2. `b2SolverTask` 的阶段注释提到独立 restitution stage，但实际枚举和执行路径只有 Relax 中的恢复处理。

阅读源码不是只读注释。应沿“分支条件 → 实际调用 → 数据写回”验证。

3. 实验 A：自由落体，先验证积分再验证碰撞

3.1 实验设置

初始高度 10 m，初始速度 0，重力 −10 m/s²，无阻尼、无碰撞。每次积分步长 1/60 秒，执行 60 次积分。若通过公开 `b2World_Step` 对照，须设 `subStepCount=1`；若每个 1/60 秒大步使用 4 个子步，则一秒执行 240 次积分，半隐式结果约为 4.979166667 m，不能拿它与下表的 60 次积分结果直接比较。

3.2 解析解

匀加速位移为：

$$y(t) = y_0 + v_0 t + \tfrac{1}{2} g t^2.$$

作用： 提供独立于数值积分器的参考。

代入 1 秒：

$$y(1) = 10 + 0 - 5 = 5 \text{ m}.$$

3.3 显式欧拉为何偏高

第 k 步开始的速度是 $v_k = kgh$。先用它移动，再更新速度，累计高度为：

$$y_N = y_0 + gh^2 \sum_{k=0}^{N-1} k = y_0 + gh^2 \frac{N(N-1)}{2}.$$

每个量： N 为步数、h 为步长、g 为带符号加速度。求和表示每一步贡献的位移。

代入：

$$y_{60} = 10 - 10\frac{1}{3600}\frac{60 \times 59}{2} = 5.083333333\ \mathrm{m}.$$

因为每次使用下落速度较小的旧值，下落距离略少。

3.4 半隐式欧拉为何偏低

先更新速度，再积分位置，相当于使用第 1 到第 N 个速度：

$$y_N = y_0 + gh^2\sum_{k=1}^{N} k = y_0 + gh^2\frac{N(N+1)}{2}.$$

代入：

$$y_{60} = 10 - 10\frac{1}{3600}\frac{60 \times 61}{2} = 4.916666667\ \mathrm{m}.$$

两者最终速度都是 −10 m/s，位置却不同。只检查速度不能验证位置积分顺序。

3.5 已独立核对的答案

方法	1 秒高度	最终速度
解析	5	−10
显式欧拉	5.083333333	−10
半隐式欧拉	4.916666667	−10

已使用精确分数的 60 次递推与闭式求和相互验证。源码 float 运行时应采用合理容差，不要求打印小数逐位相同。

排错顺序： 先确认步长和步数，再看重力缩放、阻尼、锁轴、速度上限，最后才怀疑浮点误差。

4. 实验 B：一个完整大步的力不能只作用一个子步

第 16 章的设置：质量 2 kg，初始质心 `(5,10)` m，初始速度 `(1,0)` m/s，外力 `(4,0)` N，重力 `(0,-10)` m/s²，总步长 1/60 秒，两个子步。

已独立复核的结束值：

项目	结果
最终线速度	`(1.033333333, -0.166666667)` m/s
累计位移	`(0.017083333, -0.002083333)` m
最终质心	`(5.017083333, 9.997916667)` m

如果 x 速度只有 1.016666667 m/s，常见原因是第一个子步后提前清除了外力。

如果位移 y 是 −0.001388889 m，那是该大步的连续解析位移，不是这两次半隐式子步的结果；必须说明正在比较哪一种方法。

5. 实验 C：偏心接触，完整检查 Solve → 积分 → Relax → 摩擦

5.1 设置与简化边界

- A 静态，B 的质量 1 kg，转动惯量 0.5 kg·m²。
- 法线 `(0,1)`，切向 `(1,0)`。
- B 的杆臂 `(0.5,-0.5)` m。
- 只有一个接触点，无其他接触或关节；初始法向、切向和滚动冲量缓存均为零。
- 给定速度 `(0,-4)` m/s、角速度零，是进入 Solve 时的状态；本例此前不施加外力、重力或阻尼改变它。
- 初始分离 −0.02 m，子步 1/240 秒。
- 恢复、滚阻和切向目标速度均为零，摩擦系数 0.6。
- 为了对照原文，**人工采用** `massScale=1`、`impulseScale=0`、穿透 bias=−0.18 m/s。

这组人工软度不是默认 `b2MakeSoft` 的输出。真实软度对照在下一个实验。

5.2 有效质量

法向杆臂叉积为 0.5 m，因此：

$$K = 1 + 2 \times 0.5^2 = 1.5\ \text{kg}^{-1}, \qquad m_{eff} = 1/K = 2/3\ \text{kg}.$$

作用： 告诉我们单位法向冲量会使接触点法向速度增加多少。

5.3 带 bias 的 Solve

$$\Delta\lambda_n = -m_{eff}(v_n + b) = -\frac{2}{3}(-4 - 0.18) = 2.786666667\ \text{N·s}.$$

施加后：

- 质心 y 速度为 −1.213333333 m/s。
- 角速度为 2.786666667 rad/s。
- 接触点法向速度为 `-1.213333333 + 0.5 × 2.786666667 = 0.18` m/s。

注意质心仍向下，而接触点已经向上。因为右下角被顶起，物体开始转动。

5.4 积分位置，必须确认 Relax 采用哪个分支

该子步 y 位移为：

$$\Delta y = h v_y = -1.213333333/240 = -0.005055556\ \text{m}.$$

角增量输入为：

$$\delta = h\omega = 2.786666667/240 = 0.011611111.$$

本版 `b2IntegrateRotation` 从单位旋转出发，将 `(1,δ)` 归一化。独立计算得到：

- `deltaRotation.c = 0.999932597865`。
- `deltaRotation.s = 0.011610328497`。

按旋转锚点更新分离后为约 −0.019216690 m，仍小于零。因此 Relax 走“没有重叠修正 bias”的分支，而不是正分离的推测分支。

完整展开见第 12 章；这里强调：不能跳过几何更新就断言进入哪个分支。

5.5 Relax 先解法向

此时要去掉那 0.18 m/s 的修正速度：

$$\Delta\lambda_{relax} = -\frac{2}{3} \times 0.18 = -0.12 \text{ N·s}.$$

累计冲量从 2.786666667 降为 2.666666667 N·s。

新的质心 y 速度为 −1.333333333 m/s，角速度为 2.666666667 rad/s，接触点法向速度回到零。

负增量只是撤销先前一部分正冲量，累计值仍为正。

5.6 再算摩擦

切向有效质量同样为 2/3 kg。接触点右滑速度为：

$$v_t = 0 + 0.5 \times 2.666666667 = 1.333333333 \text{ m/s}.$$

未限幅的摩擦冲量为：

$$\Delta\lambda_t = -\frac{2}{3} \times 1.333333333 = -0.888888889 \text{ N·s}.$$

库仑上限为：

$$\mu\lambda_n = 0.6 \times 2.666666667 = 1.6 \text{ N·s}.$$

所需摩擦在允许区间内，因此不用截断。

5.7 最终状态与耦合残差

阶段	质心 x 速度	质心 y 速度	角速度	法向接触速度	切向接触速度
初始	0	−4	0	−4	0
Solve 后	0	−1.213333333	2.786666667	0.18	1.393333333
Relax 法向后	0	−1.333333333	2.666666667	0	1.333333333
摩擦后	−0.888888889	−1.333333333	1.777777778	−0.444444444	0

为什么最后法向又不是零？ 摩擦改变了角速度，角速度同时贡献法向和切向接触速度。这正是约束耦合，后续子步或迭代还需要继续协调。

如果你的推导得出“摩擦后所有约束都精确满足”，通常是漏掉了角速度对另一方向的影响。

6. 实验 D：真实软度系数与默认静态接触不是同一组

6.1 计算链与单位

对正频率：

$$\omega_0 = 2\pi f, \quad a_1 = 2\zeta + h\omega_0, \quad a_2 = h\omega_0 a_1, \quad a_3 = \frac{1}{1 + a_2}.$$

- f：频率，Hz。
- ω_0：角频率，s^{-1}，不是刚体角速度。
- ζ：阻尼比，无量纲。
- h：子步时长，s。
- a_1, a_2, a_3：无量纲中间量。

输出为：

$$\mathrm{biasRate} = \omega_0 / a_1, \qquad \mathrm{massScale} = a_2 a_3, \qquad \mathrm{impulseScale} = a_3.$$

作用： 第一项把位置误差转成速度尺度，后两项调节速度响应和累计冲量的反馈。完整弹簧阻尼推导见第 11 章。

6.2 代入 30 Hz

取 $f = 30$、$\zeta = 10$、$h = 1/240$：

中间量/结果	值
ω_0	188.495559215388
a_1	20.785398163397
a_2	16.324813543017
a_3	0.057720678928
`biasRate`	9.068652798161 s^{-1}
`massScale`	0.942279321072
`impulseScale`	0.057720678928

若分离 −0.02 m、速度上限 3 m/s：

$$b = \max(0.942279321072 \times 9.068652798161 \times (-0.02), -3) = -0.170904080034 \ \mathrm{m/s}.$$

若有效质量 2/3 kg、初始法向速度 −4 m/s、初始累计冲量零：

$$\Delta\lambda = -\frac{2}{3}\left(0.942279321072 \times (-4) - 0.170904080034\right) = 2.626680909548 \ \mathrm{N{\cdot}s}.$$

这才是该组真实软度公式的第一次更新，而不是人工 `massScale=1` 例子中的 2.786666667。

6.3 默认地面接触使用两倍频率

在默认世界、大步 1/60 秒、4 子步时，世界接触频率是 30 Hz。静态接触软度使用 60 Hz。

同样的 h、ζ、分离和速度条件下：

结果	60 Hz 静态软度
`biasRate`	17.476921700961 s^{-1}
`massScale`	0.971333010129
`impulseScale`	0.028666989871
重叠 bias	−0.339518219272 m/s
初次法向增量	2.816566839860 N·s

这些只是该时刻的首次更新，不是整个大步最终冲量。

6.4 边界测试：Hertz 恰好为零

源码显式返回三个零。不要使用正频率的极限关系来覆盖这个分支，也不要将函数注释中“两个缩放之和为一”无条件用于零频率特殊值。

7. 实验 E：二维铰点约束，不靠两个独立标量猜测

7.1 设置

A 为静态。B 的质量 1 kg、惯量 1 kg·m²，杆臂 `(1,-1)` m，初始质心速度 `(-4,-5)` m/s，角速度零。暂时不加位置 bias 和限位马达。

B 的接触/关节点要在两个坐标方向同时停止。

7.2 有效质量矩阵

这一设置的逆有效质量矩阵为：

$$K = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

对角线来自平动逆质量加旋转贡献，非对角线来自同一个角速度对 x/y 的耦合。

希望增加点速度 `(4,5)` m/s，因此要求：

$$K\lambda = \begin{bmatrix} 4 \\ 5 \end{bmatrix}.$$

行列式为 $2 \times 2 - 1 \times 1 = 3$。逐项求解：

$$\lambda_x = \frac{2 \times 4 - 1 \times 5}{3} = 1, \qquad \lambda_y = \frac{-1 \times 4 + 2 \times 5}{3} = 2.$$

冲量向量是 `(1,2)` N·s。

7.3 施加后再验证点速度

质心速度变成 `(-3,-3)` m/s。角冲量为：

$$r \times P = 1 \times 2 - (-1) \times 1 = 3 \ \text{N·m·s}.$$

除以惯量得到角速度 3 rad/s。此时转动贡献为 `(3,3)` m/s，与质心速度正好抵消，关节点速度为零。

这个例子直接说明：点停止，不代表质心和角速度都为零；求解两个方向必须考虑共同的旋转响应。

源码对应：`src/revolute_joint.c` 第 443–485 行和 `b2Solve22`。

8. 实验 F：先用解析圆扫掠验证 TOI 的时间单位

两圆圆心初距 5 m，半径均为 1 m。A 静止，B 沿两圆心连线以 10 m/s 接近。总扫掠时间 0.5 秒。

圆心距离随时间为：

$$d(t) = 5 - 10t.$$

接触时圆心距离等于半径和 2 m，因此：

$$5 - 10t = 2 \Rightarrow t = 0.3 \ \text{s}.$$

归一化扫掠比例为：

$$\alpha = t / \Delta t = 0.3 / 0.5 = 0.6.$$

检查点： 0.3 是秒，0.6 是时间比例。不要把 `fraction` 当成秒直接用于速度积分。

这是理想解析答案。Box2D 的 TOI 使用目标距离与容差，数值结果可能稍有偏差，不应把理想根强行设为逐位期望值。

源码与一般凸形求根见第 08 章、`src/distance.c` 的 `b2TimeOfImpact`。

9. 实验 G：先验证优化没有改变问题，再验证速度

9.1 全配对作为宽阶段正确性参照

对 n 个形状，不重复配对的数量为：

$$P = n(n-1)/2.$$

推导： 每个形状可和另外 $n-1$ 个配对，但 A/B 和 B/A 重复，所以除以二。

代入： 1000 个形状有 499500 对。

测试动态树时，不要求候选集合等于精确接触集合；宽阶段允许假阳性。需要验证的是：经过同样过滤后，真正可能发生接触的对没有被漏掉。

9.2 不要拿优化前后不同的仿真设置比较性能

固定物体数、形状、初始状态、随机种子、步长、子步数、睡眠开关、CCD 开关、编译模式、worker 数，再比较某一项变动。

如果优化版本只是让更多物体提前睡着，耗时下降不一定来自 SIMD。应同时记录 awake 数、接触数、候选数和 overflow 数。

9.3 内存精度实验

IEEE 单精度在一个固定二进制数量级内的间距，可按有效位数计算。对 1,000,000 附近：

$$\lfloor \log_2(1000000) \rfloor = 19, \qquad \mathrm{ULP} = 2^{19-23} = 0.0625\ \mathrm{m}.$$

作用： 估计在这个绝对坐标规模下，float 还能分辨多小的长度差。

0.005 m 的接触容差小于该间距。因此“先把绝对点放进 float，再相减”可能已经丢失想求的细节。

局部/相对计算和可选双精度世界位置各自解决什么问题，见第 20 章。使用 double 也不是所有局部计算自动变 double。

10. 当前仓库有哪些现成测试可对照

以下是已定位的测试入口；它们并不都一一对应本章的人工数字设置。

学习内容	文件与函数	基线行号
数学与变换	`test/test_math.c`，`MathTest`	14
形状质量	`test/test_shape.c`，`ShapeMassTest`	18
AABB	`test/test_collision.c`，`AABBTest`、`AABBRayCastTest`	11、29
GJK 距离	`test/test_distance.c`，`ShapeDistanceTest`	32
形状扫掠	同文件，`ShapeCastTest`	55
很短的边	同文件，`ShapeDistanceShortEdgeTest`	90
TOI	同文件，`TimeOfImpactTest`	160
恢复发生的阶段	`test/test_restitution.c`，`PhaseTest`	274
两点恢复	同文件，`TwoPointTest`	347
恢复阈值	同文件，`ThresholdTest`	553
静置接触	同文件，`RestingTest`	595
worker 对照	同文件，`WorkerParityTest`	675
确定性	`test/test_determinism.c`，`SingleMultithreadingTest`	26
大世界堆叠	`test/test_large_world.c`，`LargeWorldPyramidTest`	99
大世界 CCD	同文件，`LargeWorldBulletTest`	160

学习内容	文件与函数	基线行号
大世界查询	同文件，`LargeWorldOriginQueryTest`	308
快照	`test/test_snapshot.c`，`SnapshotTest`	227
记录回放	`test/test_recording.c`，`RecordingTest`	156
关键帧定位	同文件，`RecordingKeyframeTest`	776

构建配置依据：根 `CMakeLists.txt` 的 `BOX2D_UNIT_TESTS` 与静态链接条件；`test/CMakeLists.txt` 定义名为 `test` 的可执行目标。

注意：不能因为目录叫 `test/` 就假设已经注册了某个 CTest 测试命令。应先检查当前构建配置和实际产物。

11. 建议保存的实验记录字段

不需要一开始就做复杂可视化。每个子步输出一行以下数据，通常更容易定位错误：

类型	字段
时间	大步号、子步号、Δt、h
几何	A/B ID、法线、杆臂、分离量、特征 ID
质量	两端逆质量、逆惯量、法向/切向有效质量
速度	两端线速度、角速度、点速度、法向/切向相对速度
求解	bias、三个软度系数、增量冲量、累计冲量、摩擦限幅
位置	积分前后位移增量、旋转增量
生命周期	touching、persisted、缓存是否回收、是否发生 CCD、是否睡眠

最有用的对照不是“最后一个盒子的位置”，而是**第一个出现差异的阶段和字段。**

12. 如何定义验收指标

12.1 接触几何误差

若法线应为单位向量，检查其长度与 1 的差；若接触点应落在边上，检查到该边的距离。

例：法线 `(0,2)` 方向看似正确，却使投影速度扩大两倍、有效质量旋转项扩大四倍。只画箭头不检查长度容易漏错。

12.2 碰撞守恒必须满足实验前提

没有外力、没有摩擦、恢复系数为 1 的孤立对心碰撞，可检查总动量和动能。

若有地面、马达、阻尼、恢复耗散或位置 bias，不能要求所选动态刚体的机械能严格守恒。

例如质量都是 2 kg 的两个物体，以 3 m/s 和 −1 m/s 对心弹性碰撞后交换速度；碰前碰后总动量均为 4 kg·m/s，动能均为 10 J。这适合检验最小碰撞公式，不是复杂世界的通用能量标准。

12.3 堆叠验收要有多个指标

至少观察：

- 最大或典型穿透深度。
- 顶层位置漂移。
- 最终速度与角速度。
- 达到睡眠的步数。
- 唤醒后是否能正常响应。
- 更改子步数后的趋势。

不能只写“十个盒子不倒”。不倒也可能是摩擦被错误放大、速度被强制清零或关节意外锁住了自由度。

12.4 确定性不是“误差很小”

当前 `test_determinism.c` 第 52–58 行比较预期睡眠步数与 hash，要求精确相等。

数学算法误差通常用容差衡量；确定性测试关注同一输入与受控配置下是否产生一致结果。不要把两个目标混为一谈。

13. 故障定位顺序：从输入到输出，不先乱改参数

现象	先检查	再检查
自由落体快了数倍	单位、重力、步长是否乘了两次	是否每子步都额外施加一次本应仅一次的冲量
接触点总被吸住	法向方向、累计投影	是否把分离时的负增量误写成无界拉力
碰撞后朝错误方向旋转	叉积顺序、杆臂方向	A/B 次序、质心与原点混淆
热启动一开就爆	缓存 ID 是否匹配、质量是否更新	是否缓存既已存在于速度又被重复施加
摩擦像胶水	是否按累计法向冲量限幅	摩擦系数混合、切向方向、时间单位
穿透修正引起跳动	bias 单位、软度、限速	是否缺少 Relax、恢复符号是否反了
盒子落地不旋转	是否忘记角冲量	惯量、锁角设置、杆臂是否总为零
快物体偶尔穿墙	尺寸、运动尺度、CCD 开关及实际覆盖范围	TOI 容差、过滤、形状单面性
睡眠后唤醒异常	集合迁移和索引修复	缓存/标志、旧指针是否失效
多线程结果不一致	同色是否共享动态写目标	事件提交次序、原子操作、浮点配置

现象	先检查	再检查
增加 worker 反而更慢	场景规模、活跃颜色、overflow	同步、负载不均、缓存访问、任务系统

每次只改一类因素。把摩擦、软度、子步数、睡眠同时改掉，即使现象消失，也无法知道修对了什么。

14. 最后一组自测

读完本书，试着不看答案解释：

1. 为什么刚体质心仍向下，接触点却能已经向上？
2. 为什么有效质量可能小于任一物体的质量？
3. 为什么负冲量增量不一定是吸力？
4. 为什么位移修正完成后还要再解一次速度？
5. 为什么四个子步不等于在一个大步上扫四次固定约束？
6. 为什么碰撞 begin 与 hit 不应混用？
7. 为什么睡眠会让某些旧数组索引失效？
8. 为什么同色共享静态/运动学端可以安全，而共享动态写目标不行？
9. 为什么几何缓存、冲量缓存和 GJK 单纯形缓存必须分开理解？
10. 为什么优化前先做标量、小场景、可重复验收更省时间？

答案分别回到第 01、09、10、11、12、16、18、19、17、04/20 章。能用一个自己的数字例子解释，比能复述术语更能说明理解。

返回总目录